Metabolic engineering

66 weeks, 330 sessions — a chemical engineer’s route into the field

Every session, every task, every answer key, all 323 quiz questions with their answers, 56 worked problems with full solutions and code, and the complete resource list with live links. Printed from the interactive tracker so it can be read and annotated offline.

How to use this on an iPad

- Open it in Books, or in a PDF annotation app such as GoodNotes or Notability.
- Tick the ☐ boxes with an Apple Pencil as you finish each task.
- Use the outline or bookmarks panel to jump between phases and weeks.
- Links in the resources section are tappable.
- For streaks, spaced-repetition quizzing and automatic progress, use the interactive tracker instead — this document is the offline companion to it.

Five sessions a week, two and a half hours each, about twelve hours a week with two days spare. Phase checkpoints fall in weeks 10, 19, 51, 61 and 66; answer them from memory before reading the model answer. Weeks are tagged core, extension or optional as in the interactive tracker’s default pathway engineering and biofoundry profile: 45 core weeks (225 sessions); the rest are there when you want them and never count against you.

Phase 1 — Biology foundations for engineers

Weeks 1 to 10

Acquire the molecular vocabulary and quantitative biochemistry that metabolic engineering manipulates. By the end you can read a central-carbon map, write balanced reactions with cofactors, and explain how a gene becomes a working enzyme.

Primary texts: Lehninger 8th ed.; Alberts 7th ed.; MIT OCW 7.05 General Biochemistry.

Week 1 — The cell, the central dogma, and a map of the territory

Sessions 1–5

By the end of the week you can

- Distinguish prokaryote from eukaryote and name the compartments relevant to bioproduction
- State the central dogma and why it matters for engineering
- Say why E. coli and yeast dominate as hosts

The five sessions

Day 1 · Set up, then meet the cell · 2 h

☐ Make the study folder and the notebook file you will keep for 66 weeks; title it “Week 1, Day 1”

☐ Lehninger Ch. 1, the cellular-foundations section

☐ List five concrete differences between a bacterial and a eukaryotic cell in the notebook

> **Check your answer.** Five that matter for engineering: no membrane-bound nucleus (transcription and translation are coupled in bacteria); no mitochondria, so bacterial oxidative phosphorylation happens on the plasma membrane; no ER or Golgi, so no membrane-anchored P450s or eukaryotic secretion; circular chromosome plus plasmids rather than linear chromosomes; roughly 1–2 µm against 5–10 µm, which is a 100-fold volume difference and changes diffusion times and surface-to-volume ratio.

Day 2 · Chemical and genetic foundations · 2.5 h

☐ Lehninger Ch. 1, the rest of the chapter: chemical, physical, genetic and evolutionary foundations

☐ Watch MIT OCW 7.05 lecture 1

☐ Write down ten terms you did not recognise — you will meet all of them again by week 10

Day 3 · The central dogma · 2.5 h

☐ Alberts Ch. 1, skim; then read the central-dogma pages carefully

☐ Write out DNA → RNA → protein and name the enzyme that performs each arrow

☐ Note where each step happens in E. coli versus in yeast, and why coupling matters in bacteria

Day 4 · Compartments you can engineer · 2.5 h

☐ Alberts Ch. 2, the sections on membranes and organelles

☐ Draw an E. coli and an S. cerevisiae cell from memory and label every compartment

☐ Mark three compartments where a heterologous pathway could sit, with one reason each

Day 5 · Hosts, and consolidate · 2 h

☐ Write a paragraph answering “why E. coli, and why yeast” without looking anything up

☐ Reread your ten unfamiliar terms and define the ones you now can

☐ Daily quiz, then tidy the week's notes into one page

Topics this week covers

- Cell structure; cytosol, mitochondrion, ER, peroxisome
- Macromolecules: DNA, RNA, protein, lipids, carbohydrates
- The chemical logic of life
- Model-host overview

Sources

- Lehninger, Ch. 1 — The foundations of biochemistry
- Alberts, Ch. 1–2 — Cells and chemistry
- MIT OCW 7.05 General Biochemistry, lectures 1–2

Week 2 — Protein structure and enzyme function

Sessions 6–10

By the end of the week you can

- Explain the four levels of protein structure
- Describe how an enzyme lowers activation energy
- Relate active-site chemistry to specificity

The five sessions

Day 1 · Amino acids and the chain · 2.5 h

☐ Lehninger Ch. 3, amino acids and peptide bonds

☐ Draw the generic amino acid and group the twenty side chains into polar, non-polar, acidic and basic

☐ Note which side chains you would expect in an active site, and why

Day 2 · Folding, from chain to shape · 2.5 h

☐ Lehninger Ch. 4, the four levels of structure

☐ Write a one-line definition of primary, secondary, tertiary and quaternary structure and what determines each

☐ Watch MIT OCW 7.05 lecture 3

Day 3 · How enzymes work · 2.5 h

☐ Lehninger Ch. 6, the sections before kinetics: what enzymes do and how they lower activation energy

☐ Sketch a reaction-coordinate diagram with and without enzyme and label the activation energy

☐ Write out the six EC classes with one example reaction each

Day 4 · Cofactors and the heme case · 2 h

☐ Lehninger Ch. 6, the cofactor and coenzyme material

☐ Note the difference between an apoenzyme and a holoenzyme in your own words

☐ Look up heme: what it is, what it does, and which enzymes need it

Day 5 · Mine the databases · 2.5 h

☐ Pick one enzyme and pull its full record from BRENDA: EC number, substrate, cofactor, organism, k_cat and K_M

☐ Find the same enzyme in UniProt and note its length, organism and any domain annotation

☐ Daily quiz, then write three sentences on what you would need to express that enzyme in E. coli

Topics this week covers

- Amino acids and folding
- Enzyme classes and EC numbering
- Cofactors and prosthetic groups
- Heme as a worked example — it returns in week 55

Sources

- Lehninger, Ch. 3–6 — Amino acids, proteins, enzymes
- BRENDA enzyme database — browse one enzyme fully

- UniProt — the same enzyme's entry

Week 3 — Enzyme kinetics — your kinetics, re-pointed

Sessions 11–15

By the end of the week you can

- Derive and apply Michaelis–Menten kinetics
- Interpret k_cat, K_M and k_cat/K_M
- Explain competitive, non-competitive and allosteric inhibition

The five sessions

Day 1 · Rate laws, biological edition · 2.5 h

☐ Lehninger Ch. 6, the enzyme-kinetics section

☐ Derive Michaelis–Menten from the steady-state assumption on paper, every line of algebra

> **Check your answer.** You should end with v = V_max[S]/(K_M+[S]), where V_max = k_cat[E]_total and K_M = (k_{-1}+k_cat)/k_1. The two steps that matter: set d[ES]/dt = 0, and use the enzyme balance [E]_total = [E]+[ES]. If you got K_M = k_{-1}/k_1 you assumed rapid equilibrium instead of steady state — that is the Briggs–Haldane versus Michaelis–Menten distinction, and it only coincides when k_cat ≪ k_{-1}.

☐ Write one sentence connecting that derivation to quasi-steady state in your reaction-engineering notes

Day 2 · Reading the constants · 2.5 h

☐ Define V_max, k_cat, K_M and k_cat/K_M in your own words, each in one sentence

☐ Look up the diffusion limit for k_cat/K_M and note two enzymes that approach it

☐ Explain why K_M is not always a binding constant

Day 3 · Fit real data in Python · 3 h

☐ Install or confirm Python with numpy, scipy and matplotlib

☐ Simulate or take a v-versus-[S] dataset and fit it with scipy.optimize.curve_fit

> **Check your answer.** A good fit recovers V_max to within a few per cent and K_M less precisely, because K_M is constrained mainly by the low-[S] points. If your data stop below K_M you will not recover V_max at all — the curve never approaches its asymptote. Report standard errors from the covariance matrix (`pcov`), not just the point estimates.

☐ Report V_max and K_M with uncertainties, plot the fit, and save the script to your portfolio folder

Day 4 · Inhibition and control · 2.5 h

☐ Lehninger Ch. 6, the regulatory-enzymes section

☐ Tabulate competitive, non-competitive and uncompetitive inhibition by their effect on apparent K_M and V_max

☐ Sketch a sigmoidal cooperative curve beside a hyperbolic one and say what a Hill coefficient above one means

Day 5 · Feedback inhibition, the real obstacle · 2 h

☐ Read about allosteric feedback inhibition of a committed step

☐ Write the full answer to: why is feedback inhibition the biggest obstacle to overproducing an amino acid, and how do engineers remove it?

☐ Daily quiz

Topics this week covers

- Steady-state approximation — the same move as quasi-steady state in reaction engineering
- Lineweaver–Burk and why non-linear fitting is better
- Cooperativity and the Hill coefficient
- Feedback inhibition as the cell's control loop

Sources

- Cornish-Bowden — Fundamentals of Enzyme Kinetics, the chapters on inhibition and on fitting rate equations
- Shuler, Kargi & DeLisa — Bioprocess Engineering, the enzyme-kinetics chapter, for the engineer's treatment
- Lehninger, Ch. 6 — Enzymes
- Your own ChemE kinetics notes, read side by side

Week 4 — Bioenergetics and cofactors

Sessions 16–20

By the end of the week you can

- Use ΔG and ΔG°′ for coupled reactions
- Separate the roles of ATP, NADH, NADPH, $FADH_2$ and coenzyme A
- Explain why the NADH/NADPH distinction drives pathway design

The five sessions

Day 1 · Thermodynamics you already own · 2.5 h

☐ Lehninger Ch. 13, the bioenergetics and thermodynamics section

☐ Write the relation between ΔG, ΔG°′ and the mass-action ratio, and say what each term does

☐ Note the difference between ΔG°′ in biochemistry and ΔG° in your thermodynamics course

Day 2 · ATP and group transfer · 2.5 h

☐ Lehninger Ch. 13, phosphoryl-group transfers and ATP

☐ Explain why ATP hydrolysis is favourable in terms of the products, not "high-energy bonds"

☐ Work two coupled-reaction examples and confirm the coupled ΔG°′ is negative

> **Check your answer.** ATP hydrolysis to ADP + Pi has ΔG°′ ≈ −30.5 kJ/mol, and nearer −50 kJ/mol at cellular concentrations because the mass-action ratio is far from unity. A coupled reaction is favourable when the sum is negative: glucose + Pi → glucose-6-phosphate (+13.8) coupled to ATP hydrolysis (−30.5) gives −16.7 kJ/mol overall.

Day 3 · The two reducing powers · 2.5 h

☐ Lehninger Ch. 13, the biological oxidation–reduction section

☐ Tabulate NADH, NADPH, $FADH_2$ and coenzyme A: structure difference, redox state kept by the cell, and typical use

☐ Write why a design that confuses NADH with NADPH fails silently

Day 4 · Balance a product · 2.5 h

☐ Choose one target molecule and write its synthesis stoichiometry from glucose

☐ Close the ATP and the redox balance; state the surplus or deficit explicitly

☐ Say in one line what the cell would have to do to fix that imbalance

Day 5 · Redox balance under anaerobiosis · 2 h

☐ Read about fermentative product spectra and electron disposal

☐ Explain why anaerobic product distribution is fixed by electron bookkeeping rather than by preference

☐ Daily quiz, then consolidate the week into one page of balances

Topics this week covers

- Group-transfer potential
- Redox balance
- Catabolic versus anabolic reducing power
- Cofactor cost of a target molecule

Sources

- Lehninger, Ch. 13 — Bioenergetics and biochemical reaction types

Week 5 — Central carbon metabolism I — glycolysis and the pentose phosphate pathway

Sessions 21–25

By the end of the week you can

- Write out the Embden–Meyerhof–Parnas pathway and the PPP from memory
- Identify every node where carbon or cofactor can be redirected
- Explain why the PPP is the main NADPH source

The five sessions

Day 1 · Glycolysis, preparatory phase · 2.5 h

☐ Lehninger Ch. 14, glycolysis through the aldolase step

☐ Write the reactions in order with enzyme names; mark the two ATP invested

☐ Mark glucose-6-phosphate and fructose-6-phosphate as branch points and note what leaves there

Day 2 · Glycolysis, payoff phase · 2.5 h

☐ Lehninger Ch. 14, the payoff phase to pyruvate

☐ Finish the pathway and write the net equation including ATP and NADH

> **Check your answer.** Glucose + 2 NAD^+ + 2 ADP + 2 Pi → 2 pyruvate + 2 NADH + 2 H^+ + 2 ATP + 2 H_2O. Net 2 ATP, because 2 were invested in the preparatory phase and 4 recovered in the payoff phase.

☐ Mark PEP and pyruvate as branch points and name one product made from each

Day 3 · Pentose phosphate pathway · 3 h

☐ Lehninger Ch. 14, the pentose phosphate pathway section

☐ Draw the oxidative branch, marking both NADPH-producing steps; then the non-oxidative sugar rearrangements

> **Check your answer.** The two NADPH-producing steps are glucose-6-phosphate dehydrogenase and 6-phosphogluconate dehydrogenase; the second also releases CO_2, which is why oxidative PPP flux costs carbon. The non-oxidative branch (transketolase and transaldolase) makes no NADPH — it only rearranges 3-, 4-, 5-, 6- and 7-carbon sugars.

☐ Explain how flux splits at glucose-6-phosphate and what sets the split

Day 4 · Precursors and the twelve building blocks · 2.5 h

☐ Open KEGG map 00010 and 00030 and trace them against your drawings

☐ List the twelve precursor metabolites of biosynthesis

☐ Map each of the twenty amino acids to the precursor it comes from, using KEGG

> **Check your answer.** By family: glutamate, glutamine, proline and arginine from α-ketoglutarate; aspartate, asparagine, methionine, threonine, lysine and isoleucine from oxaloacetate; alanine, valine and leucine from pyruvate; serine, glycine and cysteine from 3-phosphoglycerate; phenylalanine, tyrosine and tryptophan from PEP plus erythrose-4-phosphate; histidine from ribose-5-phosphate (via PRPP).

Day 5 · Regulation of the pathway · 2 h

☐ Lehninger Ch. 15, principles of metabolic regulation — the sections on glycolytic control

☐ Note which glycolytic steps are regulated and which are near equilibrium

☐ Daily quiz

Topics this week covers

- Glycolysis energetics and intermediates
- Oxidative and non-oxidative PPP
- The twelve precursor metabolites

- Split ratios at the G6P node

Sources

- Lehninger, Ch. 14–15 — Glycolysis, gluconeogenesis, the pentose phosphate pathway
- KEGG pathway map 00010 and 00030

Week 6 — Central carbon metabolism II — TCA cycle and oxidative phosphorylation

Sessions 26–30

By the end of the week you can

- Trace the TCA cycle and explain anaplerosis
- Describe the electron transport chain and chemiosmosis
- Relate the P/O ratio to ATP yield

The five sessions

Day 1 · Pyruvate to acetyl-CoA · 2.5 h

☐ Lehninger Ch. 16, the pyruvate dehydrogenase complex

☐ Write the reaction with all five cofactors and the CO_2 lost

☐ Explain why this step is effectively irreversible and what that means for carbon accounting

Day 2 · The TCA cycle · 3 h

☐ Lehninger Ch. 16, the cycle itself

☐ Draw all eight reactions with enzyme names, marking every NADH, $FADH_2$ and GTP

> **Check your answer.** Per turn of the cycle: 3 NADH, 1 $FADH_2$, 1 GTP (or ATP), 2 CO_2. Per glucose that is doubled, since glycolysis gives two pyruvate. Pyruvate dehydrogenase adds a further NADH and CO_2 per pyruvate before the cycle starts.

☐ Mark the intermediates that get drained for biosynthesis

Day 3 · Anaplerosis · 2 h

☐ Lehninger Ch. 16, the anaplerotic reactions section

☐ Write the PEP carboxylase and pyruvate carboxylase reactions

☐ Explain what happens to the cycle without anaplerosis, even with plenty of glucose

Day 4 · Electron transport and chemiosmosis · 3 h

☐ Lehninger Ch. 19, through ATP synthase

☐ Draw the chain with its complexes and mark where protons are pumped

☐ Define the P/O ratio and compute ATP per glucose under full respiration

> **Check your answer.** With modern consensus stoichiometry — about 2.5 ATP per NADH and 1.5 per $FADH_2$ — complete oxidation of glucose gives roughly 30–32 ATP. Older textbooks say 36–38 using integer P/O ratios of 3 and 2. Quote the range and say which assumption you used; genome-scale models are sensitive to this choice.

Day 5 · Aerobic versus anaerobic · 2 h

☐ Compare ATP yield, product spectrum and growth rate between respiration and fermentation

☐ Answer: why would an anaerobic process still want flux through the oxidative TCA branch? Keep this answer — you will meet it again in week 54

☐ Daily quiz

Topics this week covers

- The acetyl-CoA node
- TCA as catabolic and biosynthetic hub
- Respiration versus fermentation

- The electron transport chain as an engineerable energy device

Sources

- Lehninger, Ch. 16–19 — TCA cycle, catabolism, oxidative phosphorylation
- MetaCyc central-metabolism overview

Week 7 — Biosynthetic metabolism — amino acids, lipids, nucleotides

Sessions 31–35

By the end of the week you can

- Follow the routes to amino acids, fatty acids and nucleotides
- Account for their precursor and cofactor demands
- Draw the mevalonate and MEP pathways

The five sessions

Day 1 · Nitrogen into carbon skeletons · 2.5 h

☐ Lehninger Ch. 22, nitrogen assimilation and the glutamate–glutamine node

☐ Write the glutamine synthetase and glutamate dehydrogenase reactions

☐ Note which precursor each amino-acid family branches from

Day 2 · Fatty acids and the acetyl-CoA drain · 3 h

☐ Lehninger Ch. 21, fatty-acid synthesis

☐ Write the acetyl-CoA carboxylase reaction with its ATP, biotin and bicarbonate

☐ Count the ATP and NADPH needed for one palmitate and write the total

> **Check your answer.** One palmitate (C16) needs 8 acetyl-CoA. Seven are carboxylated to malonyl-CoA at 1 ATP each, and the seven condensation cycles each consume 2 NADPH, giving 7 ATP and 14 NADPH in total. That NADPH demand is why oleochemical hosts are engineered for pentose phosphate flux.

Day 3 · Isoprenoids, mevalonate route · 2.5 h

☐ Read the mevalonate pathway in Lehninger Ch. 21 and in KEGG map 00900

☐ Draw acetyl-CoA to IPP and DMAPP, marking every NADPH and ATP

> **Check your answer.** Per IPP: three acetyl-CoA condense to HMG-CoA, HMG-CoA reductase consumes 2 NADPH to give mevalonate, then two kinase steps and a decarboxylation consume 3 ATP. So 2 NADPH and 3 ATP per five-carbon unit, before any downstream prenyltransferase or P450 chemistry.

☐ Mark HMG-CoA reductase as the committed step and note why it is the most engineered enzyme in the pathway

Day 4 · Isoprenoids, MEP route · 2 h

☐ Read the MEP pathway and compare it with mevalonate

☐ Note which organisms use which, and the cofactor difference between them

☐ Say which route you would use in E. coli and why engineers often import the mevalonate route instead

Day 5 · Nucleotides, then consolidate · 2 h

☐ Lehninger Ch. 22, nucleotide biosynthesis, read for the shape rather than every intermediate

☐ Note the precursor and the cofactor demand of purine synthesis

☐ Daily quiz, then one page summarising every biosynthetic drain on central metabolism

Topics this week covers

- Nitrogen assimilation
- Fatty-acid synthesis: acetyl-CoA, malonyl-CoA, heavy NADPH demand
- Isoprenoid precursors — mevalonate in yeast, MEP in bacteria

- Nucleotide biosynthesis

Sources

- Lehninger, Ch. 20–22 — Biosynthesis of lipids, amino acids, nucleotides

Week 8 — Molecular biology and gene regulation

Sessions 36–40

By the end of the week you can

- Describe replication, transcription and translation mechanistically
- Explain operons, promoters, ribosome binding sites, terminators and transcription factors
- Explain how expression level is tuned

The five sessions

Day 1 · Replication and transcription · 2.5 h

☐ Alberts Ch. 5 on replication, then Ch. 6 on transcription

☐ Write what a promoter is, in terms of what the RNA polymerase actually does there

☐ Note the role of sigma factors in bacterial promoter recognition

Day 2 · Translation and the code · 2.5 h

☐ Alberts Ch. 6, translation

☐ Explain what a ribosome binding site does and how it differs in role from a promoter

☐ Write why codon usage differs between organisms and what codon optimisation changes — and does not change

Day 3 · Gene regulation and the lac operon · 2.5 h

☐ Alberts Ch. 7, transcriptional control; then read the lac operon specifically

☐ Draw the operon with its promoter, operator, repressor and the three genes

☐ Explain mechanistically what IPTG does

Day 4 · Parts of an expression cassette · 2.5 h

☐ List the parts of a bacterial expression cassette in order, with the job of each

☐ Browse the iGEM registry for three promoters and three RBSs and note their reported strengths

☐ Write two reasons a large-scale process might avoid an inducible promoter

Day 5 · Annotate a real plasmid · 2 h

☐ Open any plasmid map in Addgene or Benchling

☐ Annotate it fully: origin, marker, promoter, RBS, open reading frame, terminator

☐ Daily quiz

Topics this week covers

- The lac operon
- Sigma factors
- Inducible versus constitutive promoters
- Parts of an expression cassette; codon usage

Sources

- Alberts, Ch. 4–8 (selectively) — DNA, chromosomes, gene control
- Lehninger, Ch. 24–28 (selectively) — Information pathways
- iGEM Registry — browse promoters and RBSs

Week 9 — Regulation: global networks, catabolite repression, proteome allocation

Sessions 41–45

By the end of the week you can

- Explain how a cell decides what to express on a given carbon source
- Name the global regulators that will fight your design, and how
- Relate proteome allocation to growth rate, and to the burden your pathway imposes

The five sessions

Day 1 · Catabolite repression and diauxie · 2.5 h

☐ Read a review of carbon catabolite repression; focus on CRP and cAMP in E. coli and CcpA in B. subtilis

☐ Draw the circuit: how glucose lowers cAMP, and what that does to the lac and other catabolic promoters

☐ Explain the diauxic growth curve from the circuit, and say what it means for a strain fed a mixed sugar stream such as hydrolysate

Day 2 · The stringent response · 2.5 h

☐ Read on ppGpp and the stringent response

☐ Write what the cell does to ribosome synthesis when it runs out of an amino acid, and why

☐ Note how a production pathway that drains an amino-acid precursor can trigger this by accident

Day 3 · Sensing oxygen and redox · 2.5 h

☐ Read on the ArcA/ArcB and FNR systems and on two-component signalling generally

☐ Draw one two-component system: sensor kinase, response regulator, and what it switches

☐ Write why an aerobic-to-anaerobic transition rewires expression before any enzyme is engineered

Day 4 · Growth laws and the proteome budget · 3 h

☐ Read Scott et al. 2010 on the interdependence of growth and gene expression

☐ Write the central result in your own words: protein mass is a fixed budget, and the ribosome fraction scales with growth rate

☐ Estimate the proteome fraction occupied by one pathway enzyme at high expression, and convert it into a growth-rate cost — this is the quantitative form of burden

> **Check your answer.** Order of magnitude: a pathway enzyme at strong expression commonly reaches 5–20% of total protein. Since the growth laws make protein mass a fixed budget, taking 10% of the proteome removes roughly 10% of the ribosomal and metabolic machinery that would otherwise support growth, so expect a growth-rate penalty of a similar order. That is why volumetric productivity, which is titer times growth, often peaks at intermediate expression rather than maximum.

Day 5 · Burden, and what regulation does to your design · 2 h

☐ Read a review of metabolic burden in engineered cells

☐ Write why a stronger promoter can lower titer, with the mechanism, not just the observation

☐ List three engineering moves you might make and name the global regulator that would oppose each

☐ Daily quiz

Topics this week covers

- Carbon catabolite repression: CRP and cAMP in E. coli, CcpA in B. subtilis, and diauxic growth
- The stringent response and ppGpp — what the cell does when starved
- Sigma factor competition and the general stress response
- Two-component systems as the cell's sensors
- Oxygen and redox regulation: ArcA/ArcB and FNR

- Bacterial growth laws: ribosome fraction rises with growth rate, so protein mass is a budget
- The burden of heterologous expression, and why a strong promoter can lower titer

Sources

- Alberts Ch. 7, the bacterial gene-regulation sections, reread with regulators in mind
- A review of carbon catabolite repression in bacteria
- Scott et al., “Interdependence of cell growth and gene expression”, Science 330:1099 (2010)
- A review of metabolic burden in engineered cells

Week 10 — Microbial genetics and microbiology

Sessions 46–50

By the end of the week you can

- Explain plasmids, mutation, recombination, horizontal gene transfer and selection
- Describe microbial growth and physiology quantitatively
- Define a chassis and what makes one industrially acceptable

The five sessions

Day 1 · Plasmids, copy number and burden · 2.5 h

☐ Read on plasmid replicons, copy number and segregational stability

☐ Explain how copy number affects gene dosage and metabolic burden

☐ Write two reasons industry prefers genomic integration to a plasmid, and one cost of doing so

Day 2 · Mutation, recombination, transfer · 2.5 h

☐ MIT OCW 7.03 Genetics, the lectures on recombination and gene transfer

☐ Define homologous recombination, transformation, conjugation and transduction in one line each

☐ Note how each is exploited as a laboratory tool

Day 3 · Selection and markers · 2.5 h

☐ Read on antibiotic markers, auxotrophic complementation and counterselection

☐ Define auxotroph and prototroph

☐ Write why antibiotic selection is avoided at scale and what replaces it

Day 4 · Growth, quantitatively · 2 h

☐ Review the batch growth curve and the specific growth rate from your bioprocess notes

☐ Write $\mu = (1/X)(dX/dt)$ and derive the doubling time

> **Check your answer.** $\mu = (1/X)(dX/dt)$, units of reciprocal time. Integrating gives $X = X_0 e^{\mu t}$, so the doubling time is $t_d = \ln 2/\mu \approx 0.693/\mu$. An E. coli μ of 1.0 h^{-1} is a 42-minute doubling.

☐ Define what makes a host a chassis, and what makes one industrially acceptable

Day 5 · Phase 1 checkpoint · 3 h

☐ Write the full stoichiometry of aerobic glucose catabolism to CO_2 with ATP, NADH and $FADH_2$, from memory

☐ Trace carbon from glucose to IPP, and draw how a plasmid gene becomes a cytosolic enzyme

☐ Self-quiz on the twenty terms, then grade yourself honestly and reread anything shaky

Topics this week covers

- Genotype and phenotype
- Auxotrophy and prototrophy
- Genome organisation and genetic stability
- Batch growth curve and specific growth rate μ — familiar ground

Sources

- Madigan et al. — Brock Biology of Microorganisms, the chapters on microbial growth, diversity and genetics
- Snyder & Champness — Molecular Genetics of Bacteria, on selections, transformation and conjugation
- MIT OCW 7.03 Genetics, selected lectures
- MIT OCW 7.13 Experimental Microbial Genetics

Phase checkpoint — worked in day 5

Answer each from memory, on paper, before reading the model answer.

☐ Write the overall stoichiometry of aerobic glucose catabolism to CO_2 including ATP, NADH and $FADH_2$

> **Model answer.** $C_6H_{12}O_6 + 6\ O_2 \rightarrow 6\ CO_2 + 6\ H_2O$. Accounting the carriers: glycolysis gives 2 ATP and 2 NADH; two pyruvate dehydrogenase reactions give 2 NADH and 2 CO_2; two turns of the TCA cycle give 6 NADH, 2 $FADH_2$, 2 GTP and 4 CO_2. Total 10 NADH, 2 $FADH_2$, 4 ATP equivalents from substrate-level phosphorylation, and about 30–32 ATP after oxidative phosphorylation at 2.5 and 1.5 ATP per carrier.

☐ Trace carbon from glucose to the terpenoid precursor IPP

> **Model answer.** Glucose → glycolysis → pyruvate → acetyl-CoA (losing C1 of pyruvate as CO_2). Three acetyl-CoA condense to HMG-CoA, HMG-CoA reductase reduces it with 2 NADPH to mevalonate, then two phosphorylations and a decarboxylation at 3 ATP total give IPP, which isomerises to DMAPP. Six carbons of glucose therefore become five carbons of IPP at best, which caps terpenoid yield.

☐ Explain with a diagram how adding a gene on a plasmid ends with a new enzyme in the cytosol

> **Model answer.** Plasmid enters by transformation and is retained under selection; its origin of replication sets copy number. RNA polymerase, directed by a sigma factor, binds the promoter and transcribes the gene; the ribosome binds the Shine–Dalgarno sequence and translates the open reading frame; the terminator stops transcription. The polypeptide folds, acquires any cofactor, and is active in the cytosol. Each of those steps is an engineering handle: copy number, promoter, RBS, codon usage, cofactor supply, folding.

☐ Self-quiz on 20 terms: operon, promoter, RBS, codon, plasmid, allosteric inhibition, anaplerosis, NADPH, acetyl-CoA, chassis and the rest

> **Model answer.** Mark yourself honestly on the twenty terms. Anything you could not define without looking it up should be reread before week 11 — Phase 2 assumes all of it.

Phase 2 — Core principles

Weeks 11 to 19

Acquire the conceptual and quantitative framework of the field itself. This is where chemical-engineering fluency accelerates you most — S·v = 0 is a mass balance you already know how to write.

Primary texts: Stephanopoulos, Aristidou & Nielsen (1998); Lee, Nielsen & Stephanopoulos (2021).

Week 11 — Definition, history and the design–build–test–learn cycle

Sessions 51–55

By the end of the week you can

- Define metabolic engineering and distinguish it from classical genetic engineering
- Narrate the field's history
- Articulate the DBTL cycle as an engineering design loop

The five sessions

Day 1 · Bailey, 1991 · 2.5 h

☐ Read Bailey, “Toward a science of metabolic engineering”, Science 252:1668

☐ Write his definition in your own words and underline the part that distinguishes it from classical strain improvement

☐ Note each thing he predicted would be hard

Day 2 · Network rigidity · 2.5 h

☐ Read Stephanopoulos & Vallino, Science 252:1675

☐ Write what network rigidity means and give one mechanism that causes it

☐ Connect it to the feedback-inhibition answer you wrote in week 3

Day 3 · The modern synthesis · 2.5 h

☐ Read Nielsen & Keasling, “Engineering cellular metabolism”, Cell 164:1185

☐ Note what changed between 1991 and 2016: tools, scale, and what stayed hard

☐ Log this paper in the paper log using the seven-point protocol

Day 4 · The DBTL cycle · 2 h

☐ Draw the design–build–test–learn cycle and write what happens at each stage

☐ Mark the stage that bottlenecks and justify the choice

☐ List the technologies that exist mainly to widen that bottleneck

Day 5 · Judge the predictions · 2 h

☐ Write one paragraph each on what Bailey predicted would be hard and whether it proved hard

☐ Skim the current issue of Metabolic Engineering and note three paper titles you want to understand by week 51

☐ Daily quiz

Topics this week covers

- Bailey's founding vision
- Stephanopoulos and Vallino on network rigidity
- From single-gene to systems and synthetic metabolic engineering
- Where DBTL actually bottlenecks

Sources

- Bailey, “Toward a science of metabolic engineering”, Science 252:1668 (1991)

- Stephanopoulos & Vallino, “Network rigidity and metabolic engineering in metabolite overproduction”, Science 252:1675 (1991)
- Nielsen & Keasling, “Engineering cellular metabolism”, Cell 164:1185 (2016)

Week 12 — Python for modelling I: numpy, scipy, and a notebook a stranger can run

Sessions 56–60

By the end of the week you can

- Manipulate matrices and solve linear systems in numpy without guessing
- Integrate differential equations and fit parameters with scipy, and read the uncertainties
- Produce a notebook and a repository that someone else can clone and run

The five sessions

Day 1 · Environment and arrays · 2.5 h

☐ Install Python 3.11 or later and create a project virtual environment; install numpy, scipy, pandas and matplotlib and record the exact versions

☐ Work the NumPy absolute-basics guide: array creation, shape, dtype, indexing, slicing, broadcasting

☐ Create the GitHub repository you will use for the rest of the curriculum and commit a README saying what it is for

Day 2 · The linear algebra you will actually use · 3 h

☐ Read the NumPy linear-algebra guide; practise np.linalg.solve, np.linalg.lstsq and np.linalg.matrix_rank

☐ Take a three-metabolite, five-reaction network, write S by hand, and find its null space with scipy.linalg.null_space; confirm the dimension matches your hand count

☐ Write down why you should use solve rather than inv(A) @ b, and check the condition number of a nearly singular matrix to see what goes wrong

Day 3 · Integrating differential equations · 2.5 h

☐ Read the scipy.integrate documentation for solve_ivp; note the f(t, y) argument order and what the solution object contains

☐ Integrate dA/dt = k1 − k2·A and a two-equation pair; plot the approach to steady state

☐ Add a terminal event that stops integration when a variable hits zero, then rerun with method='LSODA' and compare with 'RK45' on a deliberately stiff system

Day 4 · Fitting and reading the uncertainty · 2.5 h

☐ Read the scipy.optimize documentation for curve_fit and least_squares

☐ Fit Michaelis–Menten to the dataset from problem B1 and recover the standard errors from the covariance matrix

☐ Add bounds so no parameter can go negative, and write what changes — and what does not — when you do

Day 5 · Tables, figures, and a repository someone can use · 2.5 h

☐ Put your fitted results into a pandas DataFrame and export it to CSV

☐ Make one matplotlib figure with axis labels, units and a legend, saved to file rather than only shown

☐ Restart the kernel and run the notebook top to bottom; freeze requirements with pip freeze; commit and push

☐ Daily quiz

Topics this week covers

- Environment: venv or conda, pip, pinned versions, and why the version matters

- numpy arrays against Python lists; shapes, broadcasting, indexing
- Linear algebra that metabolic modelling actually uses: solve, lstsq, matrix_rank, null_space, and why you almost never invert a matrix
- scipy.integrate.solve_ivp: the right-hand-side convention, events, and stiff versus non-stiff solvers
- scipy.optimize: curve_fit, least_squares, bounds, and standard errors from the covariance matrix
- pandas for results tables; matplotlib for a figure with labelled axes and units
- Notebook hygiene and git: a README, a requirements file, and a notebook that runs top to bottom

Sources

- NumPy — the absolute basics and linear algebra guides
- SciPy — integration and optimization reference
- pandas — 10 minutes to pandas
- Matplotlib — pyplot tutorial

Week 13 — Stoichiometric modelling and network structure

Sessions 61–65

By the end of the week you can

- Build a stoichiometric matrix S
- Understand the null space, elementary flux modes and extreme pathways
- Set up intracellular mass balances at pseudo-steady state

The five sessions

Day 1 · Build S by hand · 2.5 h

☐ Stephanopoulos Ch. 8, network stoichiometry

☐ Write the stoichiometric matrix for a five-reaction toy network on paper

☐ State which rows are metabolites and which columns are reactions, and why that is just a mass balance

Day 2 · Solve it · 3 h

☐ Find a steady-state flux vector for your toy network by hand

> **Check your answer.** Any consistent answer works, but check two things: the number of independent balances equals the number of internal metabolites, and your flux vector satisfies every row of S exactly. For a simple linear chain with one branch, flux is conserved along the chain and splits at the branch point, so you should find one free parameter — the split ratio — unless you fixed an exchange flux.

☐ Repeat it in numpy using the null space of S

> **Check your answer.** `scipy.linalg.null_space(S)` should return the same solution space you found by hand, up to scaling. If the null space has more dimensions than you expected, you have more free fluxes than measurements — that is exactly the underdetermined case, and the dimension is the number of extra measurements or assumptions you need.

☐ Check the result against your hand solution and reconcile any difference

Day 3 · Degrees of freedom · 2.5 h

☐ Stephanopoulos Ch. 12, determined and underdetermined systems

☐ Count the degrees of freedom of your toy network and say what measurements would close it

☐ Write the three ways to resolve an underdetermined network

Day 4 · Elementary flux modes · 2.5 h

☐ Stephanopoulos Ch. 13, pathway structure; or the corresponding Palsson chapters

☐ Define an elementary flux mode and an extreme pathway, and say how they differ

☐ Enumerate the modes of your toy network by hand

Day 5 · Consolidate the algebra · 2 h

☐ Write one page explaining S·v = 0 to a chemical engineer who has never seen it

☐ Note what the null space represents physically

☐ Daily quiz

Topics this week covers

- S·v = 0 as a mass balance around every metabolite
- Degrees of freedom; determined versus underdetermined systems
- Elementary flux modes
- Measured exchange rates as constraints

Sources

- Stephanopoulos, Aristidou & Nielsen, Ch. 8, 12, 13
- Palsson, Systems Biology: Constraint-based Reconstruction and Analysis, Ch. 1–4

Week 14 — Equilibrium, driving force, and transport across the membrane

Sessions 66–70

By the end of the week you can

- Decide whether a reaction can carry flux in the direction you need, from ΔG′ at real concentrations
- Write the flux expression for passive diffusion, facilitated transport and active export, and say which applies to your product
- Put a transport step into a mass balance and see what it caps

The five sessions

Day 1 · Equilibrium, properly · 2.5 h

☐ Lehninger Ch. 13, reread the thermodynamics section — this time for ΔG′ rather than ΔG°′

☐ Write ΔG′ = ΔG°′ + RT ln Q and define Q, the mass-action ratio, in your own words

☐ Using published physiological metabolite concentrations, compute ΔG′ for the hexokinase, phosphofructokinase and triosephosphate isomerase steps; classify each as near or far from equilibrium

> **Check your answer.** Using the classic erythrocyte concentrations: hexokinase about −27 kJ/mol and phosphofructokinase about −26 kJ/mol, both far from equilibrium and effectively irreversible; triosephosphate isomerase close to zero, near equilibrium. Phosphoglucose isomerase, aldolase, phosphoglycerate mutase and enolase are also near equilibrium; pyruvate kinase is far from it at about −14 kJ/mol. The three far-from-equilibrium steps are precisely the three regulated ones — that is the general rule, and the values are organism- and condition-dependent, so quote your source.

Day 2 · The driving force of a whole pathway · 2.5 h

☐ Open eQuilibrator and pull ΔG°′ for every step of the pathway you are interested in

☐ Read Noor et al. 2014 on the max-min driving force and what a thermodynamic bottleneck means

☐ Identify the least favourable step in your pathway and state which concentration changes would rescue it, and whether the cell could actually hold those concentrations

Day 3 · Across the bilayer · 2.5 h

☐ Lehninger Ch. 11, the passive-transport sections

☐ Write Fick's law for a membrane: J = P·ΔC, and say what the permeability coefficient P contains — partition coefficient, diffusivity, membrane thickness

☐ Compute the diffusive flux of a small neutral product across 1 m² of membrane for a 10 mM inside-to-outside difference, then work out the pH partitioning of a weak organic acid and why it gets trapped as the anion inside

> **Check your answer.** With a permeability coefficient of about 10^{-6} cm/s (10^{-8} m/s), typical of a small neutral solute, and ΔC = 10 mM = 10 mol/m³: J = P·ΔC = 10^{-7} mol·m^{-2}·s^{-1}, about 0.36 mmol per square metre per hour. Compare that with a cell's production rate before assuming diffusion can clear it. For a charged species P falls by three to six orders of magnitude and diffusion is simply not an option.

Day 4 · Carriers, pumps, and the proton motive force · 3 h

☐ Lehninger Ch. 11, the active-transport sections

☐ Write the saturable rate law for a carrier and explain why a transporter, unlike diffusion, has a V_max you can hit

☐ Compute the proton motive force or ATP cost of exporting one mole of product against a tenfold concentration gradient, and tabulate which of your candidate products would need an active exporter rather than diffusion

> **Check your answer.** The thermodynamic minimum is RT·ln(c_out/c_in) = 8.314 × 310 × ln10 ≈ 5.9 kJ/mol per tenfold gradient at 37 °C. Against ATP hydrolysis at roughly 50 kJ/mol in vivo, one ATP could in principle drive about eight orders of magnitude; real transporters run far from reversibility, so budget one ATP, or one to two protons of the motive force, per molecule exported. Either way the cost belongs in the yield calculation.

Day 5 · Transport inside the balance · 2.5 h

☐ Add an export reaction to your week 13 toy network as a boundary flux and solve it again; constrain that flux and watch the steady state change

☐ Compute the characteristic diffusion time L²/D across a 1 µm cell and compare it with a typical enzyme turnover time; then do the same for a 10 m vessel and explain the difference in one line

> **Check your answer.** t ≈ L²/D. With D ≈ 5×10^{-10} m²/s and L = 1 µm: t ≈ 2 ms, far faster than an enzyme turnover of tens of milliseconds, which is why the cytosol is treated as well mixed. Put L = 10 m into the same expression and you get about 6,000 years — diffusion does nothing at vessel scale, which is why mixing is mechanical and why gradients exist at all.

☐ Write half a page on where transport, not pathway chemistry, sets the ceiling — keep it, section 5 of your capstone will need it

Topics this week covers

- Mass-action ratio Q against the equilibrium constant; ΔG′ = ΔG°′ + RT ln Q
- Reactions near equilibrium versus far from it, and why that decides where control can sit
- Thermodynamic feasibility of a designed pathway; the max-min driving force
- Fick's law across a bilayer: permeability coefficient, partition coefficient, membrane thickness
- pH partitioning and ion trapping of weak organic acids
- Carrier kinetics as a saturable rate law; symport, antiport, ABC transporters
- The proton motive force and the energetic cost of exporting against a gradient
- Characteristic diffusion time L²/D against turnover time — why a cell is well mixed and a 100 m³ vessel is not

Sources

- Lehninger Ch. 11 — Biological membranes and transport
- Lehninger Ch. 13, reread the thermodynamics section with concentrations in mind
- eQuilibrator — ΔG°′ for any biochemical reaction
- Noor et al., “Pathway thermodynamics highlights kinetic obstacles in central metabolism”, PLoS Comput. Biol. 10:e1003483 (2014)

Week 15 — Flux control and metabolic control analysis

Sessions 71–75

By the end of the week you can

- Compute flux control coefficients and elasticities
- State the summation and connectivity theorems
- Explain why the rate-limiting step is usually a myth

The five sessions

Day 1 · Control coefficients · 2.5 h

☐ Stephanopoulos Ch. 11, metabolic control analysis, the first half

☐ Define a flux control coefficient and compute one for a two-step toy pathway

☐ State the summation theorem and what it forbids

Day 2 · Elasticities and connectivity · 2.5 h

☐ Stephanopoulos Ch. 11, the rest

☐ Define an elasticity coefficient and distinguish local from systemic quantities

☐ State the connectivity theorem and what it lets you calculate

Day 3 · Work the numbers · 2.5 h

☐ Work two or three MCA problems from the chapter all the way through

☐ Verify that your coefficients satisfy the summation theorem

☐ Note where the assumptions of MCA would break in a real strain

Day 4 · Why single overexpression fails · 2 h

☐ Write the full answer to: why does overexpressing the rate-limiting enzyme usually not raise flux?

☐ Find one paper where overexpressing a single gene gave a small gain, and note the authors' explanation

☐ Connect this to multiplexed editing, which arrives in week 23

Day 5 · Consolidate · 2 h

☐ Write the interview answer: why does distributed control motivate multiplexed genome editing?

☐ Reread your week 11 notes on network rigidity and reconcile them with MCA

☐ Daily quiz

Topics this week covers

- The MCA formalism
- Distributed control
- Why overexpressing one enzyme rarely helps
- Implications for choosing targets

Sources

- Fell — Understanding the Control of Metabolism, where the summation and connectivity theorems are derived
- Stephanopoulos, Aristidou & Nielsen, Ch. 11 — Metabolic control analysis

Week 16 — Kinetic and dynamic modelling of pathways

Sessions 76–80

By the end of the week you can

- Turn a stoichiometric network plus rate laws into a system of ordinary differential equations

- Estimate parameters, and recognise when they are not identifiable from your data
- Run a sensitivity analysis and recover control coefficients from the model

The five sessions

Day 1 · From stoichiometry to differential equations · 2.5 h

☐ Write dc/dt = S·v(c) and say exactly what changed from the steady-state form you solved in week 13

☐ Write irreversible and reversible Michaelis–Menten rate laws, and one with competitive inhibition

☐ Assemble by hand the ODE system for a three-enzyme linear pathway with one feedback-inhibited step

Day 2 · Integrate and explore · 3 h

☐ Code that three-enzyme model in Python and integrate it with scipy.integrate.solve_ivp

☐ Plot the approach to steady state; verify the steady state matches what your stoichiometric solution predicted

☐ Change one V_max by tenfold and record what actually happens to flux and to the intermediate pools — the MCA result, seen rather than proved

> **Check your answer.** Flux should rise by only a few per cent, not tenfold, and the intermediate pools should shift more than the flux does. That is a flux control coefficient well below one, obtained numerically. If flux rose tenfold your model has a single irreversible step with no downstream saturation — check whether you left out the feedback term.

Day 3 · Parameter estimation · 3 h

☐ Generate noisy synthetic data from your model, then fit the parameters back with scipy.optimize least squares

☐ Plot the fit and the residuals; report the parameters you recovered against the truth

☐ Write which parameters came back wrong and why — this is your first encounter with identifiability

Day 4 · Identifiability and sensitivity · 2.5 h

☐ Read on structural versus practical identifiability

☐ Run a local sensitivity analysis: perturb each parameter by 1% and record the effect on steady-state flux

☐ Compute flux control coefficients numerically from those sensitivities and compare with the hand calculation from week 15

> **Check your answer.** C^J_E = (∂J/∂E)(E/J), computed from your 1% perturbations. They should sum to approximately 1 across all enzymes. If they sum to something else, the usual causes are a perturbation too large to be local, or a conserved moiety or fixed boundary concentration in your model that absorbs part of the response.

Day 5 · Dynamic flux balance analysis · 2.5 h

☐ Work through the COBRApy dynamic FBA documentation

☐ Simulate a batch culture: substrate depletion, growth, product accumulation over time

☐ Write in half a page when a kinetic model earns its parameters and when constraint-based modelling is the honest choice — a real judgement call you will be asked about

Topics this week covers

- Rate laws: irreversible and reversible Michaelis–Menten, convenience kinetics, inhibition terms
- dc/dt = S·v(c) — the dynamic form of the balance you wrote in week 13
- Numerical integration, stiffness, and why metabolic models are stiff
- Parameter estimation by least squares; practical and structural identifiability
- Local and global sensitivity analysis, and its relation to metabolic control analysis
- Ensemble modelling when parameters are unknown
- Dynamic flux balance analysis for batch and fed-batch

Sources

- Stephanopoulos, Aristidou & Nielsen, the chapters on cellular-reaction kinetics
- COBRApy — dynamic flux balance analysis
- A review of kinetic modelling in metabolic engineering
- SciPy integrate and optimize documentation

Week 17 — Experimental design, statistics, and when a difference is real

Sessions 81–85

By the end of the week you can

- Say what an error bar means, and design the replication that earns it
- Decide whether two strains really differ, and how many replicates that decision needs
- Propagate measurement error into a reported yield, and size a screen honestly

The five sessions

Day 1 · Replicates, and what an error bar means · 2.5 h

☐ Read the Nature Methods Points of Significance pieces on replication and on error bars

☐ Write the difference between a biological and a technical replicate, then say why three flasks inoculated from one colony is, for most questions, n = 1

☐ Decide for your own capstone experiment what the independent unit is, and state what standard deviation and standard error would each mean on that figure

Day 2 · Comparing two strains honestly · 2.5 h

☐ Take two strains measured in triplicate — A at 0.60, 0.64, 0.62 g/g and B at 0.65, 0.69, 0.64 — and run a t-test in scipy.stats

☐ Report the effect size and the 95% confidence interval on the difference alongside the p-value, and write which of the three you would put in a paper

☐ List the assumptions you just made, and note when Welch's correction or a non-parametric test is the honest choice

Day 3 · Power, and how many clones to screen · 3 h

☐ Compute the statistical power of that n = 3 comparison, then find the replicate number that would reach 80%

☐ Compute library coverage: for a library of N variants and k clones screened, the expected fraction seen is $1 - \exp(-k/N)$; evaluate it for a three-gene RBS library

☐ Write the sentence you would say in an interview about screening 96 wells of a large combinatorial library

☐ Compute the replicate numbers and the smallest detectable difference across a grid of effect sizes and noise, and the power of the usual n = 3 — problem U1

☐ Compute what technical replicates buy against biological ones, and write which you would add to your own experiment

Day 4 · Error propagation into a reported number · 2.5 h

☐ List every measurement behind one reported yield: HPLC peak area and calibration, dry weight or OD conversion, volume, dilution factor

☐ Propagate their relative errors into the yield, by hand and then in Python, and report the result to a defensible number of significant figures

☐ Identify which single measurement dominates the uncertainty, and say what you would improve first

Day 5 · Design of experiments · 2.5 h

☐ Read a review of design of experiments for media and feed optimisation

☐ Lay out a 2^3 full factorial for three medium components, then the fractional design that halves the runs, and state exactly which interaction you gave up

☐ Sketch how you would follow a screening design with a response-surface step, and when one-factor-at-a-time is defensible — which is rarer than it is practised

☐ Compare one factor at a time with a factorial design on the same invented truth, in two different testing orders — problem U2

☐ Write a randomisation and blocking plan for your own experiment, naming the nuisance factors it protects against

☐ Daily quiz

Topics this week covers

- Biological against technical replicates, and why n = 3 flasks from one colony is n = 1
- Standard deviation against standard error, and which belongs on which figure
- Comparing two strains: t-test assumptions, Welch's correction, non-parametric alternatives
- Effect size and confidence intervals against p-values; why a p-value alone is not a result
- Multiple comparisons when screening many strains, and false discovery rate control
- Statistical power: how many replicates a given difference needs before you can see it
- Library coverage: how many clones you must screen to have seen a variant at all
- Error propagation through a yield: HPLC calibration, dry weight, volume, dilution
- Design of experiments: full and fractional factorial screens, interactions, response-surface optimisation of media and feed
- One factor at a time is not merely inefficient: its answer depends on the order you happen to test in, and it cannot report an interaction at all
- Randomisation and blocking: plate position, day, batch, operator and instrument are real factors, and randomising is what stops them being mistaken for your effect (week 34)
- Stating the detectable effect size turns a negative result into information rather than an absence of one

Sources

- A statistics primer for biologists, from the Nature Methods Points of Significance series
- A review of design of experiments in bioprocess and media optimisation
- SciPy — statistical functions reference
- statsmodels — statistics in Python

Week 18 — Titer, rate, yield — and the design constraints

Sessions 86–90

By the end of the week you can

- Define and compute titer, rate and yield
- Compute theoretical maximum yield from stoichiometry
- Reason about the trade-offs that decide industrial viability

The five sessions

Day 1 · Define the metrics · 2 h

☐ Write precise definitions of titer, rate and yield with units

☐ Explain which of the three a shake-flask experiment actually measures well

☐ Note the difference between carbon yield, mass yield and molar yield

Day 2 · Theoretical maxima · 3 h

☐ Compute the theoretical maximum molar and mass yield of ethanol on glucose by hand

> **Check your answer.** Ethanol: 2 mol per mol glucose, so 2 × 46.07 / 180.16 = 0.511 g/g, with 2 CO_2 carrying the lost carbon. Real processes reach roughly 90–93% of this because carbon goes to biomass and glycerol. Always state the redox balance alongside: this one closes exactly, which is why it is a fermentation at all.

☐ Do the same for one product you care about, closing the redox balance

☐ Compare your numbers with published values and explain any gap

Day 3 · Read the BDO paper for numbers · 2.5 h

☐ Read Yim et al., Nat. Chem. Biol. 7:445, focusing on the reported titers and yields

☐ Note the lab titer and what commercial scale required

☐ Log the paper in the paper log

Day 4 · Which metric binds · 2.5 h

☐ Write why yield dominates for a commodity and titer for a pharmaceutical

☐ Sketch the cost consequence of a dilute broth: water to heat, separate and dispose of

☐ Write a back-of-envelope feedstock cost per kilogram of product at two different yields

Day 5 · Bottlenecks and byproducts · 2 h

☐ List the usual byproducts of an E. coli fermentation and where each comes from

☐ Explain how a byproduct destroys yield even when titer looks fine

☐ Daily quiz

Topics this week covers

- Carbon yield and thermodynamic feasibility
- Byproduct formation
- Which metric binds for a bulk chemical versus a pharmaceutical
- The gap between 18 g/L in a paper and 100+ g/L in a plant

Sources

- Doran — Bioprocess Engineering Principles, the material- and energy-balance chapters
- Yim et al., “Metabolic engineering of E. coli for direct production of 1,4-butanediol”, Nat. Chem. Biol. 7:445 (2011)
- A recent TRY-focused review in Metabolic Engineering

Week 19 — Cofactor balancing, redox and toxicity

Sessions 91–95

By the end of the week you can

- Diagnose cofactor imbalance and redox constraints in a design
- Reason about product and intermediate toxicity
- Name the standard fixes and their costs

The five sessions

Day 1 · Diagnose a cofactor imbalance · 2.5 h

☐ Take your week 4 product balance and state the NADPH or NADH imbalance precisely

☐ List what the cell must do per mole of product to close it

☐ Write what that costs in carbon

Day 2 · Three fixes · 2.5 h

☐ Read on pentose-phosphate rerouting, transhydrogenases and cofactor swapping

☐ Write each fix with its mechanism and its cost

☐ Choose the fix you would try first for your product and justify it

Day 3 · Driving force and thermodynamics · 2.5 h

☐ Read on thermodynamic bottlenecks and driving-force engineering in pathway design

☐ Identify the least favourable step in your chosen pathway

☐ Note what could be changed to pull flux through it

Day 4 · Toxicity and tolerance · 2.5 h

☐ Read Hartline et al., Metabolic Engineering 63:126, on dynamic control

☐ Write why dynamic regulation can beat static overexpression when an intermediate is toxic

☐ Note how compartmentalisation addresses the same problem differently

Day 5 · Phase 2 checkpoint · 2.5 h

☐ Write the maximum-yield stoichiometry for your target and state the cofactor surplus or deficit

☐ State the summation theorem and its engineering implication, from memory

☐ Read one Metabolic Engineering abstract, predict the design logic, then check yourself against the paper

Topics this week covers

- Cofactor swapping and transhydrogenases
- Driving-force engineering
- Tolerance, and why it is usually evolved rather than designed
- Compartmentalisation as a toxicity and cofactor strategy

Sources

- Hartline et al., “Dynamic control in metabolic engineering”, Metabolic Engineering 63:126 (2021)
- Nielsen & Keasling (2016), re-read with cofactors in mind

Phase checkpoint — worked in day 5

Answer each from memory, on paper, before reading the model answer.

☐ For a target of your choice, write the maximum-yield stoichiometry from glucose and identify cofactor surplus or deficit

> **Model answer.** Method: write the balanced synthesis from glucose, close the carbon balance, then close the redox balance with NADH and NADPH separately, then the ATP balance. A surplus of NADH means the pathway is more oxidised than the substrate and needs an electron sink or respiration; a deficit of NADPH means pentose phosphate flux or a transhydrogenase must supply it, at a carbon or energy cost you should state in the answer.

☐ State the MCA summation theorem and its engineering implication

> **Model answer.** The flux control coefficients of all enzymes in a pathway sum to 1. The implication is that control is shared: no single enzyme has a coefficient of 1, so raising one enzyme gives a sublinear flux gain and the limitation simply moves. Useful improvements require several coordinated changes, which is the argument for multiplexed editing and for library-based tuning.

☐ Define TRY and say which metric dominates for a bulk chemical versus a high-value pharmaceutical

> **Model answer.** Titer is concentration (g/L), rate is volumetric productivity (g/L/h), yield is product per substrate (g/g or mol/mol). For a bulk chemical, yield binds because feedstock dominates cost; titer matters next because a dilute broth makes separation expensive. For a high-value pharmaceutical, total output and purity dominate, and a few per cent of sugar is irrelevant.

☐ Read one Metabolic Engineering abstract and predict the design logic before reading the paper

> **Model answer.** No fixed answer. You are checking whether, from the abstract alone, you could name the host, the likely bottleneck and the type of intervention before reading the paper. If you could not, reread weeks 11 to 19 rather than pressing on.

Phase 3 — Widely used techniques

Weeks 20 to 52

For every major technique, know what it does, why it is used, and how common it is. You also start producing computational artifacts other people can inspect.

Primary texts: Addgene protocols; COBRApy docs; Palsson (2015); methods reviews.

Week 20 — DNA assembly and cloning

Sessions 96–100

By the end of the week you can

- Compare restriction–ligation, Gibson assembly, Golden Gate and MoClo
- Choose an assembly method for a given construct and justify it
- Design a working expression construct in silico

The five sessions

Day 1 · Why assembly methods matter · 2 h

☐ Read Addgene's “Plasmids 101: Golden Gate cloning”

☐ Write what a scar is and why it matters in a multi-part construct

☐ List the four assembly methods you will compare this week

Day 2 · Restriction, ligation, Gibson · 2.5 h

☐ Read Gibson et al., Nat. Methods 6:343

☐ Write the three enzymes in a Gibson reaction and the job of each

☐ Note the homology length required and where reliability falls off

Day 3 · Golden Gate and MoClo · 2.5 h

☐ Read a Golden Gate or MoClo methods guide

☐ Explain why a Type IIS enzyme makes one-pot, scarless, hierarchical assembly possible

☐ Draw a four-base overhang scheme that joins five parts in a defined order

Day 4 · Design the construct · 3 h

☐ Open Benchling and create a promoter–RBS–ORF–terminator construct on a real backbone

☐ Simulate a Gibson assembly of it and check the junctions

☐ Export the map to your portfolio folder

Day 5 · Choose and justify · 2 h

☐ Write the answer: six parts, each to be swapped independently later — which method, and why?

☐ Note the one case where you would still use restriction–ligation

☐ Daily quiz

Topics this week covers

- Restriction–ligation: classical, now largely superseded for multi-part work
- Gibson: isothermal, 15–40 bp homology, exonuclease plus polymerase plus ligase, scarless
- Golden Gate and MoClo: Type IIS enzymes cutting outside their recognition site, 4 bp overhangs, one-pot hierarchical assembly
- Why biofoundries standardised on modular assembly

Sources

- Addgene, “Plasmids 101: Golden Gate cloning”
- Gibson et al., Nat. Methods 6:343 (2009)

Week 21 — Multigene pathway construction — sequence tooling, Golden Gate design, and yeast in vivo assembly

Sessions 101–105

By the end of the week you can

- Read, search and edit sequences in code, so that a design check is a script and not an afternoon with a viewer
- Design a Golden Gate assembly whose overhangs, cut sites and part boundaries will actually behave
- Plan a yeast in vivo assembly from overlap homology, and judge how many parts it can carry

The five sessions

Day 1 · Sequences as data · 2.5 h

☐ Reread your week 20 notes on Golden Gate and Gibson before starting

☐ Install Biopython with pip install biopython and confirm it imports; open the Biopython Tutorial to the Bio.Seq and Bio.SeqIO chapters

☐ Download a plasmid as a GenBank file from Addgene or Benchling, read it with SeqIO.read, and list every feature with its type, location and strand

☐ Extract each coding sequence, translate it, and confirm the protein length matches the annotation; then reverse-complement one feature and check it by hand

☐ Put the feature list into a pandas DataFrame, sorted by position, and save it to CSV

Day 2 · Cutting geometry, and finding sites on both strands · 3 h

☐ Draw BsaI on paper: recognition GGTCTC, cut one base after it on the top strand and five on the bottom, leaving a four-base 5′ overhang; repeat for BsmBI (CGTCTC) and BbsI (GAAGAC, with spacing two and six)

☐ Write a function that finds a recognition site on both strands — the reverse complement of GGTCTC is GAGACC — and reports the overhang it would produce

☐ Run it on a coding sequence with a hidden internal site, remove the site with a synonymous codon change, and confirm by translation that the protein is unchanged — this is problem H1

☐ Repeat with the Bio.Restriction module and compare its report with your own function; reconcile any difference, since Biopython reports cutting positions rather than recognition positions

Day 3 · Overhang design · 3 h

☐ Read the ligation-fidelity paper’s main result: not all four-base overhangs ligate equally faithfully, and mismatched pairs can ligate

☐ Write a checker for an overhang set: no palindromes, no duplicates, no reverse-complement pairs, and no two overhangs one base apart on either strand

☐ Write down a set of seven by hand and see what the checker says; then generate one by search, adding a complexity filter that rejects homopolymers and extreme GC content

☐ Assign your overhangs to the junctions of a promoter–CDS–terminator transcription unit and write the assembly out as a table of parts and the overhangs at each end

Day 4 · Yeast in vivo assembly, and connecting transcription units · 2.5 h

☐ Read the abstract and introduction of the 2026 Trends in Biotechnology paper: why Golden Gate alone and yeast recombination alone each struggle with many transcription units, and what a connector sequence does

☐ Write out how four transcription units would be joined: which ends carry which overlap, and what the first and last overlaps do with the vector

☐ Compute the per-junction success implied by 80% overall for eight transcription units, then the overall success for three, five and eight units under that per-junction figure — problem H2

☐ Note what the independence model leaves out: mispairing between similar overlaps, recombination between repeated sequences, and mutations in correct-looking clones

Day 5 · Check the build before you order it · 2.5 h

☐ Write a script that assembles your four-unit design in silico from the part sequences, in order, with the overlaps you designed

☐ Translate every open reading frame in the assembled sequence and confirm each junction reads in frame; scan the final construct for stray BsaI and BsmBI sites

☐ List the primers that would verify each junction by colony PCR, with the expected band sizes, and commit the script and its output to your repository

☐ Daily quiz

Topics this week covers

- Sequences as data: FASTA and GenBank records, features, reverse complements and translation with Biopython
- Tables for parts and checks with pandas: one row per part, one column per property
- Type IIS cutting geometry: BsaI recognises GGTCTC and cuts one base downstream on the top strand and five on the bottom, leaving a four-base 5′ overhang; BsmBI and BbsI work the same way with different spacing
- Domesticating a part: finding internal sites on both strands and removing them with a synonymous change
- Overhang design: unique, non-palindromic, and at least two bases different from every other overhang and from its reverse complement
- Hierarchical assembly: parts into transcription units, units into pathways, in the MoClo and Yeast Toolkit tradition
- Yeast in vivo assembly: homologous overlaps between neighbouring fragments recombine inside the cell, and the number of junctions sets the efficiency
- Combining the two: build transcription units by Golden Gate in vitro, then join them in yeast through connector sequences — the approach of a 2026 paper from KRIBB, whose abstract reports up to eight transcription units at 80% efficiency in one transformation
- Verifying a build in silico: assemble the sequence in code, translate every open reading frame, and check every junction before anything is ordered

Sources

- Biopython Tutorial and Cookbook
- Biopython — working with restriction enzymes
- Seong et al., Connector-enabled integration of Golden Gate Assembly and yeast recombination for streamlined multigene pathway construction in the biofoundry workflow, Trends Biotechnol. (2026)
- Addgene — a destination vector deposited with that paper (academic and non-profit use)
- Potapov et al., Comprehensive profiling of four base overhang ligation fidelity by T4 DNA ligase, ACS Synth. Biol. (2018)
- Lee et al., A highly characterized yeast toolkit for modular, multipart assembly, ACS Synth. Biol. (2015)

Week 22 — Everyday assays: what the daily measurements actually mean [extension]

Sessions 106–110

By the end of the week you can

- Read a qPCR, a gel, a blot and a microscopy image critically, including their controls
- Say what each routine assay measures and what it cannot tell you
- Recognise the standard artefacts well enough to question a result in a lab meeting

The five sessions

Day 1 · qPCR, and what a fold change rests on · 3 h

☐ Compute the efficiency of two assays from their standard curves, and the fold change by delta-delta-Ct and by the Pfaffl method — problem U3

☐ Work out how much assuming perfect efficiency misleads you, and when that matters

☐ Write the controls a qPCR run needs and what each one rules out: no template, no reverse transcriptase, melt curve, standard curve

☐ Write why a reference gene has to be validated for your own conditions

Day 2 · Gels and what they show · 2.5 h

☐ Write what each of these means on an agarose gel: a band at the wrong size, a smear, two bands where you expected one, a bright front

☐ Write how you would tell a failed PCR from a failed gel, and what control distinguishes them

☐ Connect this to construct verification in week 21: what a gel can confirm and what needs sequencing

☐ Daily quiz

Day 3 · Protein: expression, solubility and blots · 2.5 h

☐ Write what an SDS-PAGE comparison of soluble and insoluble fractions shows, and link it to the triage in week 30

☐ Write what a Western blot establishes and what it does not, and why a loading control is required for any quantitative claim

☐ Write three reasons a band can be present while the enzyme is inactive

☐ Write the assay you would add to show activity rather than presence

Day 4 · Cells: flow cytometry and microscopy · 2.5 h

☐ Write what a dot plot shows, how gating is chosen, and why two people can gate the same data differently

☐ Write what a population average hides that a single-cell distribution reveals, and connect it to the biosensor work of week 26

☐ Write what you could and could not conclude from a fluorescence image of cells with inclusion bodies

☐ Write why optical density, colony counts and live/dead staining can disagree, and which you would trust for which question

Day 5 · Reading a results figure like a reviewer · 2.5 h

☐ Take four figures from your paper log, one of each type, and write for each: what was measured, which controls appear, and what claim it actually supports

☐ Write the question you would ask in a lab meeting about each one

☐ Write the artefact you would most expect to be mistaken for a result in your own planned work

☐ Daily quiz

Topics this week covers

- The point of this week: these are the measurements a lab makes every day, and not recognising them marks someone who has not been at a bench. You are learning to read them, not yet to run them
- qPCR: the amplification curve, the cycle threshold, a standard curve and its slope, efficiency, melt curves for specificity, and the no-template and no-reverse-transcriptase controls
- Relative quantification: the delta-delta-Ct method, its assumption that both targets amplify equally well, the Pfaffl correction when they do not, and why a reference gene must be validated and not assumed
- Gels: what size, smearing, a doublet, and primer dimers each tell you, and why a ladder is a control
- Protein: SDS-PAGE, the soluble and insoluble fractions (week 30), Western blotting, the loading control, and why a band is evidence of size and antibody binding rather than of activity
- Enzyme and reporter assays: initial rates, the linear range, blanks, and why an endpoint reading can mislead (week 3)
- Flow cytometry: what a dot plot and a histogram show, gating and its subjectivity, autofluorescence, and single cells against a population average (week 26)
- Microscopy: brightfield, phase contrast and fluorescence, what magnification does and does not buy, inclusion bodies as visible objects, and the difference between an image and a measurement
- Growth and viability: optical density is not cell number, colony counts measure what can grow, and live/dead staining measures membrane integrity
- The thread through all of it: every assay has a control that makes it interpretable, and the first question about any result is which controls were run

Sources

- PubMed search: the MIQE guidelines for reporting quantitative PCR experiments
- Pfaffl, a new mathematical model for relative quantification in real-time RT-PCR
- PubMed search: reference gene validation for qPCR normalisation
- PubMed search: Western blot loading controls and quantification pitfalls

Week 23 — Genome engineering I — recombineering, MAGE, CRISPR

Sessions 111–115

By the end of the week you can

- Explain lambda Red recombineering, MAGE and CRISPR nuclease editing
- Distinguish cutting from editing from regulating
- Design guide RNAs for a real target

The five sessions

Day 1 · Recombineering · 2.5 h

☐ Read Datsenko & Wanner, PNAS 97:6640

☐ Write the job of Exo, Beta and Gam, and why short homology arms suffice

☐ Draw the one-step knockout workflow from primer design to verified colony

Day 2 · MAGE · 2.5 h

☐ Read Wang et al., Nature 460:894

☐ Explain how cycled ssDNA oligos achieve multiplexing and what limits it

☐ Connect this to your week 15 answer on distributed control

Day 3 · CRISPR nucleases · 2.5 h

☐ Read a recent review of CRISPR in metabolic engineering

☐ Write the PAM requirements of SpCas9 and of Cas12a and when each is preferred

☐ Explain why an unrepaired Cas9 cut is lethal in bacteria

Day 4 · Design guides · 3 h

☐ Choose an E. coli gene to knock out and design three gRNAs in Benchling or CHOPCHOP

☐ Record every PAM site and the predicted off-targets for each guide

☐ Write the repair template you would supply alongside

Day 5 · Why CRISPR plus recombineering · 2 h

☐ Write the full answer on Cas9 as counterselection rather than as the editing enzyme in bacteria

☐ Write up the guide design as a portfolio item with your reasoning

☐ Daily quiz

Topics this week covers

- Lambda Red: Exo, Beta and Gam; one-step inactivation with short homology arms
- MAGE: cycled ssDNA oligos and Beta recombinase, many loci at once
- Cas9 and Cas12a; PAM requirements; why bacteria need a repair template
- CRMAGE and other hybrids

Sources

- Snyder & Champness — Molecular Genetics of Bacteria, on recombination, selection and counterselection
- Datsenko & Wanner, PNAS 97:6640 (2000)
- Wang et al., “Programming cells by multiplex genome engineering”, Nature 460:894 (2009)
- A CRISPR-in-metabolic-engineering review of your choice (2023 or later)

Week 24 — Genome engineering II — CRISPRi, base editing, dynamic control

Sessions 116–120

By the end of the week you can

- Explain CRISPR interference and activation
- Explain base editing and when it beats a double-strand break
- Explain dynamic regulation and metabolite biosensors

The five sessions

Day 1 · CRISPR interference · 2.5 h

☐ Read Cleto et al., ACS Synth. Biol. 5:375, on CRISPRi in C. glutamicum

☐ Explain how dCas9 represses without cutting

☐ Write why CRISPRi beats a knockout for essential genes

Day 2 · Activation and base editing · 2.5 h

☐ Read on CRISPRa and on base editing with deaminase fusions

☐ Write when a single-base change is preferable to a double-strand break

☐ Note what each technique cannot do

Day 3 · Biosensors · 2.5 h

☐ Read a metabolite-biosensor review

☐ Draw how a transcription-factor biosensor converts a metabolite concentration into fluorescence

☐ List three metabolites for which good sensors exist

Day 4 · Dynamic control · 2.5 h

☐ Reread the Hartline dynamic-control review with the tools now in hand

☐ Draw a feedback loop that throttles an upstream enzyme when an intermediate accumulates

☐ Write what you would measure to prove the loop works

Day 5 · Consolidate the toolkit · 2 h

☐ Write one sentence per tool from weeks 20 to 24: what it does and when you choose it

☐ Decide which tools your capstone will use and note why

☐ Daily quiz

Topics this week covers

- dCas9 repression: tunable, reversible, multiplexable
- CRISPRa in bacteria and yeast
- Deaminase fusions for single-base changes
- Feedback control inside the cell

Sources

- Cleto et al., “C. glutamicum metabolic engineering with CRISPR interference”, ACS Synth. Biol. 5:375 (2016)
- A metabolite-biosensor review

Week 25 — Genome design, synthesis and non-model host toolkits

Sessions 121–125

By the end of the week you can

- Explain how a genome is designed, synthesised and installed, and what recoding buys and costs
- Choose a non-model host for a product and say what toolkit you would have to build first
- Compute the edit counts, the error budget and the time consequences of a rewriting or host-switching project

The five sessions

Day 1 · Why rewrite a genome · 2.5 h

☐ Read the abstract of Fredens et al. (2019) and write the recoding rules: the two serine codons TCG and TCA to AGC and AGT, and the TAG stop codon to TAA

☐ List what recoding is for: free codons for new chemistry, a barrier against viruses and foreign DNA, and a test of what is essential

☐ Write the costs you can see: 18,214 edits, overlapping genes that needed hand correction, and a strain that grew more slowly than its parent

☐ Log the paper in the paper log with one question

Day 2 · Counting the edits and the error budget · 3 h

☐ Work out the edits per kb, the share of codons changed and the number of sections — problem N1

☐ Compute the chance that a whole genome is error-free at three residual error rates

☐ Write why sequence verification after every stage, not only at the end, is the only workable design

☐ Daily quiz

Day 3 · Building, installing and minimising · 2.5 h

☐ Write the build hierarchy: oligonucleotides, fragments, sections of 91 to 136 kb, then replacement of the genome piece by piece

☐ Read the KRIBB project listing on ScienceON (the title and participants only, there is no abstract) and Byung-Kwan Cho’s entry on the PhD prep tab, and note what each says genome design is for

☐ Write why “essential” depends on the growth conditions, and what that does to a minimal-genome project

☐ Revisit the integration and counterselection material of week 24 and say which tools a section-by-section replacement would need

Day 4 · Choosing a non-model host · 3 h

☐ Compute the time saved by a faster host and the number of cycles needed to repay a toolkit — problem N2

☐ Compute the DNA needed to cover a library at three transformation efficiencies

☐ Fill one line for each host — E. coli, Vibrio natriegens, Corynebacterium glutamicum, Yarrowia lipolytica, Cupriavidus necator, Streptomyces — with why it is attractive, what it lacks, and which person on your shortlist works with it

☐ Write when you would stay in E. coli and move only the final strain

Day 5 · A toolkit plan · 2.5 h

☐ Pick one non-model host and write its toolkit checklist: transformation method, replicon, marker, characterised promoters and RBS, an editing tool, curing, a reporter, an annotated genome

☐ Write the first three experiments, each with a control and a pass mark

☐ Write what would make you abandon the host

☐ Daily quiz

Topics this week covers

- Why rewrite a genome: free codons for new chemistry, a barrier against viruses and foreign DNA, and a test of what is essential
- The 61-codon E. coli: a 4 Mb synthetic genome with 18,214 codons recoded, built from sections of 91 to 136 kb (Fredens et al., 2019)
- Design before synthesis: choose target codons, check overlapping genes, keep regulatory elements, and refactor where the design requires it
- Build hierarchy: oligonucleotides to fragments to sections to a genome, with sequence verification at every stage
- Installing a synthetic genome: replace the natural one piece by piece, with selection and counterselection
- Minimal genomes: what counts as essential depends on the conditions, so a minimal genome is minimal for something
- Non-model hosts: growth rate, tolerance, substrate range, native pathways, secretion and regulatory status are the reasons to leave E. coli
- A toolkit checklist: transformation method, replicon, marker, characterised promoters and RBS, an editing tool, curing, a reporter, an annotated genome
- Fast growth in practice: Vibrio natriegens divides about twice as fast as E. coli, is naturally competent, and grows on formate and acetate

Sources

- Fredens et al., Total synthesis of Escherichia coli with a recoded genome, Nature (2019)
- Weinstock et al., Vibrio natriegens as a fast-growing host for molecular biology (preprint, 2016)
- Efficient natural plasmid transformation of Vibrio natriegens enables zero-capital molecular biology, PNAS Nexus (2024)

- KRIBB project listing: Platform technologies for genome design and synthetic biology (title and participants only)
- PubMed search: Standard European Vector Architecture (SEVA) and broad-host-range vectors

Week 26 — Genetic parts, transcription-factor biosensors, and circuits

Sessions 126–130

By the end of the week you can

- Characterise a part so that its number means something outside your own plate reader
- Fit a biosensor's dose–response and state its dynamic range and operating window with honest uncertainty
- Say what a sensor can and cannot do in a screen or in a feedback circuit

The five sessions

Day 1 · Parts, and what characterisation means · 2.5 h

☐ Read how a promoter's strength is reported relative to a reference standard, and why absolute fluorescence units do not travel between instruments

☐ Write a measurement plan for three promoters with a reference included on every plate: the replicates, the growth phase you measure in, and the normalisation by cell density

☐ List three ways the same promoter can behave differently in another plasmid or host — copy number, upstream sequence, ribosome binding site context — and what you would hold fixed

☐ Write how you would detect that a part slows growth

Day 2 · Transcription-factor biosensors · 2.5 h

☐ Read a review of transcription-factor-based biosensors and draw the generic architecture: regulator, operator, promoter, reporter

☐ List four metabolite sensors with their regulators and the organism each was built in, and the ligand analogues each also responds to

☐ Write which sensor you would use for your capstone product, and what you would do if none exists — screen a regulator library, or evolve one

☐ Note the failure modes: leaky baseline, saturation at low concentrations, host-specific behaviour, and ligand uptake

Day 3 · Fit a dose–response · 3 h

☐ Take the simulated dose–response data from problem I1 and fit the four-parameter Hill function with scipy.optimize.curve_fit, bounding the parameters sensibly

☐ Report K, n, baseline and maximum with their standard errors, then compute the dynamic range and the ligand window between 10% and 90% of maximum response

☐ Refit with and without zero-ligand replicates and compare how well the baseline and the dynamic range are determined

☐ Write which single extra measurement would have tightened the estimate most

Day 4 · From sensor to screen · 2.5 h

☐ Read how fluorescence-activated sorting enriches a library: the fraction of true hits recovered against the fraction of background that gets through the gate

☐ Compute the enrichment per round for a gate that recovers 70% of hits and 1% of background, and the hit prevalence after one, two and three rounds from one in ten thousand

☐ Compute how many events you must sort for ten-fold coverage of a million-member library, and how long that takes at your instrument's rate

☐ Write why a sorted hit is a hypothesis: the sensor reports the intracellular signal, not necessarily the titer

Day 5 · Circuits, burden, and feedback · 2.5 h

☐ Write the steady state of a Hill-activated gene with production, dilution and decay, and see how it depends on the input

☐ Sketch a sensor-driven feedback loop around a toxic intermediate, starting from week 24, and write what you would measure to prove it works

☐ Write the burden penalty a strong constitutive promoter imposes, and under what conditions a sensor-controlled one would win

☐ Daily quiz

Topics this week covers

- Promoters, ribosome binding sites and terminators as parts; strength reported relative to a reference standard, and why it depends on context
- Burden: expression costs growth, and a circuit that slows its host is selected against
- Architecture of a transcription-factor biosensor: the regulator, its operator, a promoter, and a reporter
- Dose–response: baseline, maximum, half-maximal concentration and Hill coefficient
- Dynamic range, operating window and leakiness — and why dynamic range is the worst-determined of them
- Specificity and cross-talk: ligands that are close analogues of the target
- From sensor to screen: fluorescence-activated sorting, enrichment per round, and how many cells you must sort to cover a library
- From sensor to control: using the sensor to drive an enzyme so that flux follows the cell's own state, as in week 24
- A first circuit model: Hill-type activation, production and decay, and the steady state they imply

Sources

- A review of transcription-factor-based biosensors for metabolic engineering
- Kelly et al., Measuring the activity of BioBrick promoters using an in vivo reference standard, J. Biol. Eng. (2009)
- A paper using a malonyl-CoA biosensor for dynamic pathway regulation
- SciPy — curve_fit documentation
- iGEM Registry of Standard Biological Parts

Week 27 — Regulatory control: sRNA, riboswitches, quorum sensing and CRISPRi [extension]

Sessions 131–135

By the end of the week you can

- Choose a control layer for a given engineering goal and defend the choice
- Compute how far a repressor or an sRNA can turn a gene down, and what limits it
- Read an sRNA, CRISPRi or quorum-sensing metabolic-engineering paper critically

The five sessions

Day 1 · Choosing a control layer · 2.5 h

☐ For each goal choose a layer and say why: turn a competing pathway down, switch production on at high density, tune an enzyme across a ten-fold range, knock down an essential gene

☐ List what each layer costs: genome edits, burden, leak, speed of response, how portable it is to another host

☐ Read the abstracts of the sRNA and CRISPRi papers and note which goal each was built for

☐ Log one of them in the paper log with a question

Day 2 · How far will a regulator turn a gene down? · 3 h

☐ Compute the extra decay rate that gives 2-, 10- and 100-fold mRNA knockdown — problem O1

☐ Compute the best possible repression of a repressor with a given leak, and the amount needed for each target

☐ Write what the 100-fold figure implies for the design, and what you would combine instead

☐ Daily quiz

Day 3 · CRISPRi and multiplex repression · 2.5 h

☐ Write how dCas-based repression works and where it differs from deleting a gene

☐ Read the multiplex-repression paper's abstract and write what flux change it reported and how it was measured

☐ Revisit the CRISPR design material of week 23 and say what you would reuse for guide design

☐ Write the controls a CRISPRi experiment needs

Day 4 · Dynamic control and quorum sensing · 2.5 h

☐ Write the growth–production trade-off and how a two-stage or density-triggered switch addresses it

☐ Read the abstract of a quorum-sensing metabolic-engineering paper and note what triggers expression and what is controlled

☐ Connect it to the biosensor material of week 26: when would you use a sensor and when a clock

☐ Write one failure mode of dynamic control

Day 5 · A knockdown plan · 2.5 h

☐ Design a knockdown of a competing gene in your capstone pathway: the layer, the fold you aim for, and the arithmetic behind it

☐ Write what you would measure at mRNA, protein and flux, and what you would conclude if flux did not move

☐ Add the controls and a pass mark

☐ Daily quiz

Topics this week covers

- Control layers: transcription (promoter and repressor), RNA (riboswitch and sRNA), translation (RBS strength), protein (degradation tags), population (quorum sensing)
- Why tune and not delete: essential genes, competing pathways that need reducing and not removing, and the growth–production trade-off
- Synthetic small regulatory RNAs for metabolic engineering in E. coli: many genes knocked down without editing the genome (Sang Yup Lee's group, 2013)
- CRISPRi: catalytically dead Cas proteins with guide RNAs repress several competing genes at once (Dae-Hee Lee's group, 2017); Sang Woo Seo's group has a CRISPRi toolkit for Vibrio natriegens
- Quorum sensing: expression triggered by cell density, used by Min-Kyu Oh's group to control a natural product in Streptomyces coelicolor
- Static and dynamic control: two-stage processes and biosensor-driven feedback (week 26)
- Steady-state arithmetic: fold repression, leak, and the ceiling a regulator can reach
- Judging a knockdown: measure mRNA, protein and flux, and watch for burden

Sources

- Yoo et al., Metabolic engineering of Escherichia coli using synthetic small regulatory RNAs, Nat. Biotechnol. (2013)
- CRISPR interference-guided multiplex repression of endogenous competing pathway genes for redirecting metabolic flux in Escherichia coli, Microb. Cell Fact. (2017)
- PubMed search: quorum-sensing-based metabolic engineering in Streptomyces coelicolor (Min-Kyu Oh's group)
- PubMed search: CRISPR interference in Vibrio natriegens (Sang Woo Seo's group)

- KRIBB Synthetic Biology Research Center — research areas and publications

Week 28 — Libraries, expression tuning, protein engineering, evolution

Sessions 136–140

By the end of the week you can

- Use promoter and RBS libraries to tune expression quantitatively
- Explain directed evolution and what makes a screen work
- Explain adaptive laboratory evolution and how causal mutations are found

The five sessions

Day 1 · Expression libraries · 2.5 h

☐ Read on promoter and RBS libraries for pathway balancing

☐ Open the Salis Lab RBS Calculator and work through its documentation

☐ Note what the calculator predicts and how well it is reported to do

Day 2 · Design an RBS library · 2.5 h

☐ Design three RBSs spanning a predicted hundred-fold translation-rate range for one gene

☐ Record the predicted rates and the sequences

☐ Write how you would combine them combinatorially across a three-enzyme pathway

Day 3 · Directed evolution · 2.5 h

☐ Watch or read MIT OCW 7.344 on directed evolution

☐ Write the mutagenesis-and-screening loop and the rule that you get what you screen for

☐ Give one example where a screen optimised the wrong property

Day 4 · Adaptive laboratory evolution · 2.5 h

☐ Read an ALE review, ideally one on solvent tolerance

☐ Write how causal mutations are identified after evolution

☐ Explain why tolerance is usually evolved rather than designed

Day 5 · Growth-coupled thinking · 2 h

☐ Write how a selection differs from a screen, and why selections scale better

☐ Sketch a selection that would enrich for your capstone product

☐ Daily quiz

Topics this week covers

- Promoter libraries and the RBS calculator
- You get what you screen for
- ALE: impose pressure, sequence the survivors
- Growth-coupled selection

Sources

- Salis Lab RBS Calculator
- MIT OCW 7.344 Directed Evolution: Engineering Biocatalysts
- An adaptive laboratory evolution review

Week 29 — Structure prediction and computational enzyme design — literacy

Sessions 141–145

By the end of the week you can

- Read a predicted structure and its confidence metrics without over-trusting it
- Say which tool answers which question, and what each cannot do
- Write a defensible enzyme-engineering paragraph for your own proposal

The five sessions

Day 1 · Read a predicted structure properly · 2.5 h

☐ Fetch an AlphaFold prediction for one enzyme in your pathway from the AlphaFold database

☐ Interpret the per-residue pLDDT and the predicted aligned error: which regions are trustworthy, which are effectively a guess

☐ Write why a high-confidence fold tells you little about the active site geometry that matters for catalysis

Day 2 · What each tool answers · 2.5 h

☐ Read the ColabFold paper for what structure prediction does and does not give you

☐ Build a one-page table: question asked, tool, output, and the thing that output cannot establish

☐ Note explicitly that ProteinMPNN is ligand-blind and LigandMPNN is not — this single distinction is the most common error in student proposals

Day 3 · Ligand-aware design, conceptually · 3 h

☐ Read the LigandMPNN paper and the IPD announcement

☐ Write how modelling cofactor atoms changes a pocket redesign, and why an NADPH-to-NADH cofactor swap needs ligand-aware design

☐ Sketch, on paper, the redesign you would request for one step of your pathway: which residues, which cofactor, what you would measure

Day 4 · Variant effect prediction and its limits · 2.5 h

☐ Read on protein language models and zero-shot variant effect scores

☐ Write what a high score does and does not predict, and why correlation with measured activity is partial

☐ State how you would use such a score — to rank a library you will still screen, not to pick one variant

Day 5 · Write the paragraph · 2 h

☐ Draft the enzyme-engineering paragraph for your proposal: the step, the problem, the tool, the experiment that tests the prediction

☐ Have it name a specific tool correctly and include the screen that follows — a design with no assay is not a plan

☐ Daily quiz

Topics this week covers

- AlphaFold2, ColabFold and AlphaFold3: what each predicts, and the access terms that differ between them
- Confidence metrics: pLDDT, PAE, ipTM — and why a confident fold is not a confident active site
- Inverse folding: ProteinMPNN works from backbone atoms only, so it is ligand-blind
- LigandMPNN models cofactors, metals and small molecules explicitly — the right tool for a pocket or a cofactor swap
- Protein language models and zero-shot variant effect prediction, and the limits of a score with no experiment behind it
- Docking as a hypothesis generator, not a measurement
- Where this sits in the real workflow: narrow a library from 10^6 to 10^2, then still screen

Sources

- Dauparas et al., Atomic context-conditioned protein sequence design using LigandMPNN, Nat. Methods (2025)
- Institute for Protein Design — introducing LigandMPNN
- LigandMPNN source and weights
- ColabFold — the paper, for what it does and does not predict

Week 30 — Enzyme and protein engineering for pathways

Sessions 146–150

By the end of the week you can

- Decide whether an enzyme is worth engineering before spending a campaign on it
- Judge an assay, and size a library so that the screen can actually find what you want
- Read a protein-engineering paper the way a biocatalysis group would

The five sessions

Day 1 · Is this enzyme worth engineering? · 2.5 h

☐ Revisit your week 15 control coefficients and write which enzyme in your capstone pathway holds the most control

☐ Compute how much flux an activity improvement buys at different control coefficients — problem L2

☐ Write the sentence “I would engineer X because it holds Y of the control, and Z-fold improvement would raise flux by W%”, with real numbers in it

☐ Write what you would check first if the enzyme turned out to hold little control: expression, cofactor supply, a competing pathway

Day 2 · Assays you can trust · 3 h

☐ Design an assay for one enzyme in your pathway: substrate, detection method, linear range, positive and negative controls

☐ Compute the Z′ factor for the two example assays in problem L1 and say which could be used for screening

☐ Connect it to the calibration and validation material of week 50: what would you need to show before trusting a hit

☐ Daily quiz

Day 3 · Library size and screening effort · 3 h

☐ Work out clones needed for 95% coverage of one, two and three saturated sites with NNK and with NDT — problem L1

☐ Compute the false hits a 10,000-clone screen produces at two, three and four standard deviations

☐ Decide which sites you would randomise first, and how you would avoid randomising three at once

☐ Write the time the screen takes at your own throughput

Day 4 · P450 and cofactor-dependent enzymes · 2.5 h

☐ Read the abstracts of the Tyrian purple and CYP105D18 indigo papers, and note what the enzymes need: cofactor, redox partner or peroxide

☐ Draw how NADPH or hydrogen peroxide feeds the reaction, and list three things that could limit titer

☐ List what can go wrong between a plant P450 gene and a working enzyme, in E. coli and in yeast: the N-terminal membrane anchor, the reductase partner, haem supply, codon use, and the endoplasmic reticulum in yeast

☐ Write what you would measure to tell an enzyme limit from a supply limit

☐ Log both papers in the paper log with one question each

Day 5 · From a hit to a result · 2.5 h

☐ Write the validation plan for a hit: purify, measure kinetic constants, check expression and solubility, return it to the host, measure titer against the parent

☐ Add the controls that stop a screen artefact passing as an improvement

☐ Write when a structure or sequence model from week 29 would help you choose variants, and what it cannot tell you

☐ Write a triage for an enzyme that is made but inactive: compare the soluble and insoluble fractions on a gel, lower the temperature, lower the inducer, try a solubility tag, co-express a chaperone, and change one thing at a time

☐ Write why “optimise the codons” can backfire, and what you would test: harmonising codon use is a hypothesis about translation speed and folding, not a rule

☐ Daily quiz

Topics this week covers

- Where enzymes limit a pathway: low activity, poor expression or solubility, cofactor dependence, product inhibition, the wrong specificity
- Flux control: an enzyme is worth engineering in proportion to the control it holds (week 15)
- Assay design: initial rates, linear range, controls, and the Z′ factor as a measure of screening quality
- Library design: NNK and NDT degeneracy, coverage arithmetic, and why three saturated sites at once is rarely practical
- Screening and selection: plate assays, biosensor-coupled FACS (week 26), growth-coupled selection
- Cytochrome P450 enzymes: dependence on NADPH and a redox partner, or a peroxygenase route, as in indigo and Tyrian purple production
- Structure and sequence models as a prior, not an answer (week 29)
- From a hit to a result: characterise the variant in vitro, then test it in the pathway host
- Getting a heterologous enzyme to fold: lower the expression temperature and the inducer, try a solubility tag (MBP, NusA or SUMO), co-express a chaperone (GroEL–GroES or DnaK–DnaJ–GrpE), and change one thing at a time
- Plant cytochrome P450 enzymes are membrane-bound: in E. coli the N-terminal anchor, the reductase partner and haem supply decide whether they work, and yeast, with its endoplasmic reticulum, is often the easier host
- Codon optimisation is not a guarantee: fast translation can hurt folding, and codon harmonisation is a hypothesis about that, to be tested and not assumed

Sources

- Cornish-Bowden — Fundamentals of Enzyme Kinetics, for what a kinetic constant does and does not tell you
- Lee et al., Production of Tyrian purple indigoid dye from tryptophan in Escherichia coli, Nat. Chem. Biol. (2021)
- In-vitro optimization and active-site mutagenesis of CYP105D18 peroxygenase enhance the production of indigo, Enzyme Microb. Technol. (2025)
- Zhang, Chung & Oldenburg, A simple statistical parameter for use in evaluation and validation of high throughput screening assays, J. Biomol. Screen. (1999)
- PubMed search: directed evolution and library design for biocatalysts
- PubMed search: solubility tags (MBP, NusA, SUMO) and soluble expression in E. coli
- PubMed search: chaperone co-expression (GroEL–GroES, DnaK) and inclusion bodies
- PubMed search: expression of plant cytochrome P450 enzymes in E. coli and yeast

Week 31 — Streptomyces and natural-product pathway engineering [extension]

Sessions 151–155

By the end of the week you can

- Explain how a natural-product pathway is found, cloned, expressed and improved in Streptomyces
- Compute the precursor, reducing-power and clone-library demands of a polyketide project
- Read a gene-cluster paper critically, as a member of a natural-product group would

The five sessions

Day 1 · From cluster to compound · 2.5 h

☐ Read the IUBMB entry for the 6-deoxyerythronolide B synthase: one propanoyl-CoA starter, six methylmalonyl-CoA extender units, six NADPH

☐ Write the net equation and the product formula, and check the element balance — problem L3

☐ Write which parts of the assembly line use NADPH, and why a precursor or cofactor limit shows up as low titer, not as a failed enzyme

☐ Log the in-vitro reconstitution paper in the paper log with one question

Day 2 · Streptomyces genetics and vectors · 2.5 h

☐ Write how DNA gets into Streptomyces: conjugation from E. coli, integrative vectors, and what differs from E. coli

☐ Reread the integrase and landing-pad material of week 32 and say which vectors would suit a large cluster

☐ Read the abstract of the Streptomyces artificial chromosome paper and note what it adds beyond a plasmid

☐ Daily quiz

Day 3 · Cloning a giant cluster · 3 h

☐ Compute the clones needed to capture a 60 kb cluster whole from an 8.7 Mb genome with 100 kb inserts at 99% confidence — problem L4

☐ Repeat for 30 kb and 90 kb clusters, and see why cluster size matters as much as genome size

☐ Compare library cloning with direct cloning by yeast recombination from week 21, and write which you would try first

☐ Write the verification plan: restriction pattern, junction PCR and sequencing, and what each can miss

Day 4 · Editing and tuning expression · 2.5 h

☐ Read the abstracts of the 2025 Cas9 paper and the 2023 CRISPR review for Streptomyces, and note what had to be engineered to make editing work

☐ Design a knockout with a homology template, controls, and a verification step

☐ List four ways to raise expression of a silent cluster: replace promoters, activate a regulator, use a heterologous host, add a quorum-sensing control

☐ Write what you would measure to tell a cluster that is not transcribed from one that is transcribed but not made

Day 5 · Titer, process and economics · 2.5 h

☐ Compute the propionate, NADPH and carbon demand for a titer target and see where toxicity or cost begins — problem L3

☐ Write why filamentous growth changes the bioreactor problem of week 44: pellets, viscosity and oxygen transfer

☐ Write one page: for a cluster of your choice, the precursor limit, the cloning plan with clone numbers, the editing plan with controls, and the price the product must reach

☐ Daily quiz

Topics this week covers

- Natural products as a source of drugs and agrochemicals: polyketides, nonribosomal peptides, ribosomal peptides, terpenoids
- A biosynthetic gene cluster encodes the enzymes, the regulators and the resistance and export functions together
- Genome mining finds clusters; most are silent under laboratory conditions
- Streptomyces biology: filamentous growth, a large high-GC linear genome, conjugation from E. coli, integrative vectors (week 32)
- Cloning large clusters: BAC and Streptomyces artificial chromosome systems, direct cloning by yeast recombination (week 21), and the library arithmetic
- Heterologous expression and refactoring: promoter replacement, regulator engineering, quorum-sensing control
- Precursor and cofactor supply: methylmalonyl-CoA and NADPH as the usual limits for polyketides
- Genome editing in Streptomyces: Cas9 and CRISPRi, homology templates, and why double-strand breaks are harsh
- Process: filamentous cultures, pellets and viscosity (week 44), and what a high-value natural product must earn (week 49)

Sources

- IUBMB enzyme entry EC 2.3.1.94, 6-deoxyerythronolide-B synthase
- Lowry et al., In vitro reconstitution and analysis of the 6-deoxyerythronolide B synthase
- Pyeon et al., Heterologous expression of pikromycin biosynthetic gene cluster using the Streptomyces artificial chromosome system, Microb. Cell Fact. (2017)
- Kim and colleagues, BAC cloning and heterologous expression of a giant biosynthetic gene cluster encoding antifungal neotetrafibricin in Streptomyces rubrisoli, Front. Bioeng. Biotechnol. (2022)
- PubMed search: engineered CRISPR-Cas9 for Streptomyces genome editing to improve specialized metabolite production (Min-Kyu Oh’s group, 2025)
- PubMed search: CRISPR-aided genome engineering for secondary metabolite biosynthesis in Streptomyces (Byung-Kwan Cho’s group, 2023)

Week 32 — Genomic integration, sequencing data, and wet-lab reality

Sessions 156–160

By the end of the week you can

- Integrate a multi-gene pathway into a chromosomal landing pad rather than carrying it on a plasmid
- Process your own resequencing data from reads to a mutation table
- Answer the troubleshooting questions an interviewer will actually ask about a failed assembly

The five sessions

Day 1 · Landing pads and serine integrases · 2.5 h

☐ Read a review of serine integrases and landing pads for pathway integration

☐ Write how Bxb1 or φC31 recombines attB with attP, why the product sites attL and attR are not substrates, and what that irreversibility buys you

☐ Design on paper a one-step integration of a four-gene cassette into a named safe-harbour locus: mark the att sites, the selection marker, and how you would remove the marker afterwards

Day 2 · Editing without a double-strand break · 2.5 h

☐ Reread the base-editing material from week 24 and add the bacterial context: no efficient non-homologous end joining, so an unrepaired cut is lethal

☐ Tabulate base editing against nuclease editing against recombineering: what each changes, what it needs, and its failure mode

☐ Note why prime editing, despite the attention, is a mammalian tool that transfers poorly to bacteria — and what bacterial labs use instead

Day 3 · Sequencing data from the command line · 3 h

☐ Install or access FastQC, bwa or bowtie2, and samtools — or use a Galaxy server if you cannot install locally

☐ Download one small FASTQ pair from the SRA and run FastQC; read the per-base quality and adapter plots and say what you would trim

☐ Align the reads to a reference, convert SAM to sorted indexed BAM with samtools, and view the coverage; write down what each file format holds

Day 4 · breseq on a real evolved strain · 3 h

☐ Install breseq and read its documentation on consensus versus polymorphic mode

☐ Run it on a public evolved E. coli dataset against the REL606 reference — for example `breseq -j 8 -r REL606.gbk SRR2584524.fastq.gz` — confirming current accessions on the SRA page first

☐ Open the HTML output and build a mutation table: point mutations, indels, and new junctions from mobile-element insertions; then argue which one or two could plausibly be causal and how you would test that

Day 5 · Wet-lab controls and troubleshooting · 2.5 h

☐ Write the control set for a Gibson or Golden Gate reaction: a positive transformation control with standard pUC19, a negative with linearised vector and no insert to quantify background, and the vector-to-insert molar ratio you would use

☐ Answer in full: your assembly gave zero colonies; and separately, it gave only background colonies that all contain empty vector. For each, list the likely causes in order and the control that distinguishes them

☐ Design a colony PCR with one primer in the backbone and one in the insert, say what band size confirms the junction and the orientation, and note what a Sanger .ab1 trace adds that a gel cannot

☐ Read your prospective institution's biosafety framework: biosafety levels, what BSL-1 and BSL-2 actually require, GMO containment and disposal, and what an institutional biosafety committee approves before work starts — you will meet one in your first month

☐ Write the strain, plasmid and data conventions you will use: a naming scheme, an inventory with freezer positions, an electronic notebook, and where raw data lives — then apply it retrospectively to everything you have built so far

☐ Daily quiz

Topics this week covers

- Why industrial strains do not keep pathways on plasmids: dosage, stability, antibiotic-free operation
- Serine integrases — Bxb1 and φC31 — attB/attP recombination, directionality, and why it is irreversible

- Landing pads and safe-harbour loci; integrating 5–10 gene cassettes in one step; multiplexed landing pads
- Editing without a double-strand break: base editing in bacteria, and why prime editing is a mammalian tool that transfers badly
- Short-read data: FASTQ and quality scores, FastQC, alignment with bwa or bowtie2, SAM to BAM with samtools
- breseq: mutation calling for haploid microbial genomes, including new junctions from mobile-element insertions
- Reading an adaptive-evolution result: consensus versus polymorphic mode, and which mutations are causal
- Wet-lab controls: positive and negative transformation controls, vector-to-insert molar ratio, background self-ligation
- Diagnostic screening: colony PCR with one primer in the backbone and one in the insert; reading an .ab1 Sanger trace
- Biosafety: containment levels, GMO handling and disposal, and what an institutional biosafety committee approves before work begins
- Strain, plasmid and data management: naming conventions, inventories, electronic notebooks, and where raw data lives

Sources

- breseq — documentation and downloads
- breseq on GitHub
- Deatherage & Barrick, “Identification of mutations in laboratory-evolved microbes … using breseq”, Methods Mol. Biol. 1151:165 (2014)
- NCBI Sequence Read Archive
- A review of serine integrases and landing pads for pathway integration
- Addgene — protocols for colony PCR and cloning troubleshooting

Week 33 — RNA-seq and differential expression [extension]

Sessions 161–165

By the end of the week you can

- Take raw reads to a count matrix and know what each step assumed
- Run a differential expression analysis and read its output sceptically
- Say what a transcriptome can and cannot tell you about flux

The five sessions

Day 1 · From reads to counts · 2.5 h

☐ Read how a sequencing library becomes a count matrix: quantification against alignment, and what strandedness and depth constrain

☐ Read the salmon paper for what selective alignment does and what bias it corrects

☐ Write the assumptions baked into a count matrix before any statistics touch it

Day 2 · Why raw counts lie · 2.5 h

☐ Read on library size and composition bias; work out by hand why a single highly expressed gene distorts every other gene’s apparent level

☐ Write what median-of-ratios normalisation does and when it fails

☐ Note the consequence for a production strain: a pathway expressed to 20% of the transcriptome breaks the assumption most normalisation relies on

Day 3 · Differential expression end to end · 3 h

☐ Read the DESeq2 paper, focusing on dispersion estimation and shrinkage rather than the software

☐ Download a public count matrix from GEO for a strain comparison and run the analysis in R, or in Python with pydeseq2

☐ Produce a volcano plot and a ranked table, and record how many genes pass at your thresholds

Day 4 · Thresholds, and what they hide · 2.5 h

☐ Read on false discovery rate control and why an unadjusted p-value list of 6,000 genes is meaningless

☐ Vary the adjusted-p and fold-change cutoffs and record how the gene list changes — then defend the pair you chose

☐ Write why a two-fold cutoff is a convention rather than a biological threshold

Day 5 · Transcripts against flux · 2.5 h

☐ Write the three reasons a transcript level can rise while flux does not: translation, enzyme activity, and metabolite control

☐ Take your top differentially expressed genes and say which measurement you would make next to establish they matter

☐ Connect this to the week 51 session on turning omics into model constraints, and to the ^{13}C work in week 43

☐ Daily quiz

Topics this week covers

- Library preparation choices that constrain the analysis: rRNA depletion, strandedness, read length, depth
- Quantification: alignment-based against selective alignment with salmon or kallisto
- Why raw counts are not comparable: library size, composition bias, and what normalisation fixes
- DESeq2 and edgeR: dispersion estimation, shrinkage, and why n = 3 is the practical floor
- Multiple testing: false discovery rate, and why a fold-change cutoff on its own is not a filter
- Reading the result: volcano plots, enrichment, and the gap between transcript level and enzyme activity
- Transcript abundance against flux — the week 51 lesson, met from the data side

Sources

- Love, Huber & Anders, Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2, Genome Biol. (2014)
- Bioconductor — DESeq2 package and vignette
- Patro et al., Salmon provides fast and bias-aware quantification of transcript expression, Nat. Methods (2017)
- NCBI Gene Expression Omnibus — public count matrices to reanalyse

Week 34 — Biofoundry workflows — design, build and test as a data pipeline

Sessions 166–170

By the end of the week you can

- Turn a designed variant list into a plate plan with volumes, tips and dead volume worked out
- Read and write an automation protocol, and say what a simulator cannot verify
- Handle plate data so that a position effect does not pass for biology

The five sessions

Day 1 · The cycle as a data pipeline · 2.5 h

☐ Read about K-Biofoundry and the Global Biofoundry Alliance on their sites, and note what a biofoundry offers an academic group that a single lab does not

☐ Draw the pipeline from a design file to a results table, marking where each sample identifier is created and where it could be lost

☐ Write the identifier scheme you would use for strains, plasmids, plates and wells, so that any result traces back to its design

☐ Write what goes wrong if two plates share a barcode

Day 2 · From a variant list to a plate plan · 3 h

☐ Take a 24-variant combinatorial assembly — four promoters, three coding sequences, two terminators — and compute the reactions, the master mix with overage and the volume needed in each source well including dead volume

☐ Compute the number of transfers and the tips needed with a fresh tip for every part, and the destination wells used

☐ Write the plan as a table a person or a script could follow — problem J1

☐ State which parts of the plan would change if you moved the reactions to a 384-well plate

Day 3 · Writing the protocol · 3 h

☐ Read the Opentrons Python API documentation: labware, pipettes, transfers, and how a protocol declares its deck

☐ Translate your plate plan into a protocol skeleton, and run it through the simulator if you can install it

☐ Write down everything the simulator cannot verify — calibration, viscosity, tip ejection, evaporation, liquid levels — and who in a lab is responsible for each

☐ Read the PyLabRobot paper’s argument for hardware-agnostic scripts and note what problem it solves

Day 4 · Miniaturisation, and what it costs · 2 h

☐ Compute the reagent saving of running a 10 µL assembly at 2 µL and at 500 nL, and for a 384-variant library the total enzyme-and-buffer volume at each scale

☐ Write why evaporation grows more important as volume falls and what you would do about it

☐ Note which steps cannot be miniaturised at all, such as transformation and colony picking, and what that does to the throughput gain

Day 5 · Test and learn: plate data, honestly · 2.5 h

☐ Take a simulated 96-well fluorescence plate in which edge wells read high, and rank the variants raw — note how many of the top ten are edge wells

☐ Estimate the edge multiplier from the plate itself, correct for it, and compare the ranking against the known truth — problem J2

☐ Write the controls you would add to the plate design so that the edge effect can be measured directly rather than inferred

☐ Daily quiz

Topics this week covers

- What a biofoundry is: automated design–build–test cycles run with shared standards. KRIBB’s synthetic biology centre describes K-Biofoundry and membership of the Global Biofoundry Alliance
- The cycle as a pipeline: a design file in, an identifier attached to every sample, a results table out
- Plate-planning arithmetic: reaction volumes, master mixes, overage, dead volume and tip budgets
- Liquid-handling scripts: the Opentrons Python API and PyLabRobot, with protocols treated as versioned code
- Miniaturisation: what nanolitre dispensing saves, and what it makes harder — mainly evaporation
- What a simulator verifies and what only the machine can: calibration, liquid classes, tip behaviour

- Position effects: edge wells evaporate and read high, so raw plate data are biased by location
- Closing the loop: a results table keyed by design identifier feeds the next round, which is where machine learning enters (week 41)

Sources

- K-Biofoundry
- Global Biofoundry Alliance
- Opentrons Python API documentation
- PyLabRobot: an open-source, hardware-agnostic interface for liquid-handling robots and accessories
- Seong et al., Trends Biotechnol. (2026) — the biofoundry workflow section
- pandas — 10 minutes to pandas

Week 35 — Constraint-based modelling — FBA and genome-scale models

Sessions 171–175

By the end of the week you can

- Formulate and solve flux balance analysis
- Understand genome-scale model reconstruction and its limits
- Run flux variability and gene-deletion analyses

The five sessions

Day 1 · Set up the environment · 2.5 h

☐ Install COBRApy and a solver; confirm the install by loading the textbook model

☐ Create the public GitHub repository that will hold everything from here on

☐ Commit a README saying what the repo is for

Day 2 · Read the FBA primer · 2.5 h

☐ Read Orth, Thiele & Palsson, “What is flux balance analysis?”, Nat. Biotechnol. 28:245

☐ Write why FBA needs no kinetic parameters and what it therefore cannot tell you

☐ Write the optimisation problem in full: objective, equality constraints, bounds

Day 3 · Run growth · 3 h

☐ Work the COBRApy getting-started tutorial on the E. coli core model

☐ Compute aerobic and anaerobic growth rates and explain the difference from your week 6 notes

> **Check your answer.** On the textbook E. coli core model with glucose uptake at 10 mmol·gDW^{-1}·h^{-1}, maximum growth is about 0.87 h^{-1} aerobically and about 0.21 h^{-1} anaerobically. The roughly four-fold drop is the ATP yield difference from week 6, reproduced by the model, and anaerobic solutions carry large fermentative fluxes to acetate, formate, ethanol and lactate to close the redox balance.

☐ Change the glucose uptake bound and plot growth against uptake

☐ List the model's exchange and transport reactions; note which ones decide what can leave the cell

Day 4 · Deletions and variability · 3 h

☐ Run single and double gene deletion analyses on the core model

☐ Run flux variability analysis and explain what a wide range means for a given flux

☐ Identify which deletions are lethal and check two of them against the biology you know

> **Check your answer.** Deletions in the central glycolytic trunk that have no bypass — enolase and phosphoglycerate kinase are the clearest — abolish growth on glucose in the core model. Deletions with a parallel route, such as individual fermentative branches under aerobic conditions, are tolerated. If a deletion you expected to be lethal is not, look for the bypass the model has that the real organism may not.

Day 5 · Ship the notebook · 2 h

☐ Clean the notebook: headings, commentary, a conclusion a stranger could follow

☐ Push it to GitHub and confirm it renders

☐ Daily quiz

Topics this week covers

- S·v = 0 plus flux bounds plus a linear objective
- Why FBA needs no kinetic parameters
- iJO1366 and the E. coli core model
- What FBA cannot tell you

Sources

- Orth, Thiele & Palsson, “What is flux balance analysis?”, Nat. Biotechnol. 28:245 (2010)
- COBRApy documentation and tutorials
- BiGG Models — download the core and iJO1366 models

Week 36 — Python for modelling II: COBRApy in depth

Sessions 176–180

By the end of the week you can

- Build a metabolic model from scratch out of COBRApy objects, not only load one
- Change media, bounds and objectives properly, and undo changes safely
- Move models in and out of SBML and draw fluxes on a map

The five sessions

Day 1 · Model objects from scratch · 3 h

☐ Work the COBRApy building-a-model guide

☐ Build the five-reaction toy network from problem A1 out of Metabolite and Reaction objects by hand, set bounds and an objective, and optimise it

☐ Check the answer against the hand solution you wrote in week 13 — if they differ, the sign convention on one reaction is the usual culprit

Day 2 · Media, bounds, and changes that undo themselves · 2.5 h

☐ Load the E. coli core model and inspect model.medium; identify every exchange reaction

☐ Switch the model between aerobic and anaerobic by editing the medium alone, and reproduce growth rates of about 0.87 and 0.21 per hour

☐ Use `with model:` to apply a temporary knockout, optimise inside the block, and confirm the bound is restored afterwards — then write why this matters in a loop over hundreds of deletions

Day 3 · The analysis functions, and constraining the solver thermodynamically · 3.5 h

☐ Run pFBA and loopless FBA on the core model and explain what each removes from the plain FBA solution

☐ Find a loop in the plain FBA solution that loopless FBA eliminates, and write why a thermodynamically impossible cycle can satisfy S·v = 0 perfectly

☐ Read how ΔG is turned into a constraint rather than a diagnostic — thermodynamics-based flux analysis, and the loopless formulation as its cheap special case

☐ Pull ΔG°′ values from eQuilibrator for a handful of core reactions and fix the direction of the clearly irreversible ones as bounds; rerun and report what changed

☐ Run flux_variability_analysis and single_gene_deletion, export both DataFrames to CSV, and plot the deletion results as a histogram of growth rates

Day 4 · SBML in and out, and a map · 2.5 h

☐ Write your model to SBML with cobra.io.write_sbml_model, reload it, and compare reaction and metabolite counts, bounds and objective

☐ Note what did not survive the round trip, if anything, and why SBML is still the interchange format everyone uses

☐ Load a core-model map in Escher and overlay a flux vector from your own solution; export the figure

Day 5 · Make the whole thing reproducible · 2.5 h

☐ Restructure the week's work into one notebook that runs top to bottom from a fresh kernel

☐ Pin requirements, seed anything random, and write a README section saying what the notebook shows and what the headline numbers are

☐ Write a one-page data management plan for your capstone: where raw data, processed data and models live, how they are named and versioned, what gets deposited where on publication, and what the FAIR principles ask of each — funders now require this document, so writing one before you are asked is cheap practice

☐ Commit and push; then clone it into a clean directory and run it there, which is the only real test

☐ Daily quiz

Topics this week covers

- The object model: Model, Reaction, Metabolite, Gene, and gene–protein–reaction rules
- Building reactions by hand: add_metabolites, bounds, reversibility, and the sign convention
- Objective functions, and why the biomass reaction is a reaction like any other
- Exchange reactions and model.medium — the right way to set a carbon source
- optimize against slim_optimize; the solution object; model.summary()
- The context manager: with model: for temporary knockouts that undo themselves
- flux_analysis: pFBA, loopless FBA, flux_variability_analysis, single and double deletions, all returning DataFrames
- Thermodynamic constraints in the optimisation: ΔG as a bound on direction, not only a diagnostic; loopless FBA as its cheap special case
- SBML and JSON in and out; what survives a round trip and what does not
- Escher for drawing a flux vector on a pathway map

Sources

- COBRApy documentation — building a model, simulating, deletions, FVA
- COBRApy — flux analysis guide
- BiGG Models — download the core model and iJO1366 as SBML and JSON
- Escher — draw flux data on a metabolic map

Week 37 — Reconstructing a genome-scale model, and testing whether it is any good [extension]

Sessions 181–185

By the end of the week you can

- Build a draft metabolic model for an organism nobody has curated
- Test a model against a standard and read the report
- Know which gaps matter and which are cosmetic

The five sessions

Day 1 · Where models come from · 2.5 h

☐ Read the Thiele and Palsson protocol for what manual reconstruction actually involves — it is a year of work, which is why automation exists

☐ Write the steps that cannot be automated and why

☐ Check whether your capstone organism has a curated model in BiGG; most do not

Day 2 · Build a draft · 3 h

☐ Install CarveMe or gapseq and reconstruct a draft model from a genome for a non-model organism — an acetogen, a thermophile or a non-conventional yeast

☐ Load the draft in COBRApy and check it grows on a defined medium; if it does not, find out whether the medium or the model is at fault

☐ Record how many reactions and genes the draft contains against a curated model of a related organism

Day 3 · Gap filling is a hypothesis · 2.5 h

☐ Read what automated gap filling does: it adds reactions to make biomass producible, choosing the smallest set

☐ List the reactions gap filling added to your draft and look up whether any evidence supports them in this organism

☐ Write why a gap-filled reaction with no genomic evidence is a prediction to test, not a finding

Day 4 · Quality control with MEMOTE · 3 h

☐ Run MEMOTE on your draft and on a curated model such as iJO1366

☐ Compare the two reports on stoichiometric consistency, mass and charge balance, and annotation coverage

☐ Write which failures would change a flux prediction and which are only metadata — this judgement is the point of the session

Day 5 · Curate what matters · 2.5 h

☐ Fix the three issues from your MEMOTE report that could change a prediction, and document each change with its reason

☐ List the issues you deliberately left unfixed and why

☐ Write a paragraph on when a draft model is adequate: for a yield ceiling it often is, for a gene-deletion prediction it often is not

☐ Daily quiz

Topics this week covers

- Where curated models come from, and why your organism of interest probably has none
- Automated draft reconstruction: CarveMe from a genome, gapseq from annotation and pathway evidence
- Gap filling: what it adds, and why an automatically filled gap is a hypothesis rather than a fact
- Biomass objective functions: where the coefficients come from and how wrong they can be
- Quality control with MEMOTE: stoichiometric consistency, mass and charge balance, annotation coverage, SBML conformance
- Manual curation: the handful of reactions that actually change the answer
- Knowing when a draft model is good enough for the question you are asking

Sources

- Machado et al., Fast automated reconstruction of genome-scale metabolic models for microbial species and communities (CarveMe), Nucleic Acids Res. (2018)
- Lieven et al., MEMOTE for standardized genome-scale metabolic model testing, Nat. Biotechnol. (2020)

- Zimmermann et al., gapseq: informed prediction of bacterial metabolic pathways and reconstruction of genome-scale models, Genome Biol. (2021)
- Thiele & Palsson, A protocol for generating a high-quality genome-scale metabolic reconstruction, Nat. Protoc. (2010)

Week 38 — MATLAB and the COBRA Toolbox: reading other people's code [optional]

Sessions 186–190

By the end of the week you can

- Read, run and modify a MATLAB metabolic script without fighting the language
- Do the same constraint-based analyses in the COBRA Toolbox that you do in COBRApy
- Move a model between platforms through SBML and know what survives

The five sessions

Day 1 · MATLAB for a Python user · 2.5 h

☐ Get access: a university licence, MATLAB Online, or install GNU Octave if you have neither

☐ Work the getting-started material with your Python knowledge in mind: one-based indexing, matrices as the default type, element-wise operators with a dot, scripts against functions, and the debugger

☐ Translate your week 12 linear-algebra script into MATLAB — A\b in place of np.linalg.solve, null in place of null_space — and confirm identical answers

Day 2 · Installing the COBRA Toolbox · 2.5 h

☐ Install the COBRA Toolbox and run initCobraToolbox; set a solver, glpk if you have no Gurobi academic licence

☐ Load the E. coli core model with readCbModel and inspect the model struct: S, lb, ub, c, rxns, mets, genes

☐ Run optimizeCbModel and reproduce the aerobic growth rate you got in COBRApy — if it differs, check the medium and the objective before suspecting the solver

Day 3 · The same analyses under different names · 3 h

☐ Run singleGeneDeletion and fluxVariability; map each function onto its COBRApy equivalent in a two-column table

☐ Use changeRxnBounds and changeObjective to run the anaerobic case and a knockout

☐ Put your COBRApy and COBRA Toolbox results side by side and reconcile every difference; a mismatch is nearly always the medium, a bound, or the objective, not the mathematics

Day 4 · ode45, ode15s, and where Simulink fits · 2.5 h

☐ Translate the three-enzyme kinetic model from week 16 into MATLAB and integrate it with ode15s

☐ Confirm the same steady state and the same flux control coefficients to three decimal places

☐ Read enough about Simulink to say what it is for and why process and control engineers use it, without building anything in it

Day 5 · Crossing platforms deliberately · 2.5 h

☐ Write a model to SBML from one platform and read it in the other; compare reaction counts, bounds, objective and gene rules

☐ Write a short note for your repository on what survived, what did not, and which direction is safer

☐ Commit both the MATLAB and the Python versions of the same analysis with a README explaining why both exist — this is a small but genuine portfolio item

☐ Daily quiz

Topics this week covers

- MATLAB for someone who already writes Python: one-based indexing, matrices as the native type, scripts against functions, structs, cell arrays, the debugger
- Access: a university licence, MATLAB Online, or GNU Octave for most of what you need
- The COBRA Toolbox: installation, initCobraToolbox, and choosing a solver (glpk, Gurobi with an academic licence)
- readCbModel and writeCbModel; the model struct and what each field holds
- optimizeCbModel, changeObjective, changeRxnBounds, singleGeneDeletion, fluxVariability — the same analyses under different names
- ode45 and ode15s for kinetic models; where Simulink fits and why process engineers reach for it
- Translating a script in both directions, and the traps: indexing, column against row vectors, and copy-on-write

Sources

- The COBRA Toolbox — documentation and installation
- MATLAB — getting started documentation
- GNU Octave — the free alternative for most toolbox-free work
- MATLAB ODE solver documentation (ode45, ode15s)

Week 39 — Strain-design algorithms, ^{13}C-MFA, kinetic models

Sessions 191–195

By the end of the week you can

- Explain OptKnock and growth-coupled design
- Explain ^{13}C metabolic flux analysis and how it differs from FBA
- Know what enzyme-constrained and kinetic models add

The five sessions

Day 1 · OptKnock, the idea · 2.5 h

☐ Read Burgard, Pharkya & Maranas, Biotechnol. Bioeng. 84:647

☐ Write the bilevel structure: what the outer problem chooses and what the inner problem does

☐ Explain growth coupling in one sentence a non-specialist would understand

Day 2 · Design a coupled strain · 3 h

☐ Use COBRApy to search deletions that couple succinate or lactate production to growth

☐ Record the knockout set and the predicted coupled yield

☐ Plot the production envelope before and after

☐ Constrain the product's export bound to a finite rate, rerun, and record how the maximum yield changes — the transport limit made numerical

Day 3 · Compare with the literature · 2.5 h

☐ Find a published strain for the same product and compare your knockout set with theirs

☐ Write where they agree, where they differ, and why

☐ Note what the model missed that the experiment found

Day 4 · ^{13}C flux analysis · 3 h

☐ Read a ^{13}C-MFA methods review

☐ Explain how labelling patterns resolve the split between glycolysis and the pentose phosphate pathway

☐ Write the difference between an FBA-feasible flux and a ^{13}C-measured one

Day 5 · Kinetic and constrained models · 2 h

☐ Read on enzyme-constrained models such as GECKO and on kinetic models

☐ Note what each adds and what it costs in parameters

☐ Push the strain-design notebook to GitHub, then daily quiz

Topics this week covers

- Bilevel optimisation: make the product obligatory for growth
- Labelling patterns resolve split ratios between parallel pathways
- Feasible flux versus measured flux
- GECKO and proteome constraints

Sources

- Burgard, Pharkya & Maranas, “OptKnock”, Biotechnol. Bioeng. 84:647 (2003)
- A ^{13}C-MFA methods review

Week 40 — Retrosynthesis, pathway prediction and machine learning

Sessions 196–200

By the end of the week you can

- Explain computational pathway design for non-natural targets
- Explain where machine learning enters the DBTL cycle
- Propose a route for a molecule with no known pathway

The five sessions

Day 1 · Retrosynthesis · 2.5 h

☐ Read Delépine et al., Metabolic Engineering 45:158, on RetroPath2.0

☐ Write how reaction rules expand a network backwards from a target

☐ Note how candidate routes are ranked

Day 2 · Propose a route · 3 h

☐ Choose a non-natural target and propose two routes from a central metabolite by hand

☐ Source a candidate enzyme for every step from BRENDA or MetaCyc

☐ Mark each step you could not find an enzyme for — these are your real risks

Day 3 · Machine learning in the loop · 2.5 h

☐ Read a 2025 review on machine learning or AI-guided biofoundries

☐ Write where ML actually enters DBTL and what data it needs

☐ Mark every forward-looking claim in the review as demonstrated or projected

Day 4 · Biofoundries · 2 h

☐ Read how a biofoundry runs a DBTL round end to end

☐ List the steps that are automated and the ones that are not

☐ Note what this means for a PhD student's daily work

Day 5 · Consolidate the computational stack · 2 h

☐ Draw one diagram linking retrosynthesis, FBA, strain design, build tools and ML

☐ Decide which parts your capstone will actually use

☐ Daily quiz

Topics this week covers

- Reaction rules; RetroPath2.0 and BNICE

- Ranking routes by thermodynamics, enzyme availability and host fit
- Surrogate models and active learning
- AI-guided biofoundries — read the tense carefully

Sources

- Delépine et al., “RetroPath2.0”, Metabolic Engineering 45:158 (2018)
- A 2025 review on machine learning or AI-powered biofoundries

Week 41 — Bayesian optimisation and machine-learning-guided DBTL

Sessions 201–205

By the end of the week you can

- Choose the next experiment from a small, noisy dataset rather than from intuition
- Fit a surrogate model and use an acquisition function that balances exploring and exploiting
- Say honestly when a dataset is too small for machine learning to help

The five sessions

Day 1 · The problem machine learning is actually solving here · 2.5 h

☐ Read the ART paper for the framing: a handful of expensive experiments, not a big-data problem

☐ Write why this differs from the machine learning most courses teach, where data are cheap and plentiful

☐ List the experiments in your own capstone plan and count them — that number is the whole constraint

Day 2 · Gaussian process surrogates · 3 h

☐ Work through the scikit-learn Gaussian process documentation; fit one to five noisy points and plot the posterior mean with its uncertainty band

☐ Write what the kernel assumes about smoothness, and what happens to the uncertainty far from any data

☐ Note why a model that reports uncertainty is what makes sequential design possible at all

Day 3 · Choosing the next experiment · 3 h

☐ Implement expected improvement and use it to pick the next point from your fitted surrogate

☐ Run the full loop on a two-factor medium optimisation and record the best titer found after 8, 11 and 16 runs

☐ Compare against a 16-run factorial grid on the same budget — this is problem G2

Day 4 · Honest limits · 2.5 h

☐ Repeat the loop with several random seeds and report the spread, not your best run

☐ Find a seed where it gets stuck on a local optimum and explain what the acquisition function did

☐ Write the conditions under which you would not use this: too few runs, a noisy or drifting assay, confounded factors, or a readout that does not track what you care about

Day 5 · From factors to sequences · 2.5 h

☐ Read the tryptophan paper for a mechanistic and machine-learning hybrid on a real pathway

☐ Write how you would encode a promoter or RBS choice so a model can learn across variants

☐ Add two sentences to your capstone build-and-test plan describing the learn step concretely — most proposals leave it as a word

☐ Daily quiz

Topics this week covers

- The real constraint in strain engineering: tens of experiments, not thousands

- Gaussian process surrogates: what the posterior mean and variance mean, and what the kernel assumes
- Acquisition functions: expected improvement, upper confidence bound, and the exploration-exploitation trade
- Why factorial designs lose to sequential designs once factors interact and runs are scarce
- Local optima, and the failure mode of an acquisition function that stops exploring
- Scaling to sequences: encoding a genotype so a model can learn from it
- The Automated Recommendation Tool and the published record of ML-guided DBTL
- When not to: fewer than a dozen runs, unmeasured confounders, or a readout you do not trust

Sources

- Radivojević et al., A machine learning Automated Recommendation Tool for synthetic biology, Nat. Commun. (2020)
- Zhang et al., Combining mechanistic and machine learning models for predictive engineering and optimization of tryptophan metabolism, Nat. Commun. (2020)
- scikit-learn — Gaussian process regression
- A review of machine learning for metabolic engineering, 2024 or later

Week 42 — Machine learning for biology

Sessions 206–210

By the end of the week you can

- Build a small supervised model and judge it without fooling yourself
- Recognise overfitting and data leakage, and choose a split that matches the question
- Say what pretrained sequence models and active learning add to a design–build–test loop, and what they cannot promise

The five sessions

Day 1 · What a model is, and a baseline · 2.5 h

☐ Fit the mean and a linear model on a small table of your own, or a toy set, and report the held-out error of each

☐ Write what a feature is for a sequence, a strain and a plate

☐ Write why a baseline comes before any complex model

☐ Log one machine-learning paper from the reading list with a question

Day 2 · How much data is enough · 3 h

☐ Run the overfitting simulation and read the training and held-out scores at five sample sizes — problem P1

☐ Run it again with the labels shuffled and say what that tells you about a high training score

☐ Draw a learning curve for your own example and say where it flattens

☐ Daily quiz

Day 3 · Leakage · 3 h

☐ Run the family-split simulation and compare the random and grouped scores — problem P2

☐ Write which split matches each of three questions: rank variants within known families, predict a new family, predict a new plate

☐ Write one way your own data could leak

☐ Write what you would report so a reader can tell

Day 4 · Pretrained models and active learning · 2.5 h

☐ Read the abstracts of the language-model and machine-learning-guided evolution papers and write what a zero-shot score is

☐ Connect active learning to the Bayesian optimisation week: what does each use to choose the next batch

☐ Write what a pretrained model cannot tell you about your enzyme in your host

☐ Revisit the structure-literacy week and say when a model prior and a structure prior agree or conflict

Day 5 · A notebook that stays honest · 2.5 h

☐ Write one page: the question, the data, the split, the baseline, the metric, the uncertainty, and what would change your mind

☐ Add seeds and pinned versions, and check the notebook runs from a fresh clone

☐ Write the sentence you would put in a methods section about the split

☐ If an AI assistant wrote or suggested any of the code or text, write what it did and how you checked it: run it on a case with a known answer, read every line that touches the split or the metric, and confirm that any citation exists

☐ Daily quiz

Topics this week covers

- A model is a function fitted to examples; features for biology include one-hot sequence, physicochemical descriptors and embeddings from pretrained protein language models
- Overfitting: more features than samples, training error against held-out error, cross-validation and learning curves
- Baselines first: the mean, a linear model, nearest neighbours. A deep network has to beat them on held-out data
- Leakage: random splits across related sequences, replicates or plates inflate scores. Split by family, cluster, plate or time, according to what you need to predict
- Uncertainty and active learning: choose the next batch to learn the most (week 41)
- Pretrained models as priors: zero-shot scores and fine-tuning on a little data, and what they cannot guarantee
- Biofoundry data: replicates, batch effects, units and metadata (week 34); KRIBB's biofoundry group lists AI-based bio data and autonomous labs among its aims
- Reproducibility: seeds, pinned versions, and a notebook that runs from a fresh clone
- Working with AI assistants: they write plausible code and prose that can be wrong, so every output needs a check you could explain to a reviewer

Sources

- PubMed search: machine-learning-guided directed evolution of proteins
- PubMed search: protein language models and fitness prediction
- PubMed search: data leakage and homology-aware splitting for sequence machine learning
- KRIBB National Biofoundry Project Group

Week 43 — Fluxomics: ^{13}C metabolic flux analysis in practice [extension]

Sessions 211–215

By the end of the week you can

- Design a labelling experiment that resolves the split you care about
- Read a mass isotopomer distribution and say what it tells you
- Fit fluxes, judge the fit, and report confidence intervals honestly

The five sessions

Day 1 · What labelling buys you · 2.5 h

☐ Read an Antoniewicz review of ^{13}C-MFA

☐ Write the difference between a flux FBA calls feasible, a flux a measured uptake rate constrains, and a flux labelling resolves

☐ List three things in central metabolism that only labelling can settle: the G6P split, reversible exchange fluxes, and anaplerotic versus TCA flux

Day 2 · Tracer design · 2.5 h

☐ Read how tracer choice changes what is resolvable

☐ Write what [1-^{13}C] glucose reports on, what [U-^{13}C] glucose reports on, and why mixtures are used

☐ Choose the tracer you would use to resolve the pentose phosphate split and justify it in two sentences

Day 3 · Atom transitions and isotopomers · 3 h

☐ Write the carbon atom transitions for glycolysis and the oxidative pentose phosphate pathway, carbon by carbon

☐ Show on paper why carbon 1 of glucose is lost as CO_2 in the oxidative branch but retained through glycolysis

> **Check your answer.** Carbon 1 of glucose becomes the carbon released as CO_2 by 6-phosphogluconate dehydrogenase in the oxidative branch. Through glycolysis it survives: glucose carbons 3 and 4 become the carboxyl carbons of pyruvate, carbons 2 and 5 the carbonyl carbons, and carbons 1 and 6 the methyl carbons. So the labelling of downstream metabolites reports directly on the split at glucose-6-phosphate.

☐ Define mass isotopomer distribution and explain why the raw MS data must be corrected for natural abundance

Day 4 · Fitting fluxes and reading the statistics · 3 h

☐ Read how flux fitting works as nonlinear least squares over the atom transition network

☐ Write what a poor goodness of fit means — bad data, or a network that is missing a reaction

☐ Take a published flux map and write down what its confidence intervals establish, and what a narrow interval on a badly specified network is worth

Day 5 · Read a fluxomics paper properly · 2 h

☐ Pick a ^{13}C-MFA paper on a host you care about and read it closely

☐ Log it with the nine-field protocol, and add the tracer used and the reported confidence intervals

☐ Write one sentence on what the measured fluxes contradicted in the authors' model — there is almost always one

☐ Daily quiz

Topics this week covers

- Why labelling resolves what stoichiometry alone cannot
- Tracer choice: [1-^{13}C] versus [U-^{13}C] glucose and mixtures, and what each resolves
- Isotopic steady state, metabolic steady state, and when neither holds
- Measuring mass isotopomer distributions by GC-MS and LC-MS; natural abundance correction
- Atom transition networks and how a labelling pattern is predicted
- Flux fitting as nonlinear least squares; goodness of fit and confidence intervals
- Non-stationary ^{13}C-MFA for photoautotrophs and slow systems
- Software: INCA, OpenFlux, mfapy

Sources

- An Antoniewicz review of ^{13}C metabolic flux analysis
- A ^{13}C-MFA experimental protocol paper
- A review of non-stationary ^{13}C-MFA

Week 44 — Bioreactor engineering, scale-up and downstream

Sessions 216–220

By the end of the week you can

- Size mixing, power input and oxygen transfer for a real vessel
- Predict where a shake-flask or 1 L result will break at cubic-metre scale, and why
- Lay out a downstream train for your product and cost it roughly

The five sessions

Day 1 · Mixing and power · 2.5 h

☐ Write the impeller Reynolds number and the power number, and the relation $P = N_p \rho n^3 d^5$

☐ Compute power per unit volume and mixing time for a 1 L vessel and for a 100 m³ vessel at the same tip speed

> **Check your answer.** With $P = N_p \rho n^3 d^5$ and a Rushton turbine ($N_p \approx 5$), holding P/V constant under geometric similarity gives $n \propto d^{-2/3}$, so tip speed $nd \propto d^{1/3}$ rises and mixing time grows roughly as $d^{2/3}$. A 1 L vessel mixes in a second or two; a 100 m³ vessel takes tens of seconds to minutes at the same power per volume. That ratio is the whole scale-up problem in one number.

☐ Write why mixing time grows with scale even when power per volume is held constant — the single most important number in this week

Day 2 · Gas–liquid mass transfer, and feeding it into the model · 3.5 h

☐ Write $OTR = kLa\,(C^* - C_L)$ and list what each term depends on

☐ Compute the oxygen uptake rate of a culture at 50 g/L dry weight and a typical specific uptake rate, then the kLa needed to supply it

> **Check your answer.** Take 50 gDW/L and a specific oxygen uptake of 5 mmol·gDW^{-1}·h^{-1}: OUR = 250 mmol·L^{-1}·h^{-1}. With $C^* \approx 0.21$ mM for air at 37 °C and C_L held at 0.05 mM, the required kLa = 250/0.16 ≈ 1,500 h^{-1}. Large stirred tanks achieve roughly 100–500 h^{-1}, so this culture cannot be supplied with air at atmospheric pressure. The options are oxygen enrichment, raised head pressure, or a strain or feeding strategy with lower oxygen demand — usually the last.

☐ Compare that with achievable kLa values and state what you would change — stirrer speed, air flow, oxygen enrichment, pressure, or the strain's oxygen demand

☐ Convert your achievable OTR into a per-biomass flux and set it as the oxygen exchange bound in the core model; rerun FBA and report what happens to growth and to the byproducts — this is problem D6, and it is where your transport calculation and your genome-scale model finally meet

Day 3 · What the cell actually experiences · 2.5 h

☐ Read a scale-up and scale-down review

☐ Estimate the circulation time in a large vessel and compare it with the timescale of the cell's metabolic response

☐ Sketch a two-compartment scale-down reactor that would reproduce the feast-famine cycle in a 1 L experiment, and say which measurement would show it mattered

Day 4 · Feeding strategy · 2.5 h

☐ Write the substrate balance for a fed-batch and derive the exponential feed that holds a constant specific growth rate

> **Check your answer.** Substrate balance: $d(SV)/dt = F \cdot S_{feed} - (\mu/Y_{X/S} + m) \cdot X \cdot V$. To hold μ constant while X grows exponentially, the feed must grow at the same rate: $F(t) = F_0 \cdot e^{\mu t}$. The practical consequence is that the pump, not the organism, sets the growth rate — which is the point of fed-batch, since it keeps substrate low enough to avoid overflow metabolism and oxygen limitation.

☐ Compare constant, exponential, DO-stat and pH-stat feeding, and the failure mode of each

☐ Choose a feeding strategy for your capstone product and justify it from the strain's physiology, not from convention

Day 5 · Downstream and the cost sketch · 3 h

☐ List the unit operations your product needs from broth to specification, in order, and the physical property each exploits

☐ Read a techno-economic analysis of a fermentation process and note which cost line dominates

> **Check your answer.** Feedstock plus the separation energy a dilute broth forces usually dominates, which is why yield and titer are the binding metrics for a commodity. If your sketch says anything else dominates, check whether you costed the water: evaporating or distilling a broth at 20 g/L moves fifty kilograms of water per kilogram of product.

☐ Write a back-of-envelope cost per kilogram at your theoretical and your realistic yield, then state the titer below which the process cannot work

☐ Daily quiz

Topics this week covers

- Power input per unit volume, impeller Reynolds number, power number, and mixing time
- kLa: correlations, measurement methods, and what sets it in a stirred tank
- Oxygen uptake rate against oxygen transfer rate; the critical dissolved-oxygen concentration
- Turning OTR into a constraint on the model: kLa(C* − C_L) divided by biomass becomes the oxygen exchange bound in mmol·gDW^{-1}·h^{-1}
- CO_2 accumulation and stripping, and hydrostatic pressure with height
- Shear, bubble rupture, and which organisms actually care
- Scale-down reactors: two-compartment and stirred-tank-plus-plug-flow designs for reproducing gradients
- Fed-batch feeding strategies: constant, exponential, DO-stat, pH-stat; why feeding beats batching
- Sterility, contamination, and non-sterile processes
- Downstream unit operations: centrifugation, filtration, homogenisation, extraction, chromatography, distillation, crystallisation
- Techno-economic analysis and life-cycle assessment as the real gatekeepers

Sources

- Doran — Bioprocess Engineering Principles, 2nd ed., the mass-transfer and reactor chapters, and work its problems
- Shuler, Kargi & DeLisa — Bioprocess Engineering, the chapters on reactor operation and scale-up
- Blanch & Clark — Biochemical Engineering, for extra depth on bioreactor design
- Your own transport phenomena and unit operations notes — this week is mostly translation, not new theory
- A review of bioreactor scale-up and scale-down
- A techno-economic analysis of a fermentation process
- Coursera — Introduction to Industrial Bioprocess Development (DTU), the reactor modules

Week 45 — Product toxicity, export and in situ removal

Sessions 221–225

By the end of the week you can

- Decide whether a product is limited by toxicity, by export or by loss, and what each calls for
- Compute how a second phase changes the concentration the cells see, and what the solvent costs
- Read an efflux-pump or two-phase fermentation paper critically

The five sessions

Day 1 · What limits the titer: toxicity, export or loss · 2.5 h

☐ Classify four products by their main limit and say why: a limonene-like terpene, a volatile sesquiterpene, a short-chain organic acid, and a flavonoid

☐ Revisit your week 14 calculations of diffusion and active export, and say which of your candidate products would need an exporter

☐ Write three experiments that separate the causes: grow the host with the product spiked in and compare with the producer; compare intracellular and extracellular amounts; compare a closed vessel with an open one

☐ Write what each result would mean, and what you would do next in each case

Day 2 · Efflux pumps and membrane engineering · 3 h

☐ Read the abstract of Dunlop et al. (2011) and write what they screened, how they told pumps apart, and what happened for n-butanol and isopentanol

☐ Fill a table of the pump families: RND, ABC and MFS, with their components, their energy source, and the hosts you would use them in

☐ Write the costs of putting a pump in a production strain and how you would measure whether it paid

☐ List the membrane levers (saturated against unsaturated fatty acids, cis–trans isomerisation) and mark which are established and which are research-stage

Day 3 · Two-phase partitioning: what do the cells see? · 3 h

☐ Derive the total titer from the aqueous concentration, the partition coefficient and the phase fraction, and compute the table — problem Q1

☐ Compute the solvent fraction a target titer needs, and the share of the product that sits in the organic phase

☐ Check the solubility of your product in the solvent: a high partition coefficient can push the organic-phase concentration past what the solvent can hold

☐ Daily quiz

Day 4 · Loss, capture and what it costs · 2.5 h

☐ Compute how much of a volatile product is still in the broth after 1, 2, 6 and 24 hours, and what titer you would measure from a steadily producing culture — problem Q2

☐ Compute the solvent inventory and the cost per kilogram of product under three assumptions about reuse

☐ Write the difference between a solvent's log P (does it harm the cells?) and the product's partition coefficient K (does it capture the product?), and choose a solvent for your product using both

☐ Compare an organic overlay, an adsorbent resin and gas stripping on cost, simplicity, product purity and scale-up

☐ Write what changes at 100 m^3: oxygen transfer, emulsions, solvent loss, recovery and safety

Day 5 · A strategy for a toxic product · 2.5 h

☐ Pick a product from your capstone list and write the evidence you would need that toxicity, export or loss limits it

☐ Choose the levers (a pump, a membrane change, an overlay or a resin) with the numbers that justify them

☐ Write the controls, a pass mark, and what would make you drop the lever

☐ Daily quiz

Topics this week covers

- Why hydrophobic, detergent-like products stop growth: they partition into the membrane and disturb it, so the partition coefficient matters as much as the titer
- Export, revisited (week 14): when diffusion suffices and when an active exporter is needed, and what export costs

- Efflux pump families: RND pumps (tripartite, as in AcrAB–TolC), ABC transporters and major facilitator superfamily (MFS) pumps. RND and MFS pumps are typically driven by the proton motive force and ABC pumps by ATP; in yeast the ABC pumps of the pleiotropic drug resistance family are the main candidates
- Pump engineering is a screening problem: Dunlop et al. (2011) tested 43 pumps against seven biofuels in E. coli, found pumps that helped for five, and found none that helped for n-butanol or isopentanol
- Pumps cost something: expression burden, a limited substrate range, and the outer-membrane and adaptor components that tripartite pumps need
- Membrane composition as a lever: the balance of saturated and unsaturated fatty acids, and cis–trans isomerisation, which is a known adaptation to solvents in some bacteria. Hopanoid and sterol engineering is research-stage, not routine
- Tolerance by adaptive evolution (week 28), and the trade-off it can carry
- In situ product removal (ISPR): a second phase or an adsorbent takes the product away as it is made. It relieves toxicity, captures volatile products, and can pull a product-inhibited reaction forward
- Choosing the capture, as in a two-phase partitioning bioreactor (TPPB) and its relatives: an organic overlay (such as dodecane), an adsorbent resin, gas stripping or extraction. Two different numbers decide it: the product's partition coefficient K, which sets how much is captured, and the solvent's log P (octanol–water), a rule-of-thumb guide to whether the cells tolerate it. Solvents with a log P below about 2 tend to be toxic and those above about 4 tend to be tolerated, with a grey zone between and exceptions
- What it costs at scale: solvent price and loss, emulsions, oxygen transfer through an organic layer, recovering the product from the solvent, and safety (week 49)

Sources

- Dunlop et al., Engineering microbial biofuel tolerance and export using efflux pumps, Mol. Syst. Biol. (2011)
- Newman et al., High-level production of amorpha-4,11-diene in a two-phase partitioning bioreactor of metabolically engineered Escherichia coli, Biotechnol. Bioeng. (2006)
- Tsuruta et al., High-level production of amorpha-4,11-diene, a precursor of the antimalarial agent artemisinin, in Escherichia coli, PLoS ONE (2009)
- PubMed search: in situ product removal and two-phase partitioning bioreactors
- PubMed search: membrane engineering for solvent and product tolerance in bacteria

Week 46 — In situ product removal: choosing a strategy, and sorbents [extension]

Sessions 226–230

By the end of the week you can

- Choose an in situ removal strategy from the product's own properties, and say what each costs
- Size a sorbent from an adsorption isotherm, and tell an equilibrium limit from a rate limit
- Judge an ISPR paper on whether its gain survives the scale-up questions

The five sessions

Day 1 · Why remove the product, and the menu · 2.5 h

☐ Write the four reasons to remove a product in situ, with one example each from your own candidate list

☐ Tabulate the methods (overlay, beads or resin, gas stripping, pervaporation, external column, crystallisation) against the product property that each suits

☐ For four products — a volatile sesquiterpene, a phenolic, a short-chain organic acid, a hydrophobic carotenoid — pick a method and say what measurement would confirm the choice

☐ Write what you would do if the product is charged at the fermentation pH, where a solvent is usually the wrong answer

Day 2 · Isotherms and sizing a sorbent · 3 h

☐ Write the linear and Langmuir isotherms and say what each parameter means physically

☐ Compute the total product a sorbent loading reaches, and the capacity ceiling at the inhibition limit — problem S3

☐ Compute the sorbent needed per kilogram of product, and say at what price that stops making sense

☐ Daily quiz

Day 3 · Inside the bead · 2.5 h

☐ Write why uptake into a solid is usually limited by diffusion inside the polymer rather than by the film outside, and what that means for bead size

☐ Read the abstracts of the Hytrel polymer-selection papers and note the range of partition coefficients reported between polymers for one compound

☐ Run the stress test in problem S3 where production doubles, and say when the uptake rate starts to matter rather than the capacity

☐ Write the experiment that would measure the uptake rate constant for your own product and polymer

Day 4 · Recovery, reuse and the external loop · 2.5 h

☐ Write how the product comes back off: solvent evaporation or distillation, elution from a resin, or a temperature or pH swing, and what each costs

☐ Compare an internal phase with an external column in a recirculation loop on regeneration, cell damage, pumping and control

☐ Write the cycles a resin must survive for its cost to disappear into the product, using the arithmetic of week 45

☐ Write what fouling by medium components or cells would do to a resin, and how you would detect it

Day 5 · Read a paper, then write your own plan · 2.5 h

☐ Read one ISPR paper from this week’s list and write what was gained, against what control, and whether the comparison was fair

☐ List the scale-up questions it does not answer: oxygen transfer, emulsions, solvent loss, recovery, safety

☐ Write a one-page removal plan for your capstone product, with the isotherm or partition coefficient you would measure first

☐ Daily quiz

Topics this week covers

- The vocabulary, which is used loosely in the literature: in situ product removal (ISPR) is the goal; a two-phase partitioning bioreactor (TPPB) is the reactor; liquid–liquid means a solvent, solid–liquid means polymer beads or resin. Immobilising the cells is a different thing entirely (week 48)
- Four reasons to remove a product as it is made: it inhibits the cells, it is volatile and lost, it degrades, or it inhibits its own enzyme and holds back an equilibrium

- The menu: an organic overlay, polymer beads or resin, gas stripping, pervaporation, an external loop with a column, crystallisation, or an ion-exchange step
- Choosing by the product: partition coefficient, volatility, charge and pKa, solubility, and heat sensitivity decide the method, not fashion
- Choosing by the cells: a solvent can itself be toxic or be eaten, which is why solid beads can be used with mixed cultures while solvents usually cannot
- Solid sorbents and the polymer that fits: partition coefficients for one compound span orders of magnitude between polymers, so selection is a screening job rather than a default choice
- Isotherms: a linear partition coefficient is the simple case; the Langmuir form adds a capacity ceiling, which is what sets the sorbent mass you need
- Inside the bead: uptake into a solid is usually limited by diffusion within the polymer rather than by the film outside it, which is a mass-transfer problem in your own discipline
- Recovery and reuse: the product has to come back off the phase, and the solvent or resin has to survive many cycles for the cost to work (week 49)
- External against internal: a column in a recirculation loop is easier to regenerate and keeps the solvent away from the cells, at the cost of pumping and of cells in the loop

Sources

- Prpich and Daugulis, Polymer development for enhanced delivery of phenol in a solid–liquid two-phase partitioning bioreactor, Biotechnol. Prog. (2004)
- Rehmann, Sun and Daugulis, Polymer selection for biphenyl degradation in a solid–liquid two-phase partitioning bioreactor
- Prpich and Daugulis, A novel solid–liquid two-phase partitioning bioreactor for the enhanced bioproduction of 3-methylcatechol, Biotechnol. Bioeng. (2007)
- Hydrophobic microspheres for in situ removal of 2-phenylethanol from yeast fermentation
- PubMed search: in situ product recovery, reviews and strategy comparisons

Week 47 — Modelling a two-phase fermentation, and fitting it to data [extension]

Sessions 231–235

By the end of the week you can

- Write and solve the equations for a fermentation with a second phase, and check the solution with a mass balance
- Separate an equilibrium limit from a transfer limit, and know when the simple shortcut is enough
- Fit inhibition parameters to data and say honestly whether the data identify them

The five sessions

Day 1 · The base model, and the balance that checks it · 3 h

☐ Write the single-phase equations: Monod growth, Luedeking–Piret production, and product inhibition. Say what each parameter does

☐ Solve them and read the ceiling the inhibition imposes, the unused substrate, and the productivity — problem S1

☐ Check the substrate balance closes: substrate used equals biomass over its yield plus product over its yield

☐ Write why a model that runs is not a model that is right, and what the closure check does and does not prove

Day 2 · Adding the second phase · 3 h

☐ Write the second-phase equations, including the $(1 - \varphi)/\varphi$ factor, and say what each concentration is per litre of

☐ Compute the titer over a grid of partition coefficient and phase fraction, and explain the row where more solvent makes things worse — problem S2

☐ Derive the shortcut $R = 1 + K\varphi/(1 - \varphi)$ and check it against the full model

☐ Daily quiz

Day 3 · Transfer limits and stiffness · 2.5 h

☐ Scan the transfer coefficient and find where the aqueous concentration rises above equilibrium — problem S2

☐ Compare the predicted offset, the production rate over the relaxation rate, with what the model shows

☐ Write why a large kLa makes the system stiff, and what a stiff solver does about it

☐ Write the one-line check you would run on any new two-phase model you are given

Day 4 · Sorbents, losses and the net effect · 3 h

☐ Swap the liquid phase for a Langmuir sorbent and compare the two formulations — problem S3

☐ Add evaporation and a solvent penalty, then scan the phase fraction for the best harvest titer — problem S4

☐ Write what the hold-time result means for when you harvest, and what the penalty assumption is doing to the answer

☐ Say which single measurement would most reduce your uncertainty in the whole model

Day 5 · Fitting, and whether the data identify the parameters · 3 h

☐ Fit the inhibition parameters to the synthetic data and report intervals and the parameter correlation — problem S5

☐ Fit from two different starting points and say what it would mean if they disagreed

☐ Add the second experiment and read how much the intervals tighten

☐ Write the sentence a methods section needs about how the parameters were obtained and what their intervals mean

☐ Daily quiz

Topics this week covers

- The model, phase by phase: biomass, substrate and product in the aqueous phase, product in the second phase, with transfer between them
- Growth and production: Monod saturation, product inhibition (the Levenspiel form), and the Luedeking–Piret split into growth-associated and non-growth-associated production
- The transfer term: $N = kLa \cdot (Pa - Po/K)$. The driving force is the distance from equilibrium, not the concentration itself
- Volume bookkeeping, which is where most errors live: concentrations per litre of aqueous against per litre of whole broth, and the $(1 - \varphi)/\varphi$ factor when transferring between phases of different size
- The equilibrium shortcut: with fast transfer the two phases behave as one, with the inhibition limit multiplied by $R = 1 + K\varphi/(1 - \varphi)$. Derive it, then check it against the full model
- When the shortcut fails: the relaxation rate $\lambda = kLa \cdot (1 + (1 - \varphi)/(K\varphi))$ sets the offset from equilibrium needed to carry the production rate
- Competing losses: evaporation and degradation draw product out of the same aqueous pool, so a model with both shows the real trade-off
- Solving it: stiff systems, why a stiff solver is needed when kLa is large, and the mass-balance and closure checks that catch a wrong sign
- Fitting and identifiability: a Jacobian, confidence intervals, parameter correlation, and what a second experiment adds

- Simulation is not evidence: every number here rests on assumed parameters, and the model earns trust only against measurements

Sources

- Newman et al., High-level production of amorpha-4,11-diene in a two-phase partitioning bioreactor of metabolically engineered Escherichia coli, Biotechnol. Bioeng. (2006)
- Substrate mass transport in two-phase partitioning bioreactors employing liquid and solid non-aqueous phases, Bioprocess Biosyst. Eng. (2012)
- PubMed search: Luedeking–Piret and product-inhibition kinetic models in fermentation
- SciPy documentation: solve_ivp, stiff solvers (Radau, LSODA) and least_squares

Week 48 — Cell immobilisation: diffusion, dead cores and continuous operation [extension]

Sessions 236–240

By the end of the week you can

- Say what immobilising cells is for, and what it is not for
- Compute whether a bead is fully alive, using the Thiele modulus and the effectiveness factor
- Judge whether retaining biomass actually beats free cells for a given product

The five sessions

Day 1 · What it is for, and the materials · 2.5 h

☐ Write the reasons to immobilise cells, and the separate reasons to remove a product, and say which problem each solves

☐ Tabulate alginate, carrageenan, polyvinyl alcohol and polyacrylamide on gentleness, strength, chemical weaknesses and toxicity to the cells

☐ Write why calcium alginate beads dissolve in phosphate buffers and in the presence of chelators, and what that rules out

☐ Write which of your candidate products could even be made this way, given that the cells must be largely non-growing

Day 2 · Diffusion inside the bead · 3 h

☐ Derive the Thiele modulus for a sphere and write what large and small values mean physically

☐ Compute the oxygen penetration depth and the dead-core fraction across bead sizes and cell loadings — problem T1

☐ Compare the first-order effectiveness factor with the zero-order dead-core picture and say when each applies

☐ Daily quiz

Day 3 · Oxygen, and why it is the problem · 2.5 h

☐ Compare the driving force for oxygen with that for glucose, and say why one penetrates and the other does not

☐ Write what you would change first to shrink a dead core: bead diameter, cell loading, or the oxygen in the bulk, and the cost of each

☐ Write what an anaerobic centre does to a strictly aerobic process and what it might do to a facultative one

☐ Write the measurement that would show a dead core exists, rather than assuming it from a model

Day 4 · Continuous operation: what retention buys · 3 h

☐ Compute the washout dilution rate for free cells and the productivity just below it — problem T2

☐ Compute the steady state with retained biomass across dilution rates, bead fractions and effectiveness factors

☐ Write why the comparison in that problem favours free cells, and what would have to change for immobilisation to win

☐ Write the stability argument: operating just below washout against operating at a high dilution rate with retention

Day 5 · Failure modes, and a decision · 2.5 h

☐ List how an immobilised system fails: bead breakage from cell growth and gas, leakage, fouling, channelling and pressure drop in a packed bed, and lost working volume

☐ Write what you would monitor to catch each one early

☐ Write a one-page decision for your capstone product: immobilise or not, with the Thiele modulus, the dead-core estimate and the productivity comparison behind it

☐ Daily quiz

Topics this week covers

- What immobilisation is for: retaining biomass so that dilution no longer washes it out, reusing a biocatalyst, easier separation, continuous and high-dilution operation, and some protection from shear and from inhibitors
- What it is not for: it does not remove the product from the broth, which is ISPR (week 46). Confusing the two is the commonest error in this area
- Materials: calcium alginate and carrageenan are easy and gentle but mechanically weak and sensitive to phosphate and chelators; polyvinyl alcohol and polyacrylamide are tougher, and the acrylamide monomer is toxic to the cells it entraps
- The methods: entrapment in a gel, adsorption to a carrier, covalent attachment, and self-aggregation as a biofilm or a pellet
- Diffusion inside the bead: the Thiele modulus compares reaction with diffusion, and the effectiveness factor is the fraction of the rate a bead achieves against the same cells in free suspension
- Dead cores: with zero-order consumption, oxygen runs out at a depth, so the centre is anaerobic and the live volume is only a shell. Bead diameter and cell loading decide it
- Why oxygen is the usual culprit: it is sparingly soluble, so the driving force is small, while glucose at tens of grams per litre penetrates easily
- The costs: cells grow and burst the beads, leakage into the broth, fouling, pressure drop in a packed bed, and the bead volume that displaces working volume
- Reactor forms: packed bed, fluidised bed, airlift and stirred tank with retention, each with its own oxygen and shear problem
- Honest comparison: immobilisation only pays when the specific productivity of the retained, largely non-growing cells is high enough to beat a free-cell process run near its best dilution rate

Sources

- PubMed search: cell immobilisation in alginate and PVA, reviews of methods and limitations
- PubMed search: effectiveness factor and oxygen diffusion limitation in immobilised-cell beads
- Hydrogel immobilised microalgae–alginate beads modelling phenol fermentation to biohydrogen, with a derived kinetic model
- PubMed search: biofilm and pellet morphology in bioprocesses

Week 49 — Techno-economic analysis and life-cycle assessment [extension]

Sessions 241–245

By the end of the week you can

- Build a flowsheet-level cost model of your own process rather than a back-of-envelope
- Compute a minimum product selling price and show what moves it
- Produce a cradle-to-gate greenhouse-gas number next to the fossil incumbent

The five sessions

Day 1 · Install BioSTEAM and run someone else's biorefinery · 2.5 h

☐ pip install biosteam, then work the first pages of its documentation — Stream, Unit, System, TEA

☐ Run one of the Bioindustrial-Park configurations unchanged and read off its reported metrics

☐ Write down what the model contains that your week 44 envelope did not: utilities, waste, installation factors, working capital

Day 2 · Capital and operating cost, honestly · 3 h

☐ Read how equipment is sized and costed, and what an installation factor covers

☐ Build the operating-cost side for your own product: feedstock at your real yield, utilities, oxygen and cooling duty, labour

☐ Write why a Lang factor is a reasonable first pass and a poor final answer

Day 3 · Minimum product selling price · 3 h

☐ Set up the discounted cash flow: capital charge, plant life, discount rate, and the price at which net present value is zero

☐ Compute MPSP for your product at 20, 50 and 100 g/L and tabulate the cost breakdown at each

☐ Compare against the market price of the incumbent and state plainly whether the process clears it — this is problem G1

Day 4 · Sensitivity, and where the uncertainty really sits · 2.5 h

☐ Vary yield, feedstock price and capital by a defensible range and build a tornado plot

☐ Run a Monte Carlo over the two or three inputs you genuinely cannot pin down, and report a price distribution rather than a point

☐ Name the one assumption carrying your conclusion, and say what experiment would retire it

Day 5 · Life-cycle assessment · 2.5 h

☐ Read on cradle-to-gate LCA: system boundary, functional unit, allocation between co-products

☐ Compute a first-pass kg CO_2e per kg of product from your own mass and energy balances, and compare with the fossil route

☐ Write what your boundary excludes and how that flatters or penalises your process — reviewers go here first

☐ Daily quiz

Topics this week covers

- From the week 44 cost sketch to a unit-operation model: seed train, fermenter, separation, waste treatment
- Capital estimation: equipment sizing, installation factors, and why a Lang factor is a blunt instrument
- Operating cost: feedstock, utilities, oxygen and cooling duty, labour, waste
- Discounted cash flow, and minimum product selling price as the output that matters
- Sensitivity: tornado plots, and Monte Carlo when inputs are uncertain rather than unknown
- Life-cycle assessment: system boundary, functional unit, allocation, and why the boundary choice decides the answer

- Reading a published TEA critically: which assumption is carrying the conclusion

Sources

- BioSTEAM — documentation
- BioSTEAM — source and biorefinery models
- Cortes-Peña et al., BioSTEAM: a fast and flexible platform … under uncertainty, ACS Sustain. Chem. Eng. (2020)
- A published techno-economic analysis of a fermentation product, for critical reading

Week 50 — Analytical method validation — trusting your own numbers

Sessions 246–250

By the end of the week you can

- Build and defend a calibration curve
- State a detection and quantification limit, and what they do not mean
- Design the controls that make a titer credible

The five sessions

Day 1 · Calibration, properly · 2.5 h

☐ Build a calibration curve from a dilution series and plot the residuals, not just the R^2

☐ Write why a high R^2 can hide curvature and heteroscedasticity, and when weighted regression is the honest choice

☐ Decide the working range for your assay and say what happens to samples above and below it

Day 2 · Detection and quantification limits · 2.5 h

☐ Read how limits of detection and quantification are defined from the blank and from the calibration slope

☐ Compute both for your assay and write what each permits you to claim

☐ Note the common abuse: reporting a value below the quantification limit as though it were a measurement

Day 3 · Accuracy, precision, and recovery · 3 h

☐ Distinguish repeatability from intermediate precision, and design the experiment that measures each

☐ Design a spike-and-recovery experiment in fermentation broth rather than in water

☐ Write why matrix effects make an internal standard necessary for mass-spectrometry quantification of an intracellular metabolite

Day 4 · Before the instrument · 2.5 h

☐ Read on quenching and extraction for intracellular metabolites and the losses each step causes

☐ Write the quenching method you would use for your host and why, including its known artefacts

☐ Note that a 30% extraction loss applied evenly is survivable, while one that varies between strains invalidates the comparison

Day 5 · Write the method section · 2.5 h

☐ Write the analytical method section for your capstone assay: instrument, column, standards, internal standard, calibration range, limits, controls, and the sample queue

☐ Add the controls that would reveal carryover and drift

☐ Read one published methods section and list what it omitted that you would have wanted

☐ Daily quiz

Topics this week covers

- Calibration: range, linearity, weighting, and why forcing through zero is usually wrong
- Internal standards against external calibration, and when matrix effects demand the former
- Limits of detection and quantification: how they are defined and how they are abused
- Accuracy, precision, repeatability and intermediate precision — different words for different experiments
- Matrix effects and recovery: fermentation broth is not water
- Carryover, drift and the sample-queue design that reveals them
- Sample handling: quenching, extraction and the losses that happen before the instrument
- What the method validation section of a good paper contains

Sources

- A tutorial on analytical method validation for chromatographic assays
- A review of quenching and extraction for microbial metabolomics
- A paper reporting fermentation titers with a full methods section, for critical reading

Week 51 — Omics, screening, and turning data into model constraints

Sessions 251–255

By the end of the week you can

- Explain transcriptomics, proteomics, metabolomics and fluxomics and what each closes in the loop
- Know the core analytics: HPLC, GC-MS, LC-MS, FACS
- Turn an omics dataset into a constraint on a model rather than a figure in a paper

The five sessions

Day 1 · Omics, one by one · 3 h

☐ Read Ko, Lee et al., Chem. Soc. Rev. 49:4615, the omics sections

☐ Write one line per layer: transcriptomics, proteomics, metabolomics, fluxomics — what it measures and what it cannot

☐ Say which layer you would measure first for a strain that grows well but makes nothing

Day 2 · Analytics · 2.5 h

☐ Read how HPLC, GC-MS and LC-MS are used for titer and intracellular metabolites

☐ Write which method you would use for your capstone product and why

☐ Note the sample-preparation step that most often ruins metabolomics data

Day 3 · Screening and sorting · 2 h

☐ Read on biosensor-coupled FACS and plate-based screening

☐ Compute roughly how many variants each format can handle per day

☐ Connect this back to the Test bottleneck from week 11

Day 4 · Omics into models · 3 h

☐ Read on enzyme-constrained models such as GECKO and on transcriptomics-informed flux models

☐ Write how a proteomics dataset becomes a constraint — a bound on a flux through an enzyme's abundance and turnover number — rather than a decoration

☐ Take each omics layer in turn and name exactly which model parameter or bound it could set, and which it cannot

Day 5 · Phase 3 checkpoint · 2.5 h

☐ Write one sentence per technique from weeks 20 to 51: what it does, when you choose it

☐ Confirm both notebooks are pushed, documented and readable by a stranger in five minutes

☐ Write why techno-economic analysis kills strains that look good in the lab

Topics this week covers

- RNA-seq, mass-spectrometry proteomics, metabolite pools, ^{13}C fluxes
- Biosensor-coupled single-cell screening
- Biofoundries and lab automation
- Enzyme-constrained models such as GECKO; proteome allocation as a model constraint — the growth laws from week 9, made usable
- Transcriptomics-informed and omics-integrated flux models, and their limits

Sources

- Ko, Lee et al., “Tools and strategies of systems metabolic engineering”, Chem. Soc. Rev. 49:4615 (2020)
- A gas-fermentation or scale-up review

Phase checkpoint — worked in day 5

Answer each from memory, on paper, before reading the model answer.

☐ One sentence per technique: what it does and when you would choose it

> **Model answer.** No fixed answer, but the sentence for each technique should name what it does and the condition under which you would pick it — for example: CRISPRi, tunable and reversible knockdown, chosen when the gene is essential or when you want a dose-response rather than an on-off answer.

☐ Submit the COBRApy notebook — growth, deletions, flux variability — and the reproduced growth-coupled design

> **Model answer.** A reviewer should be able to open your repository and, in five minutes, see what the notebook does, run it, and read a conclusion. If a stranger has to ask what a cell does, it is not finished.

☐ Explain the difference between a flux FBA calls feasible and one ^{13}C-MFA calls actual

> **Model answer.** FBA returns one flux vector consistent with stoichiometry and an assumed objective — a hypothesis about what could happen. ^{13}C-MFA infers fluxes from measured labelling patterns — evidence about what did happen, including split ratios and reversibilities a model must assume. A design justified only by FBA should be validated by labelling before being believed.

Week 52 — Optional — host defences and pooled screens [optional]

Sessions 256–260

By the end of the week you can

- Identify an industrial strain’s restriction barriers and plan a way past them
- Estimate how likely a construct is to survive them before you build it
- Understand what a pooled barcode screen measures and how its counts are analysed

The five sessions

Day 1 · Restriction–modification systems · 2.5 h

☐ Read a review of restriction–modification barriers in non-model bacteria and write the four types in a line each, with how each recognises and cuts

☐ Look up a strain in REBASE and list its restriction enzymes and methyltransferases with their recognition motifs

☐ Count how many times each motif occurs in a plasmid you have built, on both strands

☐ Write which of your constructs would be destroyed on entry, and why

Day 2 · Getting DNA past the barrier · 3 h

☐ Compute the expected number of sites for 4-, 6- and 8-base motifs in a 6 kb plasmid, and the chance that a plasmid carries none — problem M1

☐ Write the bypass options with their costs: methylate the plasmid in the cloning host with the strain's methyltransferases, remove the motifs from the sequence, use a helper plasmid, or change how the DNA is delivered

☐ Say how you would confirm a bypass worked: transformation efficiency of methylated against unmethylated preparations of the same plasmid

☐ Write why the number of distinct systems in a strain matters more than any single one

Day 3 · Pooled screens, conceptually · 2.5 h

☐ Read the RB-TnSeq paper for how barcodes decouple library construction from fitness measurement

☐ Write why a pooled format beats plate screening once the library exceeds your plate capacity, and what it cannot tell you about individual clones

☐ Note which questions need isolated clones anyway

Day 4 · Counting barcodes · 2.5 h

☐ Take a public barcode or CRISPRi-seq count table and compute log fold changes between conditions

☐ Apply the normalisation and multiple-testing control you met in week 33, and make a volcano plot of gene fitness

☐ Write which hits you would follow up as isolates

Day 5 · From a pooled hit to a validated phenotype · 2 h

☐ Design the follow-up for one hit: reconstruct the mutant cleanly, measure it alone, and measure it in competition with the parent

☐ Use the replicate and power reasoning from week 17 to say how many replicates the follow-up needs

☐ Write what result would make you drop the hit

☐ Daily quiz

Topics this week covers

- Restriction–modification systems: Types I to IV, recognition motifs, and how a wild strain destroys incoming DNA
- REBASE, and predicting a strain's methylation requirements from its genome
- How often a motif occurs by chance, and why a strain with several systems destroys nearly everything foreign
- Bypass strategies: in vivo methylation in a cloning host, helper plasmids, site removal, and delivery method
- Pooled screens: RB-TnSeq and CRISPRi-seq, barcode counting, and fitness scores as log fold changes
- Why pooled formats beat plate screening once a library is larger than your plate capacity, and what they cannot tell you
- From a pooled hit to a validated phenotype

Sources

- REBASE — the restriction enzyme database
- Wetmore et al., Rapid quantification of mutant fitness in diverse bacteria by sequencing randomly bar-coded transposons (RB-TnSeq), mBio (2015)
- A review of restriction-modification barriers to engineering non-model bacteria

Phase 4 — Applications and PhD prep

Weeks 53 to 66

Build a mental library of what the field has actually achieved, then convert it into a proposal, a portfolio and a lab shortlist.

Primary texts: Landmark papers; 2025–2026 reviews; journal editorial boards.

Week 53 — Pharmaceuticals, terpenoids and natural products

Sessions 261–265

By the end of the week you can

- Reconstruct the semi-synthetic artemisinin story end to end
- Explain what made complete opioid biosynthesis in yeast hard
- Describe the isoprenoid platform

The five sessions

Day 1 · Artemisinin, the build · 3 h

☐ Read Paddon et al., Nature 496:528

☐ Draw the full pathway from acetyl-CoA to artemisinic acid with every engineered gene marked

☐ Note the titer and the host strain

Day 2 · Artemisinin, the lessons · 2.5 h

☐ Read Paddon & Keasling, Nat. Rev. Microbiol. 12:355

☐ Write why the process is called semi-synthetic and what the chemistry step does

☐ Log the paper in the paper log with the seven-point protocol

Day 3 · Opioids in yeast · 3 h

☐ Read Galanie et al., Science 349:1095

☐ Count the engineered genes and note their source organisms

☐ Write the two hardest problems the authors reported and how they addressed them

☐ This paper is the field's standard dual-use case: read what was published alongside it on governance of yeast strains that make controlled substances, and write your own position in a paragraph — you may be asked it in an interview, and a considered answer beats both alarm and dismissal

Day 4 · The isoprenoid platform · 2 h

☐ Read a recent terpenoid review

☐ List five commercial product classes built on the isoprenoid platform

☐ Note which enzyme families recur across them

Day 5 · Technical success versus market success · 2 h

☐ Write why semi-synthetic artemisinin struggled commercially despite working

☐ Draft the interview answer that uses this example without sounding cynical

☐ Daily quiz

Topics this week covers

- Mevalonate pathway plus amorphadiene synthase plus a P450 in S. cerevisiae; 25 g/L artemisinic acid; chemical conversion to artemisinin
- Galanie et al.: 21–23 genes from plants, bacteria and rat
- Enzyme promiscuity and low-flux bottlenecks
- Commercial reality: plant-derived supply stayed competitive

Sources

- Paddon et al., Nature 496:528 (2013)
- Paddon & Keasling, Nat. Rev. Microbiol. 12:355 (2014)
- Galanie et al., “Complete biosynthesis of opioids in yeast”, Science 349:1095 (2015)

Week 54 — Commodity chemicals, bioplastics and amino acids [extension]

Sessions 266–270

By the end of the week you can

- Explain the two archetypes: 1,3-propanediol and 1,4-butanediol
- Explain why C. glutamicum owns the amino-acid market
- Read titers critically against commercial targets

The five sessions

Day 1 · Bio-propanediol · 2.5 h

☐ Read the EPA 2003 Greener Reaction Conditions award summary and one bio-PDO review

☐ Draw the combined pathway and name the source organism of each half

☐ Note why sugar uptake was re-engineered for energy efficiency

Day 2 · 1,4-Butanediol, the design · 3 h

☐ Reread Yim et al. with weeks 35 to 40 behind you

☐ Write how the pathway was found and how the knockouts were chosen

☐ Mark where the reducing power comes from and connect it to your week 6 answer

Day 3 · 1,4-Butanediol, the scale-up · 2.5 h

☐ Read on the commercialisation of bio-BDO

☐ Write the case as host / target / pathway / tools / TRY / bottleneck / commercial status

☐ Note the titer gap between the paper and the plant

Day 4 · Amino acids and C. glutamicum · 2.5 h

☐ Read a C. glutamicum systems metabolic engineering review

☐ List the properties that make it the industrial host for amino acids

☐ Note the reported lysine titers and yields, and what feedback-resistant alleles contribute

Day 5 · Bioplastics · 2 h

☐ Read on PHA biosynthesis in Cupriavidus necator

☐ Write how carbon is partitioned between storage polymer and growth

☐ Daily quiz

Topics this week covers

- Bio-PDO: yeast glucose-to-glycerol genes plus bacterial glycerol-to-PDO genes in E. coli; Sorona; 2003 Presidential Green Chemistry Challenge Award
- 1,4-BDO: a non-natural product designed computationally, engineered anaerobic oxidative TCA branch, 18 g/L to above 100 g/L
- PHA bioplastics in Cupriavidus necator
- Lysine and glutamate at multi-million-tonne scale; feedback-resistant alleles

Sources

- Yim et al., Nat. Chem. Biol. 7:445 (2011)
- EPA, 2003 Greener Reaction Conditions Award — DuPont bio-PDO
- A C. glutamicum systems metabolic engineering review

Week 55 — Biofuels, C1 and CO_2 utilisation, food ingredients [extension]

Sessions 271–275

By the end of the week you can

- Explain the non-fermentative higher-alcohol route
- Explain gas fermentation and circular carbon
- Explain precision fermentation for food

The five sessions

Day 1 · Higher alcohols · 3 h

☐ Read Atsumi, Hanai & Liao, Nature 451:86

☐ Draw the 2-ketoacid route and the Ehrlich degradation steps

☐ Explain why the same logic yields a family of alcohols, and note the titer and percentage of theoretical

Day 2 · Fuels at scale · 2.5 h

☐ Read on isobutanol commercialisation and on mitochondrial compartmentalisation in yeast

☐ Write why compartmentalisation helped

☐ Note what killed or slowed most advanced-biofuel ventures

Day 3 · Gas fermentation · 3 h

☐ Read a LanzaTech or acetogen review

☐ Write the organism, the feedstock, the energetics of acetogenesis and why ATP is so tight

☐ Note the commercial scale and the product range

Day 4 · C1 bioconversion · 2 h

☐ Read on methanol or formate as a feedstock

☐ Compare C1 substrates with sugar on cost, energy and engineering difficulty

☐ Note which hosts are being developed for them

Day 5 · Food ingredients · 2 h

☐ Read on soy leghemoglobin production in Komagataella phaffii and the FDA GRAS notice

☐ Write why a yeast was chosen over a bacterium for a secreted heme protein

☐ Daily quiz

Topics this week covers

- Atsumi & Liao: 2-ketoacid intermediates plus Ehrlich degradation; 22 g/L isobutanol at 86% of theoretical
- Gevo, Butamax, mitochondrial compartmentalisation
- LanzaTech: Clostridium autoethanogenum on CO, CO_2 and H_2; first commercial plant 2018
- Soy leghemoglobin in Komagataella phaffii; GRAS notice GRN 737

Sources

- Atsumi, Hanai & Liao, Nature 451:86 (2008)
- A LanzaTech or gas-fermentation review
- FDA GRAS Notice GRN 737 — soy leghemoglobin

Week 56 — PHA — from pathway to polymer [extension]

Sessions 276–280

By the end of the week you can

- Draw the glucose-to-PHB pathway with its cofactors from memory, and say what limits yield
- Read a fed-batch table and judge a strain on content, productivity and yield together
- Read a PHA paper the way a member of the lab would

The five sessions

Day 1 · The PHB pathway and its cofactor balance · 2.5 h

☐ Draw glucose to PHB: glycolysis, pyruvate dehydrogenase, then PhaA, PhaB and PhaC, marking every NADH, NADPH and ATP

☐ Write the net equation, check that it balances, and read off the surplus reduced cofactors — problem K1

☐ Compute the mass and carbon yield ceilings and the oxygen needed to clear the surplus

☐ Write in two sentences why a strain cannot reach the ceiling even with perfect enzymes

Day 2 · Strains and tools · 2.5 h

☐ Read a PHA review and list the hosts in use: Cupriavidus necator for short-chain PHA and autotrophic growth, Pseudomonas putida for medium-chain PHA, recombinant E. coli

☐ Write which genetic tools each host needs and what is harder than in E. coli, using what you learned in week 23

☐ Note where the optional host-defence material applies: a non-model host can destroy incoming DNA

☐ Daily quiz

Day 3 · Reading a run · 3 h

☐ Take the fed-batch table in problem K2 and compute PHB content, residual biomass, volumetric productivity and yield

☐ Mark the point where residual biomass stops rising and say what that implies about the nutrient feed

☐ Compare the yield in the accumulation phase against the ceiling from K1, and say what the gap is made of

☐ Write what you would change in the next run, and what measurement would tell you it worked

Day 4 · Feedstocks and copolymers · 2.5 h

☐ Read the abstract and introduction of the lab’s lignocellulosic PHB paper: why hydrolysates are hard, which inhibitors matter, and what “robust” means for a strain

☐ Read the copolymer paper’s abstract: what a 3-hydroxyvalerate fraction changes in the material, and what the engineering handle is

☐ Log both papers in the paper log, then write one question about each that a member of the lab could answer in two sentences

☐ Write the four inhibitors you would measure in a hydrolysate and the assay for each

Day 5 · Recovery, characterisation, and what you would measure · 2.5 h

☐ Write how PHA content and monomer composition are measured, and which techniques give molecular weight and thermal properties

☐ Connect this to the calibration and validation material in week 50: what would you need to show before trusting a PHA titer

☐ Write the one-page summary of this week as a brief a lab member could read in a minute

☐ Daily quiz

Topics this week covers

- PHA as a carbon and reducing-power sink: nutrient limitation stops growth while carbon keeps flowing into polymer

- The short-chain pathway in Cupriavidus necator: PhaA β-ketothiolase, PhaB NADPH-dependent reductase, PhaC synthase
- Copolymers: how a 3-hydroxyvalerate fraction is tuned, and why composition sets material properties
- Medium-chain PHA in Pseudomonas, from fatty-acid β-oxidation or de novo synthesis
- The redox surplus on sugar: three reduced cofactors left per glucose, and the oxygen it takes to clear them
- Feedstocks: sugars, lignocellulosic hydrolysates with their inhibitors, acids derived from plastic waste, and CO_2-derived substrates
- Process metrics that trade against each other: PHA content, volumetric productivity, yield on substrate
- Recovery and characterisation: extraction or digestion, GC after methanolysis, DSC, GPC molecular weight, NMR
- Reading the lab's own recent papers, named in its 2026 recruiting notice

Sources

- AIM Lab — KIST Applied and Industrial Microbiology Lab (publications page)
- Sun et al., efficient poly(3-hydroxybutyrate) production from lignocellulosic hydrolysates with a robust Cupriavidus necator RXI22 strain, Green Chemistry (2026)
- Jo et al., Enhanced production of poly(3-hydroxybutyrate-co-3-hydroxyvalerate) with modulated 3-hydroxyvalerate fraction by overexpressing acetolactate synthase in Cupriavidus necator H16, Int. J. Biol. Macromol. (2023)
- A review of polyhydroxyalkanoate production and metabolic engineering in Cupriavidus necator

Week 57 — CO_2, hydrogen and waste carbon as feedstocks [extension]

Sessions 281–285

By the end of the week you can

- Compute what a CO_2-to-PHB route demands in hydrogen, electricity and gas transfer
- Explain how bioelectrochemical and hybrid routes differ from direct gas fermentation
- Judge honestly when waste carbon or CO_2 beats sugar, and when it does not

The five sessions

Day 1 · What a CO_2 route actually needs · 3 h

☐ Write $4\ CO_2 + 9\ H_2 \rightarrow C_4H_6O_2 + 6\ H_2O$ and check that it balances — this is the floor, and it counts reducing equivalents only

☐ Tally the same thing from the pathway (Calvin cycle, then acetyl-CoA, then PhaB) and confirm net ATP and net reducing equivalents — problem K3

☐ Add the energy for CO_2 fixation by respiring extra hydrogen, and see how the total depends on the ATP yield you assume

☐ Write which of your assumptions would move the answer most

Day 2 · Electricity, and the honest comparison with sugar · 2.5 h

☐ Convert hydrogen demand into kWh per kg of PHB at the electrolysis energy you assume, and into cost at three electricity prices

☐ Compute the electricity price at which the CO_2 route matches the feedstock cost of the sugar route, at the ceiling yield and at a realistic one

☐ List what the number leaves out: electrolyser capital, CO_2 supply, gas handling, downstream

☐ Connect it to the techno-economic framework of week 49 and state where this would enter your cost model

Day 3 · Moving gas into liquid · 3 h

☐ Compute the hydrogen uptake rate for a target productivity and the kLa it needs, using the correlation approach of week 44 — problem K4

☐ Repeat for oxygen with the oxygen fraction held low, and find which gas limits

☐ Write what raises kLa in practice and what each choice costs in power and shear

☐ Write the safety statement you would put at the top of any proposal involving hydrogen and oxygen, and who has to review it before work starts

Day 4 · Bioelectrochemical and hybrid routes · 2.5 h

☐ Read the abstracts of the lab's 2024 cathode-catalyst paper and its 2025 microbial-protein paper

☐ Draw a schematic of an integrated bioelectrochemical system: cathode, membrane, anode, medium, gas, cells

☐ List what limits current density and what "biocompatible" has to mean for a catalyst in a culture

☐ Log both papers in the paper log with one question each

Day 5 · Waste carbon, and the table you would defend · 2.5 h

☐ Read the abstract of the plastic-waste paper: what the chemical step produces and what the biological step does with it

☐ Build one table with sugar, lignocellulosic hydrolysate, plastic-waste-derived acids and CO_2/H_2 as rows, and pretreatment, inhibitors, achievable yield, cost drivers and carbon footprint as columns

☐ Mark which cells you could fill from your own calculations and which you would have to look up or measure

☐ Daily quiz

Topics this week covers

- The stoichiometric floor: 4 CO_2 + 9 H_2 per 3-hydroxybutyrate unit, which counts reducing equivalents only
- The energy cost on top: ATP for CO_2 fixation, supplied by respiring more hydrogen with oxygen
- Electricity as the real feedstock: kWh per kg of product and the electricity price at which the route breaks even against sugar
- Gas–liquid transfer for sparingly soluble gases, and why oxygen can limit before hydrogen does
- Safety shapes the design: flammable hydrogen and oxygen mixtures, limiting oxygen concentration, and the need for formal review
- Bioelectrochemical routes: microbial electrosynthesis, cathode catalysts that cells tolerate, integrated systems that make protein from CO_2
- Plastic waste as a feedstock: chemical oxidation of polyethylene, then biological conversion to a biodegradable polymer
- Lignocellulosic hydrolysates: mixed sugars, inhibitors, and robust strains against detoxification
- Cost and carbon accounting for all of the above, in the framework of week 49

Sources

- AIM Lab — KIST Applied and Industrial Microbiology Lab (home)
- Firmansyah et al., Upgrading polyethylene plastic waste into a biodegradable polymer: a hybrid chemical oxidation–biological conversion approach, Chem. Eng. J. (2025)
- Lee et al., Sustainable production of microbial protein from carbon dioxide in the integrated bioelectrochemical system using recycled nitrogen sources, Water Res. (2025)
- Moon et al., Biocompatible Cu/NiMo composite electrocatalyst for hydrogen evolution reaction in microbial electrosynthesis, Adv. Sci. (2024)
- A review of Cupriavidus necator autotrophic growth and gas fermentation

Week 58 — Yeast hosts: S. cerevisiae, Y. lipolytica and K. phaffii [extension]

Sessions 286–290

By the end of the week you can

- Explain what a compartmentalised cell changes about pathway design
- Compute the lipid yield ceiling and the cofactor and ATP costs behind it
- Choose between the three yeasts for a product and defend the choice with numbers

The five sessions

Day 1 · The cell, and where acetyl-CoA lives · 2.5 h

☐ Draw a yeast cell with its compartments and mark where pyruvate dehydrogenase, ATP-citrate lyase, fatty acid synthesis and beta-oxidation each sit

☐ Write why a cytosolic pathway cannot use mitochondrial acetyl-CoA directly, and the two routes that supply the cytosolic pool

☐ Write what the secretory route adds that a bacterium lacks: signal peptide, endoplasmic reticulum folding, glycosylation, Golgi, vesicles

☐ Daily quiz

Day 2 · The lipid ceiling and the cofactor bill · 3 h

☐ Compute the carbon ceiling for lipid on glucose, the NADPH demand and the extra ATP spent moving acetyl units between compartments — problem W3

☐ Write why overexpressing a fatty acid synthase alone rarely raises titer

☐ Write what the carbon-to-nitrogen ratio does, and how you would set it in a fed-batch

☐ Connect the result to your cofactor-balancing work in week 19

Day 3 · Three yeasts, compared properly · 3 h

☐ Fill a comparison of S. cerevisiae, Y. lipolytica and K. phaffii: substrate range, Crabtree status, lipid storage, secretion, tool maturity, GRAS status, documented products

☐ Cite a source for every non-obvious cell, and mark the cells where you could not find one

☐ Write one line per yeast on what it is uniquely good at, and one on what it is bad at

☐ Add the table to your host table from week 61 rather than duplicating it

Day 4 · Tools, and what does not transfer · 2.5 h

☐ Write how DNA is delivered and maintained in each of the three: episomal plasmids, integration, markers, promoters

☐ Write why gene targeting is harder in Y. lipolytica than in S. cerevisiae, and what is done about it

☐ Write three things that work in S. cerevisiae and do not simply carry across

☐ Write the first three experiments you would run in a yeast new to you

Day 5 · Choosing, and the bottlenecks you would hit · 2.5 h

☐ Choose a yeast for your capstone product and write the argument in five sentences, with the yield number in it

☐ Write the bottleneck you would expect first, and the measurement that would confirm it

☐ Write what would make you abandon the host

☐ Daily quiz

Topics this week covers

- Why yeast at all: a eukaryote folds and glycosylates proteins a bacterium cannot, tolerates low pH and high osmolarity, has GRAS status for food use, and carries organelles a pathway can be put into
- The cell: nucleus, mitochondria, endoplasmic reticulum and Golgi for the secretory route, peroxisomes, vacuole, and a cell wall of mannoproteins and glucan

- Where acetyl-CoA sits: mitochondrial from pyruvate dehydrogenase, cytosolic from ATP-citrate lyase or from acetyl-CoA synthetase. Lipid and isoprenoid pathways need the cytosolic pool, which is why its supply is engineered first
- S. cerevisiae: the best toolkit in biology, but Crabtree-positive, so it makes ethanol from sugar even with oxygen present, and it stores little lipid. Engineering it for lipid means rewiring that
- Y. lipolytica: oleaginous, Crabtree-negative, grows on hydrophobic substrates and glycerol, strongly secretory, used for lipids, organic acids and cosmetic ingredients. Fewer tools, and it is dimorphic, so morphology shifts with conditions
- K. phaffii (formerly Pichia pastoris): high-density growth on glycerol and methanol, strong inducible promoters and excellent secretion, which is why it dominates protein production rather than small-molecule work
- Nitrogen limitation as a switch: a high carbon-to-nitrogen ratio stops protein synthesis and routes carbon into storage lipid. The trigger is a ratio, not a gene
- Compartment engineering: putting a pathway in the mitochondrion or peroxisome concentrates the precursors and contains a toxic intermediate, at the cost of transporting every substrate and cofactor across a membrane (week 45)
- Tools in practice: episomal against integrated expression, auxotrophic and dominant markers, CRISPR in each of the three, and why non-homologous end joining in Y. lipolytica makes targeting harder than in S. cerevisiae
- The honest bottlenecks: glycosylation that differs from the human pattern, protease activity in the medium, plasmid instability, and the fact that an excellent toolkit in S. cerevisiae does not transfer to the other two

Sources

- PubMed search: Yarrowia lipolytica as a host for lipids and oleochemicals
- PubMed search: cytosolic acetyl-CoA supply in yeast for lipid and isoprenoid pathways
- PubMed search: Komagataella phaffii (Pichia pastoris) protein secretion platform
- PubMed search: subcellular compartmentalisation of pathways in yeast

Week 59 — Gram-positive hosts: C. glutamicum and B. subtilis [extension]

Sessions 291–295

By the end of the week you can

- Explain what the envelope difference changes about secretion and downstream processing
- Compute an amino-acid yield ceiling and the cofactor demand that sets it
- Say when a Gram-positive host is the right choice and when it is not

The five sessions

Day 1 · The envelope, and what it changes · 2.5 h

☐ Draw both envelopes side by side and mark where a signal peptide delivers a protein in each

☐ Write what the absence of an outer membrane means for secretion, for endotoxin, and for how you would recover a protein

☐ Write what the thick peptidoglycan wall still makes difficult

☐ Daily quiz

Day 2 · The lysine case, with numbers · 3 h

☐ Compute the yield ceiling, the NADPH demand, and what the pentose phosphate pathway must carry — problem W2

☐ Compute the ATP spent on export at 50, 100 and 150 g/L, and say whether it is a real term

☐ Write why carbon yield and NADPH supply pull against each other, and name three ways the literature addresses it

☐ Connect it to your own cofactor-balancing work in week 19

Day 3 · Secretion as the deciding factor · 2.5 h

☐ Compute what a product retained inside the cells would be capped at, given a realistic cell volume fraction — problem W2

☐ Write the downstream difference between harvesting a secreted product and lysing cells for an intracellular one

☐ Write why B. subtilis dominates industrial enzyme production, and what its native proteases do to a heterologous product

☐ Write what a protease-deficient strain background buys and costs

Day 4 · Tools and failure modes · 2.5 h

☐ Write how DNA is delivered and maintained in each host, and which promoters are used

☐ List the failure modes: protease degradation, sporulation, phage, foaming, plasmid loss, and what you would monitor for each

☐ Write what you would check first if a secreted protein appeared in the medium and then disappeared

☐ Write the three experiments that would tell you whether a new product can be secreted at all

Day 5 · Choosing between hosts · 2.5 h

☐ Write when you would choose C. glutamicum or B. subtilis over E. coli, with the reason stated as a measurable property

☐ Write when a Gram-positive host is the wrong answer, and why

☐ Add both hosts to your host table from week 61 with the numbers you computed

☐ Daily quiz

Topics this week covers

- The envelope: one membrane and a thick peptidoglycan wall, against the inner membrane, periplasm and outer membrane of E. coli. A secreted protein crosses one barrier instead of two
- Lipopolysaccharide: the Gram-negative outer membrane carries an endotoxin that must be removed from any injectable product, which Gram-positive hosts avoid
- Why secretion decides host choice: a product retained inside is capped by cell volume, and recovery then needs lysis, debris removal and a harder purification
- C. glutamicum: non-pathogenic, GRAS, the industrial producer of glutamate and lysine at the million-tonne scale, with dedicated amino-acid exporters and a well-mapped central metabolism
- The lysine case: 4 mol NADPH per mol lysine, so the oxidative pentose phosphate pathway carries the demand, and carbon yield and reductant supply pull against each other
- Export as a design variable: C. glutamicum has specific amino-acid efflux carriers, and export costs energy, which is a real term at a hundred grams per litre
- B. subtilis: the industrial enzyme secretor, with native proteases and amylases exported at commercial titers, good genetics, natural competence and sporulation as both a tool and a nuisance
- Tools: replicative and integrative vectors, inducible promoters, CRISPR in both hosts, and the strain backgrounds in which protease genes have been deleted
- The bottlenecks nobody mentions first: proteases in the medium degrading your product, sporulation diverting a culture, the wall itself limiting large proteins, phage sensitivity, and foaming

from secreted protein
- Where these hosts are the wrong choice: pathways needing eukaryotic folding or glycosylation, and products that need the periplasmic space E. coli provides

Sources
- Madigan et al. — Brock Biology of Microorganisms, the cell-envelope chapter
- Theoretical maximum yields and NADPH demand for L-lysine in C. glutamicum (reported 0.81 mol/mol native, 0.86 with transhydrogenase)
- Engineering C. glutamicum with an NADPH-generating glycolytic pathway for L-lysine production, Appl. Environ. Microbiol.
- PubMed search: Bacillus subtilis secretion of industrial enzymes and protease-deficient strains
- PubMed search: amino acid exporters in Corynebacterium glutamicum

Week 60 — Phototroph hosts: cyanobacteria and microalgae [extension]

Sessions 296–300

By the end of the week you can
- Treat light as the limiting substrate and compute what a culture can actually produce
- Quantify the three bottlenecks: self-shading, slow growth and the cost of taking the water out
- Say honestly which products justify a phototroph and which do not

The five sessions

Day 1 · Light is the substrate · 3 h
☐ Compute the depth at which light falls to 10% and 1% of the surface value across a range of cell densities — problem W1
☐ Compute what fraction of a 30 cm pond is lit at each density, and write what that means for mixing
☐ Write why the right unit is grams per square metre per day rather than grams per litre per day
☐ Daily quiz

Day 2 · What a pond can actually produce · 3 h
☐ Compute the areal productivity across cell densities with a saturating light response, and find where it stops rising — problem W1
☐ Convert it to tonnes per hectare per year and compare with a land crop
☐ Write what photoinhibition adds that the saturating model leaves out
☐ Write which assumption in the model you would most want to measure

Day 3 · The two hosts and their tools · 2.5 h
☐ Write the difference between a cyanobacterium and a eukaryotic microalga: genome organisation, organelles, where a transgene can go
☐ Write what polyploidy does to a knockout in Synechocystis, and what silencing does to a nuclear transgene in Chlamydomonas
☐ Compare the design–build–test cycle time with E. coli and say what that costs a project
☐ Write the three tool gaps you would have to close before engineering a pathway

Day 4 · Harvesting, and why bulk products struggle · 2.5 h
☐ Compute the water removed per kilogram of biomass and the energy an evaporative route would need — problem W1
☐ Write the staged harvesting route and where the energy actually goes
☐ Compare the dewatering energy with the energy content of the biomass, and say what that rules out
☐ Connect it to your techno-economic work in week 49

Day 5 · What a phototroph is actually for · 2.5 h

☐ Write the product classes that justify a phototroph, with a value per kilogram for each

☐ Write the classes that do not, with the number that kills them

☐ Write five honest sentences on your own microalgal work: what it was for, what the arithmetic says about it, and why you would or would not return to it

☐ Daily quiz

Topics this week covers

- The appeal: carbon dioxide and light as the only inputs, no sugar feedstock, and products such as astaxanthin, beta-carotene and biomass for food that already sell at high value
- Cyanobacteria against eukaryotic microalgae: prokaryotic with no organelles and relatively tractable genetics (Synechocystis, Synechococcus) against eukaryotic with chloroplast and nuclear genomes (Chlamydomonas, Chlorella, Dunaliella, Nannochloropsis)
- Light as a substrate you cannot stir: Beer–Lambert attenuation means a dense culture shades itself, so photons arrive at a surface while cells sit in the dark
- Areal against volumetric productivity: the right unit for a photobioreactor is grams per square metre per day, because the land or the illuminated area is what you are buying
- Photoinhibition and saturation: more light is not more growth above the saturation irradiance, and strong light damages photosystem II
- Growth rates: doubling in hours to a day rather than in twenty minutes, which lengthens every design–build–test cycle (week 25)
- Genetic tools: natural competence and homologous recombination in some cyanobacteria, against chloroplast transformation, position effects, silencing and polyploidy in algae, with CRISPR far less routine than in E. coli
- Harvesting and dewatering: cells are small, dilute and neutrally buoyant, so separation is a major energy and cost term, and it is why bulk products struggle
- Contamination and crop protection in open ponds, against the capital cost of a closed photobioreactor
- The honest conclusion: phototrophs make economic sense for high-value products, for food and feed, and where carbon capture itself is the point, and they are hard to justify for commodity fuels (week 49)
- Your own background: the microalgal glutathione and lipid work you have done is this field, and the arithmetic in this week is how you would argue for or against returning to it

Sources

- Madigan et al. — Brock Biology of Microorganisms, the phototrophy and microbial diversity chapters
- PubMed search: cyanobacteria as photosynthetic cell factories, reviews
- PubMed search: microalgae harvesting and dewatering energy cost
- PubMed search: areal productivity and light attenuation in photobioreactors
- PubMed search: genetic tools and CRISPR in Chlamydomonas and Synechocystis

Week 61 — Hosts panorama, cell-free systems, 2025–2026 trends

Sessions 301–305

By the end of the week you can

- Compare the major hosts on tool maturity, substrate range, product fit and scale-up record
- Explain what cell-free systems buy you
- Separate demonstrated results from projections in trend reviews

The five sessions

Day 1 · Build the host table, part one · 3 h

☐ Start the one-page host comparison table: E. coli, S. cerevisiae, C. glutamicum, B. subtilis

☐ Fill columns for tool maturity, GRAS status, substrate range, product fit, scale-up record

☐ Add a column for product localisation: does the host secrete, retain or store what you want?

☐ Cite a source for every non-obvious cell

Day 2 · Build the host table, part two · 3 h

☐ Add cyanobacteria, Y. lipolytica, K. phaffii and CHO

☐ Write one line per host on what it is uniquely good at

☐ If a phototroph is on your list: write the Beer–Lambert attenuation $I(z) = I_0 \cdot \exp(-k \cdot X \cdot z)$, compute the depth at which light falls to 10% of the surface value at two biomass densities, and say why self-shading is the photobioreactor's version of an oxygen gradient

☐ If a phototroph is on your list: add a photon exchange reaction to a cyanobacterial model, constrain it with the photon flux your attenuation calculation justifies, and note what RuBisCO kinetics and the carbon-concentrating mechanism cost in energy

☐ Finish the table and add it to the portfolio folder

Day 3 · Cell-free systems · 2.5 h

☐ Read a cell-free metabolic engineering review

☐ Write what removing viability buys you and what it costs

☐ Note one product for which cell-free is genuinely the right answer

Day 4 · Current trends, read sceptically · 2.5 h

☐ Read a 2025 or 2026 review on self-driving biofoundries or systems metabolic engineering

☐ Mark every claim as demonstrated or projected

☐ Write two sentences on which trend you would want to work inside

Day 5 · Phase 4 checkpoint · 2 h

☐ Reconstruct artemisinin, 1,4-BDO and LanzaTech from memory in the seven-field format

☐ Defend your capstone host choice against one named alternative

☐ Daily quiz

Topics this week covers

- E. coli, S. cerevisiae, C. glutamicum, B. subtilis, cyanobacteria, Y. lipolytica, K. phaffii, CHO
- Phototrophs, if you are targeting them: light as the limiting substrate, Beer–Lambert self-shading in dense culture, photon exchange reactions in a model, RuBisCO kinetics and the cost of carbon-concentrating mechanisms
- Cell-free protein synthesis and cell-free metabolic engineering
- Self-driving biofoundries; enzyme-constrained models; C1 valorisation; non-conventional yeast toolkits
- Reading the future tense in review articles

Sources

- A 2025–2026 review on AI-driven biofoundries or systems metabolic engineering
- A cell-free metabolic engineering review

Phase checkpoint — worked in day 5

Answer each from memory, on paper, before reading the model answer.

☐ Reconstruct the DBTL story of artemisinin, 1,4-BDO and LanzaTech from memory

> **Model answer.** Artemisinin: S. cerevisiae, mevalonate pathway plus amorphadiene synthase and a P450, 25 g/L artemisinic acid, chemical conversion to the drug, commercially undercut by agricultural supply. 1,4-BDO: E. coli, a non-natural pathway found computationally with OptKnock growth coupling and an engineered anaerobic oxidative TCA branch, 18 g/L in the paper and above 100 g/L commercially. LanzaTech: Clostridium autoethanogenum on CO, CO_2 and H_2 from steel-mill off-gas, commercial since

2018, acetogen energetics make ATP the binding constraint.

☐ Argue host choice for a given target and defend it against one alternative

Model answer. A good answer names the property that decides it — P450 support, native lipid flux, secretion, gas uptake, GRAS status — and names the alternative you rejected and what you gave up. An answer that says a host is standard or widely used is a default, not a choice.

☐ Mark every forward-looking claim in a 2025 review as demonstrated or projected

Model answer. Check your marking against the tense: demonstrated claims report a result with a number; projected claims use may, could, promises to, or has the potential to. Reviews in this field carry a great deal of the second kind, and quoting one as though it were the first is the fastest way to lose credibility in an interview.

Week 62 — How publishing works: authorship, peer review and the record [extension]

Sessions 306–310

By the end of the week you can

- Explain how a manuscript becomes a paper, and what each stage decides
- Judge a paper as a reviewer would, and write a review that would be useful
- Know the authorship and integrity norms before you are in a position where they matter

The five sessions

Day 1 · From manuscript to paper · 2.5 h

☐ Write the stages a manuscript passes through and who decides what at each one

☐ Write what a desk rejection is, and the two or three reasons it usually happens

☐ Look up how long one journal in your field reports taking from submission to first decision, and note it

☐ Write what a preprint changes and what it does not

Day 2 · Reviewing a paper properly · 3 h

☐ Write a full referee report on one paper from your log: summary, the claim, whether the data support it, the controls, reproducibility, limitations, and a recommendation

☐ Separate your major points from your minor ones, and check that every major point names what would fix it

☐ Write the two questions you could not answer from the paper as written

☐ Daily quiz

Day 3 · Responding to reviewers · 2.5 h

☐ Invent three criticisms a reviewer would make of your capstone proposal, including one you think is wrong

☐ Write the response to each: what changed and where, or why you disagree and what evidence supports you

☐ Write the covering letter that goes with a revision

☐ Write what you would do if a reviewer asked for an experiment you could not afford

Day 4 · Authorship and integrity · 2.5 h

☐ Write the conditions for authorship under a standard such as the ICMJE criteria, and what first and corresponding authorship signal

☐ Work through three cases: a technician who ran every assay, a collaborator who supplied a strain, a supervisor who did not read the draft. Decide each and say why

☐ Write what image manipulation covers, where the line is between legitimate adjustment and misconduct, and what a journal expects

☐ Write why negative results matter and what happens to a field that does not publish them

Day 5 · Reading a lab through its papers · 2.5 h

☐ For each lab on your shortlist, write how many papers it published in the last three years, in what venues, and how often a student is first author

☐ Write what the pattern suggests about how students are trained there and what would be expected of you

☐ Write the question about publishing you would ask a current student, and the one you would ask the supervisor

☐ Daily quiz

Topics this week covers

- Why this matters now: a PhD is judged largely by papers, and knowing how they are made lets you ask a prospective supervisor better questions
- The route: submission, editorial desk decision, reviewer selection, reports, the editor’s decision, revision, and acceptance or rejection, with the time each stage takes
- What reviewers actually ask: is the claim supported by the data, are the controls there, is the method reproducible, is the novelty real, and are the limitations stated
- Writing a response to reviewers: answer each point, say what changed and where, disagree politely with evidence, and never ignore a comment
- Authorship: who qualifies, what first and corresponding authorship mean, how order is decided in practice, and the common authorship criteria such as the ICMJE conditions
- The parts of a paper and what each is for, including figure legends that stand alone and a methods section someone could follow
- Journals and venues: scope, the difference between a research paper, a letter and a review, preprints, and why conference papers and posters work differently
- Research integrity: image manipulation, selective reporting, p-hacking, salami slicing, and the duty to publish negative and null results
- Reproducibility in practice: data and code availability, a repository that runs from a fresh clone (week 37), and reporting standards for your field
- Reading a lab through its output: how many papers, where, who is first author, how often students lead, and what that predicts about your own years there

Sources

- ICMJE recommendations on authorship and contributorship
- COPE, Committee on Publication Ethics: guidance and flowcharts
- PubMed search: peer review process, reviewer guidance and common criticisms
- PubMed search: research integrity, image manipulation and selective reporting

Week 63 — Writing and talking about your work

Sessions 311–315

By the end of the week you can

- Structure a paper or proposal so a reader finds the argument without effort
- Give a ten-minute talk that survives questions
- Handle the question you cannot answer

The five sessions

Day 1 · The shape of an argument · 2.5 h

☐ Read the ten rules for structuring papers; note the context-content-conclusion pattern and the one-idea-per-paragraph rule

☐ Take your capstone and write its single central claim in one sentence, then list the evidence under it

☐ Mark every place where you asserted something the evidence does not reach — there will be several, and finding them yourself is the skill

Day 2 · Figures that argue · 3 h

☐ Read a guide to figure design, then audit the figures you have produced across the curriculum

☐ Rebuild the one figure that carries your capstone argument: one message, readable axes with units, defined error bars, no decoration

☐ Write its caption so it stands alone, including n and what the error bars are

Day 3 · Abstract and title · 2.5 h

☐ Write your capstone abstract in the standard four moves: problem, approach, result, consequence

☐ Cut it by a third without losing the claim, then have someone unfamiliar read it and tell you what you are claiming

☐ Write three candidate titles: one descriptive, one claim-making, one too clever, and say which you would submit and why

Day 4 · The ten-minute talk · 3 h

☐ Build a ten-minute talk from your proposal: one idea, minimal context, the analysis, and the limitation you raise before anyone else does

☐ Record it, watch it back, and cut everything you skipped past or apologised for

☐ Rebuild the slides so each one has a single readable message rather than a dense figure

Day 5 · Questions, and not knowing · 2 h

☐ Write the five hardest questions about your proposal and answer them aloud

☐ Practise the honest form of the answer you do not have: what you do not know, why it matters, and how you would find out

☐ Read one paper as a reviewer and write the three questions you would send back — then check whether your own proposal survives them

☐ Daily quiz

Topics this week covers

- The shape of a results argument: claim, evidence, alternative explanation, and what it does not show
- Figures that carry the argument: one message per figure, axes a stranger can read, error bars defined in the caption
- Writing the methods so someone else could repeat it, which is also how you catch your own gaps
- Abstract and title as the only parts most people read
- The ten-minute talk: one idea, three slides of context, the data, the limitation you raise yourself
- Answering questions: saying what you do not know, and why that earns more credit than improvising
- Reading a paper as a reviewer rather than a consumer
- Where this lands in an application: the proposal, the interview, and the lab meeting you will give in month two

Sources

- Mensh & Kording, Ten simple rules for structuring papers, PLoS Comput. Biol. (2017)
- A guide to presenting scientific data and figure design
- Your own paper log and capstone proposal, as the raw material

Week 64 — Planning a multi-year project: time, money and what you cannot control [extension]

Sessions 316–320

By the end of the week you can

- Build a project schedule that accounts for waiting, parallel work and the critical path
- Estimate honestly how likely a plan is to be met, and what shortens it
- Turn your industry planning experience into a visible advantage in a proposal and an interview

The five sessions

Day 1 · From an aim to a task list · 2.5 h

☐ Break your capstone’s first year into tasks, each with a duration, its predecessors, and a deliverable

☐ Mark every task that is waiting rather than working, and put a real lead time on each

☐ Write which tasks could run in parallel, and what would have to be true for that to work

☐ Daily quiz

Day 2 · The critical path, and where the time actually goes · 3 h

☐ Compute the critical path, the total duration, and the float on the parallel branch — problem V1

☐ Compute how often each task lies on the critical path once durations are uncertain, and say where you would spend attention

☐ Work out what halving a waiting time is worth, and compare that with working faster

☐ Write the same analysis for your own task list

Day 3 · Why plans slip, and what to do about it · 2.5 h

☐ Run the schedule simulation and read the median, the 80th and the 95th percentile — problem V1

☐ Write the date you would put in a proposal and the date you would privately expect, and why they differ

☐ Write where you would place a buffer, and why padding every task is worse

☐ Write what you would tell a supervisor three months into a six-month slip

Day 4 · Money, and what a student actually controls · 2.5 h

☐ Price the consumables for one month of your planned work: synthesis, sequencing, enzymes, media, analytical time

☐ Write which costs are large, which are hidden, and which are shared facility charges

☐ Write what a funding programme expects a project to deliver, using the acknowledgements of one target lab’s recent papers to see which programme funds it

☐ Write one sentence connecting your capstone to what that programme is for

Day 5 · Risk, and deciding in advance when to stop · 2.5 h

☐ Write the risk table for your capstone: what could stop it, how likely, what it would cost, and the alternative

☐ Write three go/no-go points with the result that would trigger each

☐ Write the sentence about risk you would include in a proposal, and why naming a risk strengthens rather than weakens it

☐ Daily quiz

Topics this week covers

- Why this matters: a proposal with a realistic timeline and named risks reads differently from one with only science, and planning is experience you already have from engineering work
- Work breakdown: turning a research aim into tasks with durations, dependencies and a deliverable each

- The critical path: the chain that sets the end date, and the float that other tasks have. Shortening anything off the critical path changes nothing
- Waiting is not working: DNA synthesis, primer delivery, strain shipping, equipment queues, ethics or biosafety approval (week 28) and reagent lead times are a large share of real timelines
- Parallelism: what can be built while something else is being waited for, and the second near-critical branch that quietly becomes the problem
- Uncertainty: tasks slip long more often than they slip short, so a plan built on most-likely durations is usually missed. A simple simulation makes that visible
- Buffers: where to place them, and why one buffer at the end beats padding every task
- Money: what consumables, synthesis, sequencing and analytical time actually cost, and the difference between a grant budget and what a student controls
- Risk: naming what could stop the project, how likely each is, what it would cost, and what you would do instead. A proposal that names its risks is stronger, not weaker
- Milestones and go/no-go points: deciding in advance what result would make you stop or change direction, which is also how you avoid sinking two years into one idea

Sources

- PubMed search: research project management and timelines in the laboratory
- PubMed search: Monte Carlo schedule risk analysis
- KRIBB — jobs page, for intern and trainee postings and their timing
- NumPy documentation: random number generation

Week 65 — Capstone build — the mini research proposal

Sessions 321–325

By the end of the week you can

- Turn a target molecule and host choice into a defensible three-to-five page proposal
- Back the design with your own modelling
- Make the techno-economic argument your competitors cannot

The five sessions

Day 1 · Target and motivation · 2.5 h

☐ Write proposal section 1: why this molecule, with market size and the incumbent route

☐ State the sustainability argument honestly, including where it is weak

☐ Keep it to one page

Day 2 · Host and pathway · 3 h

☐ Write section 2 using your own host table, naming the property that decides it and the alternative you rejected

☐ Write section 3: the pathway from a central metabolite, every enzyme sourced and cited

☐ Add the localisation subsection to section 2: secreted, retained or stored, and what that forces on the host

☐ Mark the steps with no good candidate enzyme

Day 3 · The numbers · 3 h

☐ Write section 4: theoretical maximum mass and molar yield on glucose, worked by hand with the redox and ATP balances closed explicitly

☐ Add your heterologous reactions to a real genome-scale model in COBRApy — metabolites, stoichiometry, bounds, objective — and confirm the model reproduces your hand yield

☐ Constrain oxygen to a value your week 44 kLa calculation justifies, rerun, and report the yield you would actually get in a vessel

☐ Push the notebook to GitHub, commented so a PI can read it without running it, and put its headline figure in the proposal

Day 4 · Build and test plan · 2.5 h

☐ Write section 5: assembly method, editing tool, analytics, screening readout, DBTL iterations

☐ State the transport strategy: which efflux pump you would mine, express or evolve if secreted; which lysis and extraction train if retained

☐ Justify each choice in one clause — no tool without a reason

☐ State what result would make you abandon the design

Day 5 · Scale-up and risks · 2.5 h

☐ Write section 6: fermentation mode, scale-up risks, downstream train, back-of-envelope economics

☐ Write section 7: risks and alternatives, named specifically

☐ Read the whole draft once, aloud, and mark the weakest section

Topics this week covers

- Motivation and target; market and sustainability framing
- Host selection against your own comparison table
- Pathway design via KEGG, MetaCyc, BRENDA and retrosynthesis reasoning
- Stoichiometry, theoretical yield, FBA or OptKnock analysis, cofactor balance
- Build-and-test plan: assembly, editing, analytics, screening, DBTL iterations
- Scale-up, downstream, TEA and LCA sketch; risks and alternatives

Sources

- Your own paper log entries
- Two recent papers from each of your three target labs

Week 66 — Applications — labs, statement of purpose, interviews

Sessions 326–330

By the end of the week you can

- Finish a shortlist of labs with specific reasons and specific papers
- Write a statement of purpose that bridges chemical engineering to metabolic engineering
- Be able to discuss any paper you logged for ten minutes

The five sessions

Day 1 · Find the labs · 3 h

☐ Read the Metabolic Engineering editorial board and the last year of titles in two journals

☐ Shortlist eight to ten groups and note, for each, one recent paper and why it interests you

☐ Cut the list to three or four you would actually join

Day 2 · Read their work properly · 3 h

☐ For each shortlisted lab, read one recent paper closely and log it

☐ Write how your proposal touches each lab's current direction

☐ Note the one key figure of each paper

Day 3 · Write the statement of purpose · 3 h

☐ Draft all five parts: problem, why this field now, evidence of preparation, lab fit, goals

☐ Name your artifacts explicitly — the notebooks, the reproduced design, the proposal

☐ Cut every sentence that would survive in someone else's application

Day 4 · Polish and send · 2.5 h

☐ Revise the proposal's weakest section and tighten the whole to three to five pages

☐ Send it to one person for comment

☐ Write two short, specific emails to PIs — a real question, not an introduction

Day 5 · Final checkpoint · 2.5 h

☐ Answer aloud, in sixty seconds: why metabolic engineering, and why you?

☐ Check the GitHub repo reads clearly to a stranger in five minutes

☐ Reread your week 1 notebook page, then write what you will do in the next six months

Topics this week covers

- Reading editorial boards and conference programmes to find groups
- Framing: stoichiometric balances, kinetics, reactor design, transport, separations, TEA
- Evidence over enthusiasm — point at artifacts
- Interview drill: what would you do differently?

Sources

- Metabolic Engineering editorial board
- ACS Synthetic Biology, recent issues

Phase checkpoint — worked in day 5

Answer each from memory, on paper, before reading the model answer.

☐ Can you say in sixty seconds why metabolic engineering, and why you?

> **Model answer.** Your own answer, but it should contain a concrete problem, the reason metabolic engineering is the way to solve it, and at least one artifact by name. If it would survive unchanged in someone else's application, it is not yet specific enough.

☐ Can you name three PIs, their recent papers and how your proposal touches each?

> **Model answer.** If you cannot name each lab's recent paper and its key figure, that lab is not on your shortlist yet — it is on a list of names.

☐ Is your GitHub readable by a stranger in five minutes?

> **Model answer.** Open the repository as a stranger would: README first. If it does not say what the project is, what to run and what the result was, fix that before anything else.

☐ Does your statement of purpose cite an artifact rather than an ambition?

> **Model answer.** The test is whether the document points at something that exists — a notebook, a reproduced design, a proposal — rather than at an ambition.

The question bank

323 questions · answers included

Multiple choice answers are marked. Recall questions carry a model answer — answer out loud or in writing first, then grade yourself honestly. “I knew roughly that” counts as missed.

Week 1 — The cell, the central dogma, and a map of the territory

Which structure does E. coli lack that S. cerevisiae has?

- Ribosomes
- ✔ A membrane-bound nucleus
- A plasma membrane
- DNA

Why. Bacteria keep their chromosome in a nucleoid with no surrounding membrane. Yeast also has mitochondria, ER and peroxisomes — compartments you can later exploit or must work around.

State the central dogma and say where each step happens in a bacterium versus a yeast.

Model answer. DNA → RNA by transcription → protein by translation. In bacteria both happen in the cytoplasm and are coupled, so a ribosome can translate a transcript still being made. In yeast transcription and splicing happen in the nucleus, the mRNA is exported, and translation happens in the cytosol or on the ER.

Why is S. cerevisiae often chosen over E. coli for plant natural products?

- It grows faster
- ✔ It has endomembranes that support functional cytochrome P450 expression
- It needs no selection marker
- It has a larger genome

Why. Plant pathways lean on membrane-anchored P450s that need an ER-like membrane and a redox partner. Yeast also tolerates eukaryotic post-translational modification and carries GRAS status.

Name three compartments in yeast where a heterologous pathway could be placed, with one reason each.

Model answer. Cytosol — default, largest precursor pools, simplest expression. Mitochondrion — colocalises with 2-ketoacid and acetyl-CoA pools, used for isobutanol. Peroxisome — isolates toxic or promiscuous intermediates and offers β-oxidation-derived acetyl-CoA.

Which of these is the usual reason E. coli is picked as a chassis?

- It makes every plant product
- ✔ Fast growth, cheap defined medium and the deepest genetic toolkit
- It needs no oxygen
- It secretes everything

Why. Doubling in about twenty minutes on salts and glucose, with decades of tools and strains behind it. What it is bad at — P450s, secretion, eukaryotic modifications — is why yeast exists in this field.

Week 2 — Protein structure and enzyme function

The first digit of an EC number tells you:

- The organism
- ✔ The reaction class, such as oxidoreductase or transferase
- The cofactor
- The molecular weight

Why. EC 1 oxidoreductases, 2 transferases, 3 hydrolases, 4 lyases, 5 isomerases, 6 ligases, 7 translocases. The remaining digits narrow substrate and mechanism.

Name the four levels of protein structure and what determines each.

Model answer. Primary — the amino-acid sequence, set by the gene. Secondary — helices and sheets from backbone hydrogen bonding. Tertiary — the folded three-dimensional shape from side-chain interactions. Quaternary — the assembly of multiple subunits.

Why does expressing a plant cytochrome P450 in a microbe require more than its gene alone?

Model answer. It needs heme, a membrane to anchor in, a compatible redox partner such as a cytochrome P450 reductase, and an NADPH supply. Missing any of these gives expression without activity — a classic failure mode in natural-product pathways.

An apoenzyme is:

An enzyme with its cofactor bound

✔ The protein part without its cofactor

A denatured enzyme

An enzyme precursor

Why. Apoenzyme plus cofactor gives the active holoenzyme. Expressing the apoprotein without supplying or making the cofactor is one of the commonest reasons a cloned pathway shows no activity.

Week 3 — Enzyme kinetics — your kinetics, re-pointed

k_cat/K_M is best described as:

The maximum reaction rate

✔ The catalytic efficiency, or specificity constant

The substrate concentration at half V_max

The inhibitor binding constant

Why. It is the apparent second-order rate constant for the enzyme acting on free substrate at low [S]. Its ceiling is the diffusion limit, roughly 10^8–10^9 $M^{-1}s^{-1}$.

A competitive inhibitor changes the kinetics how?

✔ Apparent K_M rises, V_max unchanged

V_max falls, K_M unchanged

Both fall

Neither changes

Why. It competes for the free enzyme, so more substrate overcomes it — V_max is reachable, but it takes more substrate to get halfway there.

Why is feedback inhibition the biggest obstacle to overproducing an amino acid, and how do engineers get around it?

Model answer. The end product allosterically inhibits the first committed enzyme, so the cell throttles the pathway exactly when you want flux. The standard fix is a feedback-resistant allele of that enzyme — for example mutant lysC aspartokinase in C. glutamicum — together with removing transcriptional attenuation and competing branches.

State the steady-state assumption behind Michaelis–Menten and say what K_M equals.

Model answer. The enzyme-substrate complex is formed and broken down at equal rates, so d[ES]/dt ≈ 0 — the same quasi-steady-state move used in reaction engineering. $K_M = (k_{-1} + k_{cat})/k_1$, and equals the substrate concentration at half-maximal rate. It is only a binding constant when k_cat is small compared with k_{-1}.

A Hill coefficient above one indicates:

Competitive inhibition

✔ Positive cooperativity between subunits

A dead enzyme

Diffusion limitation

Why. Cooperative enzymes give sigmoidal kinetics, which makes them switch-like — useful for the cell's own control and awkward when you want smooth, tunable flux.

Week 4 — Bioenergetics and cofactors

NADPH is used mainly for:

ATP generation by the electron transport chain

✔ Reductive biosynthesis

Proton pumping

DNA replication

Why. Cells keep NADPH highly reduced for anabolism and NAD^+ oxidised for catabolism, even though the two differ by a single phosphate. Designs that confuse the two silently fail.

On glucose, the main NADPH source in E. coli is:

Glycolysis

✔ The oxidative pentose phosphate pathway

Succinate dehydrogenase

ATP synthase

Why. Glucose-6-phosphate dehydrogenase and 6-phosphogluconate dehydrogenase supply most of it, with contributions from NADP-dependent isocitrate dehydrogenase and the membrane transhydrogenase PntAB.

Write the ATP and NADH yield of glucose to two pyruvate, and say what redox balance means for an anaerobic fermentation.

Model answer. Net 2 ATP and 2 NADH per glucose. Anaerobically there is no electron transport chain to reoxidise NADH, so the pathway must consume exactly as much NADH as it makes — which is why the product spectrum of a fermentation is fixed by electron bookkeeping, not by preference.

The ΔG°′ of a reaction tells you:

How fast it goes

✔ Whether it is favourable under standard conditions

The enzyme's K_M

The ATP yield

Why. Thermodynamics sets direction and the achievable driving force; kinetics and enzyme levels set rate. A pathway can be thermodynamically fine and still carry no flux, and the reverse is impossible.

Week 5 — Central carbon metabolism I — glycolysis and the pentose phosphate pathway

Which pathway supplies both NADPH and ribose-5-phosphate?

The TCA cycle

✔ The pentose phosphate pathway

β-oxidation

The Entner–Doudoroff pathway

Why. The oxidative branch makes NADPH and CO_2; the non-oxidative branch interconverts five-, six- and seven-carbon sugars, giving ribose-5-phosphate for nucleotides and erythrose-4-phosphate for aromatics.

Aromatic amino acids are built from PEP plus:

Oxaloacetate

✔ Erythrose-4-phosphate

Acetyl-CoA

α-Ketoglutarate

Why. DAHP synthase condenses PEP with erythrose-4-phosphate — which is why aromatic overproduction strains fight over PEP with the phosphotransferase sugar uptake system.

Name the terpenoid precursor route in yeast and in E. coli, and the central metabolite each starts from.

Model answer. Yeast uses the mevalonate pathway from acetyl-CoA, consuming NADPH at HMG-CoA reductase. E. coli natively uses the MEP, or non-mevalonate, pathway from pyruvate and glyceraldehyde-3-phosphate. Both converge on IPP and DMAPP.

Phosphofructokinase matters in pathway engineering because:

It makes NADPH

✔ It is a committed, allosterically regulated step that fixes glycolytic flux

It transports glucose

It consumes oxygen

Why. Committed and regulated steps are both the obstacles and the handles. Knowing which reactions in a map are regulated tells you where the cell will fight you.

Week 6 — Central carbon metabolism II — TCA cycle and oxidative phosphorylation

In E. coli, which reaction is anaplerotic?

Pyruvate kinase

✔ PEP carboxylase making oxaloacetate

Citrate synthase

Fumarase

Why. Anaplerosis refills TCA intermediates drained for biosynthesis. Remove it and the cycle starves even with plenty of glucose.

The P/O ratio describes:

✔ ATP made per oxygen atom reduced

Protons per oxygen molecule

Phosphate per organic acid

Pyruvate per oxaloacetate

Why. It converts reducing equivalents into ATP in your energy balance. In a genome-scale model it is one of the parameters that most moves predicted maximum yield.

Why would an anaerobic process still want flux through the oxidative TCA branch?

Model answer. To generate reducing equivalents and specific intermediates — succinyl-CoA and 4-hydroxybutyrate precursors, for instance — that a purely fermentative route cannot supply. Genomatica engineered exactly this for 1,4-butanediol, forcing an oxidative branch to run without oxygen.

Switching E. coli from aerobic to anaerobic growth mainly changes:

Nothing important

✔ ATP yield per glucose, and the product spectrum needed to reoxidise NADH

Only the growth temperature

The genome sequence

Why. Without a terminal electron acceptor, ATP comes from substrate-level phosphorylation only, and every NADH must be disposed of into a reduced product — acetate, lactate, ethanol, succinate, formate.

Week 7 — Biosynthetic metabolism — amino acids, lipids, nucleotides

Malonyl-CoA is formed by:

✔ Acetyl-CoA carboxylase, using ATP, biotin and bicarbonate

Fatty-acid synthase

Citrate lyase

Pyruvate dehydrogenase

Why. It is the committed, ATP-consuming step into fatty-acid synthesis, and the usual first target when engineering any polyketide or fatty-acid-derived product.

Account for the cofactor cost of making one palmitate from acetyl-CoA.

Model answer. Seven malonyl-CoA are made from seven acetyl-CoA at the cost of seven ATP, then eight acetyl units are condensed and reduced using 14 NADPH. That NADPH demand is why oleochemical hosts are chosen and engineered for pentose phosphate flux.

Draw the mevalonate pathway from acetyl-CoA to IPP and mark the committed step.

Model answer. Two acetyl-CoA condense to acetoacetyl-CoA, a third gives HMG-CoA, HMG-CoA reductase reduces it to mevalonate using two NADPH — the committed and most-engineered step — then two phosphorylations and a decarboxylation give IPP, which isomerises to DMAPP.

Acetyl-CoA is central to engineered pathways because:

✔ It is the precursor to fatty acids, polyketides, isoprenoids via mevalonate and many others

It carries nitrogen

It replaces ATP

It only appears anaerobically

Why. It sits at the junction of catabolism and biosynthesis, which is why so much of the field is about getting more carbon to acetyl-CoA and keeping it out of the TCA cycle.

Week 8 — Molecular biology and gene regulation

Ribosome binding site strength sets:

Transcription initiation rate

✔ Translation initiation rate

Plasmid copy number

mRNA degradation rate

Why. Promoter tunes transcript abundance, RBS tunes protein per transcript. Tuning both is how pathway flux gets balanced without changing any enzyme.

IPTG induces the lac system by:

Binding the promoter directly

✔ Binding the LacI repressor so it releases the operator

Degrading LacI

Raising plasmid copy number

Why. It is a non-hydrolysable allolactose mimic. Its cost and the burden of a sudden induction are real reasons large-scale processes often prefer constitutive or self-inducing systems.

Name the parts of a bacterial expression cassette in order, and two reasons a plant might prefer constitutive to inducible expression.

Model answer. Promoter, ribosome binding site, open reading frame, terminator — on a backbone carrying an origin of replication and a selection marker. At scale, inducer cost and the difficulty of adding it uniformly to a large vessel argue for constitutive or auto-inducing systems, as does avoiding a sudden metabolic burden spike.

Codon optimisation matters because:

It changes the protein sequence

✔ Codon usage and tRNA abundance differ between organisms and affect translation

It raises plasmid copy number

It removes introns

Why. A heterologous gene can be transcribed well and still translate poorly in a new host. Note that it changes the DNA, not the protein — and that aggressive optimisation can disrupt folding or regulatory elements.

Week 9 — Regulation: global networks, catabolite repression, proteome allocation

Carbon catabolite repression in E. coli works largely through:

Direct binding of glucose to promoters

✔ Glucose lowering cAMP, so CRP cannot activate catabolic promoters

Degradation of all non-glucose enzymes

A change in DNA supercoiling only

Why. Glucose uptake through the phosphotransferase system keeps adenylate cyclase inactive, cAMP stays low, and CRP sits off its sites. That is diauxie — and the reason a strain fed mixed sugars from biomass hydrolysate eats glucose first and leaves xylose until later.

ppGpp accumulates when the cell:

Has excess carbon

✔ Runs short of an amino acid, stalling ribosomes

Enters stationary phase only

Is grown aerobically

Why. Uncharged tRNA in the ribosomal A site triggers ppGpp synthesis, which shuts down ribosome and rRNA synthesis and redirects the cell to survival. A pathway that drains an amino-acid precursor can set this off without anyone intending it.

State the central result of the bacterial growth laws and what it means for an engineered pathway.

Model answer. Protein mass is a fixed budget: the ribosomal fraction of the proteome rises roughly linearly with growth rate, so everything else must shrink. A heterologous pathway expressed strongly takes its share from that budget, which lowers growth rate and, through it, volumetric productivity. This is why burden is quantitative rather than vague, and why maximum expression is rarely the optimum.

A stronger promoter on a pathway gene can lower titer because:

The enzyme becomes less specific

✔ Expression burden slows growth and can cause misfolding or precursor drain, outweighing the extra activity

Promoters decay

Transcription stops translation

Why. The flux gain is sublinear once the enzyme is no longer limiting, while the proteome and energy cost keeps rising. Tuning with an RBS or promoter library finds the turning point instead of assuming more is better.

Name three global regulators and say which engineering move each would oppose.

Model answer. CRP–cAMP: using a non-glucose or mixed feedstock, since catabolite repression delays the second sugar. ArcA/FNR: switching between aerobic and anaerobic operation, since they rewire TCA and respiratory gene expression underneath your design. ppGpp and the stringent response: draining an amino-acid precursor, which triggers a global slowdown. Others worth naming are sigma-factor competition under stress and the specific repressors of whatever branch you delete.

Week 10 — Microbial genetics and microbiology

An auxotroph is a strain that:

Grows faster than wild type

✔ Cannot synthesise a compound it needs and must be fed it

Resists an antibiotic

Carries no plasmid

Why. Auxotrophies are workhorse selection markers in yeast and are also used industrially to force a strain to behave, replacing antibiotics.

Define the specific growth rate and relate it to doubling time. Why integrate a pathway into the genome rather than keep it on a plasmid?

Model answer. $\mu = (1/X)(dX/dt)$, and doubling time $t_d = \ln 2/\mu$. Genomic integration gives uniform gene dosage and stability without antibiotic selection, which regulators and process economics both prefer; the cost is lower copy number, usually compensated with stronger promoters or multiple integrations.

Antibiotic selection is avoided at industrial scale mainly because:

It is illegal everywhere

✔ Of cost, regulatory and waste-stream problems, and because plasmid loss still happens

Antibiotics kill the product

It slows mixing

Why. Hence genomic integration, auxotrophic complementation, and growth-coupled designs where keeping the pathway is the only way to grow.

What does it mean to call a strain a chassis, and what makes one industrially acceptable?

Model answer. A chassis is a host whose genetics, physiology and process behaviour are well enough characterised to build on predictably. Industrially acceptable adds: genetic stability over many generations, robustness in non-sterile or large-scale conditions, GRAS or regulatory precedent, growth on cheap feedstock, phage resistance, and a demonstrated scale-up record.

Week 11 — Definition, history and the design–build–test–learn cycle

Bailey's 1991 definition of metabolic engineering centres on:

Breeding better strains by mutagenesis

✔ Improving cellular activities by manipulating enzymatic, transport and regulatory functions with recombinant DNA

Optimising fermenter design

Sequencing genomes

Why. The directed, recombinant-DNA-enabled part is what separated it from classical strain improvement — and the paper correctly predicted that regulation, not gene availability, would be the hard part.

“Network rigidity” names the observation that:

Enzymes are physically rigid

✔ Metabolic networks resist flux redistribution because of regulation and node architecture

Plasmids are unstable

Genomes resist editing

Why. Stephanopoulos and Vallino showed that pushing on one branch is often absorbed by the network. It is the conceptual ancestor of distributed control and of multiplexed editing.

Name the four stages of the DBTL cycle and the stage that usually bottlenecks.

Model answer. Design, Build, Test, Learn. Test — and the Learn that depends on it — is usually the bottleneck: building variants is now cheap and fast, measuring titers and interpreting them is not. This is the whole argument for biosensors, high-throughput screening, automation and machine learning.

Week 12 — Python for modelling I: numpy, scipy, and a notebook a stranger can run

To solve A·x = b you should use:

np.linalg.inv(A) @ b

✔ np.linalg.solve(A, b)

A**-1 * b

np.transpose(A) @ b

Why. solve factorises rather than inverting, so it is faster and far better conditioned. Explicitly inverting a matrix is one of the clearest signs in a notebook that the author learned linear algebra from a textbook and not from a solver.

In scipy's solve_ivp, the function you supply must have the signature:

f(y, t)

✔ f(t, y)

f(y)

f(t, y, params) only

Why. Time first. The opposite order was the old odeint convention, and mixing them up gives a model that integrates happily and is completely wrong — a silent failure worth knowing about before it costs you a day.

Where do the uncertainties on fitted parameters come from in curve_fit, and what do they assume?

Model answer. From the covariance matrix it returns: the standard errors are the square roots of its diagonal. They assume the model is correct, the residuals are independent with constant variance, and the fit sits at a genuine minimum where the quadratic approximation holds. If you fitted with absolute_sigma left at False, the errors are also scaled by the residual variance, so they describe relative precision rather than absolute. None of that is visible in the numbers, which is why you state the assumption when you report them.

A stiff system of metabolic ODEs is best integrated with:

RK45, an explicit method

✔ LSODA or BDF, which handle stiffness

Euler's method with small steps

Any of them equally

Why. Explicit solvers take step sizes set by the fastest timescale, so a metabolic model with millisecond pools and hour-scale growth crawls or fails. LSODA switches automatically, which is why it is the sensible default here.

What makes a notebook reproducible, concretely?

Model answer. It runs top to bottom from a fresh kernel without errors; it pins the versions it needs in a requirements file; it seeds any random number generator; it reads data from a path in the repository rather than from your desktop; and it has a README saying what it does, how to run it and what the result was. A reviewer who cannot get a figure out of your repository in five minutes will assume the figure is the only thing that ever worked.

Week 13 — Stoichiometric modelling and network structure

In S·v = 0, the rows of S correspond to:

Reactions

✔ Metabolites

Genes

Measurements

Why. One row per metabolite — one mass balance each — and one column per reaction. Writing that matrix is the same exercise as writing balances around units in a flowsheet.

The pseudo-steady-state assumption on intracellular metabolites is justified because:

Cells do not change

✔ Metabolite pools turn over far faster than biomass accumulates

Enzymes are saturated

Fluxes are measured directly

Why. Pool turnover is seconds; growth is hours. The separation of timescales is what lets you drop the accumulation term and keep a linear system.

What does the null space of S represent, and how do you handle an underdetermined network?

Model answer. The set of all flux vectors satisfying every mass balance — the space of feasible steady states. If it has more dimensions than you have measurements, you either measure more exchange fluxes, add an optimisation objective as in FBA, or resolve the interior directly with ^{13}C labelling.

Week 14 — Equilibrium, driving force, and transport across the membrane

Which quantity tells you the direction a reaction will actually run inside a cell?

$\Delta G^{\circ\prime}$, the standard free-energy change

✔ $\Delta G'$, the free-energy change at the cell's real metabolite concentrations

The equilibrium constant alone

The enzyme's k_cat

Why. $\Delta G^{\circ\prime}$ is a reference value at 1 M everything. The cell runs at micromolar to millimolar concentrations, and $\Delta G' = \Delta G^{\circ\prime} + RT \ln Q$ can easily have the opposite sign — which is how glycolysis runs forward through steps that look unfavourable on paper.

Define displacement from equilibrium and say what it means for engineering a pathway.

Model answer. Displacement is how far the mass-action ratio Q sits from the equilibrium constant K — equivalently how negative $\Delta G'$ is. A reaction close to equilibrium (Q near K, $\Delta G'$ near zero) carries flux easily in either direction but cannot hold a concentration gradient, so control cannot sit there and the step reverses if downstream backs up. A step far from equilibrium is effectively irreversible, is where regulation and control normally sit, and is the one worth targeting — but it also costs the free energy you spent to push it.

The permeability coefficient P in J = P·ΔC combines:

Only the diffusivity

✔ Partition coefficient, diffusivity in the membrane, and membrane thickness

Transporter V_max and K_M

Temperature and pressure

Why. P = K·D/L. This is why lipophilic products leak out on their own and charged or polar ones do not — the partition coefficient dominates, and no amount of overexpression changes it.

A weak organic acid accumulates inside a cell at neutral internal pH mainly because:

It is actively imported

✔ The undissociated form crosses freely and then dissociates, trapping the anion inside

The membrane is impermeable to it

It binds to protein

Why. Ion trapping. Only the neutral protonated form permeates, so a lower external pH drives it in and the higher internal pH holds it as the membrane-impermeant anion — a real problem in organic-acid production, and the reason those processes fight over pH.

When does a product need an engineered exporter rather than diffusion, and what does the export cost?

Model answer. When the molecule is charged, polar or large enough that its permeability coefficient is too low to clear the production rate, or when the required internal concentration would be toxic. Active export then costs either ATP directly, as in an ABC transporter, or a share of the proton motive force in a symport or antiport, roughly RT ln(c_out/c_in) per mole against the gradient plus the coupling inefficiency. That cost belongs in the yield calculation, not just in the biology.

The characteristic time for a molecule to diffuse a distance L scales as:

L/D

✔ L^2/D

D/L^2

$L^2 \cdot D$

Why. Across a 1 µm cell this is milliseconds, far faster than enzyme turnover, which is why the cytosol is treated as well mixed. Scale L to metres and the same expression explains why a production fermenter is not — the gradients from week 44, arrived at from first principles.

Your pathway is thermodynamically feasible end to end, but one step has a max-min driving force of only −2 kJ/mol. What does that tell you, and what would you do?

Model answer. The pathway can run, but that step sits so close to equilibrium that it needs a large enzyme concentration or a steep metabolite gradient to carry the flux, and it will reverse if the downstream pool builds. Options: raise the substrate or drain the product of that step by pulling harder downstream, swap in an enzyme with a different cofactor or a coupled ATP hydrolysis to change the step's $\Delta G'^{\circ}$, or redesign that segment of the route. Noor et al. showed this is a real constraint in central metabolism, not a theoretical one.

Week 15 — Flux control and metabolic control analysis

The summation theorem of metabolic control analysis states that flux control coefficients:

Sum to zero

✔ Sum to one

Are all equal

Cannot be measured

Why. Control is shared out, not owned. If one enzyme held a coefficient of 1 the others would hold none — which is rarely what experiments find.

An elasticity coefficient measures:

✔ How a reaction rate responds locally to a metabolite concentration

How flux responds to enzyme level

The stiffness of a protein

The cost of a knockout

Why. Elasticities are local and enzymatic; control coefficients are systemic. The connectivity theorem links the two.

Why does overexpressing the supposed rate-limiting enzyme usually fail to raise flux?

Model answer. Because control is distributed: its flux control coefficient is well below one, so a large change in that enzyme produces a small change in flux, and the new bottleneck simply moves. Useful gains normally need several coordinated changes, which is what multiplexed editing and library approaches deliver.

Week 16 — Kinetic and dynamic modelling of pathways

The dynamic form of a metabolic balance is:

$S \cdot v = 0$

✔ $dc/dt = S \cdot v(c)$

$v = S^{-1}c$

$dc/dt = 0$

Why. Steady state is the special case where the left side vanishes. Keeping the accumulation term is what lets you see transients, feedback oscillation and fed-batch behaviour — at the price of needing every rate law and parameter.

Metabolic ODE models are usually stiff because:

They have too many equations

✔ Metabolite pools turn over in milliseconds while biomass changes over hours

The rate laws are nonlinear

Solvers are imprecise

Why. Timescales separated by six orders of magnitude force an implicit solver. It is also exactly the separation that justifies the pseudo-steady-state assumption behind FBA.

What is the difference between structural and practical identifiability?

Model answer. Structural: the model's form makes two or more parameters indistinguishable no matter how perfect the data — only their ratio or product can ever be determined. Practical: the form is fine, but your data are too noisy, too sparse, or taken under conditions that do not excite that parameter. The first is fixed by reparameterising or measuring a different variable; the second by better or differently designed experiments. Reporting a fitted parameter without checking either is how kinetic models lose credibility.

You raise one enzyme's V_max tenfold in a kinetic model and flux rises by 4%. That is:

A bug in the model

✔ The numerical version of a low flux control coefficient

Proof the enzyme is irrelevant

Evidence of stiffness

Why. The control coefficient is about 0.02 over that range. Seeing it emerge from a simulation you built is more convincing than being told the summation theorem in week 15.

When does a kinetic model earn its parameters, and when should you stay with constraint-based modelling?

Model answer. A kinetic model earns them when you need dynamics or regulation that stoichiometry cannot express — a toxic intermediate accumulating, a feedback loop, a fed-batch trajectory, a dynamic-control circuit — and when you have enough measurements to constrain the parameters you add. Stay constraint-based when the question is yield, feasibility or which genes to delete at steady state, where no kinetics are required and adding unmeasured parameters only adds unfalsifiable assumptions. Many published kinetic models fail this test.

Week 17 — Experimental design, statistics, and when a difference is real

An assay has a coefficient of variation of 20% and you run three replicates per group. What power do you have to detect a 20% change?

About 80%

About 50%

✔ About 16%

About 95%

Why. About 16%, so there is an 84% chance of missing a real 20% improvement. With that noise, 80% power needs about 17 replicates per group, or a better assay. Problem U1 works the grid.

With a 20% CV and six replicates per group, the smallest difference detectable at 80% power is about:

5%

20%

✔ 36%

60%

Why. About 36%; three replicates give 61% and twenty give 18%. Knowing this before you start tells you whether the experiment can answer your question.

One factor at a time found the optimum 75% of the time in one testing order and 0% in another, on identical ground truth. What happened?

Model answer. The truth contained an interaction: factor B hurt on its own but helped strongly once factor A was raised. Testing A first raises it, so B is then tested in a high-A background and correctly kept. Testing B first examines it while A is still low, sees it make things worse, discards it and never revisits it. The answer therefore depended on an arbitrary ordering, and no one-factor-at-a-time run can report the interaction at all, since A and B are never varied together. The full factorial in problem U2 recovered every effect, including the interaction at 2.49 against a true 2.50, with a spread matching the theoretical standard error of 0.200.

A half-fraction of a four-factor design uses eight runs instead of sixteen. What is the cost?

The main effects become biased

✔ Some effects are confounded, so you must assume certain interactions are zero

It needs more replicates

The noise doubles

Why. In the half used in problem U2 the AB and CD interactions cannot be told apart, so the design assumes one is negligible. Main effects were still recovered accurately, with standard errors about 40 to 50% larger.

Three shake flasks inoculated from a single colony, measured once each, are:

Three biological replicates

✔ Three technical replicates of one biological sample, for most questions

A power analysis

Sufficient for publication

Why. They share a genotype and a culture history, so they report pipetting and handling variation, not strain-to-strain variation. Independent transformants or independent colonies are what make a biological replicate, and conflating the two is the commonest statistical error in strain papers.

An error bar showing standard error rather than standard deviation tells the reader about:

How variable the biology is

✔ How precisely you know the mean

The measurement range

Nothing useful

Why. SEM shrinks as you add replicates while the underlying variability does not, which is why SEM bars look reassuring and SD bars look honest. Whichever you use, say which it is and what n was — a bar with neither is uninterpretable.

Strain A yields 0.620 ± 0.020 g/g and strain B 0.660 ± 0.026, both n = 3. Is B better?

Model answer. Not demonstrably. A two-sample t-test gives t = −2.09, p = 0.105, and the 95% confidence interval on the difference runs from −0.013 to +0.093 g/g — it includes zero, so the data are consistent with no difference at all. The effect size is large, Cohen's d ≈ 1.7, which is the useful part: the difference may well be real, the experiment is simply too small to show it. With that effect size, n = 3 has only about 36% power; you would need n = 7 per strain for 80% and n = 10 for 95%. The correct statement is not "B is better" and not "there is no difference", but "consistent with an improvement of up to about 0.09 g/g; underpowered, repeat with n = 7".

You screen 96 wells from a three-gene library with 24 RBS variants each. The fraction of the library you have seen is about:

About 70%

✔ About 0.7%

About 25%

All of it

Why. The library has $24^3 = 13{,}824$ combinations, and $1 - \exp(-96/13824) \approx 0.7\%$. Reaching 95% coverage would need roughly 41,000 clones. This is why library design matters more than library size, and why biosensor-coupled sorting exists.

Propagate the error: product 12.4 ± 0.4 g/L, substrate consumed 20.0 ± 0.5 g/L. What is the yield?

Model answer. Y = 12.4/20.0 = 0.620 g/g. For a quotient the relative errors add in quadrature: $\sqrt{(0.4/12.4)^2 + (0.5/20.0)^2} = \sqrt{0.0323^2 + 0.0250^2} = 4.1\%$, so Y = 0.620 ± 0.025 g/g. Two things follow. The product measurement dominates, contributing 0.0323 of the 0.041, so improving the HPLC calibration helps more than measuring sugar more carefully. And reporting this as 0.62 g/g is right while 0.6201 g/g is not — the uncertainty sets the significant figures, and a yield quoted to four digits tells a reviewer you have not done this calculation.

The main cost of a fractional factorial design compared with a full one is:

Less precision on each measurement

✔ Some interactions become confounded with main effects

It needs more runs

It cannot be analysed

Why. You trade resolution for runs. That is usually a good trade in a screening stage provided you know which interactions you have given up and check them later, and a bad one if you report a main effect that is really an interaction in disguise.

When is one-factor-at-a-time defensible, and why is it usually not?

Model answer. It is defensible when the factors genuinely do not interact, when you are troubleshooting a single suspected cause, or when a run is so expensive that only a handful are possible and you care about one variable. It usually is not, because medium components and process variables interact strongly — carbon and nitrogen, pH and temperature, inducer and growth rate — so varying one at a time explores a line through a space and can miss an optimum that a factorial design finds in fewer total runs. It also gives no estimate of interaction effects at all, which are often the finding.

Week 18 — Titer, rate, yield — and the design constraints

For a bulk chemical, which metric is usually most binding?

Titer

✔ Yield on feedstock

Rate

Purity

Why. Feedstock often dominates the cost of a commodity, so carbon yield sets the floor. For a high-value pharmaceutical, titer and total output matter more than a few per cent of sugar.

The theoretical maximum mass yield of ethanol on glucose is about:

0.33 g/g
✔ 0.51 g/g
0.72 g/g
1.00 g/g

Why. Two ethanol (46 g/mol) per glucose (180 g/mol) gives 92/180 ≈ 0.51 g/g. Real processes reach roughly 90–93% of this because carbon goes to biomass and glycerol.

Why does titer matter even when yield is already high?

Model answer. Because everything downstream scales with water. A dilute broth means more volume to heat, separate and dispose of per kilogram of product, so separation energy and capital often decide viability. This is the gap between an 18 g/L publication and a plant that needs over 100 g/L.

Week 19 — Cofactor balancing, redox and toxicity

Cofactor swapping means:

Changing the carbon source
✔ Altering an enzyme's preference between NADPH and NADH
Exchanging plasmids
Switching hosts

Why. If a pathway demands NADPH the cell cannot supply, you either make more NADPH or re-engineer the enzyme to accept NADH. Both are routine; the second is often cheaper in carbon.

Name three ways to fix an NADPH deficit in a production strain.

Model answer. Redirect flux through the oxidative pentose phosphate pathway, often by deleting or weakening phosphoglucose isomerase or glycolytic entry; express or overexpress a transhydrogenase such as PntAB to convert NADH to NADPH; and swap the consuming enzyme's cofactor specificity to NADH. Replacing NAD-dependent glycolytic enzymes with NADP-dependent variants is a fourth.

Compartmentalising a pathway in the yeast mitochondrion can help because:

It raises plasmid copy number
✔ It colocalises enzymes with precursor pools and away from competing cytosolic reactions
It removes the need for a promoter
It eliminates toxicity entirely

Why. Gevo used it for isobutanol: the 2-ketoacid precursors are mitochondrial, so moving the pathway there shortens the distance from precursor to product and avoids cytosolic competition.

Week 20 — DNA assembly and cloning

Type IIS restriction enzymes are central to Golden Gate because they:

Cut within their recognition site
✔ Cut outside their recognition site, leaving custom four-base overhangs
Methylate DNA
Need no buffer

Why. Cutting outside the site removes the site from the product, so assembly is scarless, one-pot and hierarchical — the basis of MoClo and of biofoundry part libraries.

Gibson assembly requires fragments that:

✔ Share overlapping homologous ends of roughly 15–40 bp
Carry compatible restriction sites
Are the same length
Contain no repeats

Why. A 5′ exonuclease chews back, the overlaps anneal, polymerase fills and ligase seals — all isothermally. It is flexible and scarless but loses reliability beyond about five fragments.

You need to assemble six parts and will later swap each one independently. Which method, and why?

Model answer. Golden Gate or MoClo. Standardised four-base overhangs let every part be swapped without redesigning its neighbours, assembly is one-pot, and the same part library serves future constructs. Gibson would work once but each new variant needs new homology arms designed against its specific neighbours.

Week 21 — Multigene pathway construction — sequence tooling, Golden Gate design, and yeast in vivo assembly

BsaI cuts outside its own recognition site. Why does that matter for assembly?

It lets the enzyme cut both strands at once

✔ The overhang is one you design, and the recognition site is lost from the product, so joins are scarless and can be made in one pot

It stops the enzyme re-cutting methylated DNA

It lets the reaction run without a ligase

Why. The overhang comes from the part's own sequence rather than from the enzyme's site, so you choose it. Once two parts are joined the recognition site is gone, so the product is not cut again while the ligase keeps working — which is what lets digestion and ligation run together in a single tube.

The reverse complement of the BsaI site GGTCTC is:

CTCTGG

✔ GAGACC

CCAGAG

GGTCTC

Why. A site on the bottom strand reads GAGACC along the top. A scan that looks only at the top strand finds half of the sites and misses the rest, which is a genuine cause of failed assemblies.

Which flaw in an overhang set will cause wrong joins?

✔ Two overhangs that are reverse complements of each other

Overhangs with different G and C content

One overhang beginning with an A

Using only seven overhangs

Why. AATG and CATT anneal to each other, so a fragment ending in one can ligate to a fragment starting with the other in the wrong place. Palindromic overhangs have the same defect with themselves, and overhangs one base apart can mispair.

Why do you remove an internal BsaI site with a synonymous codon change rather than deleting or rewriting the part?

Model answer. Because the aim is to change the DNA without changing the protein. A synonymous change — for example CTC to CTG, both leucine — breaks the recognition sequence and leaves the amino-acid sequence untouched, so the part still does its job. You then confirm it by translating the edited sequence and comparing it with the original, and by rescanning both strands for the site. Choose the replacement codon with the host's codon usage in mind, since a rare codon can lower expression.

Eight transcription units assemble in yeast with 80% overall success. If the seven junctions are independent, what does that say about each junction, and what does the model leave out?

Model answer. Each junction would have to succeed with probability 0.8 to the power 1/7, about 0.969; the same figure predicts 93.8% for three units and 88.0% for five. If each junction were only 0.95 reliable, eight units would give about 70%, and at 0.90 about 48%, which shows why per-junction reliability is the quantity that matters. The independence model leaves out mispairing between similar overlaps, recombination between repeated sequences, and mutations in clones that look correct, so use it to reason about how a build scales, not as a prediction of what a given lab will see.

Why combine in vitro Golden Gate with yeast recombination rather than use either alone?

Yeast cannot use Golden Gate parts

✔ Each is used where it is reliable: Golden Gate to build individual transcription units precisely, yeast recombination to join many of them in one transformation

It removes the need for sequencing

It avoids using a vector

Why. That is the design logic of the 2026 Trends in Biotechnology paper from KRIBB, whose abstract describes an integrated in vitro and in vivo platform assembling up to eight transcription units at 80% efficiency in a single transformation. It does not remove sequencing: every build still needs verifying.

Week 22 — Everyday assays: what the daily measurements actually mean

A qPCR standard curve has a slope of −3.32. What does it mean?

The assay failed

✔ Amplification is essentially 100% efficient: the product doubles each cycle

The efficiency is 33%

The reaction is saturated

Why. A tenfold dilution costing 3.32 cycles is exactly doubling, since 2^3.32 = 10. The usual acceptable window is about 90 to 110% efficiency, which is a slope between roughly −3.6 and −3.1.

What does the delta-delta-Ct method assume, and what happens when the assumption fails?

Model answer. It assumes the target and the reference amplify with the same, essentially perfect, efficiency, so that a cycle difference can be converted with a factor of two. When efficiencies differ, the conversion is wrong. In problem U3 a measured 95.1% and 98.8% efficiency gave 7.70-fold where the simple method gave 8.28, a 7% difference, which is tolerable. But an assay that is really 80% efficient and reported as 100% overstates the same result by 38%. The remedies are to measure the efficiency of every assay, to use the Pfaffl correction when they differ, and to redesign primers that amplify poorly.

Two technical replicates differ by 0.5 cycles. What does that correspond to in abundance?

No difference

✔ About 1.4-fold

About 5-fold

About 10-fold

Why. 2^0.5 = 1.41. Since a two-fold change is only one cycle, routine technical noise sits uncomfortably close to the effects people report, which is why replication and validated references matter more than extra wells.

A Western blot shows a band of the right size for your enzyme. What does that establish?

The enzyme is active

✔ That something of that size binds the antibody, which is evidence of presence and not of activity or folding

The protein is soluble

The pathway works

Why. Presence at a size, no more. Activity needs an assay, solubility needs a comparison of soluble and insoluble fractions, and any quantitative comparison between lanes needs a loading control.

Why can optical density, colony counts and live/dead staining disagree about the same culture?

Model answer. They measure three different things. Optical density measures light scattering, which tracks biomass and cell size rather than cell number, so a culture of filamentous or enlarged cells reads high. Colony counts measure cells that can grow on that medium, so cells that are alive but not culturable are missed. Live/dead staining measures membrane integrity, so a cell with an intact membrane but no metabolic activity counts as live. Which to trust depends on the question: biomass for yields and balances, colony counts for viability in a process, staining for rapid assessment of damage.

Which control tells you whether a qPCR signal comes from DNA contamination rather than from RNA?

- The no-template control
- ✔ The no-reverse-transcriptase control
- The melt curve
- The standard curve

Why. Without reverse transcriptase, no cDNA is made, so any signal must come from DNA in the sample. The no-template control catches reagent contamination instead, and the melt curve addresses specificity.

Week 23 — Genome engineering I — recombineering, MAGE, CRISPR

The SpCas9 protospacer-adjacent motif is:

- 5'-TTTV-3'
- ✔ 5'-NGG-3'
- 5'-GGTACC-3'
- None is required

Why. Cas12a prefers a T-rich 5′ PAM such as TTTV instead, which is why host and target sequence sometimes decide which nuclease you use.

The Gam protein of lambda Red:

- Anneals single-stranded DNA
- ✔ Inhibits host RecBCD so linear DNA survives
- Cuts double-stranded DNA
- Methylates the target

Why. Exo resects, Beta anneals, Gam protects. Together they make short homology arms on a PCR product sufficient for recombination.

Why is CRISPR usually paired with recombineering in bacteria rather than used alone?

Model answer. Bacteria lack efficient non-homologous end joining, so an unrepaired Cas9 cut is lethal. The cut is therefore used as counterselection: the recombineering machinery supplies the edit via a template, and Cas9 kills every cell that still carries the original sequence. CRMAGE pushes efficiency above 90% this way.

Week 24 — Genome engineering II — CRISPRi, base editing, dynamic control

dCas9 represses transcription by:

- Cutting the promoter
- ✔ Sterically blocking initiation or elongation
- Degrading the mRNA
- Methylating the gene

Why. Catalytically dead Cas9 is a programmable roadblock. Because it is reversible, titratable and multiplexable, it maps flux control across many genes far faster than making knockouts.

Give one clear advantage of CRISPRi over a knockout.

Model answer. Essential genes. A knockout of an essential gene is dead and tells you nothing; partial, tunable knockdown lets you find the expression level that maximises product without killing the cell — and you can do it across dozens of genes in one library.

A metabolite biosensor typically works by:

- Measuring optical density
- ✔ Coupling metabolite concentration to a transcription factor that drives fluorescence or growth
- Direct mass spectrometry in the cell
- Measuring pH

Why. That coupling is what makes single-cell screening by flow cytometry or growth selection possible, turning a slow HPLC assay into a sortable phenotype.

Week 25 — Genome design, synthesis and non-model host toolkits

The 61-codon E. coli genome described by Fredens et al. recoded about how many codons?

About 1,800

✔ About 18,000

About 180,000

About 1.8 million

Why. 18,214 codons in a 4 Mb genome, which is about 1.5% of all codons. Problem N1 works the arithmetic and the error budget.

Which replacements define the 61-codon genome?

✔ TCG and TCA (serine) to AGC and AGT, and the TAG stop codon to TAA

All arginine codons to CGT

Every start codon to GTG

All three stop codons deleted

Why. Two serine codons and one stop codon were replaced by synonyms, which leaves 61 codons in use and frees three for other purposes.

Why does the error budget force verification at every stage of building a genome?

Model answer. Because a success rate per section that looks excellent still leaves a whole genome almost certainly wrong. At a residual error rate of one in a million, a 100 kb section is clean 90.5% of the time, but the 4 Mb genome of forty such sections is clean only 1.8% of the time, with four errors expected. Checking only at the end would mean searching 4 Mb for four errors; checking each section means fixing an error in 100 kb where it is cheap to find. The error rates are assumptions, but the structure of the result holds.

At 37 °C, how do the measured generation times of V. natriegens and E. coli compare in the preprint used this week?

✔ 14.8 minutes against 31.3 minutes, about 2.1 times faster

5 minutes against 60 minutes

They are the same

30 minutes against 15 minutes

Why. That is from one preprint under particular media, so treat the ratio as typical, not fixed. It is the reason a toolkit for V. natriegens may repay itself over many cycles, and not over a few.

What must exist before a new host can carry a pathway?

Model answer. A way to get DNA in (electroporation, conjugation or natural competence), a replicon that is stable there, a selectable marker, promoters and ribosome-binding sites that have been characterised in that host and not assumed from E. coli, a method to edit its genome, a way to cure plasmids, a reporter, and an annotated genome sequence. Each missing piece is a project before the pathway project starts, which is why the break-even on a toolkit matters.

A library of 10,000 variants needs about how many transformants for 95% coverage of every variant?

About 10,000

✔ About 30,000

About 100,000

About 1,000,000

Why. $-\ln(0.05) \times 10{,}000 = 29{,}957$. In a host with a poor transformation efficiency this can need tens of micrograms of DNA, so efficiency can decide feasibility before growth rate does.

Week 26 — Genetic parts, transcription-factor biosensors, and circuits

A biosensor’s dynamic range is best defined as:

✔ The ratio of maximum output to baseline output

The span of ligand concentrations it responds to

The Hill coefficient

The ligand concentration at half maximum

Why. Maximum over baseline: a fold change. The span of ligand concentrations it responds to is the operating window, a different quantity that is often confused with it.

Why is dynamic range usually the least precisely known summary of a sensor?

It cannot be measured

✔ It divides by the baseline, which is the smallest and noisiest number in the dataset

Hill fits cannot estimate it

It depends on the plate reader brand

Why. A small error in a small number becomes a large error in the ratio. In problem I1, leaving out the zero-ligand measurements left the baseline uncertain by more than three times its own value and the dynamic range unbounded above; three zero-ligand replicates narrowed it to roughly 35 to 101-fold.

A sensor has K = 45 µM and n = 1.7. Over what ligand range would you use it, and why not outside it?

Model answer. The window between 10% and 90% of maximum response runs from K·(1/9)^(1/n) to K·9^(1/n): about 12.4 µM to 163.9 µM, a 13.3-fold range, which is also 81^(1/n). Below it the output sits near baseline and above it the sensor is saturated, so in both regions a change in ligand barely changes the signal. A higher Hill coefficient makes the window narrower and the response sharper, which suits a switch and not a quantitative readout.

A sort gate recovers 70% of true hits and lets through 1% of background. The enrichment per round is about:

7-fold

✔ 70-fold

0.7-fold

700-fold

Why. It is the ratio of the two recovery fractions. Starting from one hit in ten thousand, one round gives about 0.7% hits, two rounds about 33% and three rounds about 97% — in a model that ignores cells lost to sorting stress and cells that escape the gate.

A sensor-sorted strain glows brightly but makes no more product. Give three reasons.

Model answer. The sensor reports the intracellular concentration of its ligand, not the titer: a strain that retains product, or fails to export it, looks better to the sensor than it is. The sensor may respond to a related metabolite, so the signal can come from a precursor or a byproduct. And the selected variant may have changed the sensor, the reporter or the plasmid copy number rather than the pathway. A sorted hit is a hypothesis; confirm it by measuring product in the supernatant by HPLC or mass spectrometry, and by recloning the variant into a clean background.

Why include a reference promoter on every plate?

To improve growth

✔ So that results are reported relative to a standard and compare across plates, instruments and days

To reduce evaporation

To calibrate the pipette

Why. Absolute fluorescence depends on the instrument’s gain, the day and the plate. A ratio to an in-plate reference removes most of that, which is why part strengths are reported in relative units.

Week 27 — Regulatory control: sRNA, riboswitches, quorum sensing and CRISPRi

In the steady-state model of sRNA-mediated knockdown, the fold repression is:

✔ 1 + k·s/δ, where δ is the basal decay rate and k·s the added decay rate

k·s × δ

δ − k·s

Always about 2

Why. With production α and decay δ + k·s, the steady-state mRNA is α/(δ + k·s), so the ratio to the unregulated level is δ/(δ + k·s) and the fold repression is 1 + k·s/δ.

A repressor leaves 5% of maximum output even when saturated. What is the best repression it can give?

5-fold
✔ 20-fold
50-fold
Unlimited

Why. 1/0.05 = 20. A 50-fold target would need a 2% output, below the floor, so it is impossible however much repressor you add.

Why might you knock down a competing pathway and not knock it out?

Model answer. Because the gene may be essential or may feed biomass, so deleting it kills the strain or slows growth too much, and because the goal is often to move flux, not to remove a route. A knockdown lets you titrate the level, find the point where product flux rises before growth collapses, and apply it to several genes at once without editing the genome. The cost is leak, burden and the difficulty of knowing how far the protein actually fell.

To get a 10-fold mRNA knockdown by adding degradation, the added decay rate must be:

Equal to the basal rate
✔ Nine times the basal rate
Ten times the basal rate
Half the basal rate

Why. Fold repression = 1 + k·s/δ = 10, so k·s = 9δ. With a 3-minute mRNA half-life that is 2.08 per minute, which cuts the half-life to about 0.3 minutes.

A knockdown cuts the target mRNA ten-fold, but product flux barely changes. Give three reasons.

Model answer. The protein may turn over or dilute slowly, so its level has not fallen yet. The enzyme may be in excess, so lowering it changes little flux because it holds little control. The cell may compensate, with another enzyme or route taking over. The mRNA may not be the right thing to measure, so the protein and the flux have to be measured too. Or the knockdown may have hit growth and masked a gain. The way to tell them apart is to measure mRNA, protein and flux in the same samples.

Quorum-sensing control is best described as:

✔ Expression triggered by the density of the population
Expression triggered by temperature
A kind of antibiotic selection
A method of genome sequencing

Why. Cells release a signal that accumulates with density, and a receptor switches expression on at a threshold. It gives a built-in clock for two-stage production without adding an inducer.

Week 28 — Libraries, expression tuning, protein engineering, evolution

The guiding rule of directed evolution is:

More mutations are always better
✔ You get what you screen for
Rational design beats screening
Larger libraries always win

Why. The screen defines the outcome. If the readout drifts from the property you want — activity under process conditions, not activity in a plate — the evolved enzyme optimises the wrong thing.

Adaptive laboratory evolution identifies causal changes by:

Modelling
✔ Whole-genome resequencing of evolved clones
Proteomics alone
Guessing from phenotype

Why. Impose a pressure, let it run for hundreds of generations, sequence the survivors, then reverse-engineer which mutations matter by reintroducing them into the parent.

How would you use an RBS library to balance a three-enzyme pathway?

Model answer. Build variants spanning a wide predicted translation-rate range for each gene, combine them combinatorially, and screen the library for product. This finds the relative expression ratio that avoids both a bottleneck and the accumulation of a toxic or burdensome intermediate — without needing to know the kinetic parameters in advance.

Week 29 — Structure prediction and computational enzyme design — literacy

To redesign an active-site pocket around a bound cofactor you should use:

ProteinMPNN, which is the standard inverse-folding tool

✔ LigandMPNN, which models non-protein atoms explicitly

AlphaFold2

A language model score alone

Why. ProteinMPNN conditions on protein backbone atoms only, so the cofactor is invisible to it. LigandMPNN adds a ligand graph and was built for exactly this. Naming the wrong one of the two is a common and easily avoided error in proposals.

A predicted structure with uniformly high pLDDT tells you:

The active site geometry is accurate enough for design

✔ The backbone fold is likely correct

The enzyme is active

The substrate will bind

Why. Confidence in the fold is not confidence in side-chain placement, protonation, or the conformational state that matters for catalysis. Treat a prediction as a hypothesis to dock into and test, not as a crystal structure.

Where does computational design actually sit in an enzyme-engineering workflow?

Model answer. It narrows the search, it does not replace it. A design campaign typically goes: predict or obtain a structure, identify the residues lining the site, generate a few dozen to a few hundred sequence variants with a ligand-aware inverse-folding model, filter for predicted stability and for retained catalytic geometry, then build and screen that reduced set experimentally. The gain is going from a library of 10^6 random variants to 10^2 considered ones. The screen is still the arbiter, and a paper reporting designs with no activity data has reported a hypothesis.

Why is this week in the extension track rather than the core?

The methods are unreliable

✔ The deliverable cannot be validated without a wet lab, so it proves less on paper

It is too difficult

It is not used in the field

Why. The methods are real and increasingly standard. But an in-silico redesign scored by another model is weak evidence to a reviewer who will ask whether it worked. Literacy is worth a week; building a GPU pipeline is not, unless you are applying to an enzyme-design group.

A proposal says the pathway's third enzyme is slow, so AlphaFold will be used to redesign it. What is wrong?

Model answer. Two things. AlphaFold predicts structure, it does not design sequence, so the tool named cannot do the job described — design needs an inverse-folding model, ligand-aware if a cofactor is involved. And a slow enzyme may be slow for reasons structure cannot address: poor expression, misfolding, a missing cofactor, product inhibition, or a thermodynamically unfavourable step. The fix is to diagnose before designing, and to name the assay that will tell you which cause you are dealing with.

Week 30 — Enzyme and protein engineering for pathways

An assay has a Z′ factor of 0.1. What does that tell you?

It is excellent for screening

✔ The positive and negative controls barely separate once their spread is counted, so it is marginal for screening

It measures enzyme activity directly

The enzyme is inactive

Why. $Z' = 1 - 3(\sigma p + \sigma n) / |\mu p - \mu n|$. Above 0.5 is a good screen, between 0 and 0.5 is marginal, and below 0 is unusable. In problem L1, assay A scores 0.67 and assay B 0.10 from the same two means, only because B is noisier.

A library randomises three sites with NNK codons. About how many clones give 95% coverage?

About 100

About 3,000

✔ About 98,000

About 1,000,000

Why. NNK has 32 codons, so three sites give $32^3 = 32{,}768$ variants and about 98,163 clones for 95% coverage. At 960 clones a day that is roughly 102 days, which is why three simultaneous saturated sites are rarely practical.

An enzyme holds a flux control coefficient of 0.22 and another holds 0.77. Which would you engineer, and what does the arithmetic say?

Model answer. The one holding 0.77. In a linear pathway at fixed driving force, scaling an enzyme by a factor f changes the flux by $1 / (1 - C + C/f)$. For $C = 0.77$, $f = 2$ gives +63% and $f = 10$ gives +226%. For $C = 0.22$, $f = 2$ gives +12%, $f = 10$ gives +25%, and the ceiling is +28% however good the enzyme becomes. To get a 25% flux gain you need only a 1.4-fold improvement from the 0.77 enzyme and an 11-fold improvement from the 0.22 enzyme. The assumptions matter: this is a linear chain with additive resistances, so check the real control coefficients before committing a campaign.

You screen 10,000 clones with no real improvement present, calling hits at the wild-type mean plus three standard deviations. About how many false hits do you expect?

None

✔ About 13

About 228

About 1,000

Why. The one-sided tail beyond three standard deviations is about 0.13%, so about 13 of 10,000. At two standard deviations it is 2.3%, about 228. The screen needs the assay noise, the threshold and the library size considered together.

A variant has a higher kcat in vitro but gives no more product in the host. Give three reasons.

Model answer. The enzyme may not hold flux control, so extra activity moves flux little. Expression or solubility may be lower in the host, so the extra kcat is outweighed by less active enzyme. Cofactor or substrate supply may be the real limit, and a P450 in particular depends on NADPH and a redox partner or on peroxide. The variant may also lose stability or specificity in the cellular context. The way to tell is to measure the enzyme in the host: amount, activity, and product titer, alongside the in-vitro constants.

Which cofactor arrangement is typical for a cytochrome P450 monooxygenase?

✔ NADPH supplied through a redox partner, or in some routes hydrogen peroxide as a peroxygenase

ATP only

NAD^+ only

No cofactor

Why. P450 systems usually take electrons from NADPH through a partner protein, while peroxygenase routes use peroxide instead. That is why cofactor supply and partner choice are part of P450 engineering, not afterthoughts.

An enzyme is made at a high level in E. coli but is inactive, and most of it is in the insoluble fraction. What is a sensible first step?

Increase the inducer concentration

✔ Lower the expression temperature and the inducer concentration, and check the soluble and insoluble fractions on a gel

Sequence the gene again

Switch to a stronger promoter

Why. Slower expression gives the chain time to fold, and the gel tells you what actually happened. Tags and chaperones come next, and each change should be made alone so you know which one worked.

A plant cytochrome P450 gives no activity in E. coli despite strong expression. List the hypotheses you would test.

Model answer. The N-terminal membrane anchor may need truncating or replacing, since these are membrane-bound enzymes in plants. A reductase partner may be missing or mismatched, so electrons are not delivered. Haem supply may be limiting. The protein may be in inclusion bodies, so the soluble fraction needs checking. Codon use and the expression temperature may matter. A yeast host, whose endoplasmic reticulum suits membrane-bound P450 enzymes, may simply be the better choice. The tests are a gel of soluble against insoluble fractions, a spectrum for the reduced CO-bound enzyme, and an assay with and without the added partner.

Week 31 — Streptomyces and natural-product pathway engineering

The 6-deoxyerythronolide B synthase builds its product from:

✔ One propionyl-CoA starter and six methylmalonyl-CoA extender units

Seven malonyl-CoA units

Acetyl-CoA only

Seven methylmalonyl-CoA units

Why. That is the stoichiometry in the IUBMB entry for EC 2.3.1.94. Seven C3 units give the 21 carbons of the macrolactone, and the CO_2 fixed in making methylmalonyl-CoA is released again at each condensation.

How many moles of NADPH does the 6-deoxyerythronolide B synthase consume per mole of product?

2

4

✔ 6

12

Why. Six, from the five ketoreductase domains and the one enoyl reductase domain, as the IUBMB entry states. NADPH supply is therefore a design variable for any polyketide strain.

Why does the number of BAC clones needed to capture a cluster depend on the cluster's size and not only on the genome size?

Model answer. Because a clone only helps if it contains the whole cluster, not just part of it. With 100 kb inserts, a clone holds a 60 kb cluster whole only if the insert starts within a 40 kb window of the cluster's start, so the probability per clone is about 0.0046 in an 8.7 Mb genome. At 99% confidence that needs about 1,000 clones, against about 440 for a 10 kb cluster and about 4,000 for a 90 kb one. An estimate that ignores cluster size gives about 400 for the 60 kb case, which is 60% too few.

A biosynthetic gene cluster is silent under laboratory conditions. Which is a standard first approach?

Delete it

✔ Replace native promoters or activate a regulator, or express the cluster in a heterologous host

Raise the temperature and wait

Resequence the genome

Why. Refactoring and regulator engineering, or moving the cluster into a host where it is expressed, are the standard routes. Which one works depends on whether the block is transcription, precursor supply or the pathway itself.

Why does feeding propionate become a poor way to supply the precursors for a polyketide at a high titer?

Model answer. Because the demand scales with the titer and the toxicity and cost scale with it. Making 1 g/L of 6-deoxyerythronolide B takes at least 1.34 g/L of propionic acid, and 5 g/L takes at least 6.7 g/L, on a ceiling of 0.745 g product per g propionic acid. The cell also pays about 20 ATP-equivalents per product molecule to convert propionate to methylmalonyl-CoA units, and 6 NADPH. A strain that makes its own methylmalonyl-CoA from central metabolism avoids both the feed cost and the toxicity, which is why precursor supply is a target for strain engineering.

Why is a double-strand break from Cas9 often a harsh treatment in Streptomyces?

The cells have no DNA at all

✔ Many Streptomyces repair breaks poorly by non-homologous end joining, so editing usually needs a homology template

Cas9 cannot enter the cell

Streptomyces are haploid and cannot divide

Why. In general, editing in Streptomyces relies on homology-directed repair from a supplied template, and uncontrolled Cas9 cutting is often lethal. Engineered Cas9 and CRISPRi variants, such as those in the lab papers listed this week, exist to manage that.

Week 32 — Genomic integration, sequencing data, and wet-lab reality

Serine integrase recombination is irreversible because:

The enzyme is destroyed after use

✔ The product sites attL and attR are not substrates for the integrase alone

It needs ATP that is quickly depleted

The DNA is methylated afterwards

Why. Excision requires a separate recombination directionality factor. That one-way behaviour is exactly why Bxb1 and φC31 are used to drop multi-gene cassettes into a landing pad and leave them there.

Industrial strains keep multi-gene pathways in the chromosome rather than on plasmids mainly because:

Chromosomes allow higher copy number

✔ Uniform gene dosage and stability without antibiotic selection

Plasmids cannot hold more than three genes

Integration raises enzyme activity

Why. A plasmid population is heterogeneous in copy number and is lost without selection, and antibiotics are a cost, a waste-stream problem and a regulatory one at scale. The trade-off you accept is lower dosage, usually compensated with stronger promoters or several integration sites.

Prime editing is rarely used in bacterial metabolic engineering because:

It is patented

✔ It depends on eukaryotic repair machinery and transfers poorly to bacteria

It only edits RNA

It is too precise

Why. The attention it receives comes from mammalian genome editing. In bacteria the practical no-double-strand-break options are base editing and recombineering, with Cas9 used as counterselection rather than as the editor.

Why is breseq the standard tool after an adaptive laboratory evolution experiment, rather than a generic variant caller?

Model answer. Because it is built for the case: a haploid microbial genome a few megabases long, only slightly diverged from a good reference. It predicts new sequence junctions — mobile-element insertions, large deletions, rearrangements — which generic SNP callers miss entirely and which are a large fraction of what evolution actually produces in bacteria. It masks read ends properly so short indels are called reliably, it has a polymorphic mode for population samples as well as clones, and it outputs annotated HTML that says which gene each mutation hits. The failure mode to know: it assumes your reference is close, so it is the wrong tool for a divergent or unassembled strain.

Your Gibson assembly gives zero colonies. What are the likely causes, in order, and which control distinguishes them?

Model answer. First: the transformation itself failed — competent cells dead, heat shock wrong, wrong antibiotic. The positive control with pUC19 settles this in one plate; no pUC19 colonies means the problem is not the assembly at all. Second: insert or vector quality — wrong concentration, degraded ends, a failed PCR, or primers with too little homology; check by gel before assembling and requantify. Third: the assembly reaction — wrong molar ratio, too many fragments, or an inhibitor carried over from a purification. Fourth: the construct is toxic to the host, which shows up as zero colonies with a working positive control and a clean negative; move to a lower-copy vector or a tighter promoter. Report the control results, not just the outcome — that is what distinguishes a troubleshooting answer from a guess.

Your assembly gives plenty of colonies but they all contain empty vector. The control that would have predicted this is:

The pUC19 positive control

✔ The negative control: linearised vector with no insert

A colony PCR

A Sanger sequence

Why. That plate quantifies background from uncut or self-ligating vector. If it grows a lawn, your real plate's colonies are mostly background, and the fix is better vector linearisation or dephosphorylation before you waste a week screening.

Design the colony PCR that verifies both the junction and the orientation of an insert.

Model answer. Put one primer in the vector backbone, outside the insertion site, and the other inside the insert. A product only appears if the insert is actually joined to the backbone at that junction, so the junction is verified; and because the insert primer only points the right way in one orientation, a product of the expected size also establishes orientation. Two primers both in the insert would amplify from free insert DNA and tell you neither thing. Predict the band size before you run it, include a no-template control and a known positive, and confirm the final construct by Sanger or whole-plasmid sequencing — a gel band of the right size is consistent with the right construct, not proof of it.

Week 33 — RNA-seq and differential expression

Raw RNA-seq counts cannot be compared between samples mainly because:

Sequencers are inaccurate

✔ Library size and composition differ, so a count is a share of a total, not an amount

Genes have different lengths only

Reads are too short

Why. Counts are compositional. If one transcript takes a large share of the library, every other gene’s count falls even when its true level is unchanged — which is exactly the situation in a strain overexpressing a pathway hard.

With 6,000 genes tested, controlling the false discovery rate at 0.05 means:

5% of all genes are wrong

✔ About 5% of the genes you call significant are expected to be false positives

The p-value threshold is 0.05

No false positives

Why. The control is over the called set, not the whole list. Reporting unadjusted p-values across thousands of tests produces hundreds of false hits by construction, which is why reviewers look for the adjustment before they look at the biology.

A transcript rises fourfold but the pathway flux is unchanged. Give three explanations.

Model answer. Translation: more mRNA does not mean more protein, since ribosome availability, RBS strength and mRNA stability intervene — proteomics would settle it. Activity: the enzyme may be present but inactive through misfolding, a missing cofactor, feedback inhibition or an unfavourable thermodynamic driving force. Control: the enzyme may simply hold little flux control, so even a real increase in active enzyme moves flux very little — which is the week 15 result arriving from the data side. The practical consequence: never report a transcriptomic change as a flux change, and say which of the three you tested.

Why is n = 3 usually the practical floor for differential expression?

Software requires it

✔ Dispersion cannot be estimated per gene from fewer, so the test borrows information across genes

It guarantees significance

Sequencing is expensive

Why. Methods such as DESeq2 shrink gene-wise dispersion towards a trend fitted across all genes, which is what makes small n workable at all. It is a compromise, not a solution — and the power problem from week 17 has not gone away.

Why does this belong in your plan at all, given it is not modelling?

Model answer. Because nearly every metabolic engineering PhD generates transcriptomic data at some point, usually to explain why a strain underperforms, and the analysis is routinely handed to the student. Running the tool takes a couple of days; understanding normalisation, dispersion and multiple testing well enough not to over-claim takes longer, and misreading the output is the common failure rather than a rare one. It also pairs with the breseq work in week 32: that one calls variants in DNA, this one does count statistics, and they are genuinely different skills.

Week 34 — Biofoundry workflows — design, build and test as a data pipeline

Dead volume in a source well is:

✔ Liquid the tip cannot aspirate, which must be added to what you actually need

Volume lost to evaporation during the run

The 10% overage in the master mix

The volume of the pipette tip

Why. The last stretch of liquid in a well cannot be reached, so every source well needs the required volume plus this allowance. Forgetting it is the commonest reason a run fails on its last few reactions.

A simulator for a liquid-handling protocol cannot verify:

Whether volumes add up

Deck layout

✔ Calibration and how a viscous liquid behaves

Which wells are addressed

Why. A simulator checks the logic of a protocol, which is genuinely useful. It cannot tell you that a viscous master mix under-delivers, that a tip retains a drop, or that the deck is a millimetre out. Those need the machine, and a person who knows it.

Compute the master mix volume for 24 reactions at 6 µL each with 10% overage, and say why the overage is there.

Model answer. 24 × 6 µL × 1.10 = 158.4 µL. The overage covers liquid lost in the tip and the reservoir so that the last reaction does not come up short. Without it, the last wells in a run are the ones that fail, and the failure looks like biology.

Nine of the ten best variants on a plate sit on the edge, which holds 38% of the wells. The most likely explanation is:

Edge positions favour growth by chance

✔ A position artefact such as evaporation, not biology

Strong variants were placed there by design

The reader is faulty

Why. Edge wells lose liquid faster and read concentrated. Nine of ten from a region holding 38% of the wells is far from chance. In problem J2, estimating the edge multiplier from the plate itself and correcting for it raised the rank correlation with the truth from 0.77 to 0.96.

Why must a design identifier travel with every well from the start?

Model answer. Because the results table is only useful if each measurement can be traced to the exact design that produced it. Without a persistent identifier, results get joined to designs by position or by memory, and a single transposed plate silently assigns every measurement to the wrong variant. It also makes the learning step possible: a model can only be trained on a table in which genotype and phenotype sit in the same row.

Going from a 10 µL reaction to a 500 nL reaction saves what fraction of reagent per reaction?

50%
80%
✔ 95%
99%

Why. 500 nL is 5% of 10 µL, so the saving is 95%. The cost is that evaporation, pipetting accuracy and dead volume all matter far more at that scale.

Week 35 — Constraint-based modelling — FBA and genome-scale models

The objective function in a standard FBA is usually:

ATP production
✔ Biomass production
Product titer
Enzyme cost

Why. Maximising biomass is a reasonable proxy for an evolved wild type, but a production strain is not trying to grow optimally — which is why growth-coupled design matters.

Flux variability analysis computes:

The single optimal flux vector
✔ The range each flux can take while the objective stays optimal
Metabolite concentrations
Enzyme levels

Why. It exposes how much of an FBA solution is actually determined. A flux with a wide range is one the model is not really telling you anything about.

Why does FBA need no kinetic parameters, and name two things it therefore cannot tell you.

Model answer. It only enforces stoichiometric mass balance and flux bounds at steady state, then optimises a linear objective — no rate laws required, which is why it scales to genome-wide networks. It cannot give metabolite concentrations, and it cannot predict regulation or dynamics; it also tells you what is feasible, not what the cell actually does.

What does an exchange reaction represent in a genome-scale model, and what happens to predicted yield if you constrain one?

Model answer. An exchange reaction is a pseudo-reaction at the system boundary that lets a metabolite enter or leave the modelled cell — uptake of glucose, secretion of product. Tightening a product exchange bound caps how fast the product can leave, so the model must either slow the pathway or spill carbon elsewhere, and maximum yield falls. It is the cheapest way to see that transport, not pathway chemistry, often sets the ceiling.

Week 36 — Python for modelling II: COBRApy in depth

The right way to change a model's carbon source in COBRApy is to:

Edit the biomass reaction
✔ Set model.medium, which controls the exchange reaction bounds
Delete the glucose transporter gene
Change the objective function

Why. The medium is a dictionary of exchange reactions and their maximum uptake rates. Editing bounds by hand works too, but the medium interface makes the intent explicit and is much harder to get half-right.

What does `with model:` give you?

A faster solver
✔ Changes inside the block are reverted automatically on exit
Parallel execution
A copy of the model

Why. It is the standard idiom for deletion scans and sensitivity loops: apply a change, optimise, let the context restore the model. Without it you accumulate silent state across iterations and your 400th knockout result is meaningless.

What does parsimonious FBA add to plain FBA, and when do you want it?

Model answer. Plain FBA fixes the objective value but leaves the rest of the flux distribution underdetermined, so solvers return an arbitrary member of the optimal set, often with pointless internal cycles. pFBA keeps the optimal objective and then minimises the total absolute flux, which selects the solution using the least enzyme, and is both more biologically plausible and reproducible between runs. Use it whenever you are going to interpret or plot the internal fluxes rather than only the objective.

A flux variability analysis returns a wide range on one reaction. That means:

The reaction carries high flux

✔ The data and constraints do not determine that flux

The model is broken

The reaction is essential

Why. Wide ranges mark where the model is silent — usually a parallel route or a cycle. It is the constraint-based echo of problem A1, and the reason you reach for labelling data.

Why is SBML worth bothering with when JSON round-trips faster?

Model answer. Because SBML is the interchange format the whole field reads: COBRApy, the MATLAB COBRA Toolbox, RAVEN, COPASI and the model repositories all speak it, so a model saved as SBML can be opened by a collaborator who uses none of your tools. JSON is convenient inside one Python project and effectively private outside it. For your own portfolio, ship SBML and keep JSON as the fast local cache.

Week 37 — Reconstructing a genome-scale model, and testing whether it is any good

Automated gap filling adds reactions in order to:

Improve annotation coverage

✔ Make biomass producible, usually with the smallest set of additions

Balance charges

Match a reference model exactly

Why. It is an optimisation over a universal reaction database, not evidence about your organism. Every filled gap is a testable claim that this organism has that enzyme, and a surprising number turn out to be wrong.

A MEMOTE report shows poor annotation coverage but full stoichiometric consistency. For predicting a yield ceiling this is:

Fatal

✔ Largely cosmetic — annotations help reuse, the stoichiometry carries the prediction

Impossible

A sign the model is curated

Why. Learning which failures matter for which question is the skill. Annotation gaps hurt interoperability and future reuse; mass imbalance or stoichiometric inconsistency silently corrupts every flux you compute.

Why does the biomass objective function deserve suspicion in a draft model?

Model answer. Because its coefficients are usually copied from a related organism rather than measured for this one, and they set what growth costs in carbon, ATP and precursors. Get the ATP maintenance term or the macromolecular composition wrong and every predicted yield, every growth rate and every growth-coupled design shifts with it. For a draft model the honest practice is to state that the biomass function is inherited, to test how much your conclusion moves when you perturb the maintenance term, and to avoid quantitative claims that rest on it.

Your capstone organism has no curated model. The first move is:

Abandon the organism

✔ Reconstruct a draft from its genome and test it, treating it as provisional

Use the E. coli model and rename it

Assume the pathway works

Why. A tested draft with its weaknesses documented is a legitimate basis for a yield ceiling or a feasibility argument. Borrowing another organism's model without saying so is not, and reviewers notice.

When is a draft model good enough?

Model answer. It depends on the question. For a theoretical yield ceiling or a redox and ATP feasibility check, a draft with consistent stoichiometry and a sane biomass function is usually adequate, because those answers rest on conservation laws rather than on regulation. For gene-deletion predictions, growth-coupled strain design or anything quantitative about growth rate, it usually is not, because those depend on details the draft got by analogy — gene-protein-reaction rules, the biomass function and gap-filled reactions. State the question first, then judge the model against it, and say in the text which class your claim falls into.

Week 38 — MATLAB and the COBRA Toolbox: reading other people's code

The most common silent bug when porting Python array code to MATLAB is:

- Different floating-point precision
- ✔ One-based indexing against zero-based
- Slower loops
- Different matrix multiplication

Why. Off-by-one errors do not raise an error, they return a shifted answer. When a translated script gives a plausible but different number, check indexing before anything else.

In MATLAB, solving A·x = b idiomatically is:

- inv(A)*b
- ✔ A\b
- A/b
- solve(A,b)

Why. The backslash operator picks an appropriate factorisation, and for an overdetermined system it returns the least-squares solution. It is the direct counterpart of np.linalg.solve and lstsq together.

Why learn the COBRA Toolbox at all if COBRApy is your main tool?

Model answer. Because two decades of published strain-design work, and many of the labs you might join, run on MATLAB scripts that nobody has ported. Being able to open someone's repository, run initCobraToolbox, and reproduce their figure means you can build on their work in your first months rather than reimplementing it. You are not learning it to work in it daily; you are learning it so legacy code is readable rather than a wall.

Your COBRApy and COBRA Toolbox runs on the same model give different growth rates. The first thing to check is:

- The solver
- ✔ The medium, the bounds and the objective
- Floating-point tolerance
- The MATLAB version

Why. Both call the same kinds of linear programme and should agree to many digits. Nearly every real discrepancy is a modelling difference — a different carbon uptake bound, an oxygen exchange left open, or a different objective reaction — not a numerical one.

What survives an SBML round trip between platforms, and what tends not to?

Model answer. Reliably: the stoichiometry, reaction and metabolite identifiers, bounds, the objective coefficients and gene–protein–reaction rules. Less reliably: annotations and database cross-references, notes fields, compartment metadata, constraints added by tool-specific extensions, and anything stored only in a platform's own object attributes. The practical habit is to compare reaction and metabolite counts, the objective value under identical media, and a spot check of bounds immediately after reading a model in — before building anything on top of it.

Week 39 — Strain-design algorithms, ^{13}C-MFA, kinetic models

OptKnock searches for:

The best promoter

✔ Knockouts that make product formation obligatory for growth

The cheapest feedstock

Optimal kLa

Why. It is a bilevel problem: the outer level chooses deletions, the inner level lets the cell maximise growth. Coupling means selection works for you instead of against you — the design logic behind the Genomatica 1,4-BDO strain.

^{13}C-MFA can resolve split ratios between parallel pathways because:

It measures enzyme levels

✔ Different routes leave different labelling patterns in downstream metabolites

It is more sensitive than HPLC

It uses a genome-scale model

Why. Glycolysis and the pentose phosphate pathway rearrange carbon skeletons differently, so the mass isotopomer distribution of a shared downstream metabolite reports how much went each way.

State the difference between a flux FBA calls feasible and one ^{13}C-MFA calls actual.

Model answer. FBA returns one of many flux vectors consistent with stoichiometry and an assumed objective — a prediction about what could happen. ^{13}C-MFA infers fluxes from measured labelling patterns — evidence about what did happen. Designs justified only by FBA should be checked against measured flux before being trusted.

Week 40 — Retrosynthesis, pathway prediction and machine learning

Retrosynthesis tools such as RetroPath2.0:

Predict protein structures

✔ Enumerate and rank enzymatic routes from host metabolites to a target using reaction rules

Assemble DNA

Simulate fermenters

Why. They expand a reaction-rule network backwards from the target, then rank candidate routes by criteria such as thermodynamics, enzyme availability and host compatibility. RetroPath2.0 was validated by building pinocembrin producers.

Where does machine learning actually enter the DBTL cycle, and what does it need?

Model answer. Mostly at Learn and the next Design: a surrogate model maps genotype or expression settings to measured output and proposes the next round, often by active learning. It needs consistent, reasonably large, machine-readable datasets from a standardised build-and-test pipeline — which is why it travels with biofoundries rather than with individual benches.

Week 41 — Bayesian optimisation and machine-learning-guided DBTL

Expected improvement balances:

Speed and accuracy

✔ Sampling where the model predicts a good result against sampling where it is uncertain

Bias and variance

Training and test error

Why. Exploit the promising region, explore the unknown one. An acquisition function that only exploits converges onto the nearest local optimum, which is the failure mode worth being able to describe.

Why does a Gaussian process suit strain engineering better than a neural network?

It is more accurate

✔ It reports calibrated uncertainty from very few points, which is what sequential design needs

It trains faster

It handles images

Why. With twenty experiments, a network has nothing to learn from and no honest error bars. The uncertainty estimate is the product here, not the prediction.

A 4×4 factorial grid and a 16-run Bayesian optimisation have the same budget. When does each win?

Model answer. The grid wins when factors are few, effects are roughly additive, and you want an interpretable map of the whole space — and it is far easier to run in a single batch, which matters practically. Sequential optimisation wins when the response surface has a narrow optimum, when factors interact, or when there are more than two or three factors, because a four-level grid costs 16 runs in two factors, 64 in three and 256 in four. In a simulation of a narrow off-grid optimum, a 16-run grid reached about 68% of the true optimum while 16-run Bayesian optimisation reached at least 95% in 19 of 20 random seeds — with one seed stuck on a local optimum, which is the honest caveat.

The most common reason machine-learning-guided DBTL fails in practice is:

The wrong algorithm

✔ Too few experiments, or a readout that does not measure what matters

Insufficient compute

Overfitting

Why. A surrogate fitted to eight noisy measurements of a proxy readout will confidently recommend nonsense. The algorithm is rarely the bottleneck; the experiment count and the assay are.

Write the learn step of your own capstone concretely.

Model answer. Your own answer, and it should name: what you measure for each variant, how many variants per round, how the genotype is encoded as model input, which surrogate and acquisition function you would use, how many rounds you can afford, and what result would make you abandon the approach rather than iterate again. Most proposals say machine learning will be used to guide design and stop there, which tells a reviewer nothing.

Week 42 — Machine learning for biology

With 50 features and 30 samples, a model scores $R^2 = 0.96$ on its training data and 0.31 on new data. What does this show?

The model is excellent

✔ The model has overfitted: the training score is not evidence of performance on new data

The data are mislabelled

The model needs more features

Why. The gap of 0.65 is the signature of overfitting. In problem P1 the gap shrinks to 0.03 only at 480 samples, and the true ceiling is 0.83.

The labels are shuffled, so there is nothing to learn, and a model with more features than samples still scores $R^2 = 0.90$ on its training data. What do you conclude?

The shuffle failed

✔ A high training score proves nothing when features outnumber samples, because a model can fit noise

The model has found a real pattern

The data are too clean

Why. Held-out R^2 was -0.49 in the same run, worse than predicting the mean. Only held-out scores count.

Why is a random split misleading for variants of related sequences?

Model answer. Because variants of the same family appear in both training and test sets, so the model can recognise the family and look accurate without learning anything that transfers. In problem P2 a random 5-fold split gives $R^2 = 0.66$, while splitting by family gives -0.04 on identical data, and only 33% of the label variance was transferable at all. The split has to match the question: random is fair for ranking variants within known families, and a grouped split is needed for new families, plates or batches.

You want to predict how a model will do on a protein family it has never seen. Which split do you use?

A random split

✔ A split that holds out whole families

No split, using the training score

A split by sequence length

Why. Hold out whole families, or clusters at a chosen sequence identity, so no test sequence has a close relative in training.

A protein-language-model score correlates with your assay results. What do you check before relying on it?

Model answer. Whether the correlation holds on held-out data that the model could not have seen, whether the pretraining data included the test sequences or close relatives, whether it beats a simple baseline such as a linear model or nearest neighbour, how large the effect is against the noise between replicates, and whether it helps choose variants in practice, meaning that the top-ranked ones really are better. A correlation across a whole library can hide a model that fails at the top end, which is the only part you use.

An AI assistant writes a function that splits your data, and it runs without errors. What do you do before trusting the results?

Trust it, because it ran

✔ Test it on data where you know which rows share a family, and confirm that no family appears in both training and test

Ask the assistant to confirm it is correct

Delete the split and use all the data

Why. Code that runs is not code that is right, and an assistant asked to check its own work often agrees with itself. The check has to be independent: a case with a known answer, and a read of every line that decides what is in the test set. The same applies to any citation or number it gives you.

What is active learning for, in a design–build–test loop?

✔ To choose the next experiments that will teach the model the most, or most likely improve the target

To make the model larger

To replace replicates

To avoid a baseline

Why. It uses the model's predictions and its uncertainty to pick the next batch. The Bayesian optimisation week is the same idea applied to maximising a target.

Week 43 — Fluxomics: ^{13}C metabolic flux analysis in practice

[1-^{13}C] glucose resolves the pentose phosphate split because:

It is cheaper than other tracers

✔ Carbon 1 is released as CO_2 in the oxidative branch but retained through glycolysis

It labels only pyruvate

It prevents the TCA cycle

Why. The difference in labelling of downstream metabolites is a direct readout of how much carbon took each route. Choose the wrong tracer and the split is simply not in your data.

A mass isotopomer distribution measures:

Total ^{13}C enrichment only

✔ The fractional abundance of each mass-shifted form of a metabolite

Enzyme activity

Flux directly

Why. M+0, M+1, M+2 and so on. Fluxes are inferred from these by fitting an atom transition network — the MID is data, the flux is a model-dependent conclusion.

What does a poor goodness of fit in a ^{13}C-MFA tell you?

Model answer. Either the measurements are wrong — bad natural-abundance correction, isotopic steady state not reached, contaminated or degraded samples — or the network is wrong: a reaction is missing, a reversibility is misspecified, or a pathway you assumed absent is carrying flux. The second is the interesting case, because a failed fit is then evidence of biology you did not know about. A fit accepted without the statistic reported should not be trusted.

Isotopically non-stationary ^{13}C-MFA exists mainly because:

It is cheaper

✔ Some systems, such as photoautotrophs, never reach isotopic steady state in a usable time

Mass spectrometers are too slow

Labelled CO_2 is unavailable

Why. When the only carbon source is CO_2, or the organism grows very slowly, you must fit the transient labelling instead of the steady state — more data, more computation, and the only route to fluxes in cyanobacteria.

Why is a measured flux map worth more to a strain-design argument than an FBA solution?

Model answer. FBA returns one of many vectors consistent with stoichiometry under an assumed objective — a hypothesis about what could happen. A ^{13}C flux map is inferred from measured labelling, so it reports what the cell did under those conditions, including the splits and reversibilities a model has to assume. In a proposal or a PhD interview, saying you would validate a predicted design by labelling shows you know the difference between a prediction and a result.

Week 44 — Bioreactor engineering, scale-up and downstream

A product that stays inside the cell changes the process mainly by:

Raising the oxygen demand

✔ Forcing cell disruption and separation from cell debris downstream

Removing the need for sterility

Improving the carbon yield

Why. You pay twice: the biomass you grew is destroyed to get the product out, and the product must then be separated from DNA, protein and membrane fragments. That is why localisation belongs in the techno-economic argument, not only in the biology.

Explain how product localisation changes the engineering problem.

Model answer. Secreted: the bottleneck moves to transport mechanics and the export gradient, high-density or continuous fermentation becomes feasible, and recovery leaves the biomass intact — so you engineer or mine efflux pumps and manage feedback inhibition inside the cytoplasm. Retained or stored: the cell has a finite capacity before toxicity or osmotic stress stops growth, and recovery requires lysis and separation from cell debris — so you engineer storage capacity, compartmentalisation into peroxisomes or vacuoles, and the extraction train. Pathway and transport have to be co-designed around how the molecule will be harvested.

Holding power per unit volume constant on scale-up means mixing time:

Stays the same

✔ Increases, because the vessel is larger relative to the impeller's circulation

Decreases

Depends only on viscosity

Why. Circulation time grows with vessel size, so a cell experiences a longer trip between visits to the feed zone. This is the mechanism behind every feast-famine and gradient problem at scale, and the reason scale-down reactors exist.

In OTR = kLa (C* − C_L), raising the vessel pressure helps because it:

Increases kLa

✔ Raises C*, the saturation concentration, and so the driving force

Lowers the cell's demand

Reduces shear

Why. Henry's law. It is also why tall vessels have a dissolved-oxygen profile with height, and why the same strain behaves differently at the top and bottom of a 20 m tank.

Derive, in words, the exponential feed for a fed-batch that holds specific growth rate constant.

Model answer. A substrate balance on the vessel: accumulation equals feed minus consumption, and consumption is the biomass times the specific uptake rate, which equals μ/Y plus maintenance. To hold μ constant while biomass grows exponentially, the feed rate must itself grow exponentially at the same rate — $F(t) = F_0 \exp(\mu t)$. The practical consequence is that the feed pump, not the organism, sets the growth rate, which is the whole point of fed-batch: it keeps the substrate low enough to avoid overflow metabolism and oxygen limitation.

Which usually dominates the cost of a bulk fermentation product?

The inoculum

✔ Feedstock, together with the separation energy that a dilute broth forces

Sterilisation

Analytical testing

Why. Which is why yield and titer are the two metrics that matter most for a commodity, and why a 10 g/L titer can be fatal regardless of how elegant the pathway is.

A strain gives 40 g/L in a 1 L bioreactor and 12 g/L at 10 m³. Name four candidate explanations and how you would test each.

Model answer. Oxygen limitation — compare the required kLa with the vessel's, and check for overflow byproducts such as acetate. Substrate gradients causing feast-famine — estimate circulation time and reproduce it in a two-compartment scale-down reactor. CO_2 accumulation or pH excursions near base addition — measure dissolved CO_2 and offgas. Genetic instability over the extra generations needed to fill the vessel — sequence or plate the end-point population and check plasmid retention or pathway integrity. The honest answer names the measurement for each, not just the hypothesis.

Oxygen supply in a large aerobic fermenter is governed mainly by:

Stirrer colour

✔ kLa and the oxygen driving force

Medium pH

Inoculum size

Why. Your transport-phenomena course already covers this. Oxygen transfer rate = $kLa(C^* - C)$, and in a 100 m³ vessel it is frequently the constraint that strain data from a 1 L bioreactor hides.

Name four gradients a 100 m³ fermenter has that a 1 L bioreactor does not.

Model answer. Dissolved oxygen, substrate concentration near the feed point, pH near base addition, and dissolved CO_2 and hydrostatic pressure with height. Cells circulate through these on timescales of tens of seconds, so they experience repeated feast-famine cycles that no shake flask reproduces.

Why does techno-economic analysis often kill a strain that looked good in the lab?

Model answer. Because lab metrics ignore the costs that dominate: feedstock at yield below theoretical, oxygen and cooling duty, dilute-broth separation energy, sterility and capital charges, and the price of the petrochemical incumbent. This is exactly the judgement a chemical engineer brings to a biology group.

Week 45 — Product toxicity, export and in situ removal

Which statement about efflux pump families is correct?

✔ RND and MFS pumps are typically driven by the proton motive force, and ABC pumps by ATP hydrolysis

All efflux pumps are driven by ATP

All efflux pumps are driven by the proton motive force

Efflux pumps work by passive diffusion

Why. The energy source decides the cost to the cell. RND pumps such as AcrAB–TolC are tripartite and span the Gram-negative envelope, so a heterologous pump may need its adaptor and outer-membrane partners too.

In the 43-pump library of Dunlop et al. (2011), tested against seven biofuels in E. coli, what happened for n-butanol and isopentanol?

Every pump improved tolerance

✔ None of the pumps improved tolerance

Only ABC pumps helped

The result was not reported

Why. Pumps helped for the other five fuels. The two best were AcrAB and a pump from Alcanivorax borkumensis, and the A. borkumensis pump raised limonene yield in a production strain. Efflux is a tool to test and not a guaranteed fix.

Why can the titer measured in the broth badly underestimate how much of a volatile product the cells made?

Model answer. Because the product evaporates as it is made. Newman et al. (2006) measured a half-life of about 50 minutes for amorphadiene leaving their fermentor. With first-order loss at that rate and a constant production of 1 g/L/h, the broth can never hold more than about 1.2 g/L, so after 24 hours it shows 1.2 g/L of the 24 g/L that was made, which is 5%. Capturing the product, in the off-gas or in an organic phase, is then needed to measure it correctly as well as to recover it. The half-life is specific to that product and vessel, so measure it for yours.

A product has a partition coefficient of 1000 (organic over aqueous) and the organic phase is 10% of the broth by volume. What share of the product sits in the organic phase?

About 50%

About 91%

✔ About 99%

100%

Why. Share = $K \cdot \varphi / ((1 - \varphi) + K \cdot \varphi) = 100 / 100.9 = 99.1\%$. With K = 100 it falls to 91.7% and with K = 10 to 52.6%, which is why the coefficient has to be large before a second phase helps much.

An organic overlay works at 1 L. Why might it fail at 100 m³?

Model answer. Solvent cost and loss grow with volume, and the numbers in problem Q2 show the cost per kilogram can swing from a few cents with reuse to several dollars if the solvent is discarded. An organic layer can also slow oxygen transfer, form emulsions that are hard to break, make the product hard to recover from the solvent, and bring flammability and safety demands. At high partition coefficients the organic-phase concentration may also pass the product's solubility. A cost sketch and a small pilot decide it, not the lab result.

A solvent has a high log P and the product has a high partition coefficient K in it. What does each number tell you?

Both say how much product is captured

✔ The solvent’s log P is a rough guide to whether the cells tolerate it, and the product’s K says how much product it captures

Both say whether the cells tolerate the solvent

Log P sets the product’s titer and K sets the solvent’s price

Why. They answer different questions, and a solvent has to pass both: tolerated by the cells, and a good home for the product. High log P is only a rule of thumb for tolerance, with a grey zone between about 2 and 4 and exceptions, so test the solvent on your own strain.

A product accumulates and growth slows. Which pair of experiments best separates toxicity from an export limit?

Raise the inducer and watch the growth

✔ Spike the product into a culture of the host that does not make it and compare its growth, and compare the intracellular and extracellular amounts in the producer

Sequence the plasmid

Measure only the final titer

Why. The spike test shows whether the product itself is inhibitory at the concentration the cells reach. The inside-against-outside comparison shows whether the product is stuck in the cell. Together they say whether a pump, a membrane change or a second phase is the right lever.

Week 46 — In situ product removal: choosing a strategy, and sorbents

In the published terminology, what does a solid–liquid TPPB use as its second phase?

Cells immobilised in gel beads

✔ Polymer beads or resin that absorb the product

A gas phase

A membrane

Why. The second phase takes up the product. Immobilising the cells is a different technique with different aims, and is covered in its own week. The shared idea in a TPPB is that the product partitions into a phase the cells do not live in.

Why can polymer beads be used with mixed cultures when an organic solvent usually cannot?

Beads are cheaper

✔ A solid polymer is not bioavailable, so no organism in the culture can consume it

Beads dissolve slowly

Solvents cannot absorb products

Why. A solvent can be a carbon source or a toxin for some members of a consortium. A non-biodegradable solid avoids both, which is why the solid–liquid form opened TPPBs to mixed populations.

A sorbent follows a Langmuir isotherm. What sets the sorbent mass you need, and what would make the rate matter instead?

Model answer. Capacity sets the mass. At the aqueous concentration you are willing to let the cells see, each gram of sorbent holds q*(Pa) = qmax·KL·Pa/(1 + KL·Pa), so the mass follows from the product you expect divided by that. In problem S3, 50 g/L of a sorbent with qmax = 0.15 g/g and KL = 8 L/g reached 7.88 g/L total against 2.50 g/L in a single phase, and its ceiling at the inhibition limit was 7.14 g/L. The rate matters only when production outruns uptake: at the standard production rate, uptake constants of 0.3, 2 and 10 per hour all gave the same titer, and only the time to 90% changed, from 17.3 to 16.0 hours. When production doubled, the aqueous concentration was pushed to the inhibition limit and the slow sorbent cost time. So size on capacity, then check the rate against your production rate.

Partition coefficients for one hydrophobic compound measured across several polymers were reported to span roughly:

A factor of two

A factor of ten

✔ Orders of magnitude, from the low hundreds to the thousands

They are identical between polymers

Why. For biphenyl the reported values ranged from about 180 for Nylon 6,6 to about 11,000 for Desmopan, and phenol in silicone oil was near zero. Polymer selection is therefore a screening exercise for each product, not a default.

Your product is a carboxylic acid with a pKa of 4.8 and the fermentation runs at pH 7. Why is an organic overlay a poor first choice?

Model answer. At pH 7 the acid is almost entirely ionised, and the charged form does not partition into a non-polar solvent, so the partition coefficient collapses and the overlay captures very little. The options are to work at a lower pH, if the organism tolerates it, to use an ion-exchange resin that binds the anion, to use a reactive extraction with an amine carrier, or to remove the product by electrodialysis. The lesson is to look at the charge state at the actual pH before choosing a method, which is the same pH-partitioning argument as in week 14.

Uptake of a solute into a polymer bead is usually limited by:

The liquid film around the bead

✔ Diffusion inside the polymer

The stirrer speed

The temperature of the broth

Why. Reported work on Hytrel beads found transport was not externally mass-transfer limited but internally limited by diffusion into the polymer. That makes bead size and polymer choice the levers, not agitation.

Week 47 — Modelling a two-phase fermentation, and fitting it to data

In the transfer term N = kLa·(Pa − Po/K), what is the driving force?

The product concentration in the aqueous phase

✔ The distance of the system from equilibrium between the phases

The concentration in the organic phase

The phase fraction

Why. Transfer stops when Po = K·Pa, which is equilibrium, even though both concentrations are large. Writing the driving force as the concentration itself is a common error that makes a model transfer product for ever.

With fast transfer, a second phase behaves like a single phase whose inhibition limit is multiplied by:

K

φ

✔ 1 + Kφ/(1 − φ)

K/φ

Why. R = 1 + Kφ/(1 − φ) is the ratio of total product to aqueous product at equilibrium. In problem S2 the shortcut and the full model agreed to within 10^{-9} g/L for K = 10 and 100, which is a good check that both are coded correctly.

In the grid of problem S2, the titer at K = 100 and K = 1000 falls as the solvent fraction rises from 5% to 20%. Why?

Model answer. Because at those partition coefficients the process is no longer limited by inhibition but by substrate. The substrate sits in the aqueous phase, so raising φ from 5% to 20% shrinks the aqueous volume per litre of whole broth and therefore the substrate available, and the titer falls from 7.49 to 6.30 g/L. At K = 10 the picture is the opposite, 3.62 rising to 6.30 g/L, because there inhibition still binds and more solvent relieves it. The lesson is to check which constraint is active before adding solvent, and the diagnostic is whether substrate is left at the end.

A two-phase model with a large kLa integrates very slowly or fails with a non-stiff solver. What is happening?

The model is wrong

✔ The system is stiff: the fast exchange between phases forces tiny steps unless a stiff solver is used

The partition coefficient is too small

There is not enough substrate

Why. The exchange relaxes at λ = kLa·(1 + (1 − φ)/(Kφ)), which was 1090 per hour in one case, while the fermentation evolves over tens of hours. A stiff solver such as Radau or LSODA handles the separation of time scales.

You fit two inhibition parameters and the fit reports a correlation of 0.67 between them. What does that mean, and what would you do?

Model answer. It means the data do not separate the two parameters fully: a higher value of one can be partly compensated by a higher value of the other and still fit, so their individual intervals are wider than they look and should not be quoted as independent. In problem S5 the single-phase experiment gave an inhibition limit of 2.53 ± 0.07 g/L and an exponent of 1.09 ± 0.11 against true values of 2.5 and 1.0, with a correlation of 0.67. Adding a second experiment with a second phase tightened the exponent to 1.04 ± 0.05 and raised the smallest singular value of the Jacobian, which is what identifiability means in practice. The remedy is a designed second experiment rather than more points in the same one.

A fermentation model reproduces your data well. What does that justify?

The parameters are the true physical values

✔ The model is adequate for the conditions tested, and its predictions outside them need checking

Further experiments are unnecessary

The mechanism is proven

Why. Fitting shows consistency, not truth. Correlated parameters, a limited range of conditions and assumed yields all mean the model is a summary of what you measured. Predictions beyond that range are hypotheses.

Week 48 — Cell immobilisation: diffusion, dead cores and continuous operation

What does immobilising the cells achieve that in situ product removal does not?

- It removes the product from the broth
- ✔ It retains the biomass, so a reactor can run above the free-cell washout dilution rate
- It raises the partition coefficient
- It prevents product inhibition

Why. Retention is the point: the cells are not washed out, so the reactor can run continuously at a high dilution rate and the biocatalyst can be reused. It does nothing about a product that is toxic in the water around the beads, which is what ISPR addresses.

With oxygen consumed at a constant rate inside a bead, the whole bead stays oxygenated only while:

- The bead is stirred quickly
- ✔ $\varphi^2 = R_v \cdot R^2/(6 \cdot D \cdot Cs) \leq 1$
- The bead is larger than 2 mm
- The substrate concentration is high

Why. In problem T1 that condition gave a maximum radius of about 870 µm, so a bead above roughly 1.7 mm in diameter at 100 kg/m³ of cells carries an anaerobic core. Agitation changes the film outside the bead, not the diffusion inside it.

At a loading of 100 kg/m³, a 2 mm bead has a dead core over about 32% of its radius, yet only about 3% of its volume is dead. Why the difference, and does the small number mean it does not matter?

Model answer. Volume goes as the cube of the radius, so a core at 32% of the radius is $0.32^3 \approx 3\%$ of the volume. It still matters for two reasons. First, the number grows quickly: at 3 mm the core is 61% of the radius and 23% of the volume, and at 200 kg/m³ in a 2 mm bead it is 58% and 20%. Second, a dead core is not merely inactive. It is anaerobic, so it can produce fermentation by-products, lyse, and weaken the bead from the inside. The effectiveness factor only reports the rate loss, not these consequences.

Calcium alginate beads are a poor choice when the medium contains:

- Glucose
- ✔ Phosphate or chelating agents, which strip the calcium that holds the gel together
- Ammonium
- Trace metals

Why. Phosphate and chelators remove the cross-linking calcium and the beads soften and dissolve. This is a routine reason that an alginate process fails in a real medium, and it is why tougher materials such as polyvinyl alcohol are used despite being harder to make.

In problem T2 the immobilised reactor was less productive than free cells. What does that tell you, and what would change it?

Model answer. It tells you that immobilisation is not automatically better. With those assumed parameters, free cells reached about 5.5 g/L/h just below washout, while retained non-growing biomass at a 20% bead fraction and an effectiveness factor of 0.97 reached about 1.5 g/L/h, and that figure barely changed with dilution rate because it is set by the retained biomass and its specific rate. It would change if the retained cells had a higher specific production rate, if the bead fraction were larger (0.40 doubled it to about 3.1 g/L/h), if the product were growth-independent so that non-growing cells lose nothing, or if the free-cell process could not be held near washout in practice. The honest conclusion is to compute the comparison for your own strain rather than assume the technique helps.

Which failure mode is specific to entrapped cells rather than to free cells?

- Contamination
- ✔ The beads breaking apart as the cells grow and produce gas inside them
- Foaming
- Substrate inhibition

Why. Growth and gas inside the matrix build pressure and crack the beads, which releases cells into the broth and slowly turns the process into a free-cell one with extra solids. Monitoring free biomass in the effluent is how it is caught.

Week 49 — Techno-economic analysis and life-cycle assessment

Minimum product selling price is the price at which:

Revenue equals operating cost

✔ Net present value over the plant life is zero at the chosen discount rate

The product undercuts the incumbent

Capital is repaid in one year

Why. It folds capital, operating cost, plant life and the cost of money into one number you can compare against a market price. Quoting a cost of production without saying whether capital is in it is the commonest ambiguity in student TEAs.

In a dilute fermentation, raising titer from 20 to 100 g/L mostly reduces:

Feedstock cost per kilogram

✔ Separation energy and the capital tied to handling water

Labour

The discount rate

Why. Feedstock cost per kilogram is set by yield, not titer, so it barely moves. Water removal and the equipment that handles the volume are what shrink — which is why yield and titer are separate targets with separate fixes.

Your TEA says the process clears the incumbent price. What should a reviewer ask first?

Model answer. Which assumption is carrying the result, and what range was used for it. Usually it is one of: a yield near theoretical rather than observed, a feedstock price from a favourable year, capital scaled from a dissimilar plant, or a co-product credit that assumes a market absorbs the co-product at full price. The defence is a tornado plot showing the conclusion survives a defensible range on each, plus a sentence naming the experiment that would retire the weakest assumption. A single-point TEA with no sensitivity is not evidence.

In a life-cycle assessment, the result is most sensitive to:

The software used

✔ The system boundary and the allocation between co-products

The number of impact categories

The reference year

Why. Move the boundary and the number moves with it: counting biogenic carbon as neutral, or crediting a co-product by displacement rather than mass, can swing the answer twofold. State the boundary before the number, always.

Why does this week matter more for you than for most applicants?

Model answer. Because almost no incoming biology-trained candidate can build one, and it is an artifact a principal investigator can evaluate without a wet lab. It converts the capstone from a plausible pathway into a defensible claim about whether the pathway is worth building, with the sensitive assumptions named. It also happens to be the question funders and reviewers actually ask about a bioprocess, which is why it belongs in the core track rather than the extension.

Week 50 — Analytical method validation — trusting your own numbers

Forcing a calibration curve through the origin is usually wrong because:

It is mathematically invalid

✔ A real blank is rarely exactly zero, and forcing it biases low concentrations

It reduces R^2

Instruments are nonlinear

Why. Background, carryover and baseline offsets all give a non-zero intercept. Forcing zero pushes that error into every low sample — which is where your early-timepoint and low-titer measurements live.

An internal standard mainly corrects for:

Calibration drift only

✔ Variation in extraction recovery, injection volume and matrix suppression

Detector age

Temperature

Why. It travels with the sample through the steps that lose or suppress analyte, so losses affect standard and analyte alike. For mass-spectrometry quantification in broth it is close to mandatory, since ion suppression varies sample to sample.

Why is a titer measured without a recovery experiment weak evidence?

Model answer. Because you do not know what fraction of the product reached the detector. Broth is not water: proteins, salts, sugars and cell debris suppress ionisation, foul columns and bind analyte, and extraction loses material in a way that can differ between samples. A spike-and-recovery experiment in the actual matrix quantifies that, and an internal standard corrects it sample by sample. Without one, a reported titer carries an unknown systematic bias — and the comparison between two strains is only safe if the bias is identical in both, which is precisely what you have not checked.

Reporting a concentration below the limit of quantification as a number is:

Fine if the peak is visible

✔ Not defensible — below that limit the value is detectable but not quantifiable

Standard practice

Only a problem in clinical work

Why. Detection says something is there; quantification says how much, within a stated precision. Reporting the number anyway, or averaging several such numbers, manufactures precision that the method does not have.

Why does this week sit between the bioreactor week and the omics week?

Model answer. Because every number in both rests on it. The kLa calculation, the yield, the flux map, the differential expression list — all are downstream of a measurement whose uncertainty someone had to establish. The tracker tells you which instrument to use and how to interpret the output, but until this week it never asked whether the number going in deserves belief. In a laboratory this is what a careful student is quietly judged on in the first six months.

Week 51 — Omics, screening, and turning data into model constraints

An enzyme-constrained model such as GECKO adds which constraint?

A bound on every metabolite concentration

✔ A limit on total enzyme mass, with each flux costing enzyme in proportion to its turnover number

A thermodynamic direction for each reaction

A maximum growth rate

Why. It turns the proteome budget from week 9 into a constraint the solver obeys, which is why such models predict overflow metabolism — acetate at high glucose — that plain FBA has to be told about.

To use a proteomics dataset as a model constraint rather than a figure, you need:

Only the protein identities

✔ Protein abundances plus turnover numbers, to convert abundance into a flux bound

A transcriptome as well

Nothing further

Why. Abundance alone says how much enzyme is present, not how much flux it can carry. The turnover number converts one into the other, and missing or unreliable k_cat values are the main source of error in these models.

A strain grows well but makes almost no product. Which omics layer would you measure first, and why?

Model answer. Proteomics, or a targeted measurement of the pathway enzymes, first: the commonest cause is that an enzyme is not really being made — poor expression, misfolding, a missing cofactor — and that question is answered directly and cheaply. If the enzymes are there, measure metabolites next to find where the intermediate piles up, which localises the blocked step. Transcriptomics is the weakest first move, because a transcript present does not mean a working enzyme. Fluxomics comes last, when you need to know where the carbon went rather than why.

Week 52 — Optional — host defences and pooled screens

A construct that works in E. coli disappears when you transform a wild industrial strain. The most likely cause is:

The origin of replication is wrong

✔ The strain’s restriction–modification system recognises unmethylated motifs in your construct

The antibiotic is wrong

The promoter is inactive

Why. Restriction barriers are the classic reason a construct that works in E. coli vanishes in a non-model host. Searching REBASE for the strain’s systems and matching their motifs against your sequence takes an afternoon and saves months.

A 6 bp palindromic recognition site is expected how many times, on average, in a 6 kb plasmid?

About 0.1

✔ About 1.5

About 23

About 150

Why. 6000 / 4^6 = 1.46. The chance of no site at all is e^−1.46, about 23%, and with three independent 6 bp systems it falls to about 1% — which is why wild strains destroy most foreign DNA.

How would you get a construct past a strain’s restriction barriers?

Model answer. Identify the systems first: search the genome against REBASE for restriction endonuclease and methyltransferase genes, and find their recognition motifs. Then either make the DNA look like the host’s own — express the host methyltransferases in your cloning strain so the plasmid is methylated before transfer, or use a helper plasmid carrying them — or remove the motifs from the construct where the sequence allows. Supporting measures: heat treatment to transiently inactivate restriction activity, passaging through a methylation-matched host, and conjugation rather than electroporation. Confirm by comparing transformation efficiency of methylated and unmethylated preparations of the same plasmid.

A pooled barcode screen reports a gene as a strong hit. That is:

A validated phenotype

✔ A hypothesis about a gene, to be confirmed in an isolate

Proof of essentiality

A sequencing artefact

Why. Pooled fitness reflects competition in a mixed population, where cross-feeding, bottlenecks and barcode mapping errors all intervene. Reconstructing the mutant alone and measuring its phenotype directly is what turns a hit into a result.

This week is marked optional. When should you actually do it?

Model answer. Do the host-defence sessions if your capstone or a target lab uses a non-model chassis — an acetogen, a thermophile, a halophile, a non-conventional yeast — because restriction barriers are the first practical obstacle there and the analysis is cheap. Do the pooled-screen sessions if a target lab publishes RB-TnSeq or CRISPRi-seq work. Neither belongs in the core track, and neither is what an admissions committee will ask about. Automation, which used to sit in this week, now has its own week.

Week 53 — Pharmaceuticals, terpenoids and natural products

The semi-synthetic artemisinin process produced which molecule by fermentation?

Artemisinin itself

✔ Artemisinic acid, at about 25 g/L

Amorphadiene, at 250 g/L

Dihydroartemisinin

Why. Yeast made artemisinic acid; a chemical, partly photochemical, step converted it to artemisinin. “Semi-synthetic” names exactly that division of labour between cell and flask.

Complete opioid biosynthesis in yeast required roughly how many engineered genes?

Three

Seven

✔ Twenty-one to twenty-three

Over a hundred

Why. Galanie et al. assembled genes from plants, bacteria and rat. Titers were minute, but the point was that a pathway of that length could be reconstituted at all.

Why did semi-synthetic artemisinin struggle commercially despite working technically?

Model answer. Plant-derived artemisinin remained price-competitive, agricultural supply and price recovered, and the fermentation route carried capital and feedstock costs it could not shed. Technical success and market success are separate questions — a lesson worth stating plainly in an interview.

Week 54 — Commodity chemicals, bioplastics and amino acids

C. glutamicum suits amino acids partly because it:

Grows anaerobically

✔ Natively secretes its products into the medium at high titer

Needs no nitrogen source

Stores them as granules

Why. Secretion is why titers above 200 g/L are recoverable at all: the broth is harvested without destroying the biomass. Compare PHA in Cupriavidus necator, stored internally as granules and recovered only by lysis.

DuPont's bio-propanediol process uses:

Native Clostridium on glycerol

✔ Engineered E. coli converting glucose directly to 1,3-propanediol

Yeast on xylose

A cell-free system

Why. Yeast glucose-to-glycerol genes were combined with bacterial glycerol-to-PDO genes in one host, and sugar uptake was re-engineered for energy efficiency. The product feeds Sorona polymer.

What makes 1,4-butanediol a landmark case?

It is a natural product

✔ It is a non-natural product whose pathway was designed computationally

It needs no host

It was discovered by screening

Why. No organism makes it. A pathway-prediction algorithm plus a genome-scale model and OptKnock produced the strain — model-driven design taken all the way to commercial scale with Novamont.

Why has C. glutamicum held the amino-acid market for sixty years?

Model answer. It secretes amino acids naturally, tolerates high titers, has GRAS status and a mature genetic toolkit, resists phage, and performs robustly on cheap feedstocks at scale. Engineered lysine strains exceed 200 g/L at carbon yields around 0.65 g/g — a bar most hosts cannot approach.

Week 55 — Biofuels, C1 and CO_2 utilisation, food ingredients

LanzaTech's commercial process ferments:

Glucose with yeast

✔ CO, CO_2 and H_2 with Clostridium autoethanogenum

Methanol with Pichia

Cellulose with Trichoderma

Why. Steel-mill off-gas and gasified waste become ethanol and, increasingly, other products — the flagship of circular-carbon biomanufacturing, running at commercial scale since 2018.

Impossible Foods' heme protein is:

Bovine myoglobin from CHO cells

✔ Soy leghemoglobin from Komagataella phaffii

Porcine haemoglobin from E. coli

Synthetic haem from a cell-free system

Why. A plant gene expressed in an industrial yeast, cleared by FDA GRAS notice in 2018 — the breakthrough consumer precision-fermentation product.

Explain the logic of the non-fermentative higher-alcohol route to isobutanol.

Model answer. Instead of extending a fermentative acyl chain, it hijacks branched-chain amino-acid biosynthesis: 2-ketoacids are decarboxylated and reduced by the Ehrlich degradation route to the corresponding alcohol. Because ketoacids of several chain lengths exist, the same logic yields a family of alcohols — 22 g/L isobutanol at 86% of theoretical in the original work.

Week 56 — PHA — from pathway to polymer

In Cupriavidus necator, which enzyme converts acetoacetyl-CoA to 3-hydroxybutyryl-CoA using NADPH?

PhaA

✔ PhaB

PhaC

PhaP

Why. PhaA condenses two acetyl-CoA, PhaB reduces acetoacetyl-CoA with NADPH, and PhaC polymerises the product. The NADPH requirement at PhaB is why redox supply matters for PHB yield.

What is the theoretical mass yield of PHB on glucose through acetyl-CoA, ignoring growth and maintenance?

About 0.25 g/g

✔ About 0.48 g/g

About 0.75 g/g

About 1.0 g/g

Why. 86.1 g of PHB unit per 180.2 g of glucose is 0.478 g/g. One third of the carbon leaves as CO_2 at pyruvate dehydrogenase, so the carbon yield is 4 of 6. Real yields are lower because carbon also goes to cells and maintenance.

A strain reaches 70% PHB of cell mass but its volumetric productivity is low. What are the likely causes, and why is content alone a poor target?

Model answer. Content is a ratio, so a culture can reach a high percentage with little biomass and therefore little polymer per litre. Productivity is polymer per litre per hour, which depends on how much biomass you grew before limitation and how fast it accumulates afterwards. In the invented table of problem K2, residual biomass stopped at 43 g/L at 36 h and PHB reached 70 g/L by 60 h, 62% of cell mass, at 1.16 g/L/h overall. To raise productivity you grow more cells first, then limit the nutrient, or raise the specific rate during accumulation. A process is judged on content, productivity and yield together.

Why is a surplus of reduced cofactors a design problem when growing PHB-producing cells on sugar?

Model answer. Glycolysis and pyruvate dehydrogenase make four NAD(P)H per glucose and PhaB consumes only one, leaving three that must be re-oxidised. In a respiring cell that means oxygen: about 1.5 mol O_2 per glucose, or 0.56 kg O_2 per kg PHB at the stoichiometric minimum. So the sugar route carries an aeration cost that scales with productivity, which is why oxygen transfer and the heat it releases appear in a PHA process design from the start.

Which technique reports the molecular weight distribution of the recovered polymer?

GC after methanolysis

DSC

✔ GPC (size-exclusion chromatography)

Plate-reader absorbance

Why. GC measures content and composition, DSC reports melting and glass transition, and GPC gives molecular weight. Material properties depend on molecular weight as much as on composition, so a titer alone does not describe a PHA product.

In a lignocellulosic hydrolysate, which of these is a typical inhibitor of microbial growth?

Glucose

✔ Furfural

Ammonium

Phosphate

Why. Furfural and HMF form during pretreatment, along with acetate and phenolics. A strain described as robust for hydrolysates tolerates these, which is as important as how well it makes the polymer.

Week 57 — CO_2, hydrogen and waste carbon as feedstocks

The equation $4\ CO_2 + 9\ H_2 \rightarrow C_4H_6O_2 + 6\ H_2O$ gives:

The full hydrogen demand of the route, including energy

✔ The stoichiometric minimum for reducing equivalents, excluding the ATP needed to fix CO_2

The oxygen demand

The yield on sugar

Why. It balances atoms and counts only the reducing power. Fixing CO_2 also costs ATP, supplied by respiring additional hydrogen, so the real demand is higher: about 0.43 kg H_2 per kg PHB at the assumed ATP yield, against 0.21 kg for the floor.

Why does respiring hydrogen matter for a CO_2-fixing PHB producer?

It removes toxic CO_2

✔ It supplies the ATP the Calvin cycle needs

It lowers pH

It makes the polymer longer

Why. Fixing CO_2 costs 3 ATP per molecule, partly repaid at triose phosphate to pyruvate. The net 14 ATP per PHB unit has to come from somewhere, and a hydrogen-oxidising cell gets it by respiring hydrogen with oxygen.

Carbon dioxide is free, so why can the CO_2 route lose to sugar on cost?

Model answer. Because the feedstock is not the CO_2 but the electricity that makes the hydrogen. Under the assumptions of problem K3 (52 kWh per kg H_2, 1.5 ATP per H_2 respired), a kilogram of PHB needs about 0.43 kg H_2, or 22 kWh, which is \$0.89 at \$0.04/kWh, \$1.79 at \$0.08 and \$2.68 at \$0.12. Glucose at \$0.40/kg costs \$0.84 per kg PHB at the theoretical yield and \$1.33 at 0.30 g/g. The break-even electricity price is therefore about \$0.038/kWh against the ceiling yield and \$0.060 against a realistic one — before electrolyser capital, gas handling and downstream, which all favour sugar further. The route wins where electricity is very cheap or where carbon-neutral supply is itself valued.

In the hydrogen route modelled in problem K4, with oxygen held at 5% of the gas, which gas sets the required kLa?

Hydrogen, because it is the larger molar flow

✔ Oxygen, because its driving force is so small at low fraction

CO_2

Neither: both are equal

Why. Hydrogen uptake is four times the oxygen uptake in moles, but hydrogen's driving force at 0.7 atm is about ten times that of oxygen at 0.05 atm, so oxygen needs about 1,140 per hour against hydrogen's 450 at 1 g/L/h. The safety constraint on oxygen fraction is what makes oxygen the bottleneck.

State the safety principle that shapes the design of a gas fermentation on hydrogen and oxygen, and who decides.

Model answer. Mixtures of hydrogen and oxygen are flammable over a wide range, so the process is designed so that the gas stays outside the flammable region, typically by limiting the oxygen fraction below the limiting oxygen concentration for the mixture and by controlling pressure and ignition sources. The illustrative 5% used in problem K4 is a modelling assumption and not a safe operating condition: limits depend on the exact mixture, temperature and pressure. A formal hazard review by the institution's safety office, with the reactor design and the gas composition in front of them, has to approve the work before it starts.

In the lab's 2025 plastic-waste work, as described by its title, polyethylene is upgraded to a biodegradable polymer by:

Direct fermentation of polyethylene

✔ A chemical oxidation step followed by biological conversion

Burning and capturing CO_2

Enzymatic hydrolysis alone

Why. The title names a hybrid chemical oxidation–biological conversion approach. Polyethylene is inert to most microbes, so a chemical step produces something cells can use before the biological step makes the polymer.

Week 58 — Yeast hosts: S. cerevisiae, Y. lipolytica and K. phaffii

Why does lipid production in yeast depend on cytosolic rather than mitochondrial acetyl-CoA?

Mitochondria do not make acetyl-CoA

✔ Fatty acid synthesis happens in the cytosol, and acetyl-CoA does not cross the mitochondrial membrane freely, so it is exported as citrate and cleaved by ATP-citrate lyase

Cytosolic acetyl-CoA is cheaper to make

The mitochondrion has no carbon

Why. The citrate shuttle costs about one ATP per acetyl unit at the lyase step, so roughly 8 extra ATP per palmitate and 24 per triacylglycerol. Raising cytosolic acetyl-CoA supply is usually the first target in an oleaginous strain.

Why does overexpressing fatty acid synthase alone rarely raise lipid titer?

Model answer. Because the limit is usually reductant, not enzyme. Palmitate needs 14 NADPH and a triacylglycerol about 42, and the oxidative pentose phosphate pathway supplies only 2 NADPH per glucose that passes it. On top of the 4 glucose needed for the carbon of one palmitate, roughly 7 more would be needed purely for NADPH if the PPP were the only source. That is why oleaginous yeasts rely on malic enzyme or transhydrogenase-like cycles, and why cofactor supply, not the synthase, is the target. The carbon ceiling is separate: only 4 of glucose's 6 carbons reach acetyl-CoA, which caps lipid yield near 0.3 g/g.

S. cerevisiae is Crabtree-positive. What does that mean for a product other than ethanol?

It cannot grow aerobically

✔ It ferments sugar to ethanol even with oxygen present, so carbon is lost from your pathway unless that flux is rewired

It cannot use glucose

It stores lipid naturally

Why. Ethanol formation at high sugar competes directly with any pathway you install. Y. lipolytica is Crabtree-negative, which is one reason it is chosen for lipid and organic acids despite a thinner toolkit.

K. phaffii is used mainly for:

Lipid production

✔ Secreted recombinant protein production at high cell density

Natural products from plants

Gas fermentation

Why. Strong inducible promoters, growth to high density on glycerol or methanol, and efficient secretion make it a protein platform. It is not an oleaginous yeast.

What does compartmentalising a pathway into a peroxisome or mitochondrion buy, and what does it cost?

Model answer. It buys a high local concentration of precursors and enzymes, separation from competing cytosolic reactions, and containment of a toxic or reactive intermediate so the rest of the cell is protected. It costs transport: every substrate, product and cofactor must cross a membrane, which may need a carrier that does not exist, and the pathway's cofactor pool inside the organelle may differ from the cytosolic one. It also complicates measurement, because a whole-cell extract averages compartments that are not at the same concentration.

Week 59 — Gram-positive hosts: C. glutamicum and B. subtilis

How much NADPH does one mole of L-lysine require in C. glutamicum?

1 mol
2 mol
✔ 4 mol
8 mol

Why. Four, reported consistently across the C. glutamicum literature. That demand is why the oxidative pentose phosphate pathway carries a large share of glucose flux in producing strains, and why carbon yield and reductant supply compete.

The reported maximum theoretical yield of L-lysine on glucose in native C. glutamicum metabolism is about:

0.30 mol/mol
0.50 mol/mol
✔ 0.81 mol/mol
1.0 mol/mol

Why. About 0.81 mol per mol, which is 0.657 g/g, rising to about 0.86 mol/mol if a transhydrogenase activity is added. The 1:1 carbon bound would be 0.811 g/g, so even the theoretical maximum gives up about a fifth of it.

Why does secretion matter so much that it can decide host choice?

Model answer. Because a product that stays inside the cell is capped by cell volume. At a billion cells per millilitre and about 1 femtolitre a cell — the round figure for E. coli, whose measured volume spans roughly 0.4 to 1.8 μm^3 — the intracellular volume is only about 0.1% of the broth, so even 300 g/L inside the cells is just 0.3 g/L in the broth. A secreted product has no such ceiling, and recovery is a clarification step instead of lysis, debris removal and a purification contaminated by every intracellular protein. For amino acids and industrial enzymes this is the main reason C. glutamicum and B. subtilis are used rather than E. coli.

Which is a genuine disadvantage of B. subtilis for a heterologous protein?

It cannot secrete proteins
✔ Its native proteases in the medium can degrade the product, which is why protease-deficient strain backgrounds exist
It cannot be transformed
It has an outer membrane

Why. Secretion is its strength; the proteases that come with it are the problem. Sporulation, phage sensitivity and foaming are the other routine failure modes.

When is a Gram-positive host the wrong choice?

Model answer. When the product needs eukaryotic post-translational processing, such as complex glycosylation or disulfide-rich folding in an endoplasmic reticulum, which neither host provides. When the pathway depends on tools or parts that exist only in E. coli or yeast, so the development time outweighs the downstream advantage. When the product is best held in a periplasm, which Gram-positive bacteria do not have. And when the strain collection, the models and the published precedent for your particular chemistry all sit in another host, since starting from nothing in a less-studied organism is itself a multi-year cost.

Week 60 — Phototroph hosts: cyanobacteria and microalgae

In a culture at 1 g/L with a biomass attenuation coefficient of 0.2 m^2/g, light falls to 1% of the surface value at a depth of about:

2.3 m
23 cm
✔ 2.3 cm
2.3 mm

Why. About 0.023 m. In a 30 cm pond that is under 4% of the depth, so the overwhelming majority of the cells are in darkness at any moment and only mixing brings them into the light.

Why is areal productivity the right unit for a phototroph, and what does the arithmetic show?

Model answer. Because the culture is limited by the photons arriving on a surface, not by the volume underneath it. Once a pond is dense enough to absorb essentially all the incident light, adding more cells moves them into the dark and the areal rate stops rising: in the model it saturates near 21 g per square metre per day from about 0.1 g/L upward, while the volumetric rate falls steadily. So raising cell density does not raise output, it only makes harvesting cheaper. What you are really buying is illuminated area, and at about 21 g/m²/day over 300 days that is roughly 64 tonnes per hectare per year of dry biomass, on assumed parameters.

Why is dewatering such a large cost for microalgae?

The cells are chemically unstable

✔ The culture is dilute, so a kilogram of dry biomass is suspended in hundreds to thousands of kilograms of water

The water is toxic

Centrifuges cannot handle algae

Why. At 1 g/L there are 999 kg of water per kg of biomass. Evaporating it would take about 2,258 MJ against a biomass energy content near 20 MJ/kg, which is why harvesting is staged through settling or flocculation, then centrifugation, and why bulk fuel products struggle.

What makes a knockout harder in Synechocystis than in E. coli?

It cannot be transformed

✔ It carries many copies of its chromosome, so a mutation must be driven to full segregation across all of them

It has no homologous recombination

It grows too fast

Why. Polyploidy means a transformant is initially a mixture of wild-type and mutant chromosomes, and incomplete segregation is a common reason an apparent knockout still shows the original phenotype.

For which products does a phototroph make economic sense, and for which does it not?

Model answer. It makes sense where the product's value per kilogram is high enough to absorb the harvesting and area costs, such as astaxanthin and other pigments, nutraceuticals, and biomass sold as food or feed; and where carbon capture or wastewater treatment is itself part of the value. It does not make sense for commodity fuels, where published cost analyses put minimum selling prices several times the petroleum-derived alternative, and the combination of slow growth, self-shading, dilute culture and dewatering energy is why. The general test is to compare the value per kilogram with the areal productivity and the harvesting cost before choosing the host, rather than after.

Compared with E. coli, the design–build–test cycle in a phototroph is slower mainly because:

Plasmids do not exist

✔ Growth is measured in hours to days per doubling rather than tens of minutes, and segregation or selection takes longer still

Sequencing is impossible

There are no selectable markers

Why. Every round of building and testing waits on growth, so a project that would take weeks in E. coli takes months. That is a cost to weigh before choosing the host, as the toolkit break-even calculation in the genome-design week does for any non-model organism.

Week 61 — Hosts panorama, cell-free systems, 2025–2026 trends

In a dense phototrophic culture, light intensity falls with depth as:

Linearly with depth

✔ $I(z) = I_0 \cdot \exp(-k \cdot X \cdot z)$, so biomass itself causes the gradient

Not at all if the vessel is stirred

Only at the surface

Why. Beer–Lambert with biomass as the absorbing species. It is the photobioreactor's exact analogue of the oxygen gradient in a deep fermenter: the culture creates its own limitation as it grows, so productivity per litre falls as density rises.

What does light being the substrate change about modelling a cyanobacterium?

Model answer. The substrate uptake bound is no longer a number you set once: it depends on position in the vessel and on the culture's own density, so a single photon exchange bound represents an average over a gradient and will overestimate productivity in dense culture. The model also needs the energetic cost of the carbon-concentrating mechanism and RuBisCO's poor specificity, including photorespiration, which have no counterpart in a heterotroph. And because carbon enters only as CO_2, the culture rarely reaches isotopic steady state, which is why flux work on phototrophs needs the non-stationary ^{13}C-MFA from week 43.

Which host and product pair is the clearest case of intracellular storage?

Lysine in C. glutamicum

✔ PHA in Cupriavidus necator

Ethanol in S. cerevisiae

Leghemoglobin secreted by K. phaffii

Why. PHA accumulates as distinct granules inside the cell, so recovery means lysis and polymer extraction. The other three leave the cell, by secretion or by diffusion.

For a secreted protein at industrial scale, the usual first-choice host is:

E. coli

✔ Komagataella phaffii

Cyanobacteria

CHO

Why. It secretes to high titer in defined medium with little native secreted protein to separate from. CHO is for therapeutic proteins needing human-like glycosylation; E. coli struggles with secretion and disulfides.

The main advantage of a cell-free system is:

Lower cost at scale

✔ No viability constraint, so toxic pathways and fast prototyping become possible

Automatic purification

Higher yields always

Why. Nothing has to stay alive and nothing is diverted to biomass, and you can open the reaction to add or sample anything. The trade-off is cofactor regeneration and cost per litre.

Choose a host for an oleochemical and defend it against one alternative.

Model answer. Yarrowia lipolytica: native high lipid flux, strong acetyl-CoA and NADPH supply, tolerance of hydrophobic substrates and products, and a maturing toolkit. Against S. cerevisiae it wins on native lipid capacity; against it, S. cerevisiae has deeper tools and a better scale-up record — so the honest answer names what you give up.

Week 62 — How publishing works: authorship, peer review and the record

Under common authorship criteria, who qualifies as an author?

Anyone who worked in the lab that year

✔ Someone who contributed substantially to the work and also to drafting or revising it, approves the final version, and accepts accountability

Only the person who ran the experiments

Whoever funded the work

Why. Standards such as the ICMJE criteria require a substantial contribution, involvement in writing or revising, approval of the final version, and accountability for the work. Providing funding or a reagent alone is usually acknowledged rather than credited with authorship.

Why is reading a lab’s publication record part of choosing where to apply?

Model answer. Because it predicts your own years there better than the research description does. The number of papers tells you the pace, the venues tell you the standard the group holds itself to, and how often a student appears as first author tells you whether students lead projects or support someone else’s. A group that publishes steadily with student first authors is training students to run work; a group where only postdocs lead may be a different experience. It also gives you something specific to ask: what a student there is expected to produce, and over what timeframe.

A reviewer makes a criticism you believe is factually wrong. What is the right response?

Ignore it

✔ Address it directly and politely, explaining with evidence why you disagree, and say whether anything in the manuscript changed as a result

Withdraw the manuscript

Make the change anyway even though it is wrong

Why. Every point gets a response. Editors read the response letter to see whether criticisms were handled, and a well-argued disagreement backed by evidence is a normal and acceptable outcome.

What is salami slicing?

Separating the figures from the text

✔ Splitting one body of work into the largest possible number of minimal papers

Reviewing your own paper

Publishing in two languages

Why. It inflates a publication count while making each piece harder to interpret on its own. It is discouraged for the same reason selective reporting is: it distorts the record in the author's favour.

Why does a field suffer when negative results go unpublished?

Model answer. Because the published record then over-represents effects that appeared to work, so meta-analyses are biased, other groups repeat experiments that have already failed elsewhere, and apparent effects that were statistical accidents survive unchallenged. For a student it also distorts expectations, since the literature makes everything look like it worked the first time. The practical habits that follow are to record what did not work, to report it in the paper rather than omit it, and to read a negative result from another group as useful information rather than as a failure.

Week 63 — Writing and talking about your work

A figure caption should let a reader:

Admire the design

✔ Understand the figure without the main text, including n and what error bars show

Repeat the experiment

Find the raw data

Why. Many readers only look at the figures. A caption that omits n and the meaning of the error bars leaves the central evidence uninterpretable, which is a surprisingly common failure in otherwise careful papers.

In a ten-minute talk, the limitation of your work is best raised:

Only if asked

✔ By you, before anyone else raises it

In the written version only

Never

Why. Naming it yourself shows you understand your own evidence and sets the terms of the discussion. Waiting to be caught converts a strength into a weakness, and the audience was going to find it anyway.

You are asked a question you cannot answer. What do you say?

Model answer. Say that you do not know, say what you would need to find out, and say why the question matters — then stop. Improvising an answer is the single most damaging thing you can do in an interview or a lab meeting, because an expert will know immediately, and it puts every other claim you made in doubt. Being able to say "I do not know, here is how I would check" is read as scientific maturity rather than ignorance, and it is a rehearsable skill rather than a character trait.

Why does a communication week belong in a technical curriculum?

Model answer. Because the output of a PhD is not strains or models, it is claims other people can evaluate — papers, talks, a thesis, grant text. The tracker produces a proposal, notebooks and a portfolio, and every one of them is judged on whether a reader can find the argument. It is also the part of the application you control most directly: the statement of purpose and the interview are communication tasks, not technical ones, and they decide more outcomes than another modelling technique would.

Week 64 — Planning a multi-year project: time, money and what you cannot control

A task is not on the critical path and has four weeks of float. You make it two weeks faster. What happens to the project end date?

It moves two weeks earlier

✔ Nothing

It moves four weeks earlier

It depends on the budget

Why. Nothing, because the end date is set by the critical path. Effort spent speeding up a task with float is wasted unless the float is nearly used up, which is why the path has to be identified before deciding where to push.

A plan built from most-likely durations came to 31 weeks, but a simulation put the median at 40 weeks and the chance of finishing within 31 weeks at essentially zero. Why?

Model answer. Because task durations are not symmetric: a task can finish a little early but can overrun badly, so each one is skewed long. Adding up most-likely values therefore produces a total that is close to the best case rather than the middle. In the simulation, with each task between 0.8 and 2 times its most-likely duration, 80% of runs needed 43 weeks and 95% needed 45. The practical conclusion is to commit to a percentile rather than to the sum of most-likely values, to place a single buffer at the end rather than pad each task, and to say which number you are quoting.

In the simulated project, which task most deserved attention?

The shortest one

✔ The one that lay on the critical path most often across the simulations

The most technically interesting one

The cheapest one

Why. Some tasks were on the critical path in every run, while the cloning chain was on it 66% of the time and the parallel assay development 34%. A task that is frequently critical is where a delay most often becomes a project delay.

Why does naming risks in a research proposal strengthen it?

Model answer. Because the reviewer already knows the project can fail, and a proposal that does not mention it reads as either naive or evasive. Naming the risks shows you understand what the work depends on, and pairing each with an alternative shows the project survives a setback rather than ending with it. It also forces you to define go/no-go points in advance, which is what stops a student spending two years on an approach that stopped being promising in the third month. For a candidate from industry it also puts experience on display that most applicants do not have.

Halving a four-week synthesis wait shortened a 31-week plan by about:

Eight weeks

✔ One to three weeks

Nothing

Half the project

Why. Around one week on the deterministic plan and about two on the simulated median, because once that wait shrinks another branch becomes limiting. It is still real time for little effort, which is the general lesson: attack waiting before trying to work faster.

Week 65 — Capstone build — the mini research proposal

State your capstone target, host, theoretical yield and top two risks, in under a minute.

Model answer. Your own answer. It should contain a number for the theoretical yield, one sentence of host justification tied to a property rather than popularity, and two risks that are specific — a toxic intermediate, a cofactor deficit, a missing enzyme, a separation problem — rather than generic uncertainty.

Which section of your proposal is weakest, and what single piece of evidence would fix it?

Model answer. Your own answer. The commonest weak section is the build-and-test plan, because it is the furthest from a chemical engineer's training. The usual fix is naming the specific assembly method, editing tool, analytical method and screening readout, with a reason for each.

In a research proposal, the strongest justification for a host choice is:

It is the most widely used

✔ A specific property that the target pathway requires

It was used in a famous paper

It grows fastest

Why. Name the property — P450 support, native lipid flux, GRAS status, gas uptake, secretion capacity — and the alternative you rejected. That is the difference between a choice and a default.

Week 66 — Applications — labs, statement of purpose, interviews

Answer in sixty seconds: why metabolic engineering, and why you, given a chemical engineering degree?

Model answer. Your own answer, and it should be concrete. The field was founded by chemical engineers; stoichiometric balances, kinetics, reactor design, transport and separations transfer directly; and you closed the biology gap deliberately — point at the notebook, the reproduced design, the proposal. Enthusiasm without artifacts is the weak version of this answer.

Name three principal investigators, one recent paper each, and how your proposal touches their work.

Model answer. Your own answer. If you cannot name the paper and its key figure, the lab is not really on your shortlist yet.

In a statement of purpose, the strongest evidence of preparation is:

A list of courses taken

✔ Concrete artifacts tied to a specific lab's recent work

Enthusiasm for the field

A high grade average

Why. Committees have read every version of “I am passionate about sustainability”. They have read far fewer that say “I reproduced your 2024 growth-coupled design in COBRApy and here is where it broke”.

Worked problems

56 problems · full solutions and runnable code

Every number in these solutions was produced by the code beside it. Work each one on paper or in Python first; reading the solution first costs you the only thing these are for.

Building the balance

Where every model starts: write the stoichiometry, count what you can and cannot know from it.

A1 · Build S by hand, and find what extracellular data cannot tell you

week 13 · 45 min

A cell takes up substrate and makes product through this network:

v1: S_ext → A (uptake, measured at 10 mmol/gDW/h)

v2: A → B

v3: A → C

v4: B → P_ext (excretion, measured at 10 mmol/gDW/h)

v5: C → B

A, B and C are intracellular. S_ext and P_ext are outside the balance.

1. Write the stoichiometric matrix S with one row per intracellular metabolite and one column per reaction.
2. Write the three steady-state balances explicitly.
3. Find the dimension of the null space.
4. Apply the two measured fluxes. How many degrees of freedom remain, and what does the remaining one mean physically?

Worked solution

S has three rows (A, B, C) and five columns:

A: [1, −1, −1, 0, 0]

B: [0, 1, 0, −1, 1]

C: [0, 0, 1, 0, −1]

The balances are v1 − v2 − v3 = 0, v2 + v5 − v4 = 0, and v3 − v5 = 0.

S has rank 3, so the null space has dimension 5 − 3 = 2. Two fluxes must be supplied from outside the stoichiometry.

Now substitute. From the C balance, v5 = v3. From the A balance, v2 = v1 − v3. Put both into the B balance: v4 = (v1 − v3) + v3 = v1. So v4 = v1 identically — your second measurement was not independent, it was a consistency check, and it passed.

One degree of freedom remains: v3 is free. The route A → C → B carries any flux at all without changing a single measurable quantity outside the cell. No amount of uptake and excretion data will ever resolve it.

That is the exact situation that makes ^{13}C labelling necessary rather than optional, and problem C1 resolves this kind of split. It is also why an FBA solution with a wide flux variability range on a parallel route is telling you the data are silent there, not that the flux is zero.

Code

```
import numpy as np
from scipy.linalg import null_space

S = np.array([[ 1,-1,-1, 0, 0],   # A
              [ 0, 1, 0,-1, 1],   # B
              [ 0, 0, 1, 0,-1]])  # C

print('rank', np.linalg.matrix_rank(S))        # 3
N = null_space(S)
print('null space dimension', N.shape[1])      # 2
```

```
print(np.round(N, 3))
# add the measurements as extra rows: v1 = 10, v4 = 10
A = np.vstack([S, [1,0,0,0,0], [0,0,0,1,0]])
b = np.array([0,0,0,10,10])
print('rank with measurements', np.linalg.matrix_rank(A))   # 4, not 5
print('remaining degrees of freedom', 5 - np.linalg.matrix_rank(A))
```

A2 · Maximum theoretical yield, closed on both carbon and redox

week 19 · 45 min

Anaerobic succinate from glucose in E. coli. The reductive branch is:

PEP + CO_2 → oxaloacetate

oxaloacetate + NADH → malate

malate → fumarate + H_2O

fumarate + NADH → succinate

Glycolysis delivers 2 PEP and 2 NADH per glucose. Assume no oxygen and no other electron sink.

1. Count NADH supply and NADH demand for converting both PEP to succinate.
2. State the yield the redox balance permits through this route alone.
3. Explain how published maxima exceed it, and give the number.

Worked solution

Demand: 2 NADH per succinate, so 4 NADH to convert both PEP. Supply: 2 NADH from glycolysis. The route is short by 2 NADH per glucose.

So the reductive branch alone is redox-limited to 1 mol succinate per mol glucose — half the carbon maximum. Carbon is not the constraint here; electrons are. Note also that this route fixes CO_2, so the carbon yield can exceed what the six carbons of glucose would suggest.

The published anaerobic maximum is 12/7 = 1.71 mol/mol, equal to 1.12 g/g, reached by running part of the carbon through the oxidative TCA branch or the glyoxylate shunt to generate the extra reducing equivalents, at the cost of losing some carbon as CO_2. That combination of two routes with opposite roles — one for carbon, one for electrons — is exactly the kind of design an optimisation algorithm finds and an unaided guess does not.

The general method, worth repeating for your own target: write the synthesis equation, close carbon, then close NADH and NADPH separately, then ATP. Whichever closes last is your real constraint, and it is very often not carbon.

Kinetic and dynamic modelling

Rate laws to differential equations to a fitted, interrogated model.

B1 · Fit Michaelis–Menten to real data, and see where the data fail you

week 3 · 1 h

Initial-rate data for one enzyme. [S] in mM, v in µmol·min^{-1}·mg^{-1}:

S: 0.05, 0.10, 0.20, 0.40, 0.80, 1.60, 3.20, 6.40

v: 0.300, 0.538, 0.866, 1.246, 1.647, 1.911, 2.167, 2.367

1. Fit v = V_max[S]/(K_M+[S]) by non-linear least squares and report both parameters with standard errors.
2. Refit using only the four highest concentrations and compare.
3. Say which parameter the low-[S] points constrain, and which the high-[S] points constrain.

Worked solution

Full dataset: V_max = 2.44 ± 0.03 µmol·min^{-1}·mg^{-1} and K_M = 0.379 ± 0.020 mM.

High-[S] points only: V_max = 2.49 ± 0.06 and K_M = 0.435 ± 0.055. V_max barely moves; the uncertainty in K_M nearly triples.

The reason is structural. The high-[S] points pin the asymptote, so they constrain V_max. The curvature near and below K_M is what identifies K_M, so dropping those points leaves it poorly determined. This is practical identifiability, met early: the model is fine, the experiment was not designed to constrain that parameter.

Two habits to take from this. Always report standard errors from the covariance matrix, not just point estimates. And design the concentration series to straddle K_M — roughly 0.2 to 5 times it — rather than spreading points evenly, which is how most first datasets are wasted.

Code

```
import numpy as np
from scipy.optimize import curve_fit

S = np.array([0.05,0.10,0.20,0.40,0.80,1.60,3.20,6.40])
v = np.array([0.300,0.538,0.866,1.246,1.647,1.911,2.167,2.367])

mm = lambda s, Vmax, Km: Vmax*s/(Km+s)

p, cov = curve_fit(mm, S, v, p0=[1,1])
se = np.sqrt(np.diag(cov))
print('Vmax = %.3f +/- %.3f' % (p[0], se[0]))
print('Km   = %.3f +/- %.3f' % (p[1], se[1]))

p2, cov2 = curve_fit(mm, S[4:], v[4:], p0=[1,1])
print('high-S only:', np.round(p2,3), '+/-', np.round(np.sqrt(np.diag(cov2)),3))
```

B2 · Build a three-enzyme pathway from scratch, integrate it, and measure its control

week 16 · 2 h

S → A → B → P. Substrate S is held at 10 mM. Enzyme 1 is feedback-inhibited by B.

$v1 = V1 \cdot S/(K1+S) \cdot 1/(1+(B/Ki)^2)$

$v2 = V2 \cdot A/(K2+A)$

$v3 = V3 \cdot B/(K3+B)$

V = [1.0, 0.8, 0.6] mM/s, K = [0.5, 0.4, 0.3] mM, Ki = 0.5 mM. Start from A = B = 0.01 mM.

1. Write the two differential equations.
2. Integrate to steady state and report A, B and the pathway flux.
3. Perturb each V by 1% and compute all three flux control coefficients; check the summation theorem.
4. Now raise V2 tenfold. Predict what happens before you run it.

Worked solution

The equations are just dc/dt = S·v(c):

$dA/dt = v1 - v2$

$dB/dt = v2 - v3$

Steady state: A = 0.396 mM, B = 0.590 mM, flux J = 0.398 mM/s. Note that the flux is below every individual V — feedback has throttled the first step to match the capacity of the last.

Control coefficients from 1% perturbations: $C_1 = 0.22$, $C_2 = 0.00$, $C_3 = 0.77$. They sum to 0.997, confirming the summation theorem numerically.

Raising V2 tenfold changes the flux by 0.0%. What changes is A, which collapses from 0.396 to 0.021 mM. Enzyme 2 had no control to give, so a tenfold investment bought nothing but a smaller intermediate pool. This is the single most useful thing a kinetic model will ever show you, and it is why 'overexpress the bottleneck' fails so reliably.

Worth noticing where the control actually sits: enzyme 3, the last step, which drains B and thereby relieves the feedback on enzyme 1. Control in a feedback-regulated pathway is often downstream of where intuition puts it.

Code

```
import numpy as np
from scipy.integrate import solve_ivp

V, K, Ki, Sx = [1.0,0.8,0.6], [0.5,0.4,0.3], 0.5, 10.0

def rates(A, B, V=V):
    v1 = V[0]*Sx/(K[0]+Sx) / (1+(B/Ki)**2)
    v2 = V[1]*A/(K[1]+A)
    v3 = V[2]*B/(K[2]+B)
    return v1, v2, v3

def rhs(t, y, V=V):
    A, B = y
    v1, v2, v3 = rates(A, B, V)
    return [v1-v2, v2-v3]

sol = solve_ivp(rhs, [0,500], [0.01,0.01], method='LSODA', rtol=1e-9, atol=1e-12)
A, B = sol.y[:,-1]
J = rates(A,B)[2]
print('A=%.4f  B=%.4f  J=%.4f' % (A,B,J))

# flux control coefficients by 1% perturbation
total = 0
for i in range(3):
    Vp = list(V); Vp[i] *= 1.01
    s = solve_ivp(lambda t,y: rhs(t,y,Vp), [0,500], [A,B],
                  method='LSODA', rtol=1e-9, atol=1e-12)
    Jp = rates(*s.y[:,-1], Vp)[2]
    C = ((Jp-J)/J)/0.01
    total += C
    print('C_%d = %.3f' % (i+1, C))
print('sum = %.3f' % total)
```

B3 · A branch point, and why the split ratio is not a constant

week 16 · 45 min

Metabolite A feeds two enzymes in competition:

product route: V1 = 10 mM/s, K_M = 0.1 mM

byproduct route: V2 = 5 mM/s, K_M = 2.0 mM

Compute the split at [A] = 0.1, 1, 5 and 20 mM.

1. Write both rates and the fraction going to product.
2. Tabulate the split against [A].
3. Explain the trend from the K_M values, then say what it implies for a design that pushes more carbon to the branch point.

Worked solution

Fraction to product = v1/(v1+v2):

[A] = 0.1 mM → v1 = 5.00, v2 = 0.24, split = 0.955

[A] = 1.0 mM → v1 = 9.09, v2 = 1.67, split = 0.845

[A] = 5.0 mM → v1 = 9.80, v2 = 3.57, split = 0.733

[A] = 20 mM → v1 = 9.95, v2 = 4.55, split = 0.686

The low-K_M enzyme saturates first. Once it is at its V_max, every extra molecule of A has nowhere to go but the high-K_M byproduct route, so selectivity falls as flux rises.

The design consequence is counterintuitive and important: pushing more carbon into the branch point makes the byproduct problem worse, not better. If you want both flux and selectivity you must raise V1 — more enzyme, a better enzyme — or delete the competing route, rather than driving harder from upstream. Many strain programmes discover this the expensive way, by scaling up a strain whose byproduct fraction grows with productivity.

Code

```
for A in [0.1, 1.0, 5.0, 20.0]:
    v1 = 10*A/(0.1+A)
    v2 = 5*A/(2.0+A)
    print('[A]=%5.1f  v1=%6.3f  v2=%6.3f  split=%.3f' % (A, v1, v2, v1/(v1+v2)))
```

B4 · A batch fermentation model from scratch: Monod plus Luedeking–Piret

week 16 · 1.5 h

Build and solve the classic batch model.

$\mu = \mu_{max} \cdot S/(K_S+S)$, $\mu_{max} = 0.5\ h^{-1}$, $K_S = 0.2\ g/L$

$dX/dt = \mu X$

$dP/dt = \alpha\mu X + \beta X$, $\alpha = 0.2\ g/g$, $\beta = 0.05\ g \cdot g^{-1} \cdot h^{-1}$

$dS/dt = -(1/Y) \cdot dX/dt - dP/dt$, $Y_{X/S} = 0.45\ g/g$

Initial: $X = 0.1\ g/L$, $S = 20\ g/L$, $P = 0$.

Integrate until the substrate is exhausted.

1. Code the three differential equations and integrate with an event that stops at $S = 0$.
2. Report the time of exhaustion, final biomass and final titer.
3. Compute the observed biomass yield and compare it with $Y_{X/S} = 0.45$. Explain the difference.
4. Compute volumetric productivity, and say what you would change to raise it.

Worked solution

Substrate is exhausted at $t = 9.0$ h, with $X = 8.0$ g/L and $P = 2.43$ g/L.

Observed biomass yield is 8.0/20 = 0.400 g/g, not the 0.45 you put in. Nothing is wrong: product formation draws its own substrate, so the yield parameter you specify for biomass is not the yield you observe. Any model with more than one product has this property, and reporting an observed yield without saying which products it accounts for is a common error in the literature.

Product yield is 0.122 g/g and volumetric productivity 0.270 $g \cdot L^{-1} \cdot h^{-1}$.

What to change: the β term — product made in proportion to biomass regardless of growth — means productivity keeps running after growth stops, so extending the non-growth phase helps. That argues for fed-batch, which is problem D4. Raising α trades growth for product directly. Raising μ_{max} shortens the batch but does not change the yields at all, which is worth internalising before you try to fix a yield problem with a faster strain.

Code

```
import numpy as np
from scipy.integrate import solve_ivp

mu_max, Ks, Y, alpha, beta = 0.5, 0.2, 0.45, 0.2, 0.05

def rhs(t, y):
    X, S, P = y
    mu = mu_max*S/(Ks+S) if S > 0 else 0.0
    dX = mu*X
    dP = alpha*mu*X + beta*X
    dS = -dX/Y - dP
    return [dX, dS, dP]

out_of_sugar = lambda t, y: y[1] - 1e-4
out_of_sugar.terminal = True
out_of_sugar.direction = -1

s = solve_ivp(rhs, [0,40], [0.1,20,0], events=out_of_sugar,
              rtol=1e-9, atol=1e-12, max_step=0.05)
X, S, P = s.y[:,-1]
print('t = %.2f h   X = %.2f g/L   P = %.2f g/L' % (s.t[-1], X, P))
print('Yx/s = %.3f (you specified 0.45)' % (X/20))
print('Yp/s = %.3f   productivity = %.3f g/L/h' % (P/20, P/s.t[-1]))
```

Fluxomics

Turning labelling measurements into fluxes, with honest error bars.

C1 · Resolve the pentose phosphate split from a labelling measurement

week 43 · 1 h

You feed [1-^{13}C] glucose and measure the M+1 fraction of pyruvate as 0.35.

Simplifying model, stated so you can see what it assumes: a fraction f of glucose-6-phosphate enters the oxidative pentose phosphate pathway, where carbon 1 is released as CO_2 by 6-phosphogluconate dehydrogenase; the remaining (1−f) goes through glycolysis, where carbon 1 is retained and becomes the methyl carbon of one of the two pyruvate molecules. Treat carbon re-entering from the non-oxidative branch as unlabelled at that position, and ignore the small carbon-yield difference between routes.

1. Write the labelled fraction of pyruvate as a function of f.
2. Solve for f.
3. Repeat for measured M+1 of 0.45 and 0.25.
4. State which assumptions you would have to drop to do this properly, and what tool does it.

Worked solution

Each glucose going through glycolysis gives two pyruvate, one of which carries the label. So the M+1 fraction of pyruvate is (1−f)·1/2, and f = 1 − 2·M+1.

M+1 = 0.45 → f = 0.10

M+1 = 0.35 → f = 0.30

M+1 = 0.25 → f = 0.50

Note how steep that is: a 0.10 change in a measured fraction moves the inferred split by 0.20. Labelling data are informative here precisely because the sensitivity is high — which is also why measurement error matters, and why problem C3 puts error bars on it.

What the toy model ignores: carbon re-entering glycolysis from the non-oxidative branch really is partly labelled, the two triose pools exchange through triosephosphate isomerase, the TCA cycle scrambles label through symmetric succinate and fumarate, and the carbon yields of the two routes differ because the oxidative branch loses CO_2. A real analysis writes the full atom transition network and fits all fluxes at once — that is what INCA, OpenFlux and mfapy do. The toy version is still worth doing once by hand, because it shows you exactly why the tracer was chosen.

Code

```
for m1 in (0.45, 0.35, 0.25):
    print('M+1 = %.2f  ->  f_PPP = %.2f' % (m1, 1 - 2*m1))
```

C2 · Correct a mass isotopomer distribution for natural abundance

week 43 · 1 h

A GC-MS fragment containing 3 carbons gives raw intensities, normalised:

M+0 = 0.700, M+1 = 0.220, M+2 = 0.060, M+3 = 0.020

Natural ^{13}C abundance is 1.07%. Correct the distribution. (Ignore the derivatisation atoms; a real correction includes Si, O, N and H too.)

1. Build the 4×4 correction matrix M where M[i][j] is the probability that a species with j labelled carbons appears at mass i.
2. Solve M·x = raw for the corrected distribution x.
3. Renormalise and compare with the raw values.
4. Say in which direction the correction always moves the distribution, and why.

Worked solution

For a fragment of n = 3 carbons, a species with j labelled carbons has n−j remaining carbons that can each be naturally ^{13}C with probability p = 0.0107. The chance that k of them are gives the shift from mass j to mass j+k:

M[j+k][j] = C(n−j, k)·p^k·(1−p)^(n−j−k)

which evaluates to

j=0 j=1 j=2 j=3

i=0 0.9682 0 0 0

i=1 0.0314 0.9787 0 0

i=2 0.0003 0.0212 0.9893 0

i=3 0 0.0001 0.0107 1

Solving and renormalising gives corrected M+0 = 0.7230, M+1 = 0.2016, M+2 = 0.0561, M+3 = 0.0194.

The correction always moves intensity back down in mass, because natural ^{13}C can only add mass, never remove it. Here M+1 falls by about 8% of its value — small, but systematic, and it propagates straight into the fitted flux. Skipping the correction, or applying it with the wrong atom count for the fragment, is one of the most common silent errors in ^{13}C-MFA, and it does not announce itself in the goodness of fit.

Code

```
import numpy as np
from math import comb

p, n = 0.0107, 3
M = np.zeros((n+1, n+1))
for j in range(n+1):
    for i in range(n+1):
        k = i - j
        if 0 <= k <= n-j:
            M[i,j] = comb(n-j, k) * p**k * (1-p)**(n-j-k)

raw = np.array([0.700, 0.220, 0.060, 0.020])
x = np.linalg.solve(M, raw)
x = x / x.sum()
print('corrected:', np.round(x, 4))
```

C3 · Fit a flux to labelling data and put a confidence interval on it

week 43 · 1.5 h

A one-parameter model predicts two independent measurements from the split fraction f:

$m_1(f) = 0.5 \cdot (1 - f)$

$m_2(f) = 0.25 + 0.10 \cdot f$

You measure $m_1 = 0.330$ and $m_2 = 0.283$, each with a standard deviation of 0.010.

1. Write the chi-squared objective.
2. Find the f that minimises it, and the minimum value.
3. State the degrees of freedom and judge the fit.
4. Find the 95% confidence interval by the chi-squared increase of 3.84, and interpret it.

Worked solution

$\chi^2(f) = \Sigma[(m_i(f) - obs_i)/\sigma_i]^2$.

Best fit: f = 0.340 with $\chi^2_{min} = 0.01$. With two measurements and one parameter there is 1 degree of freedom, and a χ^2 of 0.01 against an expectation of about 1 is an excellent fit — arguably too good, which usually means the error estimate σ is conservative.

The 95% interval comes from the values of f where χ^2 rises by 3.84 above the minimum (the one-degree-of-freedom threshold): $f \in [0.301, 0.378]$. So the split is 34% ± 4 percentage points.

Three things to carry forward. First, a flux without an interval is not a result; fluxes are fitted quantities, not measurements. Second, the interval is only as meaningful as the network: narrow error bars on a model missing a reaction are precisely wrong. Third, had χ^2_{min} come back at, say, 25 with 1 degree of freedom, the right conclusion would not be 'the fit is poor' but 'either my error estimates are too small or my network is wrong' — and the second is the interesting case, because it is evidence of biology you had not modelled.

Code

```
import numpy as np
from scipy.optimize import brentq

obs, sig = np.array([0.330, 0.283]), 0.010
model = lambda f: np.array([0.5*(1-f), 0.25 + 0.10*f])
chi2  = lambda f: np.sum(((model(f) - obs)/sig)**2)

grid = np.linspace(0, 1, 100001)
fhat = grid[np.argmin([chi2(f) for f in grid])]
print('f = %.4f, chi2_min = %.3f' % (fhat, chi2(fhat)))

lo = brentq(lambda f: chi2(f) - chi2(fhat) - 3.84, 0, fhat)
hi = brentq(lambda f: chi2(f) - chi2(fhat) - 3.84, fhat, 1)
print('95%% CI = [%.3f, %.3f]' % (lo, hi))
```

Scale-up and bioreactor engineering

Your own transport phenomena, pointed at a fermenter.

D1 · Size the oxygen supply, then check a correlation to see if it is reachable

week 44 · 1.5 h

An aerobic culture at 50 gDW/L with a specific oxygen uptake of 5 mmol·gDW^{-1}·h^{-1}.

Air at 37 °C gives C* ≈ 0.21 mM; hold C_L at 0.05 mM to stay above the critical concentration.

Van't Riet's correlation for a coalescing aqueous medium: kLa = 0.026·(P/V)^0.4·v_s^0.5, with kLa in s^{-1}, P/V in W/m³ and superficial gas velocity v_s in m/s. Take v_s = 0.03 m/s.

1. Compute the oxygen uptake rate per litre per hour.
2. Compute the kLa required.
3. Invert the correlation for the P/V it would need.
4. Compute the kLa actually obtained at realistic P/V values, and the cell density each supports.
5. Propose the fix.

Worked solution

OUR = 50 × 5 = 250 mmol·L^{-1}·h^{-1}.

Required kLa = OUR/(C* − C_L) = 250/0.16 = 1,563 h^{-1} = 0.434 s^{-1}.

Inverting the correlation: (P/V)^0.4 = 0.434/(0.026 × 0.03^0.5), giving P/V ≈ 9.1 × 10⁴ W/m³. Large vessels run at 1,000–3,000 W/m³. The requirement is thirty times beyond that, and the power would cook the broth before it aerated it.

What is actually achievable:

P/V = 1,000 W/m³ → kLa = 257 h^{-1} → supports 8.2 gDW/L

P/V = 2,000 W/m³ → kLa = 339 h^{-1} → supports 10.8 gDW/L

P/V = 3,000 W/m³ → kLa = 399 h^{-1} → supports 12.8 gDW/L

So this process is capped near 10 gDW/L on air at atmospheric pressure, not 50.

The fixes, in order of how often they are the right one: reduce the oxygen demand — a lower specific uptake rate through fed-batch control of growth rate is the single most effective change, and it is a strain and process decision rather than a hardware one. Then raise C* by pressure and oxygen enrichment: 2 bar absolute with 40% O₂ raises C* about 3.8-fold and cuts the required kLa by the same factor, bringing it within reach. Only then consider more power.

This calculation, done in twenty minutes at the start of a project, is one of the most valuable things a chemical engineer brings to a biology group. It decides whether a strain's reported cell density is reachable at scale before anyone spends a year on it.

Code

```
X, qO2 = 50.0, 5.0            # gDW/L, mmol/gDW/h
Cstar, CL, vs = 0.21, 0.05, 0.03

OUR = X*qO2
kla_req = OUR/(Cstar-CL)       # 1/h
```

```
print('OUR = %.0f mmol/L/h ; kLa required = %.0f 1/h' % (OUR, kla_req))

PV_needed = ((kla_req/3600)/(0.026*vs**0.5))**(1/0.4)
print('P/V needed = %.3g W/m3' % PV_needed)

for pv in (1000, 2000, 3000):
    kla = 0.026*pv**0.4*vs**0.5*3600
    print('P/V=%5d -> kLa=%4.0f 1/h -> X_max=%.1f gDW/L'
          % (pv, kla, kla*(Cstar-CL)/qO2))
```

D2 · Mixing time from 1 L to 100 m³ at equal power per volume

week 44 · 1 h

Geometrically similar stirred tanks with H = T and impeller diameter d = T/3, Rushton turbine ($N_p \approx 5$), water-like broth ($\rho = 1000$ kg/m³), P/V held at 1,000 W/m³.

Nienow's correlation: $\theta_{95} = 5.9 \cdot T^{(2/3)} \cdot \varepsilon^{(-1/3)} \cdot (T/d)^{(1/3)}$, with ε the specific power in W/kg.

Compare 1 L, 1 m³ and 100 m³.

1. Get the tank diameter from the volume.
2. Compute θ_{95} at each scale.
3. Compute the impeller speed and tip speed at each scale from $P = N_p \cdot \rho \cdot n^3 \cdot d^5$.
4. State what a cell experiences differently at the large scale.

Worked solution

Tank diameters: 0.11 m, 1.08 m, 5.03 m.

Mixing times at 1,000 W/m³: 1.9 s, 9.0 s, 25.0 s. A 13-fold increase from 1 L to 100 m³ with power per volume held constant.

Impeller speed and tip speed: 1 L runs at 14.8 s^{-1} (889 rpm) with a tip speed of 1.7 m/s; 100 m³ at 1.15 s^{-1} (69 rpm) with a tip speed of 6.0 m/s. Power per volume is the same, but tip speed — and therefore shear at the impeller — is three to four times higher, while bulk mixing is thirteen times slower.

That combination is the whole scale-up problem in two numbers. A cell circulating through the large vessel spends tens of seconds between visits to the feed zone, which is long compared with the timescale of its own metabolic response: it experiences feast and famine in sequence rather than a steady average. Overflow metabolism, acetate formation, and the titer gap between 1 L and production scale mostly come from this, not from anything genetic.

It also explains why you cannot fix scale-up by raising power. P/V rises with the cube of tip speed; mixing time improves only as the cube root. The tractable routes are feeding strategy, vessel geometry, and strains that tolerate the gradients.

Code

```
import numpy as np

def mixing_time(V, PV=1000.0, T_over_d=3.0, rho=1000.0):
    T = (4*V/np.pi)**(1/3)        # H = T
    eps = PV/rho                  # W/kg
    return 5.9*T**(2/3)*eps**(-1/3)*T_over_d**(1/3), T

for V, lab in [(1e-3,'1 L'), (1.0,'1 m3'), (100.0,'100 m3')]:
    th, T = mixing_time(V)
    print('%-7s T=%.2f m  theta95=%.1f s' % (lab, T, th))

Np, rho = 5.0, 1000.0
for V, lab in [(1e-3,'1 L'), (100.0,'100 m3')]:
    T = (4*V/np.pi)**(1/3); d = T/3
    n = ((1000.0*V)/(Np*rho*d**5))**(1/3)
    print('%-7s d=%.3f m  n=%.2f 1/s (%.0f rpm)  tip=%.2f m/s'
          % (lab, d, n, n*60, np.pi*n*d))
```

D3 · What cell density does your vessel actually support?

week 44 · 30 min

Turn the oxygen balance around. For kLa = 100, 300, 500 and 1,000 h^{-1}, with $C^* - C_L = 0.16$ mM and a specific oxygen uptake of 5 $mmol \cdot gDW^{-1} \cdot h^{-1}$, find the maximum steady cell density.

1. Write the steady-state oxygen balance: supply equals demand.
2. Solve for X_max.
3. Tabulate.
4. Compare with the cell densities quoted in high-cell-density papers and say what must be true for those numbers.

Worked solution

At steady state $kLa \cdot (C^* - C_L) = q_{O_2} \cdot X$, so $X_{max} = kLa \cdot (C^* - C_L)/q_{O_2}$.

kLa = 100 h^{-1} → OTR = 16 $mmol \cdot L^{-1} \cdot h^{-1}$ → X_max = 3.2 gDW/L

kLa = 300 h^{-1} → 48 → 9.6 gDW/L

kLa = 500 h^{-1} → 80 → 16.0 gDW/L

kLa = 1,000 h^{-1} → 160 → 32.0 gDW/L

High-cell-density papers routinely report 50–100 gDW/L. For that to be true at these kLa values, one of three things must hold: the specific oxygen uptake is much lower than 5 $mmol \cdot gDW^{-1} \cdot h^{-1}$, which is what growth-rate-limited fed-batch achieves; C* has been raised by pressure or oxygen enrichment; or the vessel reaches a kLa far above 500 h^{-1}, which small well-aerated bioreactors can but production vessels generally cannot.

When you read a high-cell-density result, find which of the three it was. It is usually the first, and it is usually in the methods rather than the abstract.

Code

```
for kla in (100, 300, 500, 1000):
    otr = kla*0.16
    print('kLa=%4d 1/h -> OTR=%3.0f mmol/L/h -> X_max=%.1f gDW/L'
          % (kla, otr, otr/5))
```

D4 · Design an exponential fed-batch from the substrate balance

week 44 · 1.5 h

Hold the specific growth rate at $\mu = 0.10\ h^{-1}$ in fed-batch.

Start: $V_0 = 5$ L, $X_0 = 20$ g/L. Feed concentration $S_{feed} = 500$ g/L. $Y_{X/S} = 0.45$ g/g. Neglect maintenance. Run for 20 h.

1. Derive the feed rate that holds μ constant.
2. Compute F_0.
3. Tabulate F, V, X and total biomass at t = 0, 5, 10, 15 and 20 h.
4. Comment on what limits how long you can run this.

Worked solution

Substrate balance with substrate held low and quasi-steady: consumption equals supply, so $F \cdot S_{feed} = (\mu/Y) \cdot X \cdot V$. Since biomass grows as $XV = X_0V_0 \cdot e^{\{\mu t\}}$, the feed must grow at the same exponential rate:

$F(t) = (\mu \cdot X_0V_0)/(Y \cdot S_{feed}) \cdot e^{\{\mu t\}} = F_0 \cdot e^{\{\mu t\}}$

$F_0 = (0.10 \times 20 \times 5)/(0.45 \times 500) = 0.0444$ L/h.

t = 0 h: F = 0.044 L/h, V = 5.00 L, X = 20.0 g/L, biomass 100 g

t = 5 h: F = 0.073 L/h, V = 5.29 L, X = 31.2 g/L, biomass 165 g

t = 10 h: F = 0.121 L/h, V = 5.76 L, X = 47.2 g/L, biomass 272 g

t = 15 h: F = 0.199 L/h, V = 6.55 L, X = 68.4 g/L, biomass 448 g

t = 20 h: F = 0.328 L/h, V = 7.84 L, X = 94.3 g/L, biomass 739 g

Note that X rises faster than V because the feed is concentrated — this is how high cell density is reached at all.

What stops you: oxygen first. At 94 gDW/L the oxygen demand is far past what problem D3 says the vessel can supply, which is exactly why μ was set to 0.10 h^{-1} rather than μ_max — a low growth rate means a low specific oxygen uptake. Then vessel volume, since V grows and the headspace does not. Then, for real processes, broth viscosity and the accumulation of inhibitory byproducts.

The design logic to keep: in fed-batch the pump sets the growth rate, and you choose that growth rate from the oxygen balance, not from how fast the organism could grow.

Code

```
import numpy as np

mu, Y, Sf = 0.10, 0.45, 500.0
V0, X0 = 5.0, 20.0
F0 = mu*X0*V0/(Y*Sf)
print('F0 = %.4f L/h' % F0)

for t in (0, 5, 10, 15, 20):
    F = F0*np.exp(mu*t)
    V = V0 + (F0/mu)*(np.exp(mu*t) - 1)
    X = X0*V0*np.exp(mu*t)/V
    print('t=%2d h  F=%.3f L/h  V=%.2f L  X=%.1f g/L  biomass=%.0f g'
          % (t, F, V, X, X*V))
```

D5 · The cost sketch that decides whether any of it matters

week 44 · 1 h

Sugar at $0.40/kg, product yield 0.40 g/g. Thermal energy at $8/GJ, latent heat of water 2.26 MJ/kg.

Compare titers of 100, 50, 20 and 10 g/L on feedstock cost and the energy cost of removing the water.

1. Compute kilograms of sugar and feedstock cost per kilogram of product.
2. Compute kilograms of water per kilogram of product at each titer.
3. Convert to an energy cost, assuming evaporation.
4. State the conclusion for a product that sells at $2/kg.

Worked solution

Feedstock: 1/0.40 = 2.50 kg sugar per kg product, so $1.00/kg of product, independent of titer.

Water removal, taking 1 kg of product dissolved in (1000/titer) kg of broth:

100 g/L → 9.0 kg water → 20 MJ → $0.16/kg

50 g/L → 19.0 kg water → 43 MJ → $0.34/kg

20 g/L → 49.0 kg water → 111 MJ → $0.89/kg

10 g/L → 99.0 kg water → 224 MJ → $1.79/kg

For a product selling at $2/kg, feedstock alone takes half the revenue. At 10 g/L the water removal takes nearly all the rest, before capital, labour, oxygen, sterility or any purification beyond concentration. The process cannot work. At 50 g/L there is $0.66/kg left for everything else, which is tight but arguable; at 100 g/L it becomes a real conversation.

This is why commercial targets cluster above 100 g/L for commodities, and why a paper reporting 2 g/L of a bulk chemical is a proof of pathway, not a process. Evaporation is also the pessimistic case — extraction, crystallisation or distillation of a volatile product can be far cheaper — so treat this as a bound rather than a quote. Do this calculation for your capstone target in week 44 before you commit to it.

Code

```
glc, Y = 0.40, 0.40
print('feedstock: %.2f kg sugar/kg product -> $%.2f/kg' % (1/Y, glc/Y))

for titer in (100, 50, 20, 10):
    water = 1000.0/titer - 1          # kg water per kg product
    energy = water*2.26               # MJ
```

```
print('titer %3d g/L -> %5.1f kg water -> %6.1f MJ -> $%.2f/kg'
      % (titer, water, energy, energy*8/1000))
```

D6 · Turn a kLa into a flux bound and see the yield you would actually get

week 44 · 1.5 h

You have an aerobic vessel with kLa = 500 h^{-1}, C* − C_L = 0.16 mM, and a culture at 10 gDW/L. You also have the E. coli core model, with glucose uptake set to 10 mmol·gDW^{-1}·h^{-1}.

The question nobody asks early enough: what does the vessel's oxygen supply do to the model's prediction?

1. Compute the oxygen transfer rate in mmol·L^{-1}·h^{-1}.
2. Convert it to a per-biomass flux in mmol·gDW^{-1}·h^{-1}, which is the unit a genome-scale model works in.
3. Set that as the oxygen exchange bound, rerun FBA, and record growth rate and secreted products.
4. Repeat at 5, 20 and 30 gDW/L and explain the trend.
5. Say what this means for a strain whose paper reports an aerobic yield.

Worked solution

OTR = kLa·(C* − C_L) = 500 × 0.16 = 80 mmol·L^{-1}·h^{-1}.

Dividing by biomass gives the flux the model needs:

X = 5 gDW/L → 16.0 mmol·gDW^{-1}·h^{-1}

X = 10 gDW/L → 8.0

X = 20 gDW/L → 4.0

X = 30 gDW/L → 2.7

That division is the whole trick, and it is where the units finally agree: a vessel specification in mmol per litre per hour becomes a per-gram flux bound, because the same oxygen is shared among more cells as the culture gets denser.

Set EX_o2_e to those lower bounds in turn and the pattern is unmistakable. Unconstrained oxygen gives about 0.87 h^{-1} with no fermentation products. As the bound tightens, growth falls monotonically and acetate, then formate and ethanol, appear in the solution — the model switches to overflow metabolism because it must dispose of NADH it can no longer respire. By 2.7 mmol·gDW^{-1}·h^{-1} the solution is close to the anaerobic one, around 0.21 h^{-1}, even though you have an aerated tank. Find the threshold where acetate first appears from your own run rather than taking a number from me; it sits somewhere below roughly 15–20 mmol·gDW^{-1}·h^{-1} at this glucose uptake, and it depends on the model version and the maintenance ATP setting.

The lesson for reading papers: an aerobic yield measured at 2 gDW/L in a shake flask is a yield measured with effectively unlimited oxygen. The same strain at 30 gDW/L in a production vessel is operating under a bound four to ten times tighter and will make byproducts the paper never reported. Doing this calculation before you believe a yield is twenty minutes of work and it is the kind of judgement a chemical engineer is hired into a biology group to supply.

A note on this one: the arithmetic above is exact and checked. The growth rates and byproduct thresholds come from the structure of the core model and the published aerobic and anaerobic optima — run them yourself and reconcile any difference against your model version rather than assuming my numbers.

Code

```
import cobra
from cobra.io import load_model

model = load_model('textbook')          # E. coli core
kla, dC = 500.0, 0.16                   # 1/h, mM
otr = kla*dC                            # mmol/L/h

print('OTR = %.1f mmol/L/h' % otr)
```

```
for X in (5, 10, 20, 30):              # gDW/L
    bound = otr/X                      # mmol/gDW/h
    with model:                        # reverts on exit
        model.reactions.EX_o2_e.lower_bound = -bound
        sol = model.optimize()
        secreted = {r.id: round(sol.fluxes[r.id], 2)
                    for r in model.exchanges
                    if sol.fluxes[r.id] > 1e-6}
        print('X=%2d gDW/L  O2 bound=%5.1f  mu=%.3f 1/h  secreted=%s'
              % (X, bound, sol.objective_value, secreted))
```

The computational stack

The same model in two languages, and what it takes to trust both.

E1 · Build the week 13 toy network as a COBRApy model and reproduce your hand answer

week 36 · 1.5 h

Take the five-reaction network from problem A1:

v1: S_ext → A v2: A → B v3: A → C

v4: B → P_ext v5: C → B

Build it in COBRApy from Metabolite and Reaction objects. Cap v1 at 10, make the product excretion v4 the objective, and maximise.

1. Create three metabolites and five reactions, setting stoichiometry with add_metabolites and the correct signs.
2. Set bounds: all irreversible, v1 capped at 10.
3. Set v4 as the objective and optimise.
4. Compare the objective value and the flux distribution with your hand solution from A1, then run flux variability analysis and explain the result.

Worked solution

The optimum is 10, matching the hand result that v4 = v1 identically. The flux distribution the solver returns is one of infinitely many, because v3 and v5 can carry any equal flux without affecting the objective.

Flux variability analysis makes that explicit: v1, v2 and v4 come back with zero range, while v3 and v5 show a range from 0 to their upper bound. That is the same degree of freedom you found by hand in A1, now reported by the software instead of derived on paper — and seeing the two agree is the point of the exercise.

Two habits follow. Run pFBA when you intend to interpret internal fluxes, since it will pick the zero-cycle solution rather than an arbitrary one. And read a wide FVA range as 'my constraints are silent here', never as a flux value.

A note on this one: the expected numbers come from the hand solution in A1, which is exact. The code uses the standard COBRApy API, but it was not executed while writing this — run it and reconcile any API difference against the version you installed in week 12.

Code

```
import cobra
from cobra import Model, Reaction, Metabolite

m = Model('toy')
A, B, C = (Metabolite(i, compartment='c') for i in ('A','B','C'))

def rxn(idx, stoich, lb=0, ub=1000):
    r = Reaction(idx)
    r.bounds = (lb, ub)
    r.add_metabolites(stoich)
    return r

m.add_reactions([
    rxn('v1', {A: 1},            ub=10),   # uptake
    rxn('v2', {A:-1, B: 1}),
    rxn('v3', {A:-1, C: 1}),
    rxn('v4', {B:-1}),                     # excretion
```

```
    rxn('v5', {C:-1, B: 1}),
])

m.objective = 'v4'
sol = m.optimize()
print('objective', sol.objective_value)
print(sol.fluxes)

from cobra.flux_analysis import flux_variability_analysis
print(flux_variability_analysis(m))
```

E2 · Port the kinetic model to MATLAB and prove the two agree

week 38 · 1.5 h

Take the three-enzyme model from problem B2 — same rate laws, same parameters, same initial conditions — and implement it in MATLAB with ode15s.

Python gave A = 0.3957 mM, B = 0.5903 mM, J = 0.3978 mM/s, and control coefficients 0.223, 0.000, 0.774.

1. Write the right-hand side as a MATLAB function; mind the argument order and that it must return a column vector.
2. Integrate with ode15s to steady state and print A, B and the flux.
3. Repeat the 1% perturbation loop and recompute the three control coefficients.
4. Compare all five numbers with the Python result and account for any difference.

Worked solution

All five numbers should agree to at least three decimal places: A = 0.396, B = 0.590, J = 0.398, and control coefficients 0.223, 0.000, 0.774. Both solvers are solving the same stiff system to a tighter tolerance than that.

If they do not agree, work through this order. Argument order — MATLAB's odefun is f(t, y), the same as solve_ivp, but it must return a column vector; returning a row gives a confusing dimension error or, worse, silently wrong behaviour. Then indexing, if you stored parameters in arrays. Then tolerances: set RelTol to 1e-9 and AbsTol to 1e-12 to match what the Python run used, since the defaults differ and a loose tolerance shows up first in the control coefficients, which are differences of nearly equal numbers. Then integration time — both need long enough to actually reach steady state.

The real deliverable is not the numbers; it is the two-column table in your README mapping each Python call to its MATLAB equivalent. That table is what makes you useful in a lab with a decade of MATLAB scripts in its repository.

Code

```
% three_enzyme.m  —  compare against the Python run in problem B2
% NOTE: in MATLAB and Octave a script's local functions must come LAST.
V  = [1.0 0.8 0.6];  K = [0.5 0.4 0.3];  Ki = 0.5;  Sx = 10;

opts = odeset('RelTol',1e-9,'AbsTol',1e-12);
[~, y] = ode15s(@(t,y) rhs(t,y,V,K,Ki,Sx), [0 500], [0.01; 0.01], opts);
A = y(end,1);  B = y(end,2);
J = V(3)*B/(K(3)+B);
fprintf('A=%.4f  B=%.4f  J=%.4f\n', A, B, J);

for i = 1:3
    Vp = V;  Vp(i) = Vp(i)*1.01;
    [~, yp] = ode15s(@(t,y) rhs(t,y,Vp,K,Ki,Sx), [0 500], [A; B], opts);
    Jp = Vp(3)*yp(end,2)/(K(3)+yp(end,2));
    fprintf('C_%d = %.3f\n', i, ((Jp-J)/J)/0.01);
end

% ---- local functions must sit at the end of a script file ----
function dy = rhs(~, y, V, K, Ki, Sx)
    A = y(1); B = y(2);
    v1 = V(1)*Sx/(K(1)+Sx)/(1+(B/Ki)^2);
    v2 = V(2)*A/(K(2)+A);
    v3 = V(3)*B/(K(3)+B);
    dy = [v1 - v2; v2 - v3];      % column vector
end
```

Experimental design and statistics

Whether the number you are about to report means anything.

F1 · How many clones must you screen, and what does 96 wells actually buy you?

week 17 · 1 h

You build a combinatorial RBS library over three pathway genes with 24 variants each. You can screen one 96-well plate comfortably, or four plates with effort.

For a library of N equally likely variants and k clones screened, the expected fraction of the library represented is 1 − exp(−k/N), and the probability that one named variant appears is 1 − (1 − 1/N)^k.

1. Compute N.
2. Compute coverage and the named-variant probability at k = 96, 384 and 1,000.
3. Find k for 95% coverage.
4. Repeat for a single-gene 24-variant library and compare.
5. State what this means for how you would design the library rather than the screen.

Worked solution

N = 24^3 = 13,824.

k = 96 → coverage 0.7%, P(named variant) 0.7%

k = 384 → coverage 2.7%, P(named variant) 2.7%

k = 1,000 → coverage 7.0%, P(named variant) 7.0%

For 95% coverage you need roughly 3N ≈ 41,000 clones, and about 41,400 for a 95% chance of catching one specified variant.

The single-gene library is a different world: N = 24, and 96 clones give 98% coverage. One gene is screenable by hand; three genes combinatorially is not.

So the design conclusion is the opposite of the instinct to screen harder. Either reduce the library — three or four well-chosen RBS strengths per gene instead of 24, giving N = 27 to 64 — or change the readout so that large libraries become tractable: a biosensor coupled to fluorescence-activated sorting handles 10^6 to 10^8 variants, and a growth-coupled selection handles more still. 'We screened 96 clones from a combinatorial library' is, by itself, a statement that you sampled under one per cent of your design space.

One honest caveat on the arithmetic: it assumes variants are equally likely and that transformation sampled them independently. Real libraries are skewed by cloning bias and by toxicity, so the effective coverage is worse than these numbers, never better.

Code

```
import numpy as np
from math import log

for N in (24, 24**3):
    print('library of %d variants' % N)
    for k in (96, 384, 1000):
        print('  k=%5d  coverage %5.1f%%   P(named variant) %5.1f%%'
              % (k, 100*(1-np.exp(-k/N)), 100*(1-(1-1/N)**k)))
    print('  95%% coverage needs about %d clones' % round(3*N))
    print('  95%% chance of a named variant needs %d clones'
          % round(log(0.05)/log(1-1/N)))
```

F2 · Is strain B actually better? Error propagation, then power

week 17 · 1.5 h

Strain A yields, in triplicate: 0.60, 0.64, 0.62 g/g. Strain B: 0.65, 0.69, 0.64 g/g.

Separately, one of those yields came from product 12.4 ± 0.4 g/L and substrate consumed 20.0 ± 0.5 g/L.

1. Propagate the measurement errors into that single yield and report it to a defensible precision.
2. Compute the means, standard deviations, t statistic and p value for the two strains.

3. Compute the 95% confidence interval on the difference and the effect size.

4. Compute the power of this n = 3 experiment, and the n needed for 80%.

5. Write the one sentence you would actually put in a report.

Worked solution

Propagation: Y = 12.4/20.0 = 0.620 g/g, and for a quotient the relative errors add in quadrature — √((0.4/12.4)² + (0.5/20.0)²) = 4.1% — so Y = 0.620 ± 0.025 g/g. The product term dominates, so the HPLC calibration is what to improve. Note also that the uncertainty sets the significant figures: 0.62, not 0.6201.

Comparison: A has mean 0.620, SD 0.020; B has mean 0.660, SD 0.026. A two-sample t-test gives t = −2.09 with p = 0.105 (Welch gives p = 0.110). The 95% confidence interval on the difference runs from −0.013 to +0.093 g/g and therefore includes zero. Cohen's d ≈ 1.71 — a large effect.

Power at that effect size and n = 3 is only 36%. Which means this experiment had roughly a one in three chance of detecting a difference of exactly the size it was looking for, even if that difference is entirely real. To reach 80% you need n = 7 per strain; for 95%, n = 10.

The sentence to write: "Strain B gave a 0.040 g/g higher yield (95% CI −0.013 to +0.093, n = 3, p = 0.11). The experiment is underpowered for this effect size; repeating at n = 7 would give 80% power." That is both more honest and more useful than "B showed improved yield" — and it is the kind of sentence that makes a supervisor trust your numbers.

The wider lesson: most published strain comparisons are n = 3 with no power statement, which means the literature systematically over-reports small improvements and the ones that fail to replicate were never properly detected in the first place.

Code

```
import numpy as np
from scipy import stats

# error propagation on one yield
P, dP, S, dS = 12.4, 0.4, 20.0, 0.5
Y = P/S
rel = np.hypot(dP/P, dS/S)
print('Y = %.3f +/- %.3f g/g  (%.1f%%)' % (Y, Y*rel, 100*rel))

# is the difference real?
A = np.array([0.60, 0.64, 0.62])
B = np.array([0.65, 0.69, 0.64])
t, p = stats.ttest_ind(A, B)
print('A %.3f+/-%.3f  B %.3f+/-%.3f  t=%.2f  p=%.3f'
      % (A.mean(), A.std(ddof=1), B.mean(), B.std(ddof=1), t, p))

sp = np.sqrt((A.var(ddof=1)+B.var(ddof=1))/2)
d  = (B.mean()-A.mean())
se = sp*np.sqrt(2/len(A))
ci = stats.t.ppf(0.975, 2*len(A)-2)*se
print('difference %.3f, 95%% CI %.3f to %.3f, Cohen d %.2f' % (d, d-ci, d+ci, d/sp))

# power, and the n that would fix it
for n in range(2, 12):
    nc   = (d/sp)*np.sqrt(n/2)
    crit = stats.t.ppf(0.975, 2*n-2)
    pw   = stats.nct.sf(crit, 2*n-2, nc) + stats.nct.cdf(-crit, 2*n-2, nc)
    print('  n=%2d -> power %.2f' % (n, pw))
```

Economics and decision-making

Whether the thing is worth building, and which experiment to run next.

G1 · Minimum product selling price across three titers

week 49 · 2 h

Your capstone product, with these assumptions:

sugar $0.40/kg, product yield 0.40 g/g

plant output 30,000 t/y, 20-year life, 10% discount rate

base capital $120M at 50 g/L, scaling as (50/titer)^0.3

fixed operating cost 6% of capital per year

separation modelled as evaporation: 2.26 MJ/kg water at $8/GJ

Compute the minimum selling price at 20, 50 and 100 g/L, then its sensitivity.

1. Compute feedstock cost per kilogram of product. Note it does not depend on titer.
2. Compute water removed per kilogram and its energy cost at each titer.
3. Compute the annualised capital charge, using the capital recovery factor for 20 years at 10%.
4. Sum to an MPSP at each titer and tabulate the breakdown.
5. Vary yield, sugar price and capital by a defensible range and build the tornado.

Worked solution

Feedstock: 1/0.40 = 2.50 kg sugar per kg product, so $1.00/kg at every titer. This is the floor set by yield, and no amount of titer improvement touches it.

MPSP and its breakdown:

20 g/L → $2.82/kg (feed 1.00, separation 0.89, capital 0.62, fixed 0.32; CAPEX $158M)

50 g/L → $2.05/kg (feed 1.00, separation 0.34, capital 0.47, fixed 0.24; CAPEX $120M)

100 g/L → $1.74/kg (feed 1.00, separation 0.16, capital 0.38, fixed 0.19; CAPEX $97M)

Sensitivity at 50 g/L, one factor at a time:

yield 0.40 → 0.45 … $1.94/kg (−5.4%)

yield 0.40 → 0.35 … $2.20/kg (+7.0%)

sugar $0.40 → $0.30 … $1.80/kg (−12.2%)

sugar $0.40 → $0.50 … $2.30/kg (+12.2%)

CAPEX −30% … $1.84/kg (−10.4%)

CAPEX +30% … $2.27/kg (+10.4%)

Three conclusions worth carrying into the proposal. Feedstock price moves the answer more than a realistic yield improvement does, which is an uncomfortable finding for a strain-engineering project and an honest one. Going from 20 to 100 g/L cuts the price by 38%, almost all of it separation and capital. And a product selling below about $1.74/kg cannot be made by this route at any titer, because feedstock alone is $1.00 and the rest never goes to zero.

Caveats, since this is a teaching model rather than a flowsheet. Capital scaling with an exponent of 0.3 is a stand-in for a real equipment list. Evaporation is the pessimistic separation case; extraction or distillation of a volatile product can be far cheaper. Nothing here includes working capital, waste treatment, labour in detail, or a co-product credit. Build the real version in BioSTEAM in week 49 and compare it against this — the gap between the two is itself instructive.

Code

```
def mpsp(titer, cap_base=120e6, years=20, rate=0.10, plant_kg=30e6,
         sugar=0.40, yield_gg=0.40, fixed_frac=0.06):
    water    = 1000.0/titer - 1.0          # kg water per kg product
    sep_cost = water*2.26*8/1000           # $/kg, 2.26 MJ/kg at $8/GJ
    capex    = cap_base*(50.0/titer)**0.3
    annuity  = rate/(1-(1+rate)**-years)   # capital recovery factor
    cap      = capex*annuity/plant_kg
    feed     = sugar/yield_gg
    fixed    = capex*fixed_frac/plant_kg
    return feed+sep_cost+cap+fixed, dict(feed=feed, sep=sep_cost, cap=cap, fix=fixed, capex=capex)

for t in (20, 50, 100):
    v, b = mpsp(t)
    print('%3d g/L -> $%.2f/kg  feed %.2f sep %.2f cap %.2f fix %.2f  CAPEX $%.0fM'
          % (t, v, b['feed'], b['sep'], b['cap'], b['fix'], b['capex']/1e6))

base, _ = mpsp(50)
for lab, kw in [('yield 0.45', dict(yield_gg=0.45)), ('yield 0.35', dict(yield_gg=0.35)),
                ('sugar 0.30', dict(sugar=0.30)),    ('sugar 0.50', dict(sugar=0.50)),
                ('CAPEX -30%', dict(cap_base=84e6)), ('CAPEX +30%', dict(cap_base=156e6))]:
    v, _ = mpsp(50, **kw)
    print('  %-12s $%.2f/kg (%+.1f%%)' % (lab, v, 100*(v/base-1)))
```

G2 · Sixteen experiments: factorial grid against Bayesian optimisation

week 41 · 2 h

Two medium factors, each scaled to 0 to 1. The true response has a narrow optimum near (0.71, 0.29) reaching 12 g/L, plus a weaker local optimum near (0.18, 0.82). Measurement noise is 0.15 g/L.

You can afford 16 runs. Compare a 4×4 factorial grid against a Gaussian process with expected improvement, 5 random starts and 11 sequential picks.

1. Implement the response surface and the 4×4 grid; record the best titer the grid finds.
2. Fit a Gaussian process with a Matérn kernel and implement expected improvement.
3. Run the sequential loop to 16 total runs and record the best.
4. Repeat over 20 random seeds and report the spread, not your best run.
5. State when you would still prefer the grid.

Worked solution

The 4×4 grid reaches 8.16 g/L — 68% of the true optimum. Its points are 0.25 apart and the optimum is narrow, so it steps over the peak.

Bayesian optimisation on the same 16-run budget reaches a median true value of 11.95 g/L at its chosen best, and gets to at least 95% of the optimum in 19 of 20 seeds. The one failure converges on the weak local optimum and returns 4.96 g/L.

That failure is the honest part of the result. Expected improvement balances exploiting the best known region against exploring uncertain ones; with an unlucky set of random starts it can commit early and stop exploring. Reporting the median and the range across seeds, rather than a single good run, is how this should always be presented — a single run of a stochastic method is an anecdote.

Two scale points. Only 1.3% of the factor space lies within 10% of the optimum, which is why a coarse grid misses it. And a four-level full factorial costs 16 runs in two factors, 64 in three and 256 in four, so the grid becomes impossible exactly when the problem gets interesting.

When to still prefer the grid: when effects are roughly additive and you want an interpretable map; when factors are few; and, importantly, when you must run everything in one batch, since sequential methods need rounds and each round costs a fermentation cycle. A hybrid is often best — a fractional factorial to find the active factors, then sequential optimisation within them.

Code

```
import numpy as np, warnings
warnings.filterwarnings('ignore')
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern, ConstantKernel
from scipy.stats import norm

def truth(x):
    a, b = x[...,0], x[...,1]
    return (12*np.exp(-((a-0.71)**2)/0.03 - ((b-0.29)**2)/0.05)
          +  5*np.exp(-((a-0.18)**2)/0.03 - ((b-0.82)**2)/0.05))

grid = np.stack(np.meshgrid(np.linspace(0,1,201), np.linspace(0,1,201)), -1).reshape(-1,2)
best = truth(grid).max()
g4   = np.stack(np.meshgrid(np.linspace(0.125,0.875,4),
                            np.linspace(0.125,0.875,4)), -1).reshape(-1,2)
print('true optimum      %.2f' % best)
print('16-run 4x4 grid   %.2f  (%.0f%% of optimum)' % (truth(g4).max(), 100*truth(g4).max()/best))

def run_bo(seed, n_init=5, n_total=16, noise=0.15):
    rng = np.random.default_rng(seed)
    X = rng.random((n_init,2)); y = truth(X) + rng.normal(0, noise, n_init)
    while len(X) < n_total:
        gp = GaussianProcessRegressor(ConstantKernel(1.0)*Matern(length_scale=0.2, nu=2.5),
                                      alpha=noise**2, normalize_y=True,
                                      n_restarts_optimizer=3).fit(X, y)
        mu, sd = gp.predict(grid, return_std=True)
        imp = mu - y.max() - 0.01
        z   = imp/np.maximum(sd, 1e-9)
        ei  = imp*norm.cdf(z) + sd*norm.pdf(z)          # expected improvement
        nxt = grid[np.argmax(ei)]
        X = np.vstack([X, nxt]); y = np.append(y, truth(nxt[None])[0] + rng.normal(0, noise))
```

```
    return truth(X[np.argmax(y)][None])[0]

vals = np.array([run_bo(s) for s in range(20)])
print('16-run Bayesian   %.2f median (range %.2f-%.2f over 20 seeds)'
      % (np.median(vals), vals.min(), vals.max()))
print('seeds reaching >=95%% of optimum: %.0f%%' % (100*(vals >= 0.95*best).mean()))
print('fraction of space within 10%% of optimum: %.1f%%' % (100*(truth(grid) > 0.9*best).mean()))
```

Building and automating

Sequence design, assembly, biosensors and plate-scale workflows, in code you can run.

H1 · Design overhangs that will not mispair, and domesticate a site

week 21 · 1.5 h

You are assembling a transcription unit from six parts, which needs seven four-base overhangs including the two that join the vector.

A colleague hands you this set: CCCT, AACG, GCTT, ATGG, TACC, GGAA, CAGT.

Separately, this coding sequence has to be built with BsaI, which recognises GGTCTC:

ATGGGTCTCAAAGCTGAAGAAGTTCTGAAGCAGATC

Plain Python is enough; no libraries are needed.

1. Write a checker that flags palindromes, duplicates, reverse-complement pairs, overhangs only one base apart on either strand, and overhangs of low complexity or extreme GC content.
2. Run it on your colleague’s set, then on this deliberately poor set: AATG, GCGC, TTCA, TGAA, ACGT, CATT, AGCT.
3. Generate a set of seven that passes, and find out how large a set the same search can build.
4. Find every BsaI site in the coding sequence on both strands, and the overhang each would leave.
5. Remove the site with a synonymous change and prove by translation that the protein is unchanged.

Worked solution

Your colleague’s set looks fine and is not: the checker flags two near-collisions. CCCT and ATGG fail because the reverse complement of ATGG is CCAT, which differs from CCCT at a single position. TACC and GGAA fail for the same reason, since the reverse complement of GGAA is TTCC, one base from TACC. A one-base difference is enough for T4 ligase to join the wrong ends some of the time, which shows up as assemblies that look right on a gel and are wrong on sequencing. Nothing about these four-letter strings looks wrong by eye, which is why the check belongs in code.

The deliberately poor set fails on several counts: GCGC, ACGT and AGCT are palindromes (each is its own reverse complement, so it can pair with a copy of itself), and AATG with CATT, and TTCA with TGAA, are reverse-complement pairs.

Searching instead of guessing: scanning the four-base strings in order and keeping each one that passes against everything already chosen gives AAAC, AACA, AAGG, ACAA, ACCC, ACTG and AGAG, with no problems flagged. The same search can build at most 22 mutually compatible overhangs, which is a practical ceiling on how many parts one reaction can carry under these rules. Measured ligation-fidelity data are better than this rule of thumb and published tools use them, so treat the checker as a first filter.

The coding sequence has one BsaI site, at position 3 on the top strand, which would leave the overhang AAGC after the one-base spacer. Because GGTCTC spans the glycine and leucine codons, change CTC to CTG: both encode leucine, the site becomes GGTCTG, the translation is MGLKAEEVLKQI before and after, and no site remains on either strand. The same function applied to ATGAAAGAGACCTTTGGGAAA finds a site on the bottom strand, because GAGACC is the reverse complement of GGTCTC — a top-strand-only scan would have missed it.

What this does not do: it verifies logic, not chemistry. Ligation fidelity, secondary structure at overhangs and how well an enzyme handles a particular junction are empirical, so a design that

passes the checker is still tested.

Code

```
import itertools

def rc(s):  return s[::-1].translate(str.maketrans("ACGT", "TGCA"))
def ham(a, b): return sum(x != y for x, y in zip(a, b))

# standard genetic code, bases ordered T C A G
B  = "TCAG"
AA = "FFLLSSSSYY**CC*WLLLLPPPPHHQQRRRRIIIMTTTTNNKKSSRRVVVVAAAADDEEGGGG"
CODON = {a+b+c: AA[16*i+4*j+k] for i, a in enumerate(B) for j, b in enumerate(B) for k, c in enumerate(B)}
def translate(s): return "".join(CODON[s[i:i+3]] for i in range(0, len(s)-2, 3))

def single_ok(o):
    gc = o.count("G") + o.count("C")
    return o != rc(o) and len(set(o)) > 1 and 1 <= gc <= 3

def check(overhangs):
    problems = []
    for o in overhangs:
        if o == rc(o):            problems.append(("palindrome", o))
        elif not single_ok(o):    problems.append(("low complexity or extreme GC", o))
    for a, b in itertools.combinations(overhangs, 2):
        if a == b:                                problems.append(("duplicate", a))
        elif a == rc(b):                          problems.append(("reverse-complement pair", a, b))
        elif ham(a, b) <= 1 or ham(a, rc(b)) <= 1:   problems.append(("one-mismatch neighbour", a, b))
    return problems

def search(n):
    chosen = []
    for o in ("".join(p) for p in itertools.product("ACGT", repeat=4)):
        if single_ok(o) and all(ham(o, c) >= 2 and ham(o, rc(c)) >= 2 for c in chosen):
            chosen.append(o)
        if len(chosen) == n: break
    return chosen

hand = ["CCCT", "AACG", "GCTT", "ATGG", "TACC", "GGAA", "CAGT"]
bad  = ["AATG", "GCGC", "TTCA", "TGAA", "ACGT", "CATT", "AGCT"]
print("hand-picked set  :", hand)
for p in check(hand): print("   ", p)
print("a poor set       :", bad)
for p in check(bad):  print("   ", p)
good = search(7)
print("searched set of 7:", good, "->", check(good) or "no problems")
print("largest set the search can build:", len(search(99)))

def bsa_sites(seq, motif="GGTCTC"):
    """Recognition sites on both strands, with the 4-base overhang each would leave."""
    out = []
    for strand, s in (("top", seq), ("bottom", rc(seq))):
        for i in range(len(s) - len(motif) - 4):
            if s[i:i+len(motif)] == motif:
                pos = i if strand == "top" else len(seq) - i - len(motif)
                out.append((strand, pos, s[i+7:i+11]))   # cut after N1; overhang is the next four bases
    return out

orf = "ATGGGTCTCAAAGCTGAAGAAGTTCTGAAGCAGATC"
print("\nORF       :", orf, "->", translate(orf))
print("BsaI sites:", bsa_sites(orf))
fixed = orf.replace("GGTCTC", "GGTCTG")
print("domesticated:", fixed, "->", translate(fixed))
print("same protein:", translate(orf) == translate(fixed), "| sites left:", bsa_sites(fixed))
print("site on the bottom strand:", bsa_sites("ATGAAAGAGACCTTTGGGAAA"))

# --- Biopython equivalent (NOT executed where this was authored: Biopython was not installed).
# --- Check it against the version you install; Bio.Restriction reports CUT positions, not recognition positions.
# from Bio.Seq import Seq
# from Bio.Restriction import BsaI
# print(BsaI.search(Seq(orf)))          # 1-based cut positions on the top strand
```

H2 · How many transcription units can yeast assembly carry, and how do you check the overlaps?

week 21 · 1 h

A 2026 paper from KRIBB reports assembling up to eight transcription units in a single yeast transformation at 80% efficiency, using connector sequences to join units built by Golden Gate.

Treat that as a data point and ask what it implies. Then design the overlaps for a four-unit version.

1. Assuming the seven junctions succeed independently, compute the per-junction success implied by 80% overall.
2. Use that figure to predict the overall success for three, five and eight units, and the number of clones you would screen for a 95% chance of at least one correct.
3. Compute what eight units would give if each junction were only 95% or 90% reliable.
4. Design three 40 bp connectors for a four-unit assembly, check them for obvious mispairing, and assemble the design in silico.

Worked solution

Per-junction success is 0.80 to the power 1/7, which is 0.969.

Under that figure, three units (two junctions) succeed 93.8% of the time, five units (four junctions) 88.0% and eight units 80.0%. For a 95% chance of at least one correct clone you would screen about 1.1, 1.4 and 1.9 clones respectively — nearly every transformation yields a correct strain, and the screening burden is small, which is what makes such a result practical in a biofoundry.

The model's value is in its sensitivity. If each junction were only 95% reliable, eight units would give about 70%; at 90% they would give about 48%. Per-junction reliability compounds across every junction in a build, so it is the quantity to push, and it is the argument for connectors designed to be mutually distinct rather than reusing similar sequences.

Designing the connectors: three random 40 bp sequences came out at 48%, 55% and 58% GC, shared no 12-base stretch with each other (so no obvious mispairing between non-neighbouring junctions), and each occurred exactly once in the assembled 1,320 bp design (4 × 300 bp of units plus 3 × 40 bp of connectors). Random sequences are a starting point, not an answer: a real design would also screen against repeats in the units and the host genome, avoid long homopolymers and strong secondary structure, and check melting temperatures.

What the independence model leaves out: mispairing between similar overlaps, recombination between repeated sequences inside the units, and clones that look correct by junction PCR but carry a mutation. Use the numbers to reason about how a build scales, not to predict what a particular lab will see. The 80% figure comes from the paper's abstract, which is all I have read — read the paper for how it was measured.

Code

```
import math, random

# 1. what 80% overall for eight transcription units implies, if junctions were independent
overall, units = 0.80, 8
p_j = overall ** (1 / (units - 1))
print("per-junction success implied: %.4f" % p_j)
for k in (3, 5, 8):
    P = p_j ** (k - 1)
    n95 = math.log(0.05) / math.log(1 - P)
    print("  %d units, %d junctions: overall %.1f%%; clones to screen for a 95%% chance of >=1 correct: %.1f"
          % (k, k - 1, 100 * P, n95))
for pj in (0.90, 0.95):
    print("  if each junction were only %.2f: eight units -> %.1f%%" % (pj, 100 * pj ** 7))

# 2. design 40 bp connectors between four transcription units and check them
random.seed(4)
def rand(n): return "".join(random.choice("ACGT") for _ in range(n))
tus  = {"TU1": rand(300), "TU2": rand(300), "TU3": rand(300), "TU4": rand(300)}
conn = {"C1": rand(40), "C2": rand(40), "C3": rand(40)}   # C1 joins TU1-TU2, etc.
gc = lambda s: 100 * (s.count("G") + s.count("C")) / len(s)
print("\nconnector GC%:", {k: round(gc(v)) for k, v in conn.items()})
kmers = {}
for k, v in conn.items():
    for i in range(len(v) - 11):
        kmers.setdefault(v[i:i+12], set()).add(k)
shared = [m for m, ks in kmers.items() if len(ks) > 1]
print("12-mers shared between different connectors:", len(shared), "(0 means no obvious mispairing risk)")
design = tus["TU1"] + conn["C1"] + tus["TU2"] + conn["C2"] + tus["TU3"] + conn["C3"] + tus["TU4"]
print("assembled length:", len(design), "bp = 4x300 + 3x40")
print("each connector occurs exactly once:", all(design.count(v) == 1 for v in conn.values()))
```

I1 · Characterise a biosensor, and learn what a missing measurement costs you

week 26 · 2 h

A transcription-factor biosensor drives a fluorescent reporter. You measure output at 12 ligand concentrations from 0.5 to 2000 µM, with about 8% multiplicative noise. The true parameters, which you would not normally know, are baseline 60, maximum 5200, K = 45 µM and Hill coefficient 1.7.

Fit the four-parameter Hill function y = lo + (hi − lo)·x^n / (K^n + x^n).

1. Fit it with scipy.optimize.curve_fit and report each parameter with its standard error.
2. Compute the dynamic range and attach a bootstrap interval to it.
3. Repeat with three zero-ligand replicates added, and compare.
4. Compute the operating window between 10% and 90% of maximum response.
5. A sort gate recovers 70% of true hits and 1% of background; starting from one hit in ten thousand, compute the enrichment per round and the hit prevalence after three rounds.
6. How many events must you sort for ten-fold coverage of a million-member library?

Worked solution

Without a zero-ligand point the fit gives baseline 60 ± 193, maximum 5235 ± 218, K = 44.2 ± 6.6 µM and n = 1.74 ± 0.38. K and the maximum are well determined; the baseline is uncertain by more than three times its own value, because even the lowest dose (0.5 µM) is on the shoulder of the curve and gives the fit little to anchor the floor. The point estimate of dynamic range, 88-fold, happens to land close to the true 87, which would be reassuring if it meant anything — but the bootstrap 95% interval runs from 36-fold to no upper limit at all, because the baseline can be fitted as almost zero.

Adding three zero-ligand replicates changes the picture: baseline 99 ± 81, maximum 5219 ± 123, K = 46.5 ± 3.6, n = 1.96 ± 0.25, and a dynamic range of 53-fold with a bootstrap interval of 35 to 101-fold, which contains the true value. One cheap extra measurement did more than another dose point would have. The general lesson is that dynamic range divides by the smallest and noisiest number in the experiment, so it is the worst-determined summary of a sensor, and the baseline is the measurement to replicate.

Operating window, from the second fit: 10% of maximum response at K·(1/9)^(1/n) = 15.1 µM and 90% at K· 9^(1/n) = 143.0 µM, a 9.5-fold range, which equals 81^(1/n). Outside it the sensor is near its floor or saturated and cannot resolve concentration differences. A higher Hill coefficient narrows the window and sharpens the switch.

Sorting: enrichment per round is 0.70 / 0.01 = 70-fold. Starting from one hit in ten thousand, the hit prevalence is about 0.70% after one round, 32.9% after two and 97.2% after three. For ten-fold coverage of a million-member library you sort ten million events, about 8.3 minutes at 20,000 events per second. Two cautions: the model ignores cells lost to sorting stress and cells that slip through the gate by chance, and a sorted cell is only a hypothesis, because the sensor reports the intracellular signal and not necessarily the titer.

Code

```
import numpy as np
from scipy.optimize import curve_fit

hill = lambda x, lo, hi, K, n: lo + (hi - lo) * x**n / (K**n + x**n)
lo_t, hi_t, K_t, n_t = 60.0, 5200.0, 45.0, 1.7
bounds = ([0, 100, 1, 0.3], [500, 20000, 5000, 5])

def fit(conc, seed):
    rng = np.random.default_rng(seed)
    y = hill(conc, lo_t, hi_t, K_t, n_t) * rng.lognormal(0, 0.08, conc.size)   # 8% multiplicative noise
    p, cov = curve_fit(hill, conc, y, p0=[50, 4000, 30, 1.0], bounds=bounds, maxfev=20000)
    return p, np.sqrt(np.diag(cov))

def bootstrap_dynamic_range(conc, p, n=400, seed=5):
    rng = np.random.default_rng(seed); out = []
    f = hill(conc, *p)
    for _ in range(n):
        yb = f * rng.lognormal(0, 0.08, conc.size)
        try:
            pb, _ = curve_fit(hill, conc, yb, p0=p, bounds=bounds, maxfev=20000)
            out.append(pb[1] / max(pb[0], 1e-9))
        except Exception:
```

```
            pass
    return np.percentile(out, [2.5, 97.5])

A = np.array([0.5, 1, 2, 5, 10, 20, 50, 100, 200, 500, 1000, 2000.0])
B = np.concatenate([np.zeros(3), A])
for label, conc in (("no zero-ligand point", A), ("three zero-ligand replicates", B)):
    p, se = fit(conc, 11)
    ci = bootstrap_dynamic_range(conc, p)
    print("%-30s baseline %.0f±%.0f  max %.0f±%.0f  K %.1f±%.1f  n %.2f±%.2f"
          % (label, p[0], se[0], p[1], se[1], p[2], se[2], p[3], se[3]))
    print("   dynamic range %.0f-fold; bootstrap 95%% interval %.0f to %s"
          % (p[1] / p[0], ci[0], ("%.0f" % ci[1]) if ci[1] < 1e4 else "unbounded"))
    if label.startswith("three"):
        K, n = p[2], p[3]
        ec = lambda f: K * (f / (1 - f)) ** (1 / n)
        print("   operating window EC10-EC90: %.1f to %.1f uM  (a %.1f-fold range; theory 81^(1/n) = %.1f)"
              % (ec(.1), ec(.9), ec(.9) / ec(.1), 81 ** (1 / n)))

print("\nsorting")
p0, f_hit, f_bg = 1e-4, 0.70, 0.01
print("  enrichment per round: %.0f-fold" % (f_hit / f_bg))
p = p0
for r in (1, 2, 3):
    p = p * f_hit / (p * f_hit + (1 - p) * f_bg)
    print("  after round %d: hit prevalence %.4f (%.2f%%)" % (r, p, 100 * p))
lib, over, rate = 1e6, 10, 20000
print("  events for %dx coverage of a %.0e library: %.0e = %.1f minutes at %d events/s"
      % (over, lib, lib * over, lib * over / rate / 60, rate))
K, n = 45.0, 1.7
ec = lambda f: K * (f / (1 - f)) ** (1 / n)
print("\nK=45, n=1.7: EC10 %.1f, EC90 %.1f uM, window %.1f-fold" % (ec(.1), ec(.9), ec(.9) / ec(.1)))
```

J1 · From a variant list to a plate plan

week 34 · 1.5 h

Build 24 variants of a transcription unit by Golden Gate: four promoters, three coding sequences and two terminators, in a fixed vector. Each reaction is 10 µL: 6 µL of a master mix (buffer, enzymes and water) plus 1 µL each of the vector and three parts. Allow 10% overage on the master mix and 10 µL of dead volume per source well.

Produce the plan a person or a liquid handler could follow.

1. List the 24 variants and assign each to a destination well.
2. Compute the master mix to prepare.
3. Compute the volume each source well must contain.
4. Count part transfers and tips, assuming a fresh tip for every part.
5. Compute the reagent saving if the same reaction ran at 2 µL and at 500 nL, and the enzyme-and-buffer volume for a 384-variant library at each scale.

Worked solution

There are 4 × 3 × 2 = 24 variants, filling wells A1 to C8: rows A to C across columns 1 to 8, in the order promoter, coding sequence, terminator (A1 is P1-G1-T1 and C8 is P4-G3-T2).

Master mix: 24 × 6 µL × 1.10 = 158.4 µL.

Source wells need the volume used plus dead volume. A promoter appears in 6 reactions, so its well needs 6 × 1 × 1.10 + 10 = 16.6 µL. A coding sequence appears in 8 reactions: 18.8 µL. A terminator appears in 12: 23.2 µL. The vector appears in all 24: 36.4 µL. Note how dead volume dominates the small stocks: 10 µL of each 16.6 µL promoter well is unusable, which is why scarce parts are worth pre-diluting into a spare plate.

Transfers and tips: each reaction receives four part transfers, so there are 96 transfers and 96 tips with a fresh tip each time, plus one tip for the master mix if it is dispensed without touching liquid — 97 in all. Each reaction takes five dispenses: 6 µL of master mix and four 1 µL parts, which add to 10 µL.

Miniaturisation: at 2 µL a reaction uses 20% of the reagent (an 80% saving) and at 500 nL it uses 5% (a 95% saving). For 384 variants with 2 µL of enzyme and buffer per 10 µL reaction, that is 768 µL at the 10 µL scale and 38.4 µL at 500 nL. The saving is real; the catch is that evaporation, tip accuracy

and dead volume all get far worse as volume falls, and steps such as transformation and colony picking do not miniaturise at all, so the throughput gain is smaller than the reagent saving suggests.

The volumes here are illustrative, not a validated protocol: use your enzyme supplier's recommended amounts.

Code

```
import itertools

proms, cds, terms = ["P1", "P2", "P3", "P4"], ["G1", "G2", "G3"], ["T1", "T2"]
variants = list(itertools.product(proms, cds, terms))
V = len(variants)

rxn, mix_per, part, over, dead = 10.0, 6.0, 1.0, 1.10, 10.0     # all in uL
print("variants/reactions:", V)
print("master mix: %.1f uL for %d reactions at %.0f uL each with %.0f%% overage"
      % (V * mix_per * over, V, mix_per, 100 * (over - 1)))

uses = {"promoter": V // len(proms), "coding sequence": V // len(cds),
        "terminator": V // len(terms), "vector": V}
for name, u in uses.items():
    print("  %-16s used in %2d reactions -> source well needs %.1f uL (incl. %.0f uL dead volume)"
          % (name, u, u * part * over + dead, dead))

print("part transfers: %d (four per reaction), fresh tip each -> %d tips; one more for master mix: %d tips"
      % (V * 4, V * 4, V * 4 + 1))
print("per reaction: %.0f uL master mix + 4 x %.0f uL parts = %.0f uL" % (mix_per, part, mix_per + 4 * part))

# destination layout: fill rows A-C across columns 1-8
rows = "ABCDEFGH"
plan = []
for i, (p, g, t) in enumerate(variants):
    well = "%s%d" % (rows[i // 8], i % 8 + 1)
    plan.append((well, p, g, t))
print("destination wells used: %d of 96 (rows A-C, columns 1-8)" % len(plan))
print("first rows of the plan:"); [print("  ", r) for r in plan[:3]]
print("last row of the plan: ", plan[-1])

print("\nminiaturisation of the 10 uL reaction")
for vol in (10.0, 2.0, 0.5):
    print("  %4.1f uL -> reagent per reaction %.0f%% of original, saving %.0f%%"
          % (vol, 100 * vol / rxn, 100 * (1 - vol / rxn)))
print("384 variants, enzyme+buffer at 2 uL per 10 uL reaction: %.0f uL at 10 uL scale vs %.1f uL at 0.5 uL scale"
      % (384 * 2.0, 384 * 0.1))
```

J2 · An edge effect that looks like biology

week 34 · 1.5 h

A 96-well plate holds 96 variants whose true performance you want to rank. Evaporation makes the 36 edge wells read 18% too high. The true variant effects are normally distributed (mean 100, SD 12) and each reading carries about 3% noise.

Rank the variants from the raw plate, then see what it takes to correct the ranking.

1. Simulate the plate and rank the raw readings; count how many of the top ten are edge wells.
2. Estimate the edge multiplier from the plate alone and correct for it.
3. Compare the corrected ranking against the truth.
4. Say what plate design would let you measure the edge effect directly.

Worked solution

The edge wells are 36 of the 96, 38% of the plate, but 9 of the 10 highest raw readings are edge wells. With no position effect you would expect about four.

Estimating the edge multiplier from the plate itself — the median of the edge wells divided by the median of the interior wells — gives 1.194, close to the true 1.18. Dividing the edge readings by it lifts the Spearman correlation between measured and true variant effects from 0.77 to 0.96, and the corrected ranking recovers 8 of the true top ten variants against 3 from the raw ranking.

Three cautions. This works here because variants are placed at random, so edge and interior wells share the same true distribution; if you deliberately put your best candidates in the edge wells, the median trick would absorb part of their real effect. The proper fix is a design and not a correction: put

a reference strain in a scatter of edge and interior positions, randomise variant positions, or fill the edge wells with medium and leave them unused. And one multiplier is the simplest possible model; real plates often have row, column and thermal gradients, which a regression on position can handle once you have enough control wells to fit it.

Code

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(21)
rows, cols = 8, 12
R, C = np.meshgrid(np.arange(rows), np.arange(cols), indexing="ij")
edge = ((R == 0) | (R == rows - 1) | (C == 0) | (C == cols - 1)).astype(float)

true = rng.normal(100, 12, (rows, cols))                       # real variant effects
raw  = true * (1 + 0.18 * edge) * rng.lognormal(0, 0.03, (rows, cols))   # edge wells read 18% high

print("edge wells: %d of 96 (%.0f%% of the plate)" % (edge.sum(), 100 * edge.mean()))
top_raw = np.argsort(raw.ravel())[-10:]
print("of the raw top 10, %d are edge wells" % edge.ravel()[top_raw].sum())

mult = np.median(raw[edge == 1]) / np.median(raw[edge == 0])
corr = raw / np.where(edge == 1, mult, 1.0)
print("estimated edge multiplier: %.3f (true 1.18)" % mult)

rho = lambda a: stats.spearmanr(true.ravel(), a.ravel())[0]
print("Spearman correlation with the truth: raw %.3f -> corrected %.3f" % (rho(raw), rho(corr)))
top_true = set(np.argsort(true.ravel())[-10:])
print("of the true top 10: raw ranking recovers %d, corrected ranking recovers %d"
      % (len(top_true & set(top_raw)), len(top_true & set(np.argsort(corr.ravel())[-10:]))))
```

M1 · How likely is a plasmid to survive a strain’s restriction systems?

week 52 · 45 min

A wild industrial strain carries several Type II restriction systems. Your plasmid is 6,000 bp. Assume random sequence and treat each system as independent.

1. Compute the expected number of recognition sites for a 4, 6 and 8 base palindromic motif, and for a non-palindromic 6 base motif.
2. Convert each to the probability that the plasmid carries no site (a Poisson probability).
3. Compute the chance the plasmid avoids all of 1, 3 and 5 independent six-base systems.
4. Say what this implies for strategy.

Worked solution

A palindromic motif of k bases occurs by chance about L / 4^k times in L base pairs. For L = 6,000 that is 23.4 sites for a 4-base motif, 1.46 for a 6-base motif and 0.09 for an 8-base motif. A non-palindromic 6-base motif can sit on either strand, so it is expected 2.93 times.

The probability of no site is e to the power minus that number: 6.6×10^{-11} for the 4-base motif (a four-cutter essentially always cuts), 23% for a 6-base palindrome, 5.3% for a 6-base non-palindromic motif and 91% for an 8-base motif.

With three independent six-base systems, the chance of avoiding all of them is 1.2%, and with five it is 0.1%. A wild strain with several systems will therefore destroy almost any foreign DNA unless it is protected.

Strategy follows: protect rather than avoid. Methylating the plasmid in the cloning host with the strain’s own methyltransferases makes it look native; removing every site is rarely feasible once a strain has several systems; and delivery method, helper plasmids and transient heat treatment are the supporting tricks. This is a random-sequence calculation. Real genomes have codon and motif biases, and some strains avoid their own recognition sites, so the real expectation can differ — which is why the first practical step is to search REBASE for the strain’s actual systems and count the actual motifs in your actual sequence.

Code

```
import math
```

```
L = 6000   # plasmid length, bp
print("expected recognition sites in a %d bp plasmid, by motif" % L)
for k, palindromic in ((4, True), (6, True), (6, False), (8, True)):
    lam = L / 4**k if palindromic else 2 * L / 4**k     # a non-palindromic motif can sit on either strand
    print("  %d bp %-15s expected %.2f   P(none) = %.3g"
          % (k, "palindromic" if palindromic else "non-palindromic", lam, math.exp(-lam)))

lam6 = L / 4**6
print("\nprobability a plasmid avoids ALL of n independent 6 bp palindromic systems")
for n in (1, 3, 5):
    print("  n = %d: %.3f" % (n, math.exp(-n * lam6)))
```

PHA and gas-fermentation bioprocess

Yield ceilings, runs, and the economics and transfer limits of CO_2 and hydrogen routes, in code you can run.

K1 · The PHB yield ceiling, the redox surplus, and the oxygen it costs

week 56 · 1.5 h

Cupriavidus necator makes PHB from glucose. Glycolysis gives 2 ATP and 2 NADH per glucose, pyruvate dehydrogenase gives 2 NADH, PhaA joins two acetyl-CoA, PhaB reduces acetoacetyl-CoA using one NADPH, and PhaC polymerises. The PHB repeat unit is $C_4H_6O_2$, 86.09 g/mol; glucose is 180.16 g/mol.

Treat NADH and NADPH as interchangeable reducing equivalents.

1. Write the net equation from glucose to one PHB unit and check that every element balances.
2. Compute the mass yield and the carbon yield ceilings.
3. Count the reduced cofactors made and used, and give the surplus per glucose.
4. Compute the oxygen needed to re-oxidise the surplus, in kg per kg PHB.
5. Express overall yields of 0.30, 0.35 and 0.40 g/g as a percentage of the ceiling.

Worked solution

The net equation is $C_6H_{12}O_6 \rightarrow C_4H_6O_2 + 2\ CO_2 + 3\ H_2$, and it balances: 6 C, 12 H and 6 O on both sides. The 3 H_2 are the three surplus reduced cofactors written as hydrogen.

Yield ceilings: 86.09 / 180.16 = 0.478 g PHB per g glucose, so 2.09 kg of glucose per kg of PHB. The carbon yield is 4 of 6 carbons, 0.667, because two carbons leave as CO_2 at pyruvate dehydrogenase — 1.02 kg CO_2 per kg PHB.

Reducing equivalents: glycolysis and pyruvate dehydrogenase make 4, PhaB uses 1, leaving 3 per glucose. In a respiring cell they are cleared with oxygen at 0.5 O_2 each: 1.5 mol O_2 per glucose, which is 0.558 kg O_2 per kg PHB. That is the minimum, with no growth and no maintenance.

Overall yields of 0.30, 0.35 and 0.40 g/g are 63%, 73% and 84% of the ceiling. The gap is made of carbon that went to cells, to maintenance, and to by-products. A reviewer will ask which part of it your design addresses.

One caution: treating NADH and NADPH as interchangeable is a simplification. Cells have separate pools and transhydrogenase reactions, and how NADPH is supplied for PhaB is a real engineering question, so check it for your strain before you build on this number.

Code

```
# PHB from glucose: yield ceiling, carbon loss and the redox surplus
M = {"glucose": 180.156, "PHB unit": 86.089, "CO2": 44.009, "O2": 31.998}

# glucose -> 2 pyruvate (glycolysis: 2 ATP, 2 NADH) -> 2 acetyl-CoA + 2 CO2 (PDH: 2 NADH)
# 2 acetyl-CoA -> acetoacetyl-CoA (PhaA) -> 3-hydroxybutyryl-CoA (PhaB, 1 NADPH) -> PHB unit (PhaC)
nad_made  = 2 + 2
nadph_use = 1
surplus   = nad_made - nadph_use
print("reduced cofactors made %d, used by PhaB %d, surplus %d per glucose" % (nad_made, nadph_use, surplus))

# element balance of  C6H12O6 -> C4H6O2 + 2 CO2 + 3 H2   (the 3 H2 are the surplus reducing equivalents)
lhs = {"C": 6, "H": 12, "O": 6}
rhs = {"C": 4 + 2, "H": 6 + 3*2, "O": 2 + 2*2}
print("net equation balances:", lhs == rhs, "|", lhs, "->", rhs)

y_mass = M["PHB unit"] / M["glucose"]
y_C    = 4 / 6
print("mass yield ceiling : %.3f g PHB per g glucose" % y_mass)
```

```
print("carbon yield       : %.3f (one third of the carbon leaves as CO2)" % y_C)
print("glucose needed     : %.2f kg per kg PHB" % (1 / y_mass))
print("CO2 released       : %.2f kg per kg PHB" % (2 * M["CO2"] / M["PHB unit"]))

# surplus NADH must be re-oxidised with O2 (0.5 O2 per NADH)
o2_mol = surplus * 0.5
print("O2 to re-oxidise the surplus: %.1f mol per glucose = %.3f kg per kg PHB"
      % (o2_mol, o2_mol * M["O2"] / M["PHB unit"]))

# what a realistic overall yield means
for y in (0.30, 0.35, 0.40):
    print("  overall yield %.2f g/g = %.0f%% of the ceiling" % (y, 100 * y / y_mass))
```

K2 · Reading a fed-batch run: content, productivity and yield

week 56 · 1.5 h

An INVENTED fed-batch table for practice (not data from any paper). Time in h; total dry cell mass, PHB and cumulative glucose consumed in g/L:

t 0 12 24 36 48 60

X 4 22 45 72 95 113

PHB 0.2 2 10 29 52 70

Glucose 0 40 85 140 205 270

Use the ceiling yield of 0.478 g/g from problem K1.

1. Compute PHB content and residual (non-PHB) biomass at each time.
2. Compute overall volumetric productivity and productivity during the accumulation phase from 36 to 60 h.
3. Compute overall yield on glucose and yield during accumulation, as a percentage of the ceiling.
4. Say where residual biomass stops rising and what that implies about the feed.
5. Compute the total biomass needed to reach 70 g/L PHB at 50%, 70% and 80% content.

Worked solution

PHB content rises 5.0%, 9.1%, 22.2%, 40.3%, 54.7%, 61.9%. Residual biomass is 3.8, 20, 35, 43, 43 and 43 g/L: it stops rising after 36 h, which is what nutrient limitation looks like, with growth halted while polymer keeps accumulating.

Overall productivity is (70 − 0.2) / 60 = 1.16 g/L/h. During accumulation it is (70 − 29) / 24 = 1.71 g/L/h, because the cells are no longer spending carbon on growth.

Overall yield is 69.8 / 270 = 0.259 g/g, 54% of the 0.478 ceiling. During accumulation it is 41 / 130 = 0.315 g/g, 66% of the ceiling. The difference shows how much of the early carbon went into building cells.

Content and productivity are different targets. At 70 g/L PHB, a strain reaching 80% content needs only 88 g/L total biomass, 70% needs 100 g/L and 50% needs 140 g/L; the same polymer then rides on far fewer cells, which also eases oxygen and heat. This run reached 62% with 113 g/L, so its next gain is likely higher content or faster accumulation, not more cells.

The table is invented, so the numbers show the method rather than what a real culture of this kind achieves.

Code

```
import numpy as np
# ILLUSTRATIVE fed-batch table (invented for practice; not data from any paper)
t   = np.array([0, 12, 24, 36, 48, 60.0])          # h
X   = np.array([4, 22, 45, 72, 95, 113.0])         # total dry cell mass, g/L
P   = np.array([0.2, 2, 10, 29, 52, 70.0])         # PHB, g/L
S   = np.array([0, 40, 85, 140, 205, 270.0])       # glucose consumed, cumulative, g/L
Y_CEIL = 86.089 / 180.156

R = X - P                                          # residual (non-PHB) biomass
print("PHB content (% of cell mass):", np.round(100 * P / X, 1))
print("residual biomass (g/L):       ", np.round(R, 1))

print("\noverall productivity: %.2f g/L/h" % ((P[-1] - P[0]) / (t[-1] - t[0])))
```

```
i = 3                                              # start of the accumulation phase, 36 h
print("accumulation phase (36-60 h): %.2f g/L/h" % ((P[-1] - P[i]) / (t[-1] - t[i])))
print("overall yield on glucose:     %.3f g/g (%.0f%% of the %.3f ceiling)"
      % ((P[-1] - P[0]) / S[-1], 100 * ((P[-1] - P[0]) / S[-1]) / Y_CEIL, Y_CEIL))
ya = (P[-1] - P[i]) / (S[-1] - S[i])
print("yield in accumulation phase:  %.3f g/g (%.0f%% of ceiling)" % (ya, 100 * ya / Y_CEIL))

print("\nfinal PHB content: %.1f%%" % (100 * P[-1] / X[-1]))
print("residual biomass stops rising after t = %d h (%.0f g/L)" % (t[3], R[3]))
# what content would be needed at the same final PHB with less residual biomass?
for c in (0.5, 0.7, 0.8):
    print("  at %d%% content, 70 g/L PHB needs total biomass %.0f g/L" % (100*c, 70 / c))
```

K3 · What a CO_2-to-PHB route costs in hydrogen and electricity

week 57 · 2 h

Cupriavidus necator can grow autotrophically on H_2, CO_2 and O_2. Treat PHB as the product with unit $C_4H_6O_2$ (86.09 g/mol).

ASSUMPTIONS, all yours to change: net ATP per H_2 respired is 1.5; water electrolysis costs 52 kWh per kg H_2 for the whole system; glucose costs $0.40/kg. Calvin cycle: 3 ATP and 2 NADPH per CO_2 fixed; triose phosphate to pyruvate gives 2 ATP and 1 NADH; pyruvate dehydrogenase gives 1 NADH; PhaB uses 1 NADPH.

1. Write 4 CO_2 + 9 H_2 → $C_4H_6O_2$ + 6 H_2O, check the balance, and give the hydrogen floor per kg PHB.
2. Tally the same thing from the pathway: net ATP and net reducing equivalents per PHB unit.
3. Add the hydrogen respired for ATP at 1.0, 1.5 and 2.0 ATP per H_2.
4. Convert the base case to kWh per kg PHB and to electricity cost at $0.04, $0.08 and $0.12 per kWh.
5. Compute the feedstock cost of the sugar route at the ceiling yield and at 0.30 g/g, and the break-even electricity prices.

Worked solution

The floor equation balances (4 C, 8 O, 18 H on each side) and needs 9 mol H_2 per unit: 0.211 kg H_2 per kg PHB.

From the pathway: fixing 6 CO_2 costs 18 ATP and 12 NADPH; making two pyruvate returns 4 ATP and 2 NADH; two pyruvate dehydrogenase steps return 2 NADH; PhaB uses 1 NADPH. Net ATP is 14 and net reducing equivalents are 13 − 4 = 9. That matches the 9 H_2 of the floor, which is a useful independent check.

The floor counts reducing power only. The 14 ATP come from respiring more hydrogen. At 1.0, 1.5 and 2.0 ATP per H_2 the total is 23.0, 18.3 and 16.0 mol H_2 per unit, which is 0.539, 0.429 and 0.375 kg H_2 per kg PHB.

Base case, 1.5 ATP/H_2: 0.429 kg H_2 × 52 kWh = 22.3 kWh per kg PHB. Electricity alone costs $0.89, $1.79 and $2.68 per kg PHB at $0.04, $0.08 and $0.12 per kWh.

Sugar route: 0.40 / 0.478 = $0.84 per kg PHB at the ceiling, and $1.33 at an overall yield of 0.30 g/g. Break-even electricity prices are therefore $0.0375/kWh against the ceiling and $0.0597/kWh against 0.30 g/g.

So with these assumptions the CO_2 route matches sugar only where electricity is very cheap, and this counts electricity alone, with no electrolyser capital, CO_2 supply, gas handling or downstream, which add to it. The answer is sensitive to the ATP yield per hydrogen and to electrolysis efficiency; the model shows where the argument has to be won, not that the route fails. Compute your own version, and say plainly which assumption carries it.

Code

```
# CO2 + H2 -> PHB: stoichiometry, energy, and electricity cost. Every assumption is labelled.
M_PHB, M_H2 = 86.089, 2.016

# --- 1. chemical floor: 4 CO2 + 9 H2 -> C4H6O2 + 6 H2O
lhs = {"C": 4, "O": 8, "H": 18}; rhs = {"C": 4, "O": 2 + 6, "H": 6 + 12}
print("floor equation balances:", lhs == rhs)
print("floor: 9 mol H2 = %.3f kg H2 per kg PHB" % (9 * M_H2 / M_PHB))
```

```
# --- 2. tally the same thing from the pathway (Calvin cycle -> acetyl-CoA -> PHB)
co2_fixed   = 6                      # to make 2 triose phosphates; 2 CO2 are lost again at PDH
atp_cbb     = 3 * co2_fixed          # 3 ATP per CO2 fixed
nadph_cbb   = 2 * co2_fixed          # 2 NADPH per CO2 fixed
atp_gain    = 2 * 2                  # triose-P -> pyruvate: 2 ATP each
nadh_gain   = 2 * 1 + 2 * 1          # triose-P -> pyruvate (1 NADH each) + PDH (1 NADH each)
nadph_phab  = 1
atp_net     = atp_cbb - atp_gain
red_net     = nadph_cbb + nadph_phab - nadh_gain
print("pathway tally: net ATP %d, net reducing equivalents %d (should equal the 9 H2 above: %s)"
      % (atp_net, red_net, red_net == 9))

# --- 3. ATP from respiring H2 (ASSUMPTION: ATP per H2 oxidised)
print("\nATP per H2 oxidised (assumption) -> total H2 per PHB unit and per kg PHB")
for a in (1.0, 1.5, 2.0):
    mol = red_net + atp_net / a
    print("  %.1f  -> %.1f mol H2 = %.3f kg H2 per kg PHB" % (a, mol, mol * M_H2 / M_PHB))
a = 1.5
kgH2 = (red_net + atp_net / a) * M_H2 / M_PHB

# --- 4. electricity (ASSUMPTION: 52 kWh per kg H2 from electrolysis, whole system)
E = 52.0
kwh = kgH2 * E
print("\nbase case (1.5 ATP/H2): %.3f kg H2 -> %.1f kWh per kg PHB" % (kgH2, kwh))
for p in (0.04, 0.08, 0.12):
    print("  electricity at $%.2f/kWh -> $%.2f per kg PHB, electricity only" % (p, kwh * p))

# --- 5. the sugar route for comparison (ASSUMPTION: glucose $0.40/kg)
y_ceil = M_PHB / 180.156
for y, lab in ((y_ceil, "theoretical ceiling"), (0.30, "overall yield 0.30 g/g")):
    print("  glucose route, %-22s: $%.2f per kg PHB, feedstock only" % (lab, 0.40 / y))
print("break-even electricity price vs glucose at the ceiling: $%.4f/kWh" % ((0.40 / y_ceil) / kwh))
print("break-even electricity price vs glucose at 0.30 g/g:    $%.4f/kWh" % ((0.40 / 0.30) / kwh))
```

K4 · Gas transfer for hydrogen and oxygen, and which one limits

week 57 · 1.5 h

Using the stoichiometry and ATP assumption of problem K3, plan gas transfer for a PHB culture on H_2, CO_2 and O_2 at 1 and 2 g PHB per litre per hour.

ASSUMPTIONS: Henry solubilities 0.75 mM/atm for H_2 and 1.15 mM/atm for O_2 at about 30 °C; gas phase 0.70 atm H_2 and 0.05 atm O_2 (oxygen held low as a modelling assumption); dissolved concentrations 0.05 mM H_2 and 0.01 mM O_2 in the bulk. The 0.05 atm figure is NOT a safe operating recommendation.

1. Compute the hydrogen and oxygen demand per g PHB.
2. Compute the saturation concentration and driving force for each gas.
3. Compute the kLa each gas needs at 1 and at 2 g/L/h.
4. State which gas limits.
5. Find the oxygen fraction at which oxygen would need no more kLa than hydrogen.

Worked solution

Per gram of PHB the culture respires or fixes 213 mmol of H_2 and consumes 54.2 mmol of O_2 (only the hydrogen respired for ATP needs oxygen, at 0.5 O_2 each).

Hydrogen: C* = 0.75 × 0.70 = 0.525 mM, driving force 0.475 mM. Oxygen: C* = 1.15 × 0.05 = 0.0575 mM, driving force 0.0475 mM.

At 1 g/L/h the required kLa is 213 / 0.475 = 448 h^{-1} for hydrogen and 54.2 / 0.0475 = 1,141 h^{-1} for oxygen. At 2 g/L/h both double: 897 and 2,282 h^{-1}. Oxygen limits, despite the much smaller molar demand, because its driving force is about ten times smaller.

For oxygen to need no more than hydrogen's 448 h^{-1} at 1 g/L/h, the gas would need about 11.4% oxygen. That is exactly where the safety constraint bites: the more oxygen you allow, the easier the transfer and the closer the mixture is to the flammable region.

Compare these numbers with the kLa your reactor and gassing strategy can actually reach, using the correlations of the bioreactor week, and with the power and shear each option costs. This is why high productivity gas fermentation is a reactor-design problem as much as a strain problem.

Safety: flammable limits depend on the exact mixture, temperature and pressure. The 5% oxygen used here is an assumption for the arithmetic, not an operating condition. Any real hydrogen and oxygen work needs a formal hazard review first.

Code

```
# Gas transfer for a H2/CO2/O2 PHB fermentation. All parameters are ASSUMPTIONS, stated here.
M_PHB = 86.089
red_net, atp_net, a = 9, 14, 1.5            # from the K3 tally; a = ATP per H2 respired
h2_total = red_net + atp_net / a            # mol H2 per mol PHB unit
o2_resp  = (atp_net / a) * 0.5              # mol O2: only the H2 respired for ATP needs O2

H_H2, H_O2 = 0.75, 1.15                     # mM per atm (water, about 30 C)
p_H2, p_O2 = 0.70, 0.05                     # atm in the gas (O2 held at 5 vol% for safety)
CL_H2, CL_O2 = 0.05, 0.01                   # mM dissolved in the bulk

print("per g PHB: H2 %.1f mmol, O2 %.1f mmol" % (1000*h2_total/M_PHB, 1000*o2_resp/M_PHB))
for prod in (1.0, 2.0):                     # g PHB per L per h
    r_h2 = 1000 * h2_total / M_PHB * prod
    r_o2 = 1000 * o2_resp / M_PHB * prod
    cs_h2, cs_o2 = H_H2 * p_H2, H_O2 * p_O2
    k_h2 = r_h2 / (cs_h2 - CL_H2)
    k_o2 = r_o2 / (cs_o2 - CL_O2)
    print("\nproductivity %.0f g/L/h" % prod)
    print("  H2: uptake %.0f mmol/L/h, C* %.3f mM, driving force %.3f mM -> kLa %.0f 1/h" % (r_h2, cs_h2, cs_h2 - CL_H2, k_h2))
    print("  O2: uptake %.1f mmol/L/h, C* %.4f mM, driving force %.4f mM -> kLa %.0f 1/h" % (r_o2, cs_o2, cs_o2 - CL_O2, k_o2))
    print("  the limiting gas needs kLa >= %.0f 1/h (%s)" % (max(k_h2, k_o2), "O2" if k_o2 > k_h2 else "H2"))

# what O2 fraction would bring the O2 requirement down to the H2 requirement at 1 g/L/h?
r_h2 = 1000*h2_total/M_PHB; r_o2 = 1000*o2_resp/M_PHB
k_target = r_h2 / (H_H2*p_H2 - CL_H2)
p_needed = (r_o2 / k_target + CL_O2) / H_O2
print("\nfor O2 to need no more kLa than H2 does (%.0f 1/h), O2 would have to be %.1f vol%% of the gas" % (k_target, 100*p_needed))
```

Enzymes and natural products

Is an enzyme worth engineering, how big a screen is, and what a polyketide demands of a strain.

L1 · Is the screen good enough, and how big must it be?

week 30 · 1.5 h

You plan to screen an enzyme library in 96-well plates.

Assay A: positive control 1000 ± 60, negative control 100 ± 40.

Assay B: positive control 1000 ± 150, negative control 100 ± 120.

Library options: site-saturation with NNK codons (32 codons, 20 amino acids and one stop) or NDT codons (12 codons, 12 amino acids). You can screen 960 clones a day (ASSUMPTION).

1. Compute the Z′ factor of each assay and say which you would screen with.
2. For a 10,000-clone library with no real improvement present, compute the false hits at hit thresholds of two, three and four standard deviations above wild type.
3. Compute the clones needed for 95% coverage of every codon combination, for one, two and three sites, with NNK and with NDT.
4. Convert the NNK numbers into days at your throughput.
5. Say what you would do about three sites.

Worked solution

Z′ = 1 − 3(σp + σn)/|μp − μn|. Assay A gives 1 − 3(100)/900 = 0.67, a good screen. Assay B has the same means but more noise: 1 − 3(270)/900 = 0.10, which is marginal. Same signal, much worse separation, so only A is worth screening with.

False hits in 10,000 clones with no real improvement: a threshold at the wild-type mean plus two standard deviations gives 2.28% of clones, about 228 false hits; at three standard deviations 0.13%, about 13; at four, essentially none. The threshold, the noise and the library size have to be chosen together.

Coverage: NNK, one site, has 32 variants and needs 94 clones for 95% coverage; two sites have 1,024 variants and need 3,066 clones; three sites have 32,768 variants and need 98,163 clones. NDT is far

smaller: 34, 430 and 5,175 clones. NNK is also biased, because leucine, arginine and serine take three of the 32 codons each (9.4%) while single-codon amino acids take 3.1%.

At 960 clones a day, one NNK site takes 0.1 days, two take 3.2 days and three take about 102 days. Three simultaneous saturated sites are therefore rarely practical. Randomise sites in rounds, use a reduced codon set such as NDT, or use a structure or sequence model from week 29 to choose which positions to randomise at all.

A caution: coverage arithmetic assumes every variant is equally likely, which NNK is not, and it assumes each clone was sequenced correctly. Real libraries need a quality check before screening.

Code

```
import math

# ---- 1. is the assay good enough to screen with?  Z' = 1 - 3(sd_pos + sd_neg) / |mean_pos - mean_neg|
def zprime(mp, sp, mn, sn): return 1 - 3 * (sp + sn) / abs(mp - mn)
print("assay A: positive 1000 +/- 60, negative 100 +/- 40  -> Z' = %.2f" % zprime(1000, 60, 100, 40))
print("assay B: positive 1000 +/- 150, negative 100 +/- 120 -> Z' = %.2f" % zprime(1000, 150, 100, 120))
print("rule of thumb: above 0.5 is a good screen, between 0 and 0.5 marginal, below 0 unusable")

# ---- 2. false positives from noise, for a library of N clones (one-sided normal tail)
def tail(z): return 0.5 * math.erfc(z / math.sqrt(2))
N = 10_000
print("\nfalse hits among %d clones with NO real improvement:" % N)
for z in (2, 3, 4):
    print("  threshold wild-type mean + %d sd: %.2f%% of clones, about %.0f false hits" % (z, 100 * tail(z), N * tail(z)))

# ---- 3. site-saturation libraries.  NNK: 32 codons, 20 amino acids + 1 stop.  NDT: 12 codons, 12 amino acids.
def clones_for(P, variants): return math.log(1 - P) / math.log(1 - 1 / variants)
print("\nclones needed for 95% coverage of every codon combination")
for name, codons in (("NNK", 32), ("NDT", 12)):
    for k in (1, 2, 3):
        V = codons ** k
        print("  %s, %d site%s: %9d variants -> %9.0f clones" % (name, k, "" if k == 1 else "s", V, clones_for(0.95, V)))

rate = 960      # clones per day: ten 96-well plates (ASSUMPTION)
print("\nat %d clones per day:" % rate)
for k in (1, 2, 3):
    n = clones_for(0.95, 32 ** k)
    print("  NNK x %d site%s: %.1f days" % (k, "" if k == 1 else "s", n / rate))

# ---- 4. amino-acid-level view of NNK: codon degeneracy makes some residues far more likely
deg = {"Leu": 3, "Arg": 3, "Ser": 3}          # 3 of the 32 NNK codons each; every other amino acid has 1 or 2
print("\nNNK is biased: Leu, Arg and Ser each take 3 of 32 codons (%.1f%% each) against 1 of 32 (%.1f%%) for amino acids with a si
```

L2 · Is this enzyme worth engineering?

week 30 · 1.5 h

Three enzymes in a linear pathway hold flux control coefficients of 0.77, 0.22 and 0.05. You could engineer any one of them, and a campaign would raise its activity by 2-fold, 10-fold or 100-fold.

ASSUMPTION: a linear pathway at fixed driving force, where flux is set by the sum of the enzymes' resistances and an enzyme with control coefficient C carries a fraction C of that total. Scaling its activity by a factor f then changes the flux to J′/J = 1 / (1 − C + C/f).

1. Tabulate the flux gain for each enzyme at f = 2, 10 and 100, and the ceiling as f grows without limit.
2. Compare with the first-order estimate, flux gain ≈ C × (f − 1), for a small change.
3. Find the activity increase each enzyme needs to deliver a 25% flux gain.
4. Say what you would do with this before starting a campaign.

Worked solution

For the 0.77 enzyme the gains are 63% at f = 2, 226% at f = 10 and 321% at f = 100, against a ceiling of 335%. For the 0.22 enzyme they are 12%, 25% and 28%, with a ceiling of 28%. For the 0.05 enzyme they are 3%, 5% and 5%, with a ceiling of 5%.

The ceiling is 1/(1 − C) − 1, so an enzyme with little control cannot help much however good it becomes. For a small change the first-order estimate is close: at f = 1.2 and C = 0.22 it gives 4.4% against 3.8% from the full formula, and the estimate overstates as the change grows.

For a 25% flux gain you need only a 1.4-fold improvement from the 0.77 enzyme, an 11-fold improvement from the 0.22 enzyme, and it is impossible with the 0.05 enzyme because its ceiling is 5%.

Before a campaign, estimate the control coefficients of your own pathway, as in the control analysis and kinetic modelling weeks, and spend effort on the enzymes that hold it. This model is a simplification: real pathways have feedback and shifting driving forces, so control moves as you change things. Treat the table as a way to rank candidates, and measure after the first improvement to see whether control has shifted.

Code

```
# Is this enzyme worth engineering?  Linear pathway at fixed driving force: flux is set by the sum of
# "resistances", and enzyme i carries a fraction C_i of the total (its flux control coefficient).
# Scaling that enzyme's activity by f changes the flux to  J'/J = 1 / (1 - C + C/f).
def gain(C, f): return 1 / (1 - C + C / f) - 1

print("flux gain from scaling ONE enzyme's activity by a factor f (ASSUMPTION: linear pathway, additive resistances)")
print("%-34s %7s %7s %7s %9s" % ("flux control coefficient C", "f = 2", "f = 10", "f = 100", "ceiling"))
for C, label in ((0.77, "0.77 (the controlling step)"), (0.22, "0.22"), (0.05, "0.05")):
    print("%-34s %6.0f%% %6.0f%% %6.0f%% %8.0f%%" % (label, 100*gain(C, 2), 100*gain(C, 10), 100*gain(C, 100), 100*(1/(1-C) - 1)))

print("\nfirst-order check for a small change (f = 1.2, C = 0.22): linear estimate %.1f%%, formula %.1f%%" % (100*0.22*0.2, 100*ga

# how much improvement does it take to deliver a 25% flux gain from each enzyme?
print("\nactivity increase needed for a 25% flux gain:")
for C in (0.77, 0.22, 0.05):
    target = 0.25
    denom = 1/(1+target) - (1 - C)
    f = C / denom if denom > 0 else float("inf")
    print("  C = %.2f: %s" % (C, ("f = %.1f-fold" % f) if f != float("inf") and f > 0 else "impossible: the ceiling is only %.0f%%
```

L3 · What a polyketide demands of its host

week 31 · 1.5 h

The 6-deoxyerythronolide B synthase makes the macrolactone core of erythromycin. The IUBMB entry (EC 2.3.1.94) gives its reaction:

propanoyl-CoA + 6 (2S)-methylmalonyl-CoA + 6 NADPH + 6 H^+ → 6-deoxyerythronolide B + 7 CoA + 6 CO_2 + H_2O + 6 $NADP^+$

ASSUMPTIONS: every unit is supplied as propionate; converting propionate to propionyl-CoA costs 2 ATP-equivalents (ATP to AMP), and carboxylating propionyl-CoA to methylmalonyl-CoA costs 1 ATP, for the six extender units only.

1. Find the product’s formula and molar mass by element balance, using the CoA thioesters’ formulas.
2. Count the acyl units, the NADPH and the CO_2 released per product molecule.
3. Compute the ATP-equivalents needed if everything comes from propionate, and the mass-yield ceiling on propionic acid.
4. Compute the propionic acid and NADPH needed for titers of 0.5, 1 and 5 g/L.
5. Say what this implies for how a strain should be built.

Worked solution

Element balance on the IUBMB reaction gives $C_{21}H_{38}O_6$ for the product, molar mass 386.53 g/mol, which is 6-deoxyerythronolide B. That the balance closes is an independent check on the reaction as written.

Per product molecule the synthase uses 7 acyl units (one starter, six extenders), 6 NADPH and releases 6 CO_2. Seven C3 units give 21 carbons, all of which end up in the product; the CO_2 fixed in making methylmalonyl-CoA is released again at each condensation.

From propionate, activation costs 7 × 2 = 14 ATP-equivalents and carboxylation 6 × 1 = 6, so 20 in all. The mass-yield ceiling on propionic acid is 386.53 / (7 × 74.08) = 0.745 g/g.

Titers: 0.5 g/L is 1.29 mM product, needing at least 0.67 g/L propionic acid (9.1 mM) and 7.8 mM NADPH; 1 g/L is 2.59 mM, needing 1.34 g/L propionic acid (18.1 mM) and 15.5 mM NADPH; 5 g/L is 12.94 mM, needing 6.71 g/L propionic acid (90.5 mM) and 77.6 mM NADPH. At 0.05 g/L/h the NADPH

flux needed is 0.78 mmol/L/h.

The lesson: propionate demand, its toxicity and its cost all grow with the titer, and the cell also has to supply the NADPH. Strains for high titers therefore build methylmalonyl-CoA from central metabolism and engineer NADPH supply, rather than relying on feeding. The ATP counts are assumptions, so treat the ranking of constraints as the result, not the exact figures.

Code

```
# The 6-deoxyerythronolide B (6-dEB) pathway: precursor, reducing-power and ATP demand.
# Stoichiometry from the IUBMB entry for EC 2.3.1.94:
#   propanoyl-CoA + 6 (2S)-methylmalonyl-CoA + 6 NADPH + 6 H+
#       -> 6-deoxyerythronolide B + 7 CoA + 6 CO2 + H2O + 6 NADP+

# 1. check the product formula by element balance (CoA thioesters carry their own atoms)
F = {  # formulas as C, H, N, O, P, S
 "propionyl-CoA":     dict(C=24, H=40, N=7, O=17, P=3, S=1),
 "methylmalonyl-CoA": dict(C=25, H=40, N=7, O=19, P=3, S=1),
 "CoA":               dict(C=21, H=36, N=7, O=16, P=3, S=1),
 "CO2": dict(C=1, O=2), "H2O": dict(H=2, O=1),
}
left  = {}
def add(d, f, n):
    for k, v in f.items(): d[k] = d.get(k, 0) + n * v
add(left, F["propionyl-CoA"], 1); add(left, F["methylmalonyl-CoA"], 6)
left["H"] = left.get("H", 0) + 6 + 6          # 6 hydride-equivalents from NADPH, 6 H+
right = {}
add(right, F["CoA"], 7); add(right, F["CO2"], 6); add(right, F["H2O"], 1)
prod = {k: left.get(k, 0) - right.get(k, 0) for k in "CHNOPS"}
print("product by element balance:", {k: v for k, v in prod.items() if v})
M = {"C": 12.011, "H": 1.008, "O": 15.999}
M_dEB = sum(prod[k] * M[k] for k in "CHO")
print("molar mass of C%dH%dO%d = %.2f g/mol" % (prod["C"], prod["H"], prod["O"], M_dEB))

# 2. what it takes to make one mole
units, nadph, co2 = 7, 6, 6
print("\nper mol 6-dEB: %d acyl units (1 starter + 6 extender), %d NADPH, %d CO2 released" % (units, nadph, co2))

# 3. ASSUMPTIONS: every unit comes from propionate. Activation costs 2 ATP-equivalents (ATP -> AMP),
#    carboxylation to methylmalonyl-CoA costs 1 ATP each, for the six extender units.
atp_activ, atp_carb = 2 * units, 1 * 6
print("ATP-equivalents if built from propionate: activation %d + carboxylation %d = %d" % (atp_activ, atp_carb, atp_activ + atp_ca

M_prop = 74.079     # propionic acid
y_ceiling = M_dEB / (units * M_prop)
print("mass yield ceiling on propionic acid: %.3f g/g" % y_ceiling)
print("carbon check: 7 x 3 = %d carbons in, %d in the product (the CO2 fixed and released cancel)" % (units * 3, prod["C"]))

# 4. a titer target
for titer in (0.5, 1.0, 5.0):
    mM = titer / M_dEB * 1000
    prop_g = mM * units * M_prop / 1000
    print("target %.1f g/L: %.2f mM product -> propionic acid at least %.2f g/L (%.1f mM) and NADPH %.1f mM"
          % (titer, mM, prop_g, mM * units, mM * nadph))
rate = 0.05   # g/L/h
print("at %.2f g/L/h the NADPH flux needed is %.2f mmol/L/h" % (rate, rate / M_dEB * 1000 * nadph))
```

L4 · How many BAC clones to capture a cluster whole?

week 31 · 1 h

You want a 60 kb biosynthetic gene cluster from a Streptomyces genome of about 8.7 Mb (an assumption; S. coelicolor is close to this). Your BAC library has 100 kb inserts at random positions.

A clone holds the cluster whole only if the insert starts within (insert size − cluster size) of the cluster's start.

1. Compute the probability that one clone holds the cluster whole.
2. Compute the clones needed for a 99% chance of at least one such clone.
3. Repeat for 10, 30, 80 and 90 kb clusters.
4. Compute the same at 90%, 95%, 99% and 99.9% confidence for the 60 kb cluster.
5. Compare with an estimate that ignores cluster size, and say what you would do about a cluster near the insert size.

Worked solution

For a 60 kb cluster the effective window is 100 − 60 = 40 kb, so the probability per clone is 40 / 8,700 = 0.0046. The number of clones for 99% confidence is ln(0.01) / ln(1 − 0.0046) = about 999.

By cluster size at 99% confidence: 10 kb needs 443 clones, 30 kb needs 570, 60 kb needs 999, 80 kb needs 2,001 and 90 kb needs 4,004. As the cluster approaches the insert size the window collapses and the library has to grow sharply.

For the 60 kb cluster, 90%, 95%, 99% and 99.9% confidence need 500, 650, 999 and 1,499 clones. In 96-well plates, 999 clones is about 10.4 plates.

An estimate that uses only insert size over genome size gives 398 clones, which is about 60% too few. For a cluster close to the insert size, switch strategy: use larger inserts, or clone directly from genomic DNA by yeast recombination, as in the assembly week, instead of screening a random library.

This model assumes random, unbiased insert positions and complete coverage of the genome, and real libraries are biased by GC content and by toxic regions. Treat the numbers as a lower bound, and verify the clone you find by sequencing.

Code

```
import math
# How many BAC clones to catch a cluster whole? Clarke-Carbon style, with the cluster size built in.
G  = 8.7e6      # bp, a typical Streptomyces genome (ASSUMPTION; S. coelicolor is about 8.7 Mb)
L  = 100e3      # bp insert size (ASSUMPTION)

def clones(c, P):
    # a clone contains the whole cluster only if the insert starts within (L - c) of the cluster's start
    f = max(L - c, 0) / G
    return math.log(1 - P) / math.log(1 - f), f

print("genome %.1f Mb, insert %d kb" % (G / 1e6, L / 1e3))
print("\nclones needed for a 99% chance of at least one clone holding the whole cluster")
for c in (10e3, 30e3, 60e3, 80e3, 90e3):
    n, f = clones(c, 0.99)
    print("  cluster %2d kb: P(one clone holds it) = %.4f  ->  %6.0f clones" % (c / 1e3, f, n))

print("\nthe same for a 60 kb cluster at other confidence levels")
for P in (0.90, 0.95, 0.99, 0.999):
    n, f = clones(60e3, P)
    print("  P = %.3f: %5.0f clones" % (P, n))

print("\nnaive estimate that ignores cluster size (insert / genome only), for 60 kb at 99%:")
f0 = L / G; print("  %.0f clones, which is %.0f%% too few" % (math.log(0.01) / math.log(1 - f0),
      100 * (1 - (math.log(0.01) / math.log(1 - f0)) / clones(60e3, 0.99)[0])))
print("\nin 96-well plates: %.1f plates for 60 kb at 99%%" % (clones(60e3, 0.99)[0] / 96))
```

Rewriting genomes and choosing hosts

Edit counts, error budgets, host speed and whether a toolkit repays itself.

N1 · How much rewriting, and how likely is it to come out right?

week 25 · 1.5 h

A 4 Mb synthetic E. coli genome, as reported by Fredens et al. (2019), recoded 18,214 codons and was built from sections 91 to 136 kb long.

ASSUMPTIONS: about 88% of the genome is protein-coding, and the residual error rates per base after assembly and before any repair are 10^{-5}, 10^{-6} and 10^{-7}.

1. Compute the recoded codons per kb, and the number in a 100 kb section.
2. Compute the share of all codons that are changed.
3. Compute how many sections the genome splits into at 91, 100 and 136 kb.
4. For each error rate compute the expected errors and the chance of being clean for one 100 kb section and for the whole genome.
5. Say what this implies for how a genome project should be run.

Worked solution

The design recodes 18,214 codons in a 4,000 kb genome, which is 4.55 per kb, or about 455 in a 100 kb section. If 88% of the genome is coding, an assumption, it holds about 1.17 million codons, so about 1.55% of them change.

A 4,000 kb genome splits into 44 sections of 91 kb, 40 of 100 kb or 30 of 136 kb.

Error budget. At one error in 100,000 bases, a 100 kb section carries 1.0 errors on average and is clean 36.8% of the time, while the whole genome carries 40 errors and has a chance near 4×10^{-18} of being clean. At one in a million, a section has 0.10 expected errors and is clean 90.5% of the time, but the whole genome has 4 expected errors and is clean only 1.8% of the time. At one in ten million, sections are clean 99.0% of the time and the genome 67%.

The lesson is that a per-section success rate that looks excellent still leaves a whole genome almost certainly wrong. That is why genome projects verify by sequencing at every stage and fix errors in the section where they are cheap to find, not assemble everything and check at the end. The error rates are assumptions, so use the structure of the result and not the exact figures. The real project also had to correct design problems by hand, such as overlapping genes.

Code

```
import math
# Rewriting a genome: how many edits, and how likely is it to come out right?
# Verified inputs (Fredens et al., Nature 2019): a 4 Mb synthetic E. coli genome, 18,214 codons recoded,
# built from sections 91 to 136 kb long.
genome_kb = 4000.0
recoded = 18214
# ASSUMPTION: about 88% of an E. coli genome is protein-coding.
coding = 0.88
codons_total = genome_kb * 1000 * coding / 3

print("recoded codons per kb of genome: %.2f  (about %.0f per 100 kb section)" % (recoded / genome_kb, 100 * recoded / genome_kb))
print("share of all codons changed: %.2f%%  (of about %.2f million)" % (100 * recoded / codons_total, codons_total / 1e6))

print("\nnumber of sections the genome splits into")
for L in (91, 100, 136):
    print("  %3d kb sections: %.0f" % (L, math.ceil(genome_kb / L)))

# ASSUMPTION: residual error rates per base after assembly, before any repair
print("\nerror budget, for one 100 kb section and for the whole genome (Poisson)")
print("%-14s %-22s %-18s %-22s %s" % ("error per bp", "expected errors/section", "P(section clean)", "expected errors/genome", "P(
for r in (1e-5, 1e-6, 1e-7):
    lam_s = r * 100e3
    lam_g = r * genome_kb * 1000
    print("%-14s %-22.2f %-18.3f %-22.1f %.3g" % ("%.0e" % r, lam_s, math.exp(-lam_s), lam_g, math.exp(-lam_g)))
```

N2 · Is a faster host worth the toolkit?

week 25 · 1.5 h

A preprint (Weinstock et al., 2016) measured generation times at 37 °C of 14.8 minutes for Vibrio natriegens in LB3 and 31.3 minutes for E. coli in LB.

ASSUMPTIONS: one cell to about a billion cells is 30 doublings; each build–test cycle has two such growth steps; a working day is 8 hours; building the toolkit for the new host takes 40 working days; transformation efficiencies of 10^3, 10^5 and 10^7 CFU per µg of DNA.

1. Compute the speed ratio and the time for 30 doublings in each host.
2. Compute the time saved per cycle, in hours and working days.
3. Find the number of cycles at which the toolkit repays itself, and compare projects of 12 and 40 cycles.
4. Compute the transformants needed for 95% coverage of a 10,000-variant library, and the DNA needed at each efficiency.
5. Say when you would use the faster host and when you would not.

Worked solution

At 37 °C V. natriegens divides every 14.8 minutes and E. coli every 31.3 minutes, a ratio of 2.11. Thirty doublings take 7.4 hours in V. natriegens and 15.7 hours in E. coli, so one fits in a working day and the other needs an overnight step.

With two such growth steps per cycle each cycle saves 16.5 hours, which is 2.06 working days. A toolkit that costs 40 working days therefore breaks even at 19.4 cycles, so 20. A project with 12 cycles saves about 25 days and does not repay the toolkit; one with 40 cycles saves about 82 days and does.

The library needs −ln(0.05) × 10,000 = 29,957 transformants for 95% coverage. At 10^3 CFU per µg that takes about 30 µg of DNA, at 10^5 about 0.3 µg and at 10^7 about 0.003 µg, so transformation efficiency can decide feasibility before growth rate does.

Caveats: the growth numbers come from one preprint under particular media, and the toolkit cost, the number of growth steps and the efficiencies are assumptions. A faster host pays only if the toolkit exists or is cheap to build, if your pathway runs in it, and if the cycles are numerous. Often the right answer is to prototype in E. coli and move only the final strain.

Code

```
import math
# Is a faster host worth building a toolkit for?
# Verified (Weinstock et al., bioRxiv 2016): at 37 C, V. natriegens generation time 14.8 min in LB3,
# E. coli 31.3 min in LB.
t_vn, t_ec = 14.8, 31.3
print("speed ratio: %.2f x" % (t_ec / t_vn))

n_dbl = 30          # one cell to about 1e9 cells is about 30 doublings
h_vn, h_ec = n_dbl * t_vn / 60, n_dbl * t_ec / 60
print("time for %d doublings: V. natriegens %.1f h, E. coli %.1f h" % (n_dbl, h_vn, h_ec))

# ASSUMPTIONS: each build-test cycle has two such growth steps; a working day is 8 h;
# building the toolkit for the new host takes 40 working days.
steps, day_h, toolkit_days = 2, 8.0, 40
saved_h = steps * (h_ec - h_vn)
saved_d = saved_h / day_h
print("\nsaved per cycle: %.1f h = %.2f working days" % (saved_h, saved_d))
print("break-even: %.1f cycles (round up to %d)" % (toolkit_days / saved_d, math.ceil(toolkit_days / saved_d)))
print("a project with 12 cycles saves %.0f days against a %d-day toolkit; one with 40 cycles saves %.0f" % (12 * saved_d, toolkit_

# the library must also be transformable
variants = 1e4
P = 0.95
need = -math.log(1 - P) * variants
print("\ntransformants for %.0f%% coverage of a %d-variant library: %.0f" % (100 * P, variants, need))
print("DNA needed at three transformation efficiencies (ASSUMED, CFU per ug of DNA):")
for eff in (1e3, 1e5, 1e7):
    print("  %8.0e CFU/ug -> %8.3f ug" % (eff, need / eff))
```

Regulatory control

How far a repressor or an sRNA can turn a gene down.

O1 · How far will a regulator turn a gene down?

week 27 · 1.5 h

Part 1. An sRNA adds degradation to an mRNA. The mRNA balance is dm/dt = α − (δ + k·s)m, where α is production, δ the basal decay rate and k·s the extra decay rate the sRNA adds. At steady state m^*/m_0 = δ/(δ + k·s). ASSUMPTION: the mRNA half-life without the sRNA is 3 minutes.

Part 2. A repressor gives output y = y_min + (1 − y_min)/(1 + x^n), where x = R/K. ASSUMPTIONS: a 5% leak (y_min = 0.05) and a Hill coefficient n = 2.

1. Compute δ, and the added decay rate k·s needed for 2-, 10- and 100-fold knockdown, with the resulting mRNA half-lives.
2. Say what the 100-fold result implies for the design.
3. Compute the best possible repression for the leaky repressor.
4. Compute the repressor level R/K needed for outputs of 50%, 20%, 10% and 6% of maximum.
5. Say whether a 50-fold target is reachable.

Worked solution

With a 3-minute half-life the basal decay rate is δ = ln 2 / 3 = 0.231 per minute. Fold repression is 1 + k·s/δ. For 2-fold, k·s = δ = 0.23 per minute, which cuts the half-life to 1.5 minutes. For 10-fold, k·s =

9δ = 2.08 per minute, a half-life of 0.30 minutes. For 100-fold, k·s = 99δ = 22.9 per minute, a half-life near 2 seconds.

The last figure is the warning. Large knockdowns demand very fast added decay, which one regulator may not deliver, so designers combine layers, for instance a weak promoter with a repressor, instead of expecting 100-fold from one.

For a repressor with a 5% leak and a Hill coefficient of 2, the output cannot fall below 5% of maximum, so the best possible repression is 20-fold. Reaching 50% of maximum needs R/K = 1.05, 20% needs 2.31, 10% needs 4.24, and 6% (16.7-fold) needs 9.70. Each step costs more repressor than the last. A 50-fold target is impossible with this leak because it would need a 2% output.

The model is steady state and ignores protein turnover, burden and the dynamics of reaching it. Use it to see what is feasible, then measure mRNA, protein and flux separately.

Code

```
import math
# How far will a regulator turn a gene down?
# Part 1, RNA-level knockdown. mRNA balance: dm/dt = alpha - (delta + k*s) * m, so at steady state
#   m*/m0 = delta / (delta + k*s)        where k*s is the extra degradation rate the sRNA adds.
# ASSUMPTION: an mRNA half-life of 3 minutes without the sRNA.
delta = math.log(2) / 3.0
print("basal decay rate delta = %.3f per min (half-life 3.0 min)" % delta)
print("\nadded decay rate k*s needed for a given knockdown")
print("%-18s %-18s %-14s %s" % ("remaining mRNA", "fold repression", "k*s (per min)", "mRNA half-life (min)"))
for rem in (0.5, 0.1, 0.01):
    ks = delta * (1 / rem - 1)
    print("%-18s %-18.0f %-14.2f %.2f" % ("%.0f%%" % (100 * rem), 1 / rem, ks, math.log(2) / (delta + ks)))
print("fold repression = 1 + k*s/delta, so 10-fold needs k*s = 9 delta = %.2f per min" % (9 * delta))

# Part 2, transcriptional repression with leak. Output fraction y = ymin + (1 - ymin) / (1 + x^n), x = R/K.
ymin, n = 0.05, 2
print("\nrepressor with %.0f%% leak and Hill coefficient %d: the best possible repression is %.0f-fold" % (100 * ymin, n, 1 / ymin
print("%-20s %-18s %s" % ("target output", "fold repression", "repressor needed (R/K)"))
for y in (0.5, 0.2, 0.1, 0.06):
    x = ((1 - ymin) / (y - ymin) - 1) ** (1.0 / n)
    print("%-20s %-18.1f %.2f" % ("%.0f%% of max" % (100 * y), 1 / y, x))
print("a 50-fold target is impossible with this leak: it would need an output of %.0f%%, below the %.0f%% floor" % (100 / 50, 100
```

Machine learning

Overfitting, leakage, and what a score means.

P1 · How much data before a model stops fooling you?

week 42 · 1.5 h

Synthetic data, so the truth is known. There are 50 features, only the first 5 matter (weight 1 each), and the noise has standard deviation 1, so the best possible R^2 is 5/6 = 0.83. Fit ridge regression with a fixed penalty at sample sizes 30, 60, 120, 240 and 480, scoring on 2,000 new points. Average over 20 seeded repeats. Then repeat with the labels shuffled.

1. Tabulate the training and held-out R^2 at each sample size, and the gap.
2. Say where the model stops overfitting.
3. Repeat with shuffled labels at 30, 120 and 480 samples and read the training and held-out scores.
4. Say what a high training score tells you when features outnumber samples.
5. Write what you would do on a real dataset.

Worked solution

The ceiling is R^2 = 5/6 = 0.83. With 30 samples and 50 features the model scores 0.96 on its training data and 0.31 on new data, a gap of 0.65. The gap shrinks as data grows: 60 samples give 0.93 against 0.59, 120 give 0.90 against 0.73, 240 give 0.87 against 0.79, and 480 give 0.84 against 0.81. Only at the largest size do the training and held-out scores agree closely and approach the ceiling.

With the labels shuffled, so there is nothing to learn, the model still fits: training R^2 is 0.90 at 30 samples, 0.41 at 120 and 0.10 at 480, while held-out R^2 is negative (−0.49, −0.58 and −0.08), worse than predicting the mean. A high training score proves nothing when features outnumber samples.

On a real dataset: report only held-out scores, draw a learning curve before trusting a result, keep the model simple when data are scarce, and compare against the baseline of predicting the mean. These are averages over 20 seeded repeats of synthetic data, so a real dataset will differ; the pattern is the point.

Code

```
import numpy as np
# How much data before a model stops fooling you?  Synthetic data, so the truth is known.
# 50 features, only the first 5 matter (each with weight 1), noise sd 1: the best possible R^2 is 5/6 = 0.83.
# Ridge regression in closed form, fixed penalty. Everything is seeded; scores are averages over 20 repeats.
def r2(y, yhat): return 1 - ((y - yhat) ** 2).sum() / ((y - y.mean()) ** 2).sum()

def ridge(X, y, lam):
    Xc, yc = X - X.mean(0), y - y.mean()
    w = np.linalg.solve(Xc.T @ Xc + lam * np.eye(X.shape[1]), Xc.T @ yc)
    return lambda Z: (Z - X.mean(0)) @ w + y.mean()

def trial(n, seed, shuffle=False, p=50, k=5, lam=10.0, ntest=2000):
    rng = np.random.default_rng(seed)
    beta = np.zeros(p); beta[:k] = 1.0
    X = rng.normal(size=(n, p)); y = X @ beta + rng.normal(size=n)
    Xt = rng.normal(size=(ntest, p)); yt = Xt @ beta + rng.normal(size=ntest)
    if shuffle: y = rng.permutation(y)
    f = ridge(X, y, lam)
    return r2(y, f(X)), r2(yt, f(Xt))

print("ceiling R^2 = %.2f" % (5 / 6))
print("%5s %12s %14s %12s" % ("n", "train R^2", "held-out R^2", "gap"))
for n in (30, 60, 120, 240, 480):
    res = np.array([trial(n, 1000 * n + s) for s in range(20)]).mean(0)
    print("%5d %12.2f %14.2f %12.2f" % (n, res[0], res[1], res[0] - res[1]))

print("\nthe same with the labels shuffled, so there is nothing to learn:")
print("%5s %12s %14s" % ("n", "train R^2", "held-out R^2"))
for n in (30, 120, 480):
    res = np.array([trial(n, 5000 + 1000 * n + s, shuffle=True) for s in range(20)]).mean(0)
    print("%5d %12.2f %14.2f" % (n, res[0], res[1]))
```

P2 · Random split or split by family?

week 42 · 1.5 h

Synthetic data with a known structure: 40 families of 12 variants and 20 features. A variant is its family centre plus a small deviation. The label is a family offset (sd 1, not learnable for a new family) plus a real effect of the deviation (first five features) plus noise (sd 0.3). Fit kernel ridge regression and score it by 5-fold cross-validation two ways: a random split, and a split that holds out whole families. Repeat 10 times.

1. Compute the label variance and the share that is a real, transferable effect.
2. Score the model under the random split and under the family split.
3. Compute how much the random split overstates performance on a new family.
4. Say which split fits each of three questions: rank variants within known families, predict a new family, predict a new plate.
5. Say what the result does not show.

Worked solution

The label has variance 1.27, of which only 0.42 (33%) is a real effect of the variant’s deviation that would carry over to a new family; the rest is a family offset and noise. A kernel ridge model scores $R^2 = 0.66$ under a random 5-fold split, because variants of the same family appear in both training and test sets and the model recognises the family. When the test families are new, it scores −0.04, no better than predicting the mean, with a standard deviation of 0.01 to 0.02 over 10 repeats. The random split overstates new-family performance by 0.70, and the best any model could do on a new family is about 0.33.

So the honest question before choosing a split is what you will use the model for. To rank variants within known families a random split is fair. To predict new families, new plates or new batches, split that way.

A caveat: the kernel width and penalty were set once and not tuned, so a better model might capture some of the 33%. It cannot recover the part that depends on family identity.

Code

```
import numpy as np
# Random split versus a split by family. Synthetic data with a known structure.
# 40 families of 12 variants, 20 features. A variant is its family centre plus a small deviation.
# The label is a family offset (not learnable for a new family) plus a small real effect of the deviation plus noise.
rng = np.random.default_rng(7)
F, V, d = 40, 12, 20
C = rng.normal(0, 2.0, size=(F, d))
u = rng.normal(0, 1.0, size=F)                # family offsets, sd 1
w = np.zeros(d); w[:5] = 1.0
dev = rng.normal(0, 0.3, size=(F * V, d))
fam = np.repeat(np.arange(F), V)
X = C[fam] + dev
signal = dev @ w                              # the part that carries over to new families
y = u[fam] + signal + rng.normal(0, 0.3, size=F * V)
print("variance of the label: %.2f; of which the real, transferable effect is %.2f (%.0f%%)" % (y.var(), signal.var(), 100 * signa

def r2(y, yhat): return 1 - ((y - yhat) ** 2).sum() / ((y - y.mean()) ** 2).sum()
def krr_fit_predict(Xtr, ytr, Xte, sigma=3.0, lam=0.1):
    def K(A, B):
        d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
        return np.exp(-d2 / (2 * sigma ** 2))
    a = np.linalg.solve(K(Xtr, Xtr) + lam * np.eye(len(Xtr)), ytr - ytr.mean())
    return K(Xte, Xtr) @ a + ytr.mean()

def cv(groups_fn, repeats=10):
    out = []
    for r in range(repeats):
        rr = np.random.default_rng(100 + r)
        folds = groups_fn(rr)
        pred = np.zeros(len(y))
        for f in range(5):
            te = folds == f
            pred[te] = krr_fit_predict(X[~te], y[~te], X[te])
        out.append(r2(y, pred))
    return np.mean(out), np.std(out)

random_folds = lambda rr: rr.permutation(np.arange(len(y)) % 5)
def family_folds(rr):
    order = rr.permutation(F); fold_of_family = np.empty(F, int); fold_of_family[order] = np.arange(F) % 5
    return fold_of_family[fam]

m1, s1 = cv(random_folds); m2, s2 = cv(family_folds)
print("5-fold CV, random split:       R^2 = %.2f (sd %.2f over 10 repeats)" % (m1, s1))
print("5-fold CV, split by family:    R^2 = %.2f (sd %.2f over 10 repeats)" % (m2, s2))
print("the random split overstates how well the model will do on a NEW family by %.2f" % (m1 - m2))
print("ceiling for a new family is about the transferable share, %.2f" % (signal.var() / y.var()))
```

Toxicity, export and removal

What a second phase buys, what evaporation hides, and what the solvent costs.

Q1 · Two-phase partitioning: what do the cells actually see?

week 45 · 1.5 h

A product stops growth when its concentration in the aqueous phase passes 0.2 g/L. You add an organic phase that takes up a fraction φ of the broth volume, and the product partitions so that C_org = K · C_aq.

ASSUMPTIONS: instant equilibrium, a solvent that is not itself toxic, no emulsion, a K that does not change with concentration, and toxicity that depends only on the aqueous concentration.

1. Write the total product per litre of whole broth in terms of C_aq, K and φ.
2. Compute the highest total titer that keeps the aqueous phase at 0.2 g/L, for K = 1, 10, 100 and 1000 and φ = 5%, 10% and 20%.
3. Compute the gain over a single phase at φ = 10%.
4. Compute the solvent fraction needed to reach a total titer of 10 g/L.
5. Compute the share of the product in the organic phase at φ = 10%, and say what you would check before relying on it.

Worked solution

Total product per litre of whole broth is P = C_aq · ((1 − φ) + K·φ), since the aqueous phase holds C_aq in a volume (1 − φ) and the organic phase holds K·C_aq in a volume φ. Holding C_aq at 0.2 g/L gives the highest titer the cells will tolerate.

For φ = 10%, K = 1 gives 0.20 g/L (no gain), K = 10 gives 0.38, K = 100 gives 2.18 and K = 1000 gives 20.18 g/L. At φ = 5% the values are 0.20, 0.29, 1.19 and 10.19; at φ = 20% they are 0.20, 0.56, 4.16 and 40.16. A single phase is capped at 0.2 g/L. The gain at φ = 10% is 1.0, 1.9, 10.9 and 100.9-fold.

The solvent fraction needed for a total of 10 g/L is φ = (P*/c_tox − 1)/(K − 1) = 49/(K − 1): K = 10 would need 544% of the broth volume, which is impossible; K = 100 needs 49.5% and K = 1000 needs 4.9%. At φ = 10% the share of the product in the organic phase is 52.6% for K = 10, 91.7% for K = 100 and 99.1% for K = 1000.

The lesson is that K has to be large before a second phase pays, and then a small volume is enough. Before relying on it, check three things: the product's solubility in the solvent (at K = 1000 and 20 g/L total the solvent would hold about 200 g/L), whether the solvent itself is tolerated, and whether K holds at the real concentrations and temperature. The model also ignores the time to reach equilibrium and any emulsion, so test it on a small scale first.

Code

```
# Two-phase partitioning: what concentration do the cells actually see?
# Product partitions between an aqueous phase (volume fraction 1 - phi) and an organic phase (fraction phi):
#   C_org = K * C_aq        (K = partition coefficient, organic over aqueous)
# Total product per litre of whole broth:  P = C_aq * ((1 - phi) + K * phi)
# ASSUMPTIONS: instant equilibrium, a solvent that is not itself toxic, no emulsion, a K that does not change with concentration,
# and a toxicity that depends only on the aqueous concentration, here c_tox = 0.2 g/L.
c_tox = 0.2

print("highest total titer (g per litre of whole broth) with the aqueous phase held at %.1f g/L" % c_tox)
print("%-10s %12s %12s %12s" % ("K", "phi = 5%", "phi = 10%", "phi = 20%"))
for K in (1, 10, 100, 1000):
    print("%-10d %12.2f %12.2f %12.2f" % (K, *[c_tox * ((1 - phi) + K * phi) for phi in (0.05, 0.10, 0.20)]))
print("single phase (no solvent): %.1f g/L" % c_tox)

print("\ngain over single phase, phi = 10%:")
for K in (1, 10, 100, 1000):
    P = c_tox * ((1 - 0.10) + K * 0.10); print("  K = %-5d %.1f-fold" % (K, P / c_tox))

print("\nsolvent fraction needed to reach a target titer of 10 g/L:  phi = (P*/c_tox - 1) / (K - 1)")
for K in (10, 100, 1000):
    phi = (10 / c_tox - 1) / (K - 1)
    print("  K = %-5d phi = %.3f (%.1f%% v/v)%s" % (K, phi, 100 * phi, "   <- more than the whole broth: impossible" if phi >= 1 e

print("\nshare of the product that sits in the organic phase, phi = 10%:")
for K in (10, 100, 1000):
    f = K * 0.1 / ((1 - 0.1) + K * 0.1); print("  K = %-5d %.1f%%" % (K, 100 * f))
```

Q2 · What evaporation hides, and what the solvent costs

week 45 · 1.5 h

Newman et al. (2006) measured that amorphadiene evaporated from their fermentor with a half-life of about 50 minutes.

ASSUMPTIONS: first-order loss with that half-life, a constant production of 1 g/L/h, and an open vessel with no capture. For the economics: a 10 m³ batch, a 10% v/v dodecane overlay with K = 1000, a total titer of 20 g/L, dodecane density 0.75 kg/L, a solvent price of 2 USD/kg, 1% of the solvent lost per batch, and reuse over 50 batches.

1. Compute the loss rate constant and the fraction of product made at time zero that is still in the broth after 1, 2, 6 and 24 hours.
2. Compute what the broth holds after 4, 12 and 24 hours of steady production, and the most it can ever hold.
3. Compute the solvent inventory and its cost, and the product per batch.
4. Compute the solvent cost per kilogram of product if the solvent is discarded, with make-up for 1% loss, and with the first fill spread over 50 batches.
5. Say what this tells you about measured titers and about the cost of ISPR.

Worked solution

With a half-life of 50 minutes the loss constant is k = ln 2 / 0.833 h = 0.832 per hour. The fraction of product made at time zero that remains is 43.5% after 1 hour, 18.9% after 2 hours, 0.68% after 6 hours and about 2×10^{-9} after 24 hours: essentially all of it is gone.

With steady production r = 1 g/L/h the broth holds r/k · (1 − e^(−kt)). After 4 hours 4.0 g/L has been made and 1.16 g/L is measured (29%); after 12 hours 12.0 g/L has been made and 1.20 g/L is measured (10%); after 24 hours 24.0 g/L has been made and 1.20 g/L is measured (5%). The broth can never hold more than r/k = 1.20 g/L however long you run, so an uncaptured volatile product makes a strain look twenty times worse than it is.

The overlay is 10 m^3 × 10% × 0.75 kg/L = 750 kg of dodecane, worth 1,500 USD, against 200 kg of product per batch. If the solvent were used once and discarded it would add 7.50 USD per kg of product. With 1% loss per batch and make-up only it is 15 USD per batch, or 0.07 USD per kg; adding the first fill spread over 50 batches gives 0.15 USD per kg, and a total of 0.23 USD per kg.

So capture is needed to measure a volatile product at all, and its solvent cost is small only if the solvent is reused. The half-life is specific to that product and vessel, and the price, loss and reuse figures are assumptions. Solvent recovery, emulsions, oxygen transfer and safety are not in this number and belong in the techno-economic analysis of week 49.

Code

```
import math
# A volatile product, and what the solvent costs.
# Measured (Newman et al., Biotechnol. Bioeng. 2006): amorphadiene evaporated from their fermentor with a half-life of about 50 mi
# ASSUMPTIONS: first-order loss with that half-life, a constant production rate, and an open vessel with no capture.
t_half_h = 50 / 60.0
k = math.log(2) / t_half_h
print("loss rate constant k = %.3f per h" % k)
print("\nfraction of product made at time 0 that is still in the broth:")
for t in (1, 2, 6, 24):
    print("  after %2d h: %.3g" % (t, 0.5 ** (t / t_half_h)))

r = 1.0   # g/L/h, ASSUMED production rate
print("\nwith production %.1f g/L/h, the broth holds  r/k * (1 - exp(-k t))" % r)
for t in (4, 12, 24):
    made = r * t; held = r / k * (1 - math.exp(-k * t))
    print("  after %2d h: made %5.1f g/L, measured in the broth %.2f g/L (%.0f%% of what was made)" % (t, made, held, 100 * held /
print("the broth can never hold more than r/k = %.2f g/L, however long you run" % (r / k))

# solvent economics. ASSUMPTIONS: 10 m3 broth, 10% v/v dodecane overlay, K = 1000, titer 20 g/L whole broth,
# dodecane density 0.75 kg/L, solvent price 2 USD/kg, 1% of the solvent lost per batch, solvent reused for 50 batches.
V, phi, price, loss, reuse, titer = 10.0, 0.10, 2.0, 0.01, 50, 20.0
solvent_kg = V * 1000 * phi * 0.75
product_kg = titer * V
print("\nsolvent economics for a %.0f m3 batch" % V)
print("  solvent inventory %.0f kg (%.0f USD); product per batch %.0f kg" % (solvent_kg, solvent_kg * price, product_kg))
print("  if the solvent is used once and discarded: %.2f USD per kg of product" % (solvent_kg * price / product_kg))
make_up = solvent_kg * loss * price
print("  with %d%% loss per batch and make-up only: %.2f USD per batch = %.2f USD per kg" % (100 * loss, make_up, make_up / produc
print("  plus the first fill spread over %d batches: %.2f USD per kg; total %.2f USD per kg" % (reuse, solvent_kg * price / reuse
```

Two-phase and immobilised systems

Modelling product capture and cell retention, and fitting the parameters.

S1 · The base model: what product inhibition costs you

week 47 · 1.5 h

A fermentation with Monod growth, Luedeking–Piret production and product inhibition of the Levenspiel form $(1 - P/P_{crit})^n$.

ASSUMED PARAMETERS: μ_{max} 0.30 /h, Ks 1.0 g/L, Yxs 0.40 g/g, Yps 0.30 g/g, α 0.30, β 0.02 /h, P_{crit} 2.5 g/L, n 1, X_0 0.2 g/L, S_0 80 g/L, 72 h.

1. Write the three balances and solve them numerically.
2. Report the titer, the time to reach 90% of it, and the productivity on both bases.
3. Report the substrate left unused.
4. Check that the substrate balance closes.

5. Compare the titer with the yield-limited ceiling and say what the gap means.

Worked solution

The model reaches 2.500 g/L, which is exactly P_crit, so inhibition and not substrate is what stops it. It passes 90% of that titer at 18.9 hours, which makes productivity 0.119 g/L/h over the active period but only 0.035 g/L/h over the full 72 hours: the last two days add almost nothing, and that alone is an argument for harvesting early or for removing the product.

Substrate left unused is 54.6 of 80 g/L, so 68% of the feedstock is wasted. The yield on the substrate actually consumed is 0.099 g/g, well under the assumed Yps of 0.30, because much of it went to biomass. The substrate balance closes to 0.00 g/L, which confirms the equations and the integration are consistent, though it does not make the parameters right.

If inhibition were removed entirely, the ceiling would be $Yps \cdot S_0 = 24.0$ g/L. The gap between 2.5 and 24 g/L is the prize that in situ removal is chasing, and the rest of this week asks how much of it is really reachable.

Code

```
# S1. The base model: one phase, product inhibition, and what it caps.
import numpy as np
from scipy.integrate import solve_ivp

# ---- model: growth and production with product inhibition (ASSUMED parameters). Concentrations are per litre of AQUEOUS phase --
p = dict(mu_max=0.30, Ks=1.0, Yxs=0.40, Yps=0.30, alpha=0.30, beta=0.02, Pcrit=2.5, n=1.0, X0=0.2, S0=80.0, T=72.0)

def kin(S, Pa, p, pen=1.0):
    S = max(S, 0.0)
    sat = S / (p["Ks"] + S)                                  # Monod saturation
    f = max(0.0, 1.0 - Pa / p["Pcrit"]) ** p["n"] * pen      # product inhibition (Levenspiel form), and any penalty
    mu = p["mu_max"] * sat * f                               # specific growth rate
    qP = p["alpha"] * mu + p["beta"] * sat * f               # Luedeking-Piret: growth-associated + non-growth-associated
    return mu, qP

def run_single(p, kv=0.0):
    def rhs(t, y):
        X, S, Pa, Lost = y
        mu, qP = kin(S, Pa, p)
        return [mu * X, -mu * X / p["Yxs"] - qP * X / p["Yps"], qP * X - kv * Pa, kv * Pa]
    return solve_ivp(rhs, [0, p["T"]], [p["X0"], p["S0"], 0.0, 0.0], method="LSODA", rtol=1e-9, atol=1e-11, dense_output=True)

sol = run_single(p)
X, S, Pa, Lost = sol.y[:, -1]
print("%6s %8s %8s %8s" % ("t (h)", "X g/L", "S g/L", "P g/L"))
for t in (6, 12, 19, 24, 36, 48, 72):
    y = sol.sol(t); print("%6d %8.2f %8.2f %8.3f" % (t, *y[:3]))
t90 = next(t for t in np.linspace(0, 72, 7201) if sol.sol(t)[2] >= 0.9 * Pa)
print("\nfinal titer %.3f g/L, which is %.1f%% of P_crit = %.1f g/L" % (Pa, 100 * Pa / p["Pcrit"], p["Pcrit"]))
print("90%% of the final titer is reached at %.1f h; productivity over 72 h = %.3f g/L/h, over that 90%% time = %.3f g/L/h" % (t90
print("substrate left unused: %.1f of %.1f g/L (%.0f%%)" % (S, p["S0"], 100 * S / p["S0"]))

# mass-balance closure: substrate used = biomass/Yxs + product/Yps
closure = (p["S0"] - S) - ((X - p["X0"]) / p["Yxs"] + Pa / p["Yps"])
print("substrate balance closure error: %.2e g/L (should be ~0)" % closure)
print("yield on substrate used: %.3f g product per g substrate" % (Pa / (p["S0"] - S)))
print("\nupper bound if inhibition were removed: product could reach at most Yps*S0 = %.1f g/L" % (p["Yps"] * p["S0"]))
```

S2 · Adding a liquid second phase, and when the shortcut works

week 47 · 2 h

The same strain, with an organic phase of fraction φ, a partition coefficient K and transfer $N = kLa \cdot (Pa - Po/K)$ per litre of aqueous phase.

ASSUMPTIONS: as in S1, plus instant mixing within each phase and a solvent that costs the cells nothing (relaxed in S4).

1. Write the four balances, being explicit about which volume each concentration refers to, and explain the $(1 - φ)/φ$ factor.
2. Tabulate the whole-broth titer for K = 10, 100 and 1000 and φ = 5%, 10% and 20% with fast transfer.
3. Explain the rows where more solvent lowers the titer.

4. Derive the equilibrium shortcut R = 1 + Kφ/(1 − φ) and check it against the full model.

5. Scan kLa and compare the offset from equilibrium against the prediction r/λ.

Worked solution

With fast transfer the whole-broth titer rises from 2.50 g/L to 3.62, 4.75 and 6.30 g/L at K = 10 for φ = 5, 10 and 20%, and to 7.49, 7.09 and 6.30 g/L at K = 100. K = 1000 gives the same as K = 100.

The pattern reverses between the rows, and the reason is which constraint is active. At K = 10 inhibition still binds — the aqueous phase ends at 2.50 g/L with 26 g/L of substrate left — so more solvent helps. At K = 100 the aqueous phase ends at 0.65 g/L, well below P_crit, and the substrate is exhausted, so the process is substrate-limited; raising φ then shrinks the aqueous volume that holds the substrate and the titer falls. The diagnostic is simply whether substrate remains at the end.

The shortcut: at equilibrium Po = K·Pa, so the total product per litre of aqueous is Pa·R with R = 1 + Kφ/(1 − φ). Running a single-phase model with the inhibition felt at P/R reproduced the full model to within 10^{-9} g/L for K = 10 and 100, which is a strong check that both are coded correctly.

Transfer limits: the exchange relaxes at λ = kLa·(1 + (1 − φ)/(Kφ)), which is 1090 /h at kLa = 1000 and 0.5 /h at kLa = 0.5. The offset from equilibrium needed to carry the production rate r is about r/λ, and the model agrees: 0.001 against 0.001 g/L at kLa = 1000, and 1.208 against 1.177 g/L at kLa = 0.5. Even at the slowest transfer the final titer was unchanged at 7.09 g/L; what changed was the peak aqueous concentration, 0.65 rising to 1.69 g/L, and the time to 90%, 16.4 rising to 21.0 hours. So in this case transfer costs time and safety margin rather than titer, and a large kLa makes the system stiff, which is why a stiff solver is needed.

Code

```
# S2. A liquid second phase: partition coefficient K, phase fraction phi and mass-transfer coefficient kLa.
import numpy as np
from scipy.integrate import solve_ivp

# ---- model: growth and production with product inhibition (ASSUMED parameters). Concentrations are per litre of AQUEOUS phase --
p = dict(mu_max=0.30, Ks=1.0, Yxs=0.40, Yps=0.30, alpha=0.30, beta=0.02, Pcrit=2.5, n=1.0, X0=0.2, S0=80.0, T=72.0)

def kin(S, Pa, p, pen=1.0):
    S = max(S, 0.0)
    sat = S / (p["Ks"] + S)                                  # Monod saturation
    f = max(0.0, 1.0 - Pa / p["Pcrit"]) ** p["n"] * pen      # product inhibition (Levenspiel form), and any penalty
    mu = p["mu_max"] * sat * f                               # specific growth rate
    qP = p["alpha"] * mu + p["beta"] * sat * f               # Luedeking-Piret: growth-associated + non-growth-associated
    return mu, qP

def run_liquid(p, K, phi, kLa, pen=1.0, kv=0.0):
    """States per litre of aqueous (X, S, Pa) and per litre of organic (Po). The product is made in the aqueous phase.
       Transfer to the organic phase: N = kLa * (Pa - Po/K), per litre of aqueous."""
    def rhs(t, y):
        X, S, Pa, Po, Lost = y
        mu, qP = kin(S, Pa, p, pen)
        N = kLa * (Pa - Po / K)
        return [mu * X, -mu * X / p["Yxs"] - qP * X / p["Yps"], qP * X - N - kv * Pa, N * (1 - phi) / phi, kv * Pa]
    s = solve_ivp(rhs, [0, p["T"]], [p["X0"], p["S0"], 0, 0, 0], method="Radau", rtol=1e-9, atol=1e-11, dense_output=True)
    X, S, Pa, Po, Lost = s.y[:, -1]
    return s, (1 - phi) * Pa + phi * Po, Pa, S                  # whole-broth titer = (1-phi)*Pa + phi*Po

print("Whole-broth titer (g/L) at 72 h with fast transfer (kLa = 1000 /h); single phase gave 2.50 g/L")
print("%-8s %10s %10s %10s" % ("K", "phi = 5%", "phi = 10%", "phi = 20%"))
for K in (10, 100, 1000):
    row = [run_liquid(p, K, phi, 1000.0)[1] for phi in (0.05, 0.10, 0.20)]
    print("%-8d %10.2f %10.2f %10.2f" % (K, *row))
print("(for K >= 100 the titer is substrate-limited, and it FALLS as phi rises, because the substrate sits in the aqueous phase, w

print("\nAqueous concentration the cells saw at the end, and substrate left, for K = 100, phi = 10%:")
s, tit, Pa, S = run_liquid(p, 100, 0.10, 1000.0); print("  Pa = %.2f g/L (limit %.1f), substrate left %.2f g/L, titer %.2f g/L" %
s, tit, Pa, S = run_liquid(p, 10, 0.10, 1000.0);  print("  K = 10:  Pa = %.2f g/L (limit %.1f), substrate left %.2f g/L, titer %.2

print("\nShortcut: at equilibrium the whole system behaves like ONE phase whose inhibition limit is multiplied by R = 1 + K*phi/(1
for K, phi in ((10, 0.10), (100, 0.10), (100, 0.20)):
    R = 1 + K * phi / (1 - phi)
    q = dict(p); q["Pcrit"] = p["Pcrit"] * R                    # a single-phase model with the scaled limit; its P is per litre
    def rhs(t, y):
        X, S, P = y; Pa = P / R; mu, qP = kin(S, Pa, p); return [mu * X, -mu * X / p["Yxs"] - qP * X / p["Yps"], qP * X]
    ss = solve_ivp(rhs, [0, p["T"]], [p["X0"], p["S0"], 0], method="LSODA", rtol=1e-9, atol=1e-11)
    full = run_liquid(p, K, phi, 1000.0)[1]
    # lumped total per litre of aqueous is P = Pa*R, so the whole-broth titer is (1-phi)*P
    print("  K=%-4d phi=%.2f R=%5.1f  shortcut titer %.3f   full model %.3f   (difference %.1e)" % (K, phi, R, (1 - phi) * ss.y[2,
```

```
print("\nTransfer limit, K = 100, phi = 10%: the aqueous phase must sit ABOVE equilibrium to drive the transfer")
print("%-9s %10s %12s %14s %16s %16s %12s" % ("kLa (/h)", "titer g/L", "peak Pa", "lambda (/h)", "offset r/lambda", "offset, model
t = np.linspace(0, 72, 7201)
for kLa in (1000, 200, 50, 10, 2, 0.5):
    s, tit, Pa, S = run_liquid(p, 100, 0.10, kLa)
    Y = s.sol(t); X, S_, Pa_, Po_ = Y[0], Y[1], Y[2], Y[3]
    r = np.array([kin(S_[i], Pa_[i], p)[1] * X[i] for i in range(len(t))])        # production rate, g per L aqueous per h
    lam = kLa * (1 + (1 - 0.10) / (100 * 0.10))
    whole = (1 - 0.10) * Pa_ + 0.10 * Po_
    t90 = t[np.argmax(whole >= 0.9 * whole[-1])]                                 # time to reach 90% of the final whole-broth tit
    print("%-9s %10.2f %12.3f %14.1f %16.3f %16.3f %12.1f" % (("%g" % kLa), tit, Pa_.max(), lam, r.max() / lam, (Pa_ - Po_ / 100).
print("lambda = kLa*(1 + (1-phi)/(K*phi)) is the relaxation rate of the exchange; the offset needed to move product at rate r is a
```

S3 · A solid sorbent: capacity against rate

week 46 · 1.5 h

The same strain, with a solid sorbent added at w grams per litre of aqueous phase. Equilibrium follows a Langmuir isotherm q* = qmax·KL·Pa/(1 + KL·Pa), and uptake follows a linear driving force dq/dt = ks·(q* − q).

ASSUMED SORBENT: qmax 0.15 g/g, KL 8 L/g; bead volume neglected.

1. Write the balances and say why the sorbent term appears multiplied by w.
2. Tabulate the total product for w = 20, 50 and 100 g/L and ks = 0.3, 2 and 10 /h.
3. For w = 50 g/L, report the final aqueous concentration, the loading, the substrate left and the time to 90%.
4. Compute the sorbent capacity at the inhibition limit and the sorbent needed per kilogram of product.
5. Repeat with production doubled and say what changes.

Worked solution

At w = 20 g/L the total reaches 5.36 g/L and at w = 50 g/L it reaches 7.88 g/L, against 2.50 g/L with no sorbent. At w = 100 g/L it is still 7.88 g/L, because by then the substrate is exhausted and adding sorbent cannot help.

The rate constant barely matters here: 0.3, 2 and 10 /h all give 7.88 g/L at w = 50, and only the time to 90% moves, from 17.3 to 16.0 hours. The final aqueous concentration is 1.13 g/L, below the 2.5 g/L limit, the loading is 0.135 g/g, which is 90% of qmax, and the substrate is fully used. So this process is sized by capacity, not by rate.

The capacity at the inhibition limit is w·q*(P_crit) = 7.14 g/L at w = 50, which is the ceiling the sorbent itself imposes. The sorbent requirement is 6.3 kg per kg of product, and that ratio is what decides affordability: a sorbent must be cheap, or reusable over many cycles, for that to work.

When production is doubled the picture changes: the aqueous concentration is pushed to the inhibition limit of 2.50 g/L whatever the rate constant, the total reaches 9.64 g/L, and the slow sorbent now costs time, with 90% reached at 18.1 hours against 15.0 hours for the fast one. The rule is to size on capacity, then check the uptake rate against your actual production rate.

Code

```
# S3. A solid sorbent (resin, polymer bead or gel): Langmuir equilibrium and finite-rate uptake (linear driving force).
import numpy as np
from scipy.integrate import solve_ivp

# ---- model: growth and production with product inhibition (ASSUMED parameters). Concentrations are per litre of AQUEOUS phase --
p = dict(mu_max=0.30, Ks=1.0, Yxs=0.40, Yps=0.30, alpha=0.30, beta=0.02, Pcrit=2.5, n=1.0, X0=0.2, S0=80.0, T=72.0)

def kin(S, Pa, p, pen=1.0):
    S = max(S, 0.0)
    sat = S / (p["Ks"] + S)                                  # Monod saturation
    f = max(0.0, 1.0 - Pa / p["Pcrit"]) ** p["n"] * pen      # product inhibition (Levenspiel form), and any penalty
    mu = p["mu_max"] * sat * f                               # specific growth rate
    qP = p["alpha"] * mu + p["beta"] * sat * f               # Luedeking-Piret: growth-associated + non-growth-associated
    return mu, qP

qmax, KL = 0.15, 8.0           # ASSUMED sorbent: capacity g product per g sorbent; Langmuir constant, L/g

def run_sorbent(p, w, ks, pen=1.0):
```

```
    """w = g sorbent per litre of aqueous phase (bead volume neglected). q = g product per g sorbent.
       Uptake rate (linear driving force): dq/dt = ks * (q* - q), with q* = qmax*KL*Pa / (1 + KL*Pa)."""
    def rhs(t, y):
        X, S, Pa, q = y
        mu, qP = kin(S, Pa, p, pen)
        qstar = qmax * KL * Pa / (1 + KL * Pa)
        dq = ks * (qstar - q)
        return [mu * X, -mu * X / p["Yxs"] - qP * X / p["Yps"], qP * X - w * dq, dq]
    s = solve_ivp(rhs, [0, p["T"]], [p["X0"], p["S0"], 0.0, 0.0], method="Radau", rtol=1e-9, atol=1e-11, dense_output=True)
    X, S, Pa, q = s.y[:, -1]
    return s, Pa + w * q, Pa, q, S                          # total product per litre = aqueous + sorbent-bound

print("Total product (g/L) at 72 h; single phase gave 2.50 g/L. Sorbent: qmax = %.2f g/g, KL = %.0f L/g" % (qmax, KL))
print("%-10s %10s %10s %10s" % ("w (g/L)", "ks=0.3/h", "ks=2/h", "ks=10/h"))
for w in (20, 50, 100):
    print("%-10d %10.2f %10.2f %10.2f" % (w, *[run_sorbent(p, w, ks)[1] for ks in (0.3, 2.0, 10.0)]))

print("\nDetail for w = 50 g/L:")
print("%-8s %10s %10s %10s %12s %12s" % ("ks (/h)", "total", "final Pa", "final q", "S left", "t90 (h)"))
t = np.linspace(0, 72, 7201)
for ks in (0.3, 2.0, 10.0):
    s, tot, Pa, q, S = run_sorbent(p, 50, ks)
    Y = s.sol(t); total_t = Y[2] + 50 * Y[3]
    t90 = t[np.argmax(total_t >= 0.9 * total_t[-1])]
    print("%-8g %10.2f %10.3f %10.4f %12.2f %12.1f" % (ks, tot, Pa, q, max(S, 0), t90))

cap = 50 * qmax * KL * p["Pcrit"] / (1 + KL * p["Pcrit"])
print("\nsorbent capacity at the inhibition limit: w*q*(Pcrit) = %.2f g/L (this caps the total once Pa reaches Pcrit)" % cap)
print("sorbent used per kg product at w = 50 and a total of %.1f g/L: %.1f kg sorbent per kg product" % (run_sorbent(p, 50, 2.0)[1

# stress test: a strain that makes product twice as fast (alpha and beta doubled), same sorbent, w = 50 g/L
q2 = dict(p); q2["alpha"], q2["beta"] = 0.60, 0.04
print("\nStress test, production twice as fast (alpha = 0.60, beta = 0.04), w = 50 g/L:")
print("%-8s %10s %10s %12s %14s" % ("ks (/h)", "total", "peak Pa", "t90 (h)", "single phase"))
single = run_sorbent(q2, 0.0, 1.0)[1]
for ks in (0.3, 1.0, 3.0, 10.0):
    s, tot, Pa, q, S = run_sorbent(q2, 50, ks)
    Y = s.sol(t); total_t = Y[2] + 50 * Y[3]
    print("%-8g %10.2f %10.3f %12.1f %14.2f" % (ks, tot, Y[2].max(), t[np.argmax(total_t >= 0.9 * total_t[-1])], single))
```

S4 · The net effect: evaporation, a solvent penalty, and when to harvest

week 47 · 2 h

The same strain and an overlay, now with two realities added: the product evaporates from an open vessel, and the solvent costs the cells something.

ASSUMPTIONS: evaporation first-order with a 50-minute half-life (the figure measured for amorphadiene by Newman et al., 2006), K 100, kLa 200 /h, harvest at 24 h, and a penalty on growth and production of (1 − φ/0.5), so 20% at φ = 10%.

1. Compute what a closed vessel and an open vessel give with no second phase.
2. Scan φ and report the product kept at 24 h, the product made, the percentage kept, and the titer if you wait until 72 h.
3. Find the best φ at the harvest time.
4. Estimate the hold-time loss and explain it.
5. Repeat the scan with and without the penalty, closed to evaporation, and separate the two effects.

Worked solution

With no second phase, a closed vessel holds 2.44 g/L at 24 hours. Open, it holds 0.022 g/L of the 7.88 g/L that was made, which is 0.3%. Note what the evaporation did: by keeping the aqueous concentration low it removed the inhibition, so the cells kept producing. The strain was never the problem; the measurement was.

With an overlay, the product kept at 24 hours is 0.77 g/L at φ = 2%, 2.42 at 5%, 4.30 at 10%, 5.30 at 15%, then falls to 3.24 at 20% and 0.72 at 30%. The best in this scan is φ = 15%. Beyond that the penalty dominates: at φ = 30% the cells are so impaired that only 0.82 g/L has been made by 24 hours, even though 88% of it is kept.

Waiting to 72 hours is worse, not better: the kept titer at φ = 10% drops from 4.30 to 0.16 g/L. The reason is that the overlay is a store, not a seal. Product leaks back through the aqueous phase and evaporates, with a time constant of about R/kv ≈ 14.6 hours at φ = 10%. A capture phase changes

when you must harvest.

Separating the two effects, closed to evaporation: without a penalty the titer falls gently with φ (7.49, 7.09, 6.30, 4.73 g/L at φ = 5, 10, 20, 40%) purely from the shrinking aqueous volume. With the penalty the first three are identical and only φ = 40% diverges, 2.97 against 4.73 g/L, where the impairment leaves 30 g/L of substrate unused. So in this model the penalty only bites at high solvent fractions, while evaporation and harvest timing dominate everywhere. All of these rest on assumed parameters, especially the penalty, which is a guess standing in for solvent toxicity and reduced oxygen transfer; it is the first thing to measure.

Code

```
# S4. The net effect of a second phase: gain from partitioning against the cost of the solvent, with an evaporating product.
import numpy as np
from scipy.integrate import solve_ivp

# ---- model: growth and production with product inhibition (ASSUMED parameters). Concentrations are per litre of AQUEOUS phase --
p = dict(mu_max=0.30, Ks=1.0, Yxs=0.40, Yps=0.30, alpha=0.30, beta=0.02, Pcrit=2.5, n=1.0, X0=0.2, S0=80.0, T=72.0)

def kin(S, Pa, p, pen=1.0):
    S = max(S, 0.0)
    sat = S / (p["Ks"] + S)                                  # Monod saturation
    f = max(0.0, 1.0 - Pa / p["Pcrit"]) ** p["n"] * pen      # product inhibition (Levenspiel form), and any penalty
    mu = p["mu_max"] * sat * f                               # specific growth rate
    qP = p["alpha"] * mu + p["beta"] * sat * f               # Luedeking-Piret: growth-associated + non-growth-associated
    return mu, qP

def run(p, K, phi, kLa, kv, pen_fn):
    # returns the ODE solution; use at(...) below to read the product kept and made at any time
    pen = pen_fn(phi)
    def rhs(t, y):
        X, S, Pa, Po, Lost = y
        mu, qP = kin(S, Pa, p, pen)
        N = kLa * (Pa - Po / K) if phi > 0 else 0.0
        dPo = N * (1 - phi) / phi if phi > 0 else 0.0
        return [mu * X, -mu * X / p["Yxs"] - qP * X / p["Yps"], qP * X - N - kv * Pa, dPo, kv * Pa]
    s = solve_ivp(rhs, [0, p["T"]], [p["X0"], p["S0"], 0, 0, 0], method="Radau", rtol=1e-9, atol=1e-11, dense_output=True)
    return s


def at(s, phi, t):
    X, S, Pa, Po, Lost = s.sol(t)
    kept = (1 - phi) * Pa + phi * Po                  # product retained per litre of whole broth
    made = (1 - phi) * (Pa + Lost) + phi * Po          # product made per litre of whole broth (kept + evaporated)
    return kept, made, max(S, 0.0)

K, kLa, TH = 100.0, 200.0, 24.0                       # partition coefficient, transfer coefficient, harvest time (h)
kv = np.log(2) / (50.0 / 60.0)                        # evaporation: half-life 50 min (Newman et al. 2006, amorphadiene), open ae
pen_fn = lambda phi: max(0.0, 1.0 - phi / 0.5)        # ASSUMED penalty on growth and production: 20% at phi = 10% (solvent toxic
print("kv = %.3f /h; penalty = 1 - phi/0.5; K = %g; kLa = %g /h; harvest at %g h" % (kv, K, kLa, TH))

print("\nA. No second phase")
sc = run(p, K, 0.0, kLa, 0.0, pen_fn); so = run(p, K, 0.0, kLa, kv, pen_fn)
kc, mc, _ = at(sc, 0.0, TH); ko, mo, Sol = at(so, 0.0, TH)
print("   closed vessel:  kept %.2f g/L at %g h" % (kc, TH))
print("   open vessel:    kept %.3f g/L of %.2f made (%.1f%% kept); evaporation holds the aqueous level down, so the cells keep pr

print("\nB. Overlay, open vessel, harvest at %g h and again at 72 h (hold-time loss)" % TH)
print("%-6s %6s %12s %10s %10s %12s %12s" % ("phi", "R", "kept@24 g/L", "% kept", "made@24", "kept@72 g/L", "S left@24"))
best = None
for phi in (0.02, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40):
    s = run(p, K, phi, kLa, kv, pen_fn)
    k24, m24, S24 = at(s, phi, TH); k72, m72, _ = at(s, phi, 72.0)
    R = 1 + K * phi / (1 - phi)
    print("%-6.2f %6.1f %12.2f %10.0f %10.2f %12.2f %12.2f" % (phi, R, k24, 100 * k24 / m24, m24, k72, S24))
    if best is None or k24 > best[1]: best = (phi, k24)
print("best kept titer at the harvest time in this scan: phi = %.2f gives %.2f g/L" % best)
print("hold-time loss: the stored product leaves through the aqueous phase with time constant about R/kv = %.1f h at phi = 0.10" %

print("\nC. The same scan with a solvent that costs the cells nothing (no penalty), closed to evaporation")
for phi in (0.05, 0.10, 0.20, 0.40):
    s = run(p, K, phi, kLa, 0.0, lambda x: 1.0); k, m, S = at(s, phi, 72.0)
    print("   phi=%.2f kept %.2f g/L at 72 h; substrate left %.2f" % (phi, k, S))
print("\nD. The same scan WITH the penalty, closed to evaporation")
for phi in (0.05, 0.10, 0.20, 0.40):
    s = run(p, K, phi, kLa, 0.0, pen_fn); k, m, S = at(s, phi, 72.0)
    print("   phi=%.2f kept %.2f g/L at 72 h; substrate left %.2f" % (phi, k, S))
```

S5 · Fitting the inhibition parameters, and whether the data identify them

week 47 · 2 h

Synthetic data generated from the model itself, so the truth is known: P_crit 2.5 g/L and n 1.0. Twelve sampling times from 2 to 46 h, 5% relative noise on every measurement. Experiment 1 is a single-phase run; experiment 2 adds an overlay with K 100 and φ 10%.

ASSUMPTIONS: every other parameter is held at its true value while these two are fitted, the noise is independent and proportional to the measurement, and the intervals come from a linearisation (the Jacobian) rather than from a full uncertainty analysis. Because the data are synthetic, this tests the method, not reality.

1. Fit the two parameters to experiment 1 by weighted least squares.
2. Report 95% intervals from the Jacobian and the correlation between the parameters.
3. Fit from two different starting points and compare.
4. Add experiment 2 and report what changes.
5. Say what the singular values of the Jacobian tell you.

Worked solution

From experiment 1 alone the fit returns P_crit = 2.53 ± 0.07 g/L and n = 1.09 ± 0.11, against the true 2.50 and 1.00, so both are recovered within their intervals. Two very different starting points, (5.0, 2.0) and (1.0, 0.5), converge to the same answer, which suggests the objective has one clear minimum rather than several.

The correlation between the parameters is 0.67. That matters: the two are partly interchangeable, so a higher limit with a higher exponent fits almost as well, and the intervals should not be read as independent. The singular values of the Jacobian, 36.7 and 14.4, measure the same thing geometrically: the smaller one is the poorly determined direction in parameter space.

Adding the second experiment tightens the exponent from ±0.11 to ±0.05 and the limit from ±0.07 to ±0.06, lowers the correlation to about 0.55 to 0.65, and raises both singular values, to roughly 47 and 22. A designed second condition therefore buys more than extra samples in the same condition would, because it probes a different part of the inhibition curve.

Two honest limits. The data are synthetic, so this shows that the method can recover parameters it generated, not that it recovers reality. And all other parameters were held at their true values; fitting yields and μ_max simultaneously would be considerably harder and the correlations would grow.

Code

```
# S5. Estimating the inhibition parameters from data, and asking whether the data can identify them.
import numpy as np
from scipy.integrate import solve_ivp

# ---- model: growth and production with product inhibition (ASSUMED parameters). Concentrations are per litre of AQUEOUS phase --
p = dict(mu_max=0.30, Ks=1.0, Yxs=0.40, Yps=0.30, alpha=0.30, beta=0.02, Pcrit=2.5, n=1.0, X0=0.2, S0=80.0, T=72.0)

def kin(S, Pa, p, pen=1.0):
    S = max(S, 0.0)
    sat = S / (p["Ks"] + S)                                  # Monod saturation
    f = max(0.0, 1.0 - Pa / p["Pcrit"]) ** p["n"] * pen      # product inhibition (Levenspiel form), and any penalty
    mu = p["mu_max"] * sat * f                               # specific growth rate
    qP = p["alpha"] * mu + p["beta"] * sat * f               # Luedeking-Piret: growth-associated + non-growth-associated
    return mu, qP
from scipy.optimize import least_squares

truth = dict(p)                                    # the "true" strain: Pcrit = 2.5 g/L, n = 1.0
def sim_single(pp, times):
    def rhs(t, y):
        X, S, Pa = y; mu, qP = kin(S, Pa, pp); return [mu * X, -mu * X / pp["Yxs"] - qP * X / pp["Yps"], qP * X]
    s = solve_ivp(rhs, [0, max(times)], [pp["X0"], pp["S0"], 0.0], method="LSODA", rtol=1e-9, atol=1e-11, t_eval=times)
    return s.y                                      # rows X, S, Pa

def sim_liquid(pp, times, K=100.0, phi=0.10, kLa=1000.0):
    def rhs(t, y):
        X, S, Pa, Po = y; mu, qP = kin(S, Pa, pp); N = kLa * (Pa - Po / K)
        return [mu * X, -mu * X / pp["Yxs"] - qP * X / pp["Yps"], qP * X - N, N * (1 - phi) / phi]
    s = solve_ivp(rhs, [0, max(times)], [pp["X0"], pp["S0"], 0, 0], method="Radau", rtol=1e-9, atol=1e-11, t_eval=times)
    return s.y

def make_data(times, noise, seed, liquid=False, species=(0, 1, 2)):
    rng = np.random.default_rng(seed)
```

```
    y = (sim_liquid if liquid else sim_single)(truth, times)[list(species)]
    return y * (1 + noise * rng.normal(size=y.shape))

def make_residuals(designs):
    # each design: (times, data, noise, species, liquid?)
    def residuals(theta):
        pp = dict(p); pp["Pcrit"], pp["n"] = theta
        out = []
        for times, data, noise, species, liquid in designs:
            sim = (sim_liquid if liquid else sim_single)(pp, times)[list(species)]
            out.append(((sim - data) / (noise * np.maximum(data, 1e-6))).ravel())
        return np.concatenate(out)
    return residuals

def fit(designs, start):
    res = make_residuals(designs)
    sol = least_squares(res, start, bounds=([0.3, 0.2], [20.0, 4.0]), xtol=1e-12, ftol=1e-12, gtol=1e-12)
    J = sol.jac; dof = len(sol.fun) - 2
    cov = np.linalg.inv(J.T @ J) * (2 * sol.cost / dof)             # covariance from the Jacobian, scaled by the residual varianc
    sd = np.sqrt(np.diag(cov)); corr = cov[0, 1] / (sd[0] * sd[1])
    return sol.x, 1.96 * sd, corr

t12 = np.arange(2.0, 49.0, 4.0); t6 = np.array([6.0, 12.0, 18.0, 24.0, 36.0, 48.0])
A = [(t12, make_data(t12, 0.05, 11), 0.05, (0, 1, 2), False)]
B = A + [(t12, make_data(t12, 0.05, 12, liquid=True), 0.05, (0, 1, 2), True)]
C = [(t6, make_data(t6, 0.10, 13, species=(2,)), 0.10, (2,), False)]
print("truth: Pcrit = %.2f g/L, n = %.2f" % (truth["Pcrit"], truth["n"]))
for label, des in (("A. single phase; X, S and P measured; 12 times; 5% noise", A),
                   ("B. design A plus an overlay run (aqueous P measured)", B),
                   ("C. single phase; P only; 6 times; 10% noise", C)):
    print("\n" + label)
    for start in ([5.0, 2.0], [1.0, 0.5]):
        x, ci, corr = fit(des, start)
        print("   start (%.1f, %.1f): Pcrit = %.2f +/- %.2f, n = %.2f +/- %.2f, correlation %.2f" % (start[0], start[1], x[0], ci[
print("\n(plus-minus values are 95% intervals from the Jacobian; a correlation near +/-1 means the two parameters trade off)")
```

T1 · Inside an immobilisation bead: how deep does oxygen get?

week 48 · 1.5 h

Cells entrapped in spherical gel beads, respiring at a constant (zero-order) rate.

ASSUMED PARAMETERS: oxygen diffusivity in the gel 2×10^{-9} m²/s, surface oxygen 7 mg/L (air-saturated water at 30 °C), biomass in the bead 100 kg/m³, specific oxygen uptake 4 mg per g biomass per hour.

1. Compute the volumetric oxygen demand inside the bead.
2. Estimate the penetration depth with a flat-plate balance.
3. For spheres, compute the dead-core radius fraction and the live volume for diameters from 0.5 to 4 mm.
4. Find the largest bead that is fully oxygenated.
5. Scan the cell loading at a fixed 2 mm diameter, and compare with the first-order effectiveness factor.

Worked solution

The demand is 1.11×10^{-4} kg of oxygen per m³ of bead per second. A flat-plate balance, $D\cdot Cs = R_v\cdot\delta^2/2$, gives a penetration depth of about 500 µm, which is already the whole answer in outline: oxygen reaches half a millimetre into this bead and no further.

For spheres the condition for a fully live bead is $\varphi^2 = R_v\cdot R^2/(6\cdot D\cdot Cs) \le 1$, which gives a maximum radius of 869 µm, so about 1.7 mm in diameter. Beyond that a dead core appears: at 2 mm the core is 32% of the radius but only 3% of the volume; at 3 mm it is 61% of the radius and 23% of the volume; at 4 mm it is 72% and 38%. The live region is a shell of roughly 550 to 680 µm regardless of how much bigger the bead gets, which is why making beads larger adds dead weight rather than capacity.

Cell loading matters as much as size. In a 2 mm bead, loadings of 10, 30 and 50 kg/m³ leave the bead fully live; 100 kg/m³ gives a 32% core and 200 kg/m³ a 58% core with 20% of the volume dead. So the two levers are diameter and loading, and a high loading in a large bead is the worst combination.

For comparison, first-order kinetics in a sphere give $\eta = (3/\varphi)(\coth\varphi - 1/\varphi)$, which falls from 0.94 at $\varphi = 1$ to 0.27 at $\varphi = 10$. The zero-order picture is the right one for saturated respiration; the first-order one applies when the substrate is far below its half-saturation constant.

Every number here rests on the assumed diffusivity, loading and uptake rate, which vary several-fold between systems. Use the structure, then measure your own.

Code

```
# T1. Diffusion inside a bead: Thiele modulus, effectiveness factor, and how deep oxygen gets.
import numpy as np
from scipy.optimize import brentq

# ASSUMPTIONS, typical of alginate beads with aerobic cells (orders of magnitude, not measured values):
D  = 2.0e-9      # m2/s, oxygen diffusivity in the gel (about that of water)
Cs = 7.0e-3      # kg/m3 (= g/L) dissolved oxygen at the bead surface, air-saturated water at 30 C
Xb = 100.0       # kg/m3 biomass per m3 of bead (immobilized beads reach high densities; this is the variable that decides everyth
qO = 4.0e-3      # kg O2 per kg biomass per h -> convert to per second
qO_s = qO / 3600.0
Rv = Xb * qO_s   # kg O2 per m3 of bead per s, at zero order (saturated respiration)
print("volumetric oxygen demand inside the bead: %.3e kg/m3/s" % Rv)

# Zero-order reaction in a sphere: the depth the substrate penetrates before it runs out.
# Balance over a shell: D*Cs = Rv * delta^2 / 2  gives the penetration depth for a flat slab; for a sphere use the full solution b
delta_slab = np.sqrt(2 * D * Cs / Rv)
print("penetration depth (flat-plate estimate): %.0f um" % (delta_slab * 1e6))

def dead_core_fraction(R):
    """Zero-order consumption in a sphere. Returns the radius fraction that has NO oxygen (dead core),
       by solving the classic zero-order sphere balance."""
    # Dimensionless: phi2 = Rv*R^2/(6*D*Cs). If phi2 <= 1 the whole bead is oxygenated.
    phi2 = Rv * R**2 / (6 * D * Cs)
    if phi2 <= 1: return 0.0, phi2
    # with a dead core at r = rc, the profile satisfies C(rc)=0 and dC/dr(rc)=0:
    #   Cs = Rv/(6D) * (R^2 - 3*rc^2 + 2*rc^3/R)
    f = lambda rc: Rv / (6 * D) * (R**2 - 3 * rc**2 + 2 * rc**3 / R) - Cs
    rc = brentq(f, 0, R)
    return rc / R, phi2

print("\n%-12s %10s %14s %16s %14s" % ("bead dia", "phi^2", "dead core r/R", "live volume %", "live shell um"))
for d_mm in (0.5, 1.0, 2.0, 3.0, 4.0):
    R = d_mm / 2 * 1e-3
    frac, phi2 = dead_core_fraction(R)
    live = 1 - frac**3
    print("%-12s %10.2f %14.3f %16.1f %14.0f" % ("%.1f mm" % d_mm, phi2, frac, 100 * live, (R - frac * R) * 1e6))

print("\nthe whole bead stays oxygenated only while phi^2 = Rv*R^2/(6*D*Cs) <= 1, i.e. R <= %.0f um (diameter %.2f mm)"
      % (np.sqrt(6 * D * Cs / Rv) * 1e6, 2 * np.sqrt(6 * D * Cs / Rv) * 1e3))

# effectiveness factor for a FIRST-order reaction, for comparison (the standard sphere formula)
print("\nhow the biomass loading decides it, for a 2 mm bead:")
print("%-14s %10s %14s %16s" % ("Xb kg/m3", "phi^2", "dead core r/R", "live volume %"))
for xb in (10, 30, 50, 100, 200):
    Rv = xb * qO_s
    R = 1.0e-3
    phi2 = Rv * R**2 / (6 * D * Cs)
    if phi2 <= 1: frac = 0.0
    else:
        f = lambda rc: Rv / (6 * D) * (R**2 - 3 * rc**2 + 2 * rc**3 / R) - Cs
        frac = brentq(f, 0, R) / R
    print("%-14d %10.2f %14.3f %16.1f" % (xb, phi2, frac, 100 * (1 - frac**3)))
Rv = Xb * qO_s

print("\nfor comparison, first-order kinetics in a sphere: eta = (3/phi)*(1/tanh(phi) - 1/phi)")
print("%-10s %12s" % ("phi", "eta"))
for phi in (0.5, 1, 2, 5, 10, 20):
    eta = (3 / phi) * (1 / np.tanh(phi) - 1 / phi)
    print("%-10g %12.3f" % (phi, eta))
```

T2 · Does retaining the biomass actually pay?

week 48 · 2 h

A continuous reactor, feed substrate 80 g/L, the same strain as the earlier problems. Free cells wash out above a dilution rate set by their growth. Immobilised cells are retained in beads, are largely non-growing, and take up substrate for production plus maintenance.

ASSUMPTIONS: μ_max 0.30 /h, Ks 1.0 g/L, Yxs 0.40, Yps 0.30, and for the retained cells a specific production rate of 0.08 /h and maintenance of 0.004 g substrate per g biomass per hour; bead loading 100 kg/m³; effectiveness factor from T1.

1. Compute the washout dilution rate for free cells.
2. Tabulate the free-cell steady state and productivity up to washout.

3. Compute the immobilised steady state across dilution rates at a 20% bead fraction.

4. Vary the effectiveness factor and the bead fraction.

5. Compare the two and say which wins, and why.

Worked solution

Free cells wash out above D = μ_max·S₀/(Ks + S₀) = 0.296 /h. Their productivity climbs towards that point, 1.20 g/L/h at D = 0.05, 4.68 at 0.20 and a peak of about 5.54 g/L/h at D = 0.28, then collapses to zero at 0.30. The best operating point is a fraction below a cliff, which is an uncomfortable place to run a plant.

The immobilised reactor behaves completely differently. Because the biomass is retained, there is no washout at all: at a 20% bead fraction and η = 0.97 the productivity is about 1.53 g/L/h and stays there from D = 0.1 to D = 4 /h, while the conversion falls from 63% to 2%. Productivity is set by the retained biomass and its specific rate, not by the dilution rate, so raising D buys dilute product rather than more product.

The effectiveness factor costs roughly what T1 predicts: 1.58, 1.53, 1.22 and 0.98 g/L/h at η = 1.00, 0.97, 0.77 and 0.62. The bead fraction scales it almost linearly: 0.38, 0.77, 1.53 and 3.06 g/L/h at φ = 5, 10, 20 and 40%.

So with these parameters immobilisation loses, 1.53 against 5.54 g/L/h. That is the honest answer and it is worth sitting with, because the technique is often presented as an improvement in itself. It would win if the retained non-growing cells had a higher specific production rate, if the bead fraction could be pushed much higher without mass-transfer and mechanical penalties, if the product were growth-independent, or if the free-cell optimum near washout could not be held stably in practice. What immobilisation reliably buys is stability, biocatalyst reuse and easy separation, not productivity by default. Compute the comparison for your own strain before choosing.

Code

```
# T2. Why immobilize cells at all: washout in a chemostat, and what bead retention changes.
import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import brentq

mu_max, Ks, Yxs, Yps, Sf = 0.30, 1.0, 0.40, 0.30, 80.0      # ASSUMED, same strain as the ISPR problems
print("free cells in a chemostat: washout when D > mu_max*Sf/(Ks+Sf) = %.4f /h" % (mu_max * Sf / (Ks + Sf)))
print("\n%-10s %10s %10s %14s %16s" % ("D (/h)", "S g/L", "X g/L", "P g/L", "productivity g/L/h"))
for Dd in (0.05, 0.10, 0.20, 0.28, 0.295, 0.30):
    if Dd >= mu_max: S = Sf
    else:
        S = Ks * Dd / (mu_max - Dd)
        if S > Sf: S = Sf
    X = Yxs * (Sf - S); P = Yps * (Sf - S)
    print("%-10.3f %10.3f %10.2f %14.2f %16.3f" % (Dd, S, X, P, Dd * P))
print("the best free-cell productivity sits just below washout, which is an unstable place to operate")

# immobilized cells: the retained biomass is largely NON-growing, so its substrate uptake is
# production plus maintenance, not growth. This is the standard immobilized-biocatalyst case.
#   qP = beta * S/(Ks+S)        (g product per g biomass per h)
#   qS = qP/Yps + m             (m = maintenance)
beta, m = 0.08, 0.004          # ASSUMED specific production rate and maintenance coefficient

def immobilized(Dd, eps, Xb, eta):
    """Steady state: D*(Sf - S) = qS * Xv, with Xv = eps*Xb*eta the effective biomass per litre of reactor."""
    Xv = eps * Xb * eta
    def bal(S):
        qP = beta * S / (Ks + S)
        return Dd * (Sf - S) - (qP / Yps + m) * Xv
    if bal(1e-9) < 0: return 0.0, 0.0, 0.0, 0.0
    S = brentq(bal, 1e-9, Sf) if bal(Sf) < 0 else Sf
    qP = beta * S / (Ks + S)
    P = qP * Xv / Dd                                  # product leaving per litre of feed
    return S, P, Dd * P, 100 * (Sf - S) / Sf          # also return the substrate conversion

print("\nimmobilized cells, non-growing: beta = %.2f /h, maintenance m = %.3f, 100 kg/m3 in the beads, eta = 0.97 (2 mm beads from
print("%-10s %10s %12s %10s %18s %14s" % ("D (/h)", "eps", "S g/L", "P g/L", "productivity g/L/h", "conversion %"))
for Dd in (0.1, 0.3, 1.0, 2.0, 4.0):
    S, P, pr, conv = immobilized(Dd, 0.20, 100.0, 0.97)
    print("%-10.2f %10.2f %12.2f %10.2f %18.2f %14.0f" % (Dd, 0.20, S, P, pr, conv))

print("\nthe same at three effectiveness factors, D = 1 /h, eps = 0.20:")
for eta in (1.0, 0.97, 0.77, 0.62):
```

```
    S, P, pr, conv = immobilized(1.0, 0.20, 100.0, eta)
    print("   eta = %.2f -> S = %.2f g/L, P = %.2f g/L, productivity %.2f g/L/h, conversion %.0f%%" % (eta, S, P, pr, conv))
print("\nand at three bead fractions, D = 1 /h, eta = 0.97:")
for eps in (0.05, 0.10, 0.20, 0.40):
    S, P, pr, conv = immobilized(1.0, eps, 100.0, 0.97)
    print("   eps = %.2f -> S = %.2f g/L, P = %.2f g/L, productivity %.2f g/L/h, conversion %.0f%%" % (eps, S, P, pr, conv))
print("\nbest free-cell productivity above was 5.54 g/L/h at D = 0.28 /h, just below washout at %.3f /h" % (mu_max * Sf / (Ks + Sf
```

Design, statistics and assays

How many replicates, which design, and what a routine measurement means.

U1 · How many replicates, and what can this experiment even detect?

week 17 · 1.5 h

You compare an engineered strain with its control by titer, using a two-sided two-sample t-test at the 5% level.

ASSUMPTIONS: differences and noise are both expressed relative to the control mean; for the replicate comparison, a biological coefficient of variation of 18% and a technical one of 8%; power calculated from the non-central t distribution.

1. Compute the replicates per group needed for 80% power across a range of differences and coefficients of variation.
2. For a 20% CV, compute the smallest difference detectable at 80% power with 3, 4, 6, 8, 12 and 20 replicates.
3. Compute the power of the common choice of three replicates to see a 20% change at a 20% CV.
4. Compare the standard error from adding biological replicates against adding technical ones.
5. Say what you would change in your own planned experiment.

Worked solution

Replicates needed for 80% power depend on the ratio of the difference to the noise. A 50% difference at a 10% CV needs 3 per group; a 20% difference at 10% needs 6; a 10% difference at 10% needs 17. The same 20% difference needs 17 replicates at a 20% CV and 37 at a 30% CV. The assay’s noise, not the biology, usually decides the cost of the experiment.

Turned around: at a 20% CV, three replicates can only detect a difference of 61%, four can detect 48%, six 36%, eight 30%, twelve 24% and twenty 18%. Most published strain comparisons are run with three.

The common choice of three replicates has 16% power to see a 20% change at that noise level, so there is an 84% chance of reporting no difference when a real 20% improvement exists. That is the single most useful number in this problem.

On replicate type: with a biological CV of 18% and a technical CV of 8%, three biological samples measured once give a standard error of 0.114. Measuring each three times gives 0.107 (94% of the original, for nine samples) and nine times gives 0.105. Using nine biological samples measured once gives 0.066, which is 58%. Technical replicates check your technique; biological replicates buy inference.

What to do when you cannot afford enough replicates: reduce the noise rather than add samples, by improving the assay, using a paired or blocked design, or measuring a less variable proxy. And state the detectable effect size honestly, so a negative result is not reported as evidence of no effect.

Code

```
# U1. How many replicates? Power, detectable effect, and what noise costs you.
import numpy as np
from scipy import stats

def power_two_sample(n, d, alpha=0.05):
    """Two-sided two-sample t-test. d = effect size (difference / standard deviation).
       scipy's non-central t returns nan for very large non-centrality, so guard it: by then power is 1."""
    df = 2 * (n - 1); ncp = d * np.sqrt(n / 2.0); crit = stats.t.ppf(1 - alpha / 2, df)
    pw = stats.nct.sf(crit, df, ncp) + stats.nct.cdf(-crit, df, ncp)
    if not np.isfinite(pw): return 1.0 if ncp > crit else 0.0
    return float(np.clip(pw, 0.0, 1.0))
```

```
def n_for_power(d, power=0.80, alpha=0.05):
    for n in range(2, 2001):
        if power_two_sample(n, d, alpha) >= power: return n
    return None

print("Replicates per group for 80% power, two-sided t-test at alpha = 0.05")
print("%-22s %-12s %-14s %s" % ("titer difference", "CV", "effect size d", "n per group"))
for diff, cv in [(0.50, 0.10), (0.20, 0.10), (0.10, 0.10), (0.20, 0.20), (0.20, 0.30), (0.50, 0.30)]:
    d = diff / cv                      # both expressed relative to the control mean
    n = n_for_power(d)
    print("%-22s %-12s %-14.2f %s" % ("%.0f%%" % (100 * diff), "%.0f%%" % (100 * cv), d, n))

print("\nWhat you can actually detect with the n you can afford (CV = 20%, 80% power)")
print("%-10s %-24s" % ("n/group", "smallest detectable difference"))
for n in (3, 4, 6, 8, 12, 20):
    lo, hi = 0.001, 3.0                    # difference as a fraction of the control mean
    assert power_two_sample(n, hi / 0.20) >= 0.80 > power_two_sample(n, lo / 0.20)   # bracket the answer
    for _ in range(100):
        mid = (lo + hi) / 2
        if power_two_sample(n, mid / 0.20) >= 0.80: hi = mid
        else: lo = mid
    print("%-10d %-24s" % (n, "%.1f%%" % (100 * hi)))

pw3 = power_two_sample(3, 0.20 / 0.20)
print("\nThe usual mistake: n = 3 with CV = 20%% has power of only %.0f%% to see a 20%% change." % (100 * pw3))
print("Running it anyway means a %.0f%% chance of calling a real 20%% improvement 'no difference'." % (100 * (1 - pw3)))

# Technical vs biological replicates: where the variance actually comes from
sb, st = 0.18, 0.08                # ASSUMED: biological CV 18%, technical CV 8%
print("\nBiological CV %.0f%%, technical CV %.0f%%. Standard error of the mean with b biological x t technical:" % (100 * sb, 100
print("%-16s %-12s %-12s %s" % ("b x t", "SEM", "vs b=3,t=1", "total samples"))
base = np.sqrt(sb**2 / 3 + st**2 / 3)
for b, t in [(3, 1), (3, 3), (3, 9), (6, 1), (9, 1), (6, 2)]:
    sem = np.sqrt(sb**2 / b + st**2 / (b * t))
    print("%-16s %-12.4f %-12s %d" % ("%d x %d" % (b, t), sem, "%.0f%%" % (100 * sem / base), b * t))
print("adding technical replicates barely helps; biological replicates are what move the standard error")
```

U2 · One factor at a time, or a factorial design?

week 17 · 2 h

Four factors (A, B, C, D) at two levels each, and an invented ground truth so the methods can be graded: titer = 10 + 3A − 1.5B + 0.5C + 0D + 2.5·AB in coded units, with noise of standard deviation 0.8. Note the trap: B hurts on its own but helps strongly once A is raised.

ASSUMPTIONS: the truth is invented; one-factor-at-a-time starts with every factor low and keeps a change if it improves the result.

1. Run one factor at a time in two different testing orders and count how often it reaches the optimum.
2. Run a full factorial of 16 runs and estimate every main effect and the AB interaction.
3. Compare the estimates with the truth and with the theoretical standard error.
4. Check how often the factorial calls the inactive factor D important.
5. Run a half-fraction of 8 runs and say what it costs.

Worked solution

One factor at a time reached the true optimum in A and B 75% of the time when A was tested first, and 0% of the time when B was tested first. The reason is the interaction: tested while A is still low, B appears to hurt by −1.5 − 2.5, so it is discarded and never revisited. The result therefore depends on an arbitrary ordering choice, and no such run can report the interaction at all, because A and B are never varied together.

The full factorial, 16 runs, recovered everything: A 3.01 ± 0.19, B −1.49 ± 0.19, C 0.52 ± 0.21, D −0.03 ± 0.19 and AB 2.49 ± 0.18, against true values of 3.0, −1.5, 0.5, 0 and 2.5. The spread matches the theoretical standard error of 0.8/√16 = 0.200, which is a good check that the analysis is right. Using a threshold of two standard errors, the inactive factor D was called important in 4% of repeats, close to the 5% expected from chance.

The half-fraction, 8 runs, also recovered the main effects accurately but with standard errors about 40 to 50% larger, and in this fraction the AB and CD interactions are confounded: the design cannot tell

them apart and assumes one is zero. That is an acceptable trade when you have reason to believe the higher-order interactions are negligible, and a bad one when you do not.

Run counts: 5 for one factor at a time, 8 for the half-fraction, 16 for the full design. The factorial costs three times as many runs as the greedy approach and is the only one that answers the question.

Code

```
# U2. One factor at a time, or a factorial design? And how many runs a screen really needs.
import numpy as np
import itertools

# A truth we invent so we can grade the methods: titer from 4 factors, with one interaction.
# y = 10 + 3*A - 1.5*B + 0.5*C + 0*D + 2.5*A*B   (coded factors at -1 and +1), noise sd 0.8
# Note the trap: B HURTS on its own (-1.5) but helps strongly when A is high (+2.5 interaction),
# so B only pays once A is raised. This is the situation one-factor-at-a-time cannot see.
TRUE = dict(A=3.0, B=-1.5, C=0.5, D=0.0, AB=2.5); SD = 0.8; INT = 10.0
def observe(a, b, c, d, rng): return INT + TRUE["A"]*a + TRUE["B"]*b + TRUE["C"]*c + TRUE["D"]*d + TRUE["AB"]*a*b + rng.normal(0,

print("truth: A %.1f, B %.1f, C %.1f, D %.1f, interaction AB %.1f; noise sd %.1f" % (TRUE["A"], TRUE["B"], TRUE["C"], TRUE["D"], T

# --- one factor at a time: start at all -1, change each factor alone, keep what helps ---
def ofat(seed, order=(0, 1, 2, 3), reps=1):
    """Greedy: start with every factor low, raise each in turn, keep the change if it helped."""
    rng = np.random.default_rng(seed); base = [-1, -1, -1, -1]; runs = 0
    y_base = np.mean([observe(*base, rng) for _ in range(reps)]); runs += reps
    for i in order:
        trial = list(base); trial[i] = 1
        y = np.mean([observe(*trial, rng) for _ in range(reps)]); runs += reps
        if y > y_base: base, y_base = trial, y
    return base, y_base, runs

# --- full factorial, 16 runs, estimate every main effect and the AB interaction ---
def factorial(seed, reps=1):
    rng = np.random.default_rng(seed)
    design = list(itertools.product([-1, 1], repeat=4))
    Y = np.array([np.mean([observe(*row, rng) for _ in range(reps)]) for row in design])
    Xd = np.array(design, float)
    X = np.column_stack([np.ones(16), Xd, Xd[:, 0] * Xd[:, 1]])
    beta, *_ = np.linalg.lstsq(X, Y, rcond=None)
    return beta, 16 * reps

print("\nA. One factor at a time (5 runs each), 400 repeats, and the order it happens to test in")
for order, label in [((0, 1, 2, 3), "A first, then B"), ((1, 0, 2, 3), "B first, then A")]:
    hits = 0
    for s in range(400):
        pt, y, runs = ofat(1000 + s, order)
        if pt[0] == 1 and pt[1] == 1: hits += 1
    print("   %-16s reaches the true optimum in A and B: %3.0f%% of the time" % (label, 100 * hits / 400))
print("   testing B first, while A is still low, shows B HURTING (-1.5 - 2.5), so it is discarded and never revisited")
print("   and no OFAT run can report the interaction at all, because A and B are never varied together")

print("\nB. Full factorial (16 runs), 200 repeats: estimated effects (mean +/- sd over repeats)")
est = np.array([factorial(2000 + s)[0] for s in range(200)])
names = ["intercept", "A", "B", "C", "D", "AB"]; truth = [INT, TRUE["A"], TRUE["B"], TRUE["C"], TRUE["D"], TRUE["AB"]]
# the model is fitted to coded factors, so a coefficient is half the change from -1 to +1
for i, nm in enumerate(names):
    print("   %-10s %6.2f +/- %4.2f   (true %.1f)" % (nm, est[:, i].mean(), est[:, i].std(), truth[i]))
print("   standard error of a coefficient, theory: sd/sqrt(16) = %.3f" % (SD / np.sqrt(16)))

print("\nC. How often does each method identify D (the factor that does nothing) as important?")
print("   factorial, calling an effect real when |coefficient| > 2 standard errors: %.0f%% of repeats (expected about 5%%)"
      % (100 * np.mean(np.abs(est[:, 4]) > 2 * SD / np.sqrt(16))))

print("\nD. Half-fraction (8 runs): same main effects, but AB is now confounded with CD")
def half(seed):
    rng = np.random.default_rng(seed)
    design = [r for r in itertools.product([-1, 1], repeat=4) if r[0]*r[1]*r[2]*r[3] == 1]   # the 8-run half
    Y = np.array([observe(*row, rng) for row in design]); Xd = np.array(design, float)
    X = np.column_stack([np.ones(8), Xd, Xd[:, 0]*Xd[:, 1]])
    beta, *_ = np.linalg.lstsq(X, Y, rcond=None); return beta
h = np.array([half(3000 + s) for s in range(200)])
for i, nm in enumerate(names):
    print("   %-10s %6.2f +/- %4.2f   (true %.1f)" % (nm, h[:, i].mean(), h[:, i].std(), truth[i]))
print("   in this half-fraction AB and CD cannot be told apart: the design assumes one of them is zero")
print("\nruns: OFAT 5, half-fraction 8, full factorial 16")
```

U3 · Reading a qPCR result properly

week 22 · 1.5 h

A standard curve of tenfold dilutions for a target and a reference gene, and cycle-threshold values for a control and an engineered strain.

ASSUMPTIONS: the measured cycle thresholds are invented but realistic; a single reference gene; no biological replication in the worked example, which is itself a weakness to note.

1. Compute the slope, amplification factor and efficiency of each assay from its standard curve.
2. Compute delta-delta-Ct and the fold change assuming perfect doubling.
3. Recompute the fold change using the measured efficiencies and compare.
4. Compute how far a wrongly assumed efficiency would mislead you.
5. Convert a replicate standard deviation in cycles into a factor in abundance.

Worked solution

Regressing cycle threshold against log10 of relative concentration gives a slope of −3.445 for the target, an amplification factor of 1.951 and an efficiency of 95.1%; and −3.350, 1.988 and 98.8% for the reference. Both sit inside the usual 90 to 110% window. A slope of −3.32 would be exactly 100%, since 2^3.32 = 10.

Delta-Ct is −1.70 for the control and −4.75 for the strain, so delta-delta-Ct is −3.05 and the simple method reports 2^3.05 = 8.28-fold. Using the measured efficiencies, the Pfaffl calculation gives 7.70-fold, 7% lower. Here the correction is modest because both assays are efficient.

When they are not, it matters: the same cycle difference corresponds to 6.01-fold at 80% efficiency, 7.08 at 90% and 7.67 at 95%. Reporting an 80% efficient assay as if it were perfect overstates the change by 38%.

On noise, a replicate standard deviation of 0.2 cycles is a factor of 1.15 in abundance and 0.5 cycles is 1.41. Since a two-fold change is one cycle, a claimed two-fold difference sits barely above routine technical variation. The practical consequences are to measure the efficiency of every assay, validate the reference gene for your own conditions, run the no-template and no-reverse-transcriptase controls, and put the replication effort into biological rather than technical replicates, as problem U1 shows.

Code

```
# U3. Reading a qPCR result: efficiency, delta-delta-Ct, and what a wrong efficiency costs.
import numpy as np
from scipy import stats

# A standard curve: 10-fold dilutions, Ct measured. Efficiency comes from the slope.
dil = np.array([1, 10, 100, 1000, 10000], float)
ct_target = np.array([18.10, 21.55, 25.00, 28.40, 31.90])
ct_ref    = np.array([20.00, 23.35, 26.70, 30.05, 33.40])

def efficiency(dil, ct):
    """Regress Ct against log10(relative CONCENTRATION), not against the dilution factor,
       so the slope is negative: more template means a lower Ct."""
    x = np.log10(1.0 / dil); sl, ic, r, pv, se = stats.linregress(x, ct)
    E = 10 ** (-1 / sl)                      # amplification factor per cycle: 2 is perfect doubling
    return sl, E, 100 * (E - 1), r ** 2, se

for name, ct in (("target", ct_target), ("reference", ct_ref)):
    sl, E, pct, r2, se = efficiency(dil, ct)
    print("%-10s slope %.3f (se %.3f), amplification factor %.3f, efficiency %.1f%%, R2 %.4f" % (name, sl, se, E, pct, r2))
print("a slope of -3.32 is 100% efficiency; the accepted window is roughly 90-110% (slopes -3.6 to -3.1)")

# delta-delta-Ct on an engineered strain against the control
ct = dict(control_target=25.00, control_ref=26.70, strain_target=22.10, strain_ref=26.85)
dCt_c = ct["control_target"] - ct["control_ref"]; dCt_s = ct["strain_target"] - ct["strain_ref"]
ddCt = dCt_s - dCt_c
print("\ndelta Ct control %.2f, strain %.2f, delta-delta-Ct %.2f" % (dCt_c, dCt_s, ddCt))
print("fold change assuming perfect doubling (2^-ddCt): %.2f" % (2 ** -ddCt))

# the same with the measured efficiencies (Pfaffl)
_, Et, _, _, _ = efficiency(dil, ct_target); _, Er, _, _, _ = efficiency(dil, ct_ref)
pf = (Et ** (ct["control_target"] - ct["strain_target"])) / (Er ** (ct["control_ref"] - ct["strain_ref"]))
print("fold change using the measured efficiencies (Pfaffl): %.2f  (difference %.1f%%)" % (pf, 100 * (pf / 2 ** -ddCt - 1)))

print("\nHow much does assuming perfect efficiency mislead you? (true ddCt = %.2f)" % ddCt)
print("%-16s %-16s %s" % ("true efficiency", "fold, assumed 2", "fold, correct"))
for E in (1.80, 1.90, 1.95, 2.00):
```

```
    correct = E ** -ddCt
    print("%-16s %-16.2f %.2f" % ("%.0f%%" % (100 * (E - 1)), 2 ** -ddCt, correct))
over = 100 * (2 ** -ddCt / 1.80 ** -ddCt - 1)
print("reporting an 80%% efficient assay as if it were 100%% overstates the fold change by %.0f%%" % over)

# what a Ct difference means in replicates
print("\nnoise: a standard deviation of 0.2 Ct between replicates is a factor of %.2f in abundance; 0.5 Ct is %.2f" % (2 ** 0.2, 2
print("so a 'two-fold change' is only one Ct: it sits barely above routine technical noise, which is why")
print("biological replicates and a validated reference gene matter more than extra technical wells (problem U1)")
```

Planning

Critical paths, schedule risk and where the time goes.

V1 · The critical path, and why the plan will be missed

week 64 · 2 h

A strain-engineering project broken into ten tasks with durations and dependencies: design, ordering (a wait), assembly, verification, strain construction, assay development in parallel, screening, fermentation, analysis and writing.

ASSUMPTIONS: the durations are invented but typical; in the simulation each task follows a triangular distribution from 0.8 to 2.0 times its most-likely duration, which is skewed long as real tasks are.

1. Find the critical path and the planned duration.
2. Compute the float on the parallel branch.
3. Simulate 20,000 schedules and report the median and the 80th, 90th and 95th percentiles.
4. Compute how often each task lies on the critical path.
5. Compute what halving the ordering wait is worth.

Worked solution

The critical path runs design, order, assemble, verify, strain, screen, ferment, analyse, write, giving a planned 31 weeks, about 7.2 months. The four-week ordering wait is 13% of that, and it is time in which nobody is doing anything.

The parallel assay development finishes at week 8 but is not needed until week 14, so it has 6 weeks of float, and speeding it up would change nothing while that float lasts.

The simulation is the important part. The median is 40 weeks against a plan of 31, and the probability of finishing within the planned 31 weeks is essentially zero. 80% of runs finish within 43 weeks, 90% within 44 and 95% within 45. A plan built by adding most-likely durations is close to a best case, not a middle case, because each task can overrun far more than it can underrun.

Across the simulations, the final chain of screening, fermentation, analysis and writing was on the critical path in every run, the cloning chain in 66% and the parallel assay development in 34%. That ranking is where attention is worth spending, and it is not obvious from the plan alone, since the assay branch looked comfortably safe with its 6 weeks of float.

Halving the ordering wait shortens the deterministic plan by only 1 week and the simulated median by about 2, because another branch becomes limiting. It is still cheap time, which is the general rule: attack the waiting first, then the work. Two honest limits: the durations are invented, so use the structure rather than the numbers, and the simulation assumes tasks are independent, while in reality a bad month tends to delay several at once.

Code

```
# V1. Planning a PhD-scale project: the critical path, and why plans slip.
import numpy as np

# Tasks: (name, most likely duration in weeks, predecessors). ASSUMED, but the shape is typical of a strain-engineering project.
T = {
 "design":    (2,  []),
 "order":     (4,  ["design"]),              # synthesis and primer delivery: waiting, not working
 "assemble":  (3,  ["order"]),
 "verify":    (2,  ["assemble"]),
 "strain":    (3,  ["verify"]),
 "assay_dev": (11, ["design"]),              # runs in parallel with the cloning, and is nearly as long: a second near-critical br
 "screen":    (4,  ["strain", "assay_dev"]),
```

```
 "ferment":   (6,  ["screen"]),
 "analyse":   (3,  ["ferment"]),
 "write":     (4,  ["analyse"]),
}

def critical_path(dur):
    """Earliest finish by forward pass; returns total duration and the path that sets it."""
    ef, pred = {}, {}
    def finish(k):
        if k in ef: return ef[k]
        ps = T[k][1]
        if not ps: ef[k], pred[k] = dur[k], None
        else:
            best = max(ps, key=finish); ef[k], pred[k] = finish(best) + dur[k], best
        return ef[k]
    for k in T: finish(k)
    end = max(T, key=lambda k: ef[k]); path = []; k = end
    while k: path.append(k); k = pred[k]
    return ef[end], list(reversed(path)), ef

dur0 = {k: v[0] for k, v in T.items()}
total, path, ef = critical_path(dur0)
print("critical path: %s" % " -> ".join(path))
print("planned duration: %d weeks (%.1f months)" % (total, total / 4.33))
slack = {k: ef[k] for k in T}
print("\nwaiting rather than working: 'order' is %d of those weeks (%.0f%% of the critical path)" % (dur0["order"], 100 * dur0["or

print("\nfloat: how much each off-path task could slip before it matters")
for k in ("assay_dev",):
    # latest finish for assay_dev is the start of screen
    start_screen = max(ef["strain"], ef["assay_dev"])
    print("   %-10s earliest finish %d, needed by %d, float %d weeks" % (k, ef[k], start_screen, start_screen - ef[k]))

# Monte Carlo: every task uncertain, PERT-style (optimistic 0.8x, pessimistic 2x the most likely)
rng = np.random.default_rng(7)
def sample(k):
    m = dur0[k]; a, b = 0.8 * m, 2.0 * m
    # triangular distribution, skewed long, as real tasks are
    return rng.triangular(a, m, b)

N = 20000
tot = np.empty(N); crit_count = {k: 0 for k in T}
for i in range(N):
    d = {k: sample(k) for k in T}
    t, p, _ = critical_path(d)
    tot[i] = t
    for k in p: crit_count[k] += 1
print("\nMonte Carlo, %d runs, each task triangular (0.8x, 1x, 2x the most likely):" % N)
print("   planned %d weeks; median %.0f; mean %.0f" % (total, np.median(tot), tot.mean()))
for q in (50, 80, 90, 95):
    print("   %d%% of runs finish within %.0f weeks (%.1f months)" % (q, np.percentile(tot, q), np.percentile(tot, q) / 4.33))
print("   probability of finishing by the planned %d weeks: %.1f%%" % (total, 100 * np.mean(tot <= total)))
print("\nhow often each task lies on the critical path (this is where to spend attention):")
for k, c in sorted(crit_count.items(), key=lambda kv: -kv[1]):
    if c: print("   %-10s %5.1f%%" % (k, 100 * c / N))

print("\nwhat buying down the wait is worth: halve 'order' from 4 to 2 weeks")
d2 = dict(dur0); d2["order"] = 2
t2, p2, _ = critical_path(d2); print("   planned duration %d weeks, saving %d" % (t2, total - t2))
tot2 = np.empty(5000)
for i in range(5000):
    d = {k: sample(k) for k in T}; d["order"] = d["order"] / 2
    tot2[i], _, _ = critical_path(d)
print("   median with the shorter wait: %.0f weeks, saving %.0f against %.0f" % (np.median(tot2), np.median(tot) - np.median(tot2)
```

Host organisms

What each host's biology costs and buys, in numbers.

W1 · A photobioreactor is a light reactor

week 60 · 2 h

An open pond 0.30 m deep in full midday sun.

ASSUMPTIONS (orders of magnitude, not measured values): biomass attenuation coefficient 0.20 m²/g, surface irradiance 1,500 µmol photons/m²/s, saturation irradiance 200 µmol/m²/s, maximum specific growth 2.0 per day at saturating light, biomass energy content 20 MJ/kg, latent heat of water 2.26 MJ/kg, and 300 productive days a year.

1. Compute the depth at which light falls to 10% and 1% of the surface value for densities from 0.1 to 5 g/L.

2. Compute the lit fraction of the 0.30 m pond at each density.
3. Compute the areal productivity with a saturating light response, and the volumetric rate it implies.
4. Convert the areal rate to tonnes per hectare per year.
5. Compute the water to be removed per kilogram of biomass, and the energy an evaporative route would need.

Worked solution

Self-shading first. With an attenuation coefficient of 0.20 m²/g, light falls to 1% of the surface value at 0.230 m when the culture is 0.1 g/L, 0.046 m at 0.5 g/L, 0.023 m at 1 g/L and 0.005 m at 5 g/L. In a 0.30 m pond the lit fraction is 38% at 0.1 g/L but only 3.8% at 1 g/L and 0.8% at 5 g/L. In a working culture almost every cell is in the dark at any instant, and mixing is what moves them through the light.

Areal productivity, integrating a saturating response over depth, is 18.2 g/m²/day at 0.05 g/L and then saturates at about 21.4 g/m²/day from 0.1 g/L upward. The volumetric rate is only 0.071 g/L/day. The reason the areal rate stops rising is that the pond already absorbs essentially all the incident light; extra cells go into the dark. So raising density does not raise output, it only concentrates the broth and makes harvesting cheaper. At 21.4 g/m²/day over 300 days that is 64 tonnes per hectare per year.

Dewatering is the number that decides the economics. At 1 g/L there are 999 kg of water per kg of dry biomass, and evaporating it would take 2,258 MJ against a biomass energy content of 20 MJ/kg, which is 113 times the energy in the product. Even at 20 g/L it is 111 MJ/kg, still 5.6 times. That is why harvesting is staged: settling or flocculation to concentrate, then centrifugation, then drying, with a realistic mechanical figure of order 1 to 10 MJ/kg, which is still a large fraction of 20 MJ/kg.

The assumptions matter: the attenuation coefficient, the growth rate and the photon requirement all vary several-fold between species and conditions, and the model ignores photoinhibition, night respiration and temperature. Treat the structure of the result as the lesson, measure the coefficients for your own organism, and note that this arithmetic is the honest reason bulk fuel from microalgae is hard and high-value pigments are not.

Code

```
# W1. A photobioreactor is a light reactor: self-shading, areal productivity, and the cost of taking the water out.
import numpy as np
from scipy.optimize import brentq

# ASSUMPTIONS (orders of magnitude for a dense microalgal culture, not measured values):
ka   = 0.20      # m2 per g dry weight, biomass-specific light attenuation
I0   = 1500.0    # umol photons per m2 per s at the surface, full midday sun
Ik   = 200.0     # umol/m2/s, saturation irradiance
Yph  = 1.0 / 10.0  # g dry weight per mmol photons absorbed (10 mmol photons per g: ASSUMED, a generous whole-culture value)

def depth_to(frac, X):
    """Beer-Lambert: I(z) = I0 exp(-ka X z). Depth at which I falls to frac of I0."""
    return -np.log(frac) / (ka * X)

print("Beer-Lambert self-shading, attenuation coefficient %.2f m2/g" % ka)
print("%-14s %-16s %-16s %s" % ("X (g/L)", "depth to 10% I0", "depth to 1% I0", "'photic' fraction of a 0.3 m pond"))
for X in (0.1, 0.5, 1.0, 3.0, 5.0):
    Xm = X * 1000.0                      # g/L -> g/m3
    d10, d01 = depth_to(0.10, Xm), depth_to(0.01, Xm)
    print("%-14.1f %-16s %-16s %.1f%%" % (X, "%.3f m" % d10, "%.3f m" % d01, 100 * min(d10 / 0.30, 1.0)))

# Areal productivity. Local specific growth follows a saturating light response, mu(z) = mu_max * I/(I + Ik),
# so the volumetric rate is X*mu(z) and the areal rate is its integral over depth.
mu_max = 2.0     # g biomass per g biomass per day at saturating light (ASSUMED)

def areal(X, H=0.30):
    Xm = X * 1000.0                                  # g/m3
    z = np.linspace(0, H, 20001)
    I = I0 * np.exp(-ka * Xm * z)
    mu = mu_max * I / (I + Ik)                       # per day, local
    return np.trapezoid(Xm * mu, z)                  # g per m2 per day

print("\nAreal productivity of a %.2f m pond (mu_max %.1f /day, Ik %.0f umol/m2/s)" % (0.30, mu_max, Ik))
print("%-12s %-22s %-22s %s" % ("X (g/L)", "areal g/m2/day", "volumetric g/L/day", "lit depth (to 1% I0)"))
best = None
for X in (0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0):
```

```
    A = areal(X); V = A / (0.30 * 1000.0)
    print("%-12.2f %-22.1f %-22.3f %.3f m" % (X, A, V, depth_to(0.01, X * 1000.0)))
    if best is None or A > best[1]: best = (X, A)
print("highest areal rate in this scan: X = %.2f g/L giving %.1f g/m2/day" % best)
print("the areal rate saturates because the pond already absorbs nearly all the light; adding cells only moves")
print("them into the dark, which is why volumetric productivity falls steeply while areal productivity does not rise")

print("\nWhat that areal rate means per hectare and per year (ASSUMING 300 productive days)")
for X in (0.2, 1.0):
    A = areal(X)
    print("   X = %.1f g/L: %.1f g/m2/day = %.1f t/ha/year of dry biomass" % (X, A, A * 10000 * 300 / 1e6))

# Harvesting: the water you must remove per kg of biomass
print("\nDewatering: water removed per kg of dry biomass, and the energy if you evaporated it")
hv = 2.26   # MJ per kg water, latent heat of vaporisation
print("%-14s %-22s %-24s %s" % ("X (g/L)", "water per kg biomass", "energy to evaporate (MJ/kg)", "vs biomass energy content"))
Ebio = 20.0  # MJ per kg dry biomass (ASSUMED)
for X in (0.5, 1.0, 5.0, 20.0, 200.0):
    w = (1000.0 / X) - 1.0
    E = w * hv
    print("%-14.1f %-22s %-24.0f %s" % (X, "%.0f kg" % w, E, "%.1fx the biomass energy" % (E / Ebio)))
print("evaporation is absurd, which is why harvesting is staged: settling or flocculation, then centrifugation, then drying")
print("a realistic mechanical dewatering figure is of order 1-10 MJ per kg, still a large share of %.0f MJ/kg" % Ebio)
```

W2 · The lysine case: yield, cofactors and the value of secretion

week 59 · 1.5 h

L-lysine from glucose in C. glutamicum.

VERIFIED FACTS: 4 mol NADPH per mol lysine; a maximum theoretical yield of about 0.81 mol/mol on native metabolism, rising to about 0.86 mol/mol with a transhydrogenase activity; measured oxidative pentose phosphate pathway flux in wild-type cells around 60 to 69% of glucose uptake.

ASSUMPTIONS: about 1 ATP-equivalent per lysine exported; a cell volume of 1 femtolitre, the round figure for E. coli whose measured volume spans about 0.4 to 1.8 μm³, and a density of 10^9 cells per millilitre.

1. Compute the mass yield that a 1:1 molar mapping would give, and the mass yields at the reported maxima.
2. Work out how much glucose must pass the oxidative pentose phosphate pathway per lysine if it is the only NADPH source.
3. Compute the ATP-equivalents spent on export at 50, 100 and 150 g/L.
4. Compute what a non-secreted product would be capped at by cell volume.
5. State what each number implies for host choice.

Worked solution

A 1:1 molar mapping of glucose to lysine would give 146.19/180.16 = 0.811 g/g. The reported native maximum of 0.81 mol/mol is 0.657 g/g, and 0.86 mol/mol with a transhydrogenase is 0.698 g/g. So even the theoretical ceiling gives up about a fifth of the carbon bound, because carbon is lost making the reductant.

That is the tension: 4 NADPH per lysine, and the oxidative PPP delivers 2 NADPH per glucose that passes it, losing a carbon as CO_2 each time. Two moles of glucose must therefore pass the PPP per mole of lysine if it is the sole source, which is why measured PPP flux in producing strains is high and why the literature adds NADPH-generating glycolytic steps, malic enzyme or transhydrogenase instead of simply pushing the pathway enzymes.

Export costs real energy at industrial titers: 0.34 mol/L of lysine at 50 g/L, 0.68 at 100 g/L and 1.03 at 150 g/L, each costing about one ATP-equivalent, so about one mole of ATP per litre spent purely on getting the product out at the high end. It is a term worth carrying in a balance, not an afterthought.

And the reason secretion dominates host choice: at 10^9 cells per millilitre and 1 femtolitre a cell, the intracellular volume is 0.001 L per litre of broth, which is 0.1%. Even at 300 g/L inside the cells the whole broth would hold 0.3 g/L, and at the top of the measured cell-volume range still only about 0.5 g/L. A non-secreted product is capped by cell volume and recovery means lysis, debris and a harder purification, whereas a secreted one is limited only by what the cells can make and what the medium tolerates. That, together with having no outer membrane and no lipopolysaccharide, is why C.

glutamicum and B. subtilis are the industrial choices for amino acids and enzymes.

Code

```
# W2. Why a Gram-positive secretor is attractive: export, and what the envelope changes.
# Lysine from glucose in C. glutamicum. Stoichiometry of the pathway, then the cost of getting it out.
M = dict(glc=180.16, lys=146.19, nh3=17.03)

# Lysine from oxaloacetate + pyruvate route: per mol lysine, 4 NADPH and 2 NH3 are needed (standard textbook account).
# Carbon: lysine is C6, glucose is C6, but the pathway runs through PEP/pyruvate and the TCA, so the
# published maximum theoretical yield depends on which anaplerotic and PPP routes carry the flux.
print("Lysine (C6H14N2O2, %.2f g/mol) from glucose (%.2f g/mol)" % (M["lys"], M["glc"]))
print("\nmass yield if the carbon mapped 1 mol per 1 mol: %.3f g/g" % (M["lys"] / M["glc"]))
print("VERIFIED: C. glutamicum native metabolism allows a maximum theoretical yield of 0.81 mol lysine per mol glucose")
print("          (%.3f g/g); adding a transhydrogenase activity pushes that to 0.86 mol/mol (%.3f g/g)" % (0.81 * M["lys"] / M["gl
for y_mol in (0.86, 0.81, 0.60, 0.40):
    print("   %.2f mol/mol -> %.3f g/g  (%.0f%% of the 1:1 bound)" % (y_mol, y_mol * M["lys"] / M["glc"], 100 * y_mol))

# NADPH demand, which is what makes the pentose phosphate pathway central here
print("\nNADPH: 4 mol per mol lysine (VERIFIED, reported consistently in the C. glutamicum literature).")
print("The oxidative PPP yields 2 NADPH per glucose that passes it, with one carbon lost as CO2.")
print("   so 2 mol glucose must pass the oxidative PPP per mol lysine if the PPP is the only source,")
print("   and measured PPP flux in wild-type C. glutamicum is already around 60-69% of glucose uptake")
print("   that carbon leaves as CO2, which is why NADPH supply and carbon yield pull against each other")

# Export cost. ASSUMPTION: lysine export by a carrier costs about 1 ATP-equivalent per molecule
atp_export = 1.0
print("\nExport: ASSUMING about %.0f ATP-equivalent per lysine exported" % atp_export)
for titer in (50.0, 100.0, 150.0):
    mol = titer / M["lys"]
    print("   at %5.0f g/L product: %5.2f mol/L exported, so %.2f mol/L of ATP-equivalents spent purely on export" % (titer, mol,

# Why the envelope matters: what fraction of product you must break cells to recover
print("\nIf the product stayed INSIDE the cell instead, how much could a cell even hold?")
Vc = 1.0e-15      # L per cell. E. coli is measured at about 0.4 to 1.8 um3, so 1 fL is the round number;
                  # C. glutamicum is of the same order. Using 2 fL, as some write, overstates the ceiling twofold.
ncell = 1e9       # cells per mL at a dense culture -> per litre
cells_per_L = ncell * 1000
intracell_volume = Vc * cells_per_L
print("   cell volume fraction at 1e9 cells/mL: %.4f L per L of broth (%.2f%%)" % (intracell_volume, 100 * intracell_volume))
for c_in in (100.0, 300.0):      # g/L INSIDE the cell
    print("   at %.0f g/L inside the cells, the whole broth holds only %.2f g/L" % (c_in, c_in * intracell_volume))
print("   so a non-secreted product is capped by cell volume, and recovery means lysis, debris and a harder purification")
print("   secretion is the single biggest reason C. glutamicum and B. subtilis are chosen for amino acids and enzymes")

print("\nThe envelope difference, stated plainly:")
print("   Gram-negative (E. coli): inner membrane, periplasm, outer membrane. A secreted protein must cross two membranes,")
print("   and the outer membrane is also why E. coli carries lipopolysaccharide, an endotoxin to be removed downstream.")
print("   Gram-positive (C. glutamicum, B. subtilis): one membrane and a thick peptidoglycan wall, so a signal peptide")
print("   can deliver a protein straight to the medium. B. subtilis secretes native enzymes at commercial titers for this reason.")
print("   The cost: proteases in the medium, and a wall that still has to be crossed by large or folded proteins.")
```

W3 · Why lipid yield has a hard ceiling

week 58 · 1.5 h

Lipid production from glucose in an oleaginous yeast.

ASSUMPTIONS AND STOICHIOMETRY: glucose to 2 acetyl-CoA with 2 CO_2 lost; palmitate needs 8 acetyl-CoA and 14 NADPH; a triacylglycerol is three fatty acids on a glycerol backbone; the oxidative pentose phosphate pathway gives 2 NADPH per glucose decarboxylated; the citrate shuttle costs about 1 ATP per acetyl unit moved to the cytosol.

1. Count the carbon that reaches acetyl-CoA and compute the mass-yield ceiling for palmitate and for a triacylglycerol.
2. Compare with the published range for lipid on glucose.
3. Compute the NADPH demand and the glucose it implies if the PPP is the only source.
4. Compute the ATP spent moving carbon between compartments.
5. Say what this predicts about which engineering targets will work.

Worked solution

Carbon first: glucose gives 2 acetyl-CoA and loses 2 CO_2, so only 4 of its 6 carbons reach the precursor, 66.7%. Palmitate needs 8 acetyl-CoA, hence 4 glucose, giving a mass ceiling of 256.42/(4 × 180.16) = 0.356 g/g, and about 0.393 g/g for a triacylglycerol. Published theoretical maxima for lipid on glucose sit near 0.30 to 0.32 g/g once maintenance and the glycerol backbone are included,

and good strains reach roughly two thirds of that, so 100 g/L of glucose becomes of order 20 g/L of lipid, not 40.

The reductant is the real constraint: 14 NADPH per palmitate and 42 per triacylglycerol. If the oxidative PPP were the only source, about 7 further glucose would be spent per palmitate on NADPH alone, on top of the 4 spent on carbon. That is why oleaginous yeasts lean on malic enzyme or transhydrogenase-like cycles, and why overexpressing a fatty acid synthase on its own rarely raises titer: the enzyme was not what was limiting.

Compartments add their own bill. Fatty acid synthesis is cytosolic while pyruvate dehydrogenase is mitochondrial, so acetyl units leave as citrate and are cleaved by ATP-citrate lyase at about 1 ATP each: roughly 8 extra ATP per palmitate and 24 per triacylglycerol.

What it predicts: raise cytosolic acetyl-CoA supply and NADPH supply before touching the synthase, and set the carbon-to-nitrogen ratio to trigger storage, since nitrogen starvation is what slows isocitrate dehydrogenase, accumulates citrate and feeds the cytosolic pool. The stoichiometry here is textbook and the yields are bounds, not predictions of a real strain, which will also spend carbon on biomass and maintenance.

Code

```
# W3. The oleaginous yeast: why lipid yield has a hard ceiling, and what a compartment costs.
M = dict(glc=180.16, acCoA=0.0, tag=885.4, pal=256.42)   # tripalmitin as a stand-in triacylglycerol

# Carbon route: glucose -> 2 pyruvate -> 2 acetyl-CoA (2 CO2 lost) -> fatty acid.
# Palmitate (C16) needs 8 acetyl-CoA and 14 NADPH; a triacylglycerol is 3 fatty acids on a glycerol backbone.
print("Lipid from glucose, counted by carbon")
print("  glucose C6 -> 2 acetyl-CoA (C2 each) + 2 CO2, so only 4 of 6 carbons reach the precursor: %.1f%%" % (100 * 4 / 6))
print("  palmitate C16 needs 8 acetyl-CoA, hence 4 glucose, and 14 NADPH")
y_pal_mol = 1.0 / 4.0
print("  mass yield ceiling for palmitate: %.3f g/g glucose" % (M["pal"] / (4 * M["glc"])))
print("  for a triacylglycerol (3 x C16 + glycerol), about %.3f g/g" % (M["tag"] / (12 * M["glc"] + 0.5 * M["glc"])))
print("  published theoretical maxima for lipid on glucose sit near 0.30-0.32 g/g, and the best strains reach roughly two thirds o
for y in (0.32, 0.25, 0.20, 0.15):
    print("     %.2f g/g -> from 100 g/L glucose you get %.1f g/L lipid" % (y, y * 100))

print("\nNADPH is the real constraint: 14 per palmitate, 42 per triacylglycerol")
print("  the oxidative pentose phosphate pathway gives 2 NADPH per glucose decarboxylated,")
print("  so %d glucose must be spent on NADPH alone per palmitate, on top of the 4 spent on carbon" % 7)
print("  that is why oleaginous yeasts rely on a malic enzyme or a transhydrogenase-like cycle, and why")
print("  overexpressing a fatty acid synthase alone rarely raises titer: the reductant, not the enzyme, is limiting")

print("\nWhy nitrogen limitation triggers storage")
print("  on nitrogen starvation the cell cannot make protein, AMP deaminase activity rises, mitochondrial isocitrate")
print("  dehydrogenase (which needs AMP) slows, citrate accumulates and is exported to the cytosol, where ATP-citrate")
print("  lyase cleaves it to acetyl-CoA for lipid synthesis. The trigger is a C/N ratio, not a gene.")
for cn in (10, 30, 60, 100):
    print("     C/N = %3d: %s" % (cn, "growth-favouring" if cn <= 20 else ("transition" if cn <= 40 else "storage-favouring")))

print("\nCompartments: the cost and the point")
print("  acetyl-CoA for lipid is CYTOSOLIC, made by ATP-citrate lyase; acetyl-CoA from pyruvate dehydrogenase is MITOCHONDRIAL.")
print("  moving one acetyl unit out as citrate costs about 1 ATP at the citrate lyase step, so a palmitate costs about 8 extra ATP
print("  the benefit of compartmentalising a pathway instead is a high local concentration and isolation of a toxic intermediate,"
print("  at the cost of transporting every precursor and cofactor across a membrane (week on toxicity and export)")
atp_per_pal = 8
print("  per triacylglycerol that is about %d ATP spent purely on moving carbon between compartments" % (3 * atp_per_pal))

print("\nWhat this means for host choice")
print("  S. cerevisiae: best tools, but it ferments sugar to ethanol (Crabtree) and stores little lipid, so it needs rewiring")
print("  Y. lipolytica: naturally stores lipid, grows on hydrophobic substrates, is not Crabtree-positive, but has fewer tools")
print("  K. phaffii (Pichia): strong secretion and methanol or glycerol growth, used for proteins rather than for lipid")
```

What a reviewer can actually see

Portfolio · 45 artifacts

- ☐ Public GitHub repository, readable by a stranger in five minutes (due around week 35)
- ☐ COBRApy notebook: core model growth, single and double deletions, flux variability (due around week 35)
- ☐ Reproduced growth-coupled design, compared against the published strain (due around week 39)
- ☐ Michaelis–Menten fitting script with error estimates (due around week 3)
- ☐ Stoichiometric matrix and null-space exercise in numpy (due around week 13)
- ☐ Benchling construct: promoter–RBS–ORF–terminator, with an assembly simulated (due around week 20)
- ☐ CRISPR guide design write-up with PAM sites and off-target analysis (due around week 23)
- ☐ RBS library design spanning a 100-fold predicted range (due around week 28)
- ☐ One-page host comparison table (due around week 61)
- ☐ Three case studies in the seven-field format (due around week 61)
- ☐ Six to eight paper log entries (due around week 61)
- ☐ Mini research proposal, three to five pages (due around week 66)
- ☐ Statement of purpose, five-part structure (due around week 66)
- ☐ Lab shortlist with a specific reason and paper for each (due around week 66)
- ☐ A COBRApy notebook that runs top to bottom from a fresh clone, with pinned requirements (due around week 36)
- ☐ A Golden Gate design with a checked overhang set, domesticated parts and an in-silico assembly script (due around week 21)
- ☐ A biosensor characterisation notebook: dose–response fit, operating window, and an honest interval on the dynamic range (due around week 26)
- ☐ A plate plan and protocol skeleton for a 24-variant assembly, with a note on what the simulator could not check (due around week 34)
- ☐ A one-page brief for each lab you will contact: two or three of its papers read, how your own work connects, one specific question, and a draft first email (due around week 66)
- ☐ A PHB notebook: pathway with cofactors, yield ceilings, the redox surplus, and a fed-batch table analysed for content, productivity and yield (due around week 56)
- ☐ A cost and transfer model for a CO_2/H_2-to-PHB route against sugar and hydrolysate, with a break-even electricity price and the gas-transfer coefficient it needs (due around week 57)
- ☐ A two-page proposal on one of the AIM Lab’s themes, with your own yield, redox and cost numbers, written so a PI can reply to it in two sentences (due around week 65)
- ☐ A screening-quality notebook: Z′ of an assay, false hits at a threshold, site-saturation library size, and the value of improving a given enzyme (due around week 30)
- ☐ A one-page strategy to raise the titer of a natural-product cluster: precursor demand, cloning plan with clone numbers, editing plan with controls (due around week 31)
- ☐ The bench log kept up to date: techniques, counts, dates, and a named supervisor who can vouch for them (due around week 10)
- ☐ A genome-rewriting and host-choice notebook: edit counts, an error budget, growth-time savings and the toolkit break-even (due around week 25)
- ☐ A regulator-tuning notebook: the extra decay rate for a knockdown, the leak ceiling, and the repressor level for each target (due around week 27)
- ☐ A machine-learning honesty notebook: a learning curve, a shuffled-label control, and random against grouped splits on the same data (due around week 42)
- ☐ A product-toxicity notebook: the partition mass balance, the evaporation inventory, and a solvent cost sketch (due around week 45)

☐ An in situ removal plan for your target: the method chosen from the product’s properties, the isotherm or partition coefficient to measure first, and the recovery and reuse arithmetic (due around week 46)

☐ A two-phase fermentation model you wrote and solved: the full model, the equilibrium shortcut checked against it, a transfer-limit scan, and fitted inhibition parameters with honest intervals (due around week 47)

☐ An immobilisation notebook: Thiele modulus, dead-core fraction against bead size and loading, and a continuous-reactor comparison against free cells (due around week 48)

☐ A design notebook for your first real experiment: the effect size that matters, the power calculation, the replicate plan, the randomisation and the controls (due around week 17)

☐ Four figures from the literature, one each of qPCR, gel, blot and cytometry, with what each does and does not establish written underneath (due around week 22)

☐ A full referee report on one paper, with major and minor points and a recommendation, plus a response-to-reviewers letter for your own capstone (due around week 62)

☐ A project plan for your capstone’s first year: critical path, lead times, a schedule simulation, a risk table and go/no-go points (due around week 64)

☐ A host-choice notebook: the lipid ceiling and cofactor bill for a yeast, the lysine yield and secretion argument for a Gram-positive, and the light and dewatering arithmetic for a phototroph (due around week 60)

☐ A minimum product selling price for your target at three titers, with a tornado plot naming the assumption that carries it (due around week 49)

☐ A draft genome-scale model you reconstructed and tested with MEMOTE, with the curation you did and did not do documented (due around week 37)

☐ A recorded ten-minute talk on your proposal, watched back and rebuilt once (due around week 63)

☐ A statistics note for your capstone: the independent replicate unit, the effect size you can detect, and the screen size you justified (due around week 17)

☐ A breseq run on a public SRA dataset from an evolved strain, with your mutation table and which mutations you argue are causal (due around week 32)

☐ Your target pathway added to a real genome-scale model, with theoretical yield under redox and ATP closure (due around week 65)

☐ The same analysis in Python and in MATLAB, with a note on what crossed over through SBML (due around week 38)

☐ A route to bench time started: a research-intern posting, a research-assistant application, iGEM, or a summer programme (due around week 2)

Capstone proposal — seven sections

☐ Motivation and target — why this molecule, with market and sustainability framing

☐ Host selection, justified against your own comparison table — including whether it secretes, retains or stores the product

☐ Pathway design from a central metabolite, with enzymes sourced from BRENDA, KEGG and MetaCyc

☐ Stoichiometry: the heterologous pathway added to a real genome-scale model, theoretical maximum mass and molar yield on glucose with redox and ATP closure enforced, and the FBA or OptKnock analysis that supports it — published as a notebook

☐ Build and test plan: assembly, editing, analytics, screening, DBTL iterations, the transport strategy — efflux engineering if secreted, lysis and extraction if retained — and the design of the experiment itself: how many clones you will screen and what fraction of the library that covers, how many replicates and what effect size they detect

☐ Scale-up and downstream, with a flowsheet-level techno-economic model: minimum selling price at three titers, a tornado plot naming the assumption that carries the conclusion, and a cradle-to-gate greenhouse-gas number beside the fossil incumbent

☐ Risks and alternatives, named specifically

Resources

Every link opens the material itself

Start here — everything in this group is free

If you buy nothing, these cover Phase 1 end to end. Open one and read; no account, no payment.

- Fundamentals of Biochemistry (Jakubowski & Flatt), LibreTexts [free]

A full year of biochemistry, CC-licensed. Use it in place of Lehninger for weeks 2–7; chapter order follows the standard texts closely.

- Molecular Biology of the Cell, 4th edition — NCBI Bookshelf [free]

The Alberts text, free from the NIH. Dated to 2002, which does not matter for the central dogma and cell structure you need in weeks 1 and 8. Searchable; some chapters browse better than others.

- MIT OCW 7.05 General Biochemistry (Spring 2020) [free]

Full video lectures. The sessions name specific lectures in weeks 1, 2 and 4.

- MIT OCW 7.03 Genetics (Fall 2004) [free]

Lecture notes, problem sets and exams with answers. Used in week 10.

- MIT OCW 20.020 Introduction to Biological Engineering Design [free]

Synthetic biology framing, useful background before Phase 3.

- Microbiology — OpenStax [free]

Free textbook and PDF download. Covers the microbiology in week 10 more gently than a genetics course.

The four books the curriculum is built on

These are not free. Check your university library first, then the second-hand market — older editions are fine for everything except Phase 3 technique chapters.

- Stephanopoulos, Aristidou & Nielsen — Metabolic Engineering: Principles and Methodologies (1998) [library]

The founding text, and the backbone of weeks 11–19. Written by chemical engineers, so the maths will feel familiar. Many institutions have it on ScienceDirect — try the link while on your university network.

- Nelson, Cox & Hoskins — Lehninger Principles of Biochemistry, 8th ed. [paid]

Chapters 1, 3–6, 13–16, 19–22 are what the sessions use. The LibreTexts book above is the free substitute if you would rather not buy it.

- Alberts et al. — Molecular Biology of the Cell, 7th ed. [paid]

Only chapters 1–2 and 4–8 are needed. The free 4th edition on NCBI covers all of it.

- Palsson — Systems Biology: Constraint-based Reconstruction and Analysis (2015) [library]

The reference for weeks 35–39. The Orth primer below gets you through the sessions without it.

- Lee, Nielsen & Stephanopoulos (eds.) — Metabolic Engineering: Concepts and Applications (2021) [library]

Modern companion to the 1998 text. Useful in Phase 4; not required.

The papers, in the order the sessions reach them

Where a DOI link is given it goes straight to the publisher. Where it says PubMed, the link lands on the record, which carries the free full text when one exists. Check your library proxy before paying for anything.

- Bailey 1991 — Toward a science of metabolic engineering (week 11) [paywall]

Short. Read it properly; it set the field's agenda and predicted what would stay hard.

- Stephanopoulos & Vallino 1991 — Network rigidity (week 11) [paywall]
- Nielsen & Keasling 2016 — Engineering cellular metabolism (week 11) [paywall]

Log this one in the paper log; it is the best single overview of where the field stands.

- Orth, Thiele & Palsson 2010 — What is flux balance analysis? (week 35) [open]

Free full text on PubMed Central. The clearest four pages on FBA anywhere — read before touching COBRApy.

- Burgard, Pharkya & Maranas 2003 — OptKnock (week 39) [paywall]
- Datsenko & Wanner 2000 — One-step gene inactivation (week 23) [open]

Free from PNAS. The recombineering method still used daily.

- Wang et al. 2009 — Multiplex automated genome engineering (week 23) [paywall]
- Cleto et al. 2016 — CRISPRi in C. glutamicum (week 24) [paywall]
- Delépine et al. 2018 — RetroPath2.0 (week 40) [open]
- Ko, Lee et al. 2020 — Tools and strategies of systems metabolic engineering (week 51) [paywall]

Long review. Read the omics and scale-up sections in week 51, not the whole thing.

- Hartline et al. 2021 — Dynamic control in metabolic engineering (week 19 and 24) [open]
- Yim et al. 2011 — 1,4-butanediol in E. coli (weeks 18 and 54) [paywall]

The model-driven design exemplar. Worth paying attention to the titer numbers.

- Paddon et al. 2013 — Semi-synthetic artemisinin (week 53) [paywall]
- Paddon & Keasling 2014 — Semi-synthetic artemisinin, a model for the use of synthetic biology (week 53) [paywall]
- Galanie et al. 2015 — Complete biosynthesis of opioids in yeast (week 53) [open]
- Atsumi, Hanai & Liao 2008 — Non-fermentative higher alcohols (week 55) [paywall]
- EPA — 2003 Greener Reaction Conditions Award, DuPont bio-propanediol (week 54) [free]

Short official summary of the bio-PDO process and why it won.

- FDA GRAS Notice inventory — search GRN 737, soy leghemoglobin (week 55) [free]

The regulatory file behind the Impossible Foods heme protein.

Databases you will open most days

All free, no account needed. Bookmark the first four now; weeks 5 to 7 are mostly spent in them.

- KEGG — pathway maps [free]

Map 00010 glycolysis, 00030 pentose phosphate, 00020 TCA, 00900 terpenoid backbone. The sessions name these by number.

- MetaCyc — curated pathways and enzymes [free]

Better than KEGG for sourcing a specific enzyme with its literature evidence.

- BRENDA — enzyme kinetics [free]

Where k_cat, K_M, cofactor and organism come from. Week 2 session 5 is a guided tour.

- UniProt — protein sequences and annotation [free]
- NCBI — genomes, genes, literature [free]
- eQuilibrator — $\Delta G'^{\circ}$ for any biochemical reaction [free]

Week 14 lives here. Enter a reaction and get its standard transformed Gibbs energy at a chosen pH and ionic strength; it also has a Python API if you want the values inside a model.

- EMBL-EBI services [free]

Constraint-based modelling — install these in week 35

This is the part that becomes your portfolio. Everything here is free and open source.

- COBRApy — documentation and tutorials [free]

Start at Getting Started, then Simulating with FBA, then Deletions and Flux Variability. These are the exact tutorials week 35 asks for.

- COBRApy source and install instructions [download]

pip install cobra. Add a solver if the default is slow.

- BiGG Models — genome-scale models to download [download]

Get the E. coli core model for learning and iJO1366 for the real thing. SBML and JSON downloads.

- Escher — draw flux maps onto pathway diagrams [free]

Runs in the browser. Turns a COBRApy flux vector into a figure you can put in the capstone.

- The COBRA Toolbox for MATLAB [free]

Week 38. Note what it does and does not do: it has a native OptKnock, and it does not do ^{13}C-MFA — that is INCA, below.

Computational toolchain: Python

Week 12 installs all of this. Free, and the stack almost every new computational group in the field runs on.

- NumPy — absolute basics for beginners [free]

Start here on day 1 of week 12. Arrays, shapes, indexing, broadcasting.

- NumPy — linear algebra reference [free]

solve, lstsq, matrix_rank, and the functions you will use on every stoichiometric matrix from week 13 onward.

- SciPy — integration (solve_ivp) [free]

Weeks 12 and 16. Read the f(t, y) argument convention carefully; getting it backwards is a silent error.

- SciPy — optimization (curve_fit, least_squares) [free]

Parameter fitting, and the covariance matrix your standard errors come from.

- pandas — 10 minutes to pandas [free]

COBRApy's deletion and FVA functions hand you DataFrames, so this is not optional by week 36.

- Matplotlib — pyplot tutorial [free]
- Project Jupyter [free]

Notebooks are the format a PI will actually open.

- Pro Git — the free book [free]

Chapters 1–3 are enough to keep a repository a stranger can clone. Your portfolio depends on this more than on any library.

Computational toolchain: MATLAB

Week 38, and only week 38. You are learning this to read other people's code, not to work in it daily — so get access the cheapest way you can.

- MATLAB — getting started [paid]

Check for a university site licence before paying anything; many institutions include it, and MATLAB Online then needs no install.

- GNU Octave — documentation [free]

Free and largely MATLAB-compatible for the language itself. Enough for the week 38 translation work, though the COBRA Toolbox officially targets MATLAB.

- MATLAB ODE solvers (ode45, ode15s) [paid]

ode15s is the stiff solver you want for the kinetic model in problem E2.

- Gurobi academic licence [account]

Free for academics and much faster than glpk on large models. Works with both the COBRA Toolbox and COBRApy; glpk is fine for the core model.

KIST AIM Lab and PHA bioprocess

Built from the lab’s own site and its 2026 recruiting notice. Notices change; confirm on KIST’s own pages before you act.

- AIM Lab — home [free]

Five principal investigators, each with a research page and a member page.

- AIM Lab — publications [free]
- KIST — student researcher (□□□□□) notices [free]

For people enrolled in a master’s or PhD programme. Notices appear a couple of months before a September, January or March start.

- KIST — postdoc and research-intern (□□□) notices [free]

Research-intern postings open to bachelor’s and master’s degree holders. Each posting has an attached list of fields; check it for the Clean Energy Research Center.

- The AIM Lab 2026 recruiting notice, on a third-party board [free]

A third-party copy of Dr. Ko’s group’s notice, with the research themes, the five named papers and the application steps. It closed on 7 August 2026.

- phdkim.net — graduate recruiting board, to watch for the next notice [free]

Enzymes, natural products and your shortlist

Weeks 30 and 31, plus where to find research-intern postings.

- KRIBB — jobs page, for intern and trainee postings [free]

KRIBB posts research-intern and post-master's trainee rounds with a field list attached. A 2024 round included places in the Synthetic Biology Research Center and the gene-editing centre. Check each posting's eligibility, especially for foreign nationals.

- KIST — postdoc and research-intern notices [free]

KIST research-intern rounds are open to bachelor's and master's degree holders; foreign nationals need an E-3 visa or the ability to obtain one.

- IUBMB — EC 2.3.1.94, 6-deoxyerythronolide-B synthase [free]
- In vitro reconstitution of the 6-deoxyerythronolide B synthase [open]
- Heterologous expression of pikromycin cluster with the Streptomyces artificial chromosome [open]

Genome design, regulation and machine learning

Week 25, week 27 and week 42.

- Fredens et al., Total synthesis of Escherichia coli with a recoded genome [free]

The 61-codon genome. The week's edit counts and sections come from this paper.

- Weinstock et al., Vibrio natriegens as a fast-growing host (preprint) [free]

A preprint, so treat the generation times as one measurement under particular media.

- Natural plasmid transformation of Vibrio natriegens, PNAS Nexus (2024) [open]
- KRIBB National Biofoundry Project Group [free]
- KRIBB Synthetic Biology Research Center [free]

Toxicity, export and product removal

Week 45.

- Dunlop et al., Engineering microbial biofuel tolerance and export using efflux pumps [free]

A 43-pump library against seven fuels: pumps helped for five and not for n-butanol or isopentanol.

- Newman et al., High-level production of amorpha-4,11-diene in a two-phase partitioning bioreactor [free]

Amorphadiene evaporated with a half-life of about 50 minutes, so the product was captured in a dodecane phase.

- Tsuruta et al., High-level production of amorpha-4,11-diene in Escherichia coli, PLoS ONE (2009) [open]

In situ removal, two-phase reactors and immobilisation

Week 46, week 47 and week 48.

- Prpich and Daugulis, Polymer development for phenol delivery in a solid–liquid TPPB [free]
- Rehmann, Sun and Daugulis, Polymer selection for biphenyl degradation in a solid–liquid TPPB [free]

Partition coefficients for biphenyl ranged from about 180 (Nylon 6,6) to about 11,000 (Desmopan): polymer choice is a screening job.

- Prpich and Daugulis, A solid–liquid TPPB for 3-methylcatechol production [free]
- Substrate mass transport in two-phase partitioning bioreactors, Bioprocess Biosyst. Eng. (2012) [free]

Uptake into Hytrel beads was limited by diffusion inside the polymer, not by the film outside.

- SciPy solve_ivp documentation (stiff solvers) [free]

Design, assays, publishing and planning

Week 17, week 22, week 62 and week 64.

- ICMJE recommendations on authorship [free]

The authorship criteria most journals in this area point to. Read it before you are in a position where it matters.

- COPE: publication ethics guidance and flowcharts [free]
- SciPy statistical functions reference [free]
- PubMed search: MIQE guidelines for qPCR reporting [free]

What a qPCR experiment should report: efficiency, controls, reference-gene validation and replicate structure.

Host organisms in depth

Week 58, week 59 and week 60.

- NADPH demand and theoretical lysine yields in C. glutamicum [open]

4 NADPH per lysine; maximum theoretical yield about 0.81 mol/mol native and 0.86 with a transhydrogenase.

- An NADPH-generating glycolytic pathway for L-lysine production [open]
- PubMed search: Yarrowia lipolytica for lipids and oleochemicals [free]
- PubMed search: microalgae harvesting and dewatering energy [free]

The reference shelf: standard textbooks in this field

The curriculum is built on Lehninger, Alberts, Stephanopoulos, Palsson and Nielsen & Villadsen. These are the other books students in this field are routinely taught from. You do not need to buy them; use them as references for the weeks named, and check your library or a legal electronic copy first.

- Shuler, Kargi & DeLisa — Bioprocess Engineering: Basic Concepts, 3rd ed. (Pearson, 2017) [paid]

The standard undergraduate bioprocess text and the closest book to your own training. Use it for weeks on kinetics, bioreactors and scale-up; its enzyme-kinetics and growth-kinetics chapters cover the same ground as the early weeks from an engineer's side.

- Doran — Bioprocess Engineering Principles, 2nd ed. (Academic Press, 2013) [paid]

The worked-problem book for this field: material and energy balances, fluid flow, mass transfer, reactor design, downstream processing, with over 150 problems and worked examples. If you want to drill transport and reactor calculations, work its problems rather than reading it through.

- Madigan et al. — Brock Biology of Microorganisms (Pearson, latest edition) [paid]

The standard microbiology text worldwide, and the reference for your largest background gap. Use it alongside weeks 1 to 10 and the host weeks, as a lookup rather than a read-through.

- Cornish-Bowden — Fundamentals of Enzyme Kinetics (Wiley-VCH) [paid]

The reference for enzyme kinetics beyond a single Lehninger chapter: inhibition patterns, two-substrate mechanisms, and how to fit and misfit rate equations. Use it in weeks on enzyme kinetics and enzyme engineering.

- Segel — Enzyme Kinetics: Behavior and Analysis of Rapid Equilibrium and Steady-State Enzyme Systems (Wiley) [paid]

The exhaustive treatment of kinetic mechanisms and inhibition. Older, and heavier than you usually need, but it is the book that answers a mechanism question completely.

- Fell — Understanding the Control of Metabolism (Portland Press) [paid]

The dedicated text on metabolic control analysis, by one of the people who built the field. Use it in the control-analysis week; it is where the summation and connectivity theorems are properly derived.

- Blanch & Clark — Biochemical Engineering (CRC Press / Marcel Dekker) [paid]

An alternative to Shuler and Doran with more depth on bioreactor engineering and separations. Worth consulting if a particular chapter in the others is too brief.

- Snyder, Peters, Henkin & Champness — Molecular Genetics of Bacteria (ASM Press) [paid]

The standard bacterial genetics text: transformation, conjugation, transposons, recombination and selection design. Use it alongside the genome-engineering weeks, where thinking in terms of selections and screens matters more than protocol detail.

Synthetic biology and biofoundry

Weeks 21 to 34. The first three links are the research centre's own pages and the paper that motivated these weeks.

- KRIBB — Synthetic Biology and Bioengineering Research Center [free]

The centre's own description of its research areas, including genetic devices, modelling and CRISPR, enzyme engineering and K-Biofoundry.

- K-Biofoundry [free]
- Global Biofoundry Alliance [free]
- Seong et al., Connector-enabled integration of Golden Gate Assembly and yeast recombination, Trends Biotechnol. (2026) [library]
- Addgene — destination vector deposited with that paper [account]

Academic and non-profit use only; the page lists the deposit terms.

- Biopython Tutorial and Cookbook [free]

- Biopython — restriction enzymes [free]
- Opentrons Python API documentation [free]
- SnapGene — plasmid design and visualisation [paid]

A commercial alternative to Benchling. Benchling's free academic tier covers everything in this curriculum.

Economics, and models you did not write

Weeks 37 and 49. All free and open source; the commercial alternatives are named where they matter.

- BioSTEAM — documentation [free]

Flowsheet-level process simulation and techno-economic analysis in Python, with uncertainty built in. Its paper reports economics closely matching SuperPro Designer and Aspen Plus benchmarks.

- BioSTEAM — source and the Bioindustrial-Park model library [download]

Start by running a published biorefinery configuration unchanged, before building your own.

- CarveMe — automated model reconstruction from a genome [download]

Draft a genome-scale model for an organism nobody has curated. Treat every gap-filled reaction as a hypothesis.

- gapseq — pathway prediction and reconstruction [download]

The evidence-driven alternative to CarveMe; slower, and usually better annotated.

- MEMOTE — standardised model testing [download]

Run it on your draft and on a curated model, then learn which failures change a prediction and which are only metadata.

- Aspen Plus and SuperPro Designer [paid]

The industry standards. Worth knowing by name, and worth learning if you are heading for industry; not needed for this curriculum.

Protein structure and design

Week 29, as literacy. You are learning to name the right tool and read its output, not to run a GPU pipeline.

- Dauparas et al., LigandMPNN, Nature Methods (2025) [open]

Free full text. The ligand-aware inverse-folding model — the right tool for a pocket or cofactor redesign, where ProteinMPNN is blind to the cofactor.

- Institute for Protein Design — introducing LigandMPNN [free]

The short readable version, with example workflows.

- LigandMPNN source and weights [download]

MIT licensed. Needs a GPU to be comfortable; reading the repository is useful even if you never run it.

- AlphaFold Protein Structure Database [free]

Predicted structures for most known proteins, with per-residue confidence. Start here rather than running anything.

Expression data and statistics

Weeks 17 and 33, plus week 52.

- Love, Huber & Anders, DESeq2, Genome Biology (2014) [open]

Read it for dispersion shrinkage and why small n is workable at all.

- Bioconductor — DESeq2 package and vignette [free]
- NCBI Gene Expression Omnibus [free]

Public count matrices to reanalyse, including many strain comparisons.

- scikit-learn — Gaussian process regression [free]

The surrogate behind week 41 and problem G2.

- REBASE — restriction enzyme database [free]

Week 52. Predict which restriction systems will destroy your construct in a wild strain.

Sequencing and command-line genomics

Week 32. All free. If you cannot install command-line tools, a public Galaxy server runs the same pipeline in a browser.

- breseq — documentation and downloads [free]

The standard tool for calling mutations after adaptive laboratory evolution in bacteria. Reads the reference, reports point mutations, indels and new junctions, and writes annotated HTML.

- breseq on GitHub — releases and issues [download]

Precompiled binaries for Linux and macOS; building from source needs a C++ toolchain and R.

- FastQC — read quality control [download]

The first thing you run on any FASTQ file. Per-base quality and adapter content tell you what to trim.

- Bowtie 2 — read alignment [free]
- BWA — read alignment [free]

Either aligner is fine for a bacterial genome; pick one and learn its output.

- SAMtools and HTSlib [free]

SAM to sorted, indexed BAM, coverage, and everything downstream handles these formats.

- NCBI Sequence Read Archive [free]

Where the public evolved-strain datasets live. Search a published ALE study's accession and reanalyse its data yourself.

- Galaxy — run the pipeline in a browser [account]

Free account. Worth knowing as a fallback, though the command line is what a lab will expect.

- Integrative Genomics Viewer (IGV) [free]

Look at the alignment around a called mutation before you believe it.

Fluxomics software

Week 43. A correction worth absorbing: ^{13}C-MFA is not a COBRA Toolbox feature. These are the tools that actually do it.

- INCA — the paper and the software [library]

MATLAB-based, does both steady-state and isotopically non-stationary MFA. Free for academic use under a Vanderbilt licence; request it through the group rather than expecting a download link.

- INCAWrapper — drive INCA from Python [free]

Set up models, run INCA and get results back as CSV without leaving Python. Needs MATLAB and an INCA licence underneath.

- OpenFLUX — the paper [open]

Free full text. The older open alternative, MATLAB algorithms with a Java model parser.

Design tools for Phase 3

Free for academic use; two need an account.

- Benchling — plasmid maps, assembly simulation, CRISPR guides [account]

Free academic tier. Weeks 20 and 23 are built around it.

- Addgene — plasmids, maps and protocols [free]

The Plasmids 101 series is the clearest free explanation of cloning methods.

- iGEM Registry of Standard Biological Parts [free]

Browse real promoters and RBSs with measured strengths; week 8.

- CHOPCHOP — CRISPR guide design [free]

Alternative to Benchling's guide tool, with off-target scoring.

- RBS Calculator — De Novo DNA [account]

Registration required for the current version. Week 28 uses it to design a library spanning a hundred-fold range.

- RBS Calculator v1.0 source — free and open [download]

GPLv3. Use this, or OSTIR, if you would rather not register.

- OSTIR — open-source translation initiation rate prediction [open]

Paper and links for the fully open alternative to the RBS Calculator.

Courses, if you want lectures alongside the reading

- Coursera — Introduction to Industrial Bioprocess Development (DTU) [account]

Free to audit. Closest to your existing bioprocess background; good company for week 44.

- Coursera — Industrial Biotechnology (Manchester) [account]

Free to audit.

- edX — synthetic biology courses [account]

Includes the MITx Principles of Synthetic Biology course; audit track is free.

- MIT OCW — find 7.344 Directed Evolution and other biology courses [free]

Week 28 uses the directed evolution seminar; search the department listing for it.

Journals to follow from week 11 onward

Skim the tables of contents monthly. By week 51 you should recognise most of the recurring groups.

- Metabolic Engineering [library]

The field's journal. Its editorial board is also your first lab shortlist, in week 66.

- Metabolic Engineering Communications [open]

Fully open access. Start here if you have no library access.

- ACS Synthetic Biology [library]
- Nature Chemical Biology [library]
- Biotechnology and Bioengineering [library]
- bioRxiv — preprints, all free [open]

Where much of this work appears months before the journal version.

Getting bench time — start this in week 2

The one gap this curriculum cannot close on its own. The bench-time portfolio item is due in week 2 for that reason.

- iGEM [free]

Team-based synthetic biology with real cloning. The single best entry point if you are near a participating university.

- NSF Research Experiences for Undergraduates [free]

US summer placements, paid, applications usually close in winter.

- Novo Nordisk Foundation Center for Biosustainability (DTU) [free]

One of the largest metabolic engineering centres; check its vacancies and student pages.

- Joint BioEnergy Institute [free]

Keasling's institute. Read its research pages before writing to anyone there.